

Data Science II with python (Class notes)

STAT 303-2-Sec20

Lizhen Shi

2026-01-02

Table of contents

2	Simple Linear Regression	10
2.1	Learning objectives	10
2.2	Dataset	10
2.3	Simple Linear Regression	11
2.3.1	Training with <code>statsmodels</code> (formula API)	12
2.3.2	Model Prediction and Error Metrics	15
2.3.3	Confidence interval vs. prediction interval	17
2.3.4	Training with <code>sklearn</code>	18
2.3.5	Training with <code>statsmodels</code> (API)	21
2.4	Summary and Key Takeaways	23
3	Multiple Linear Regression	24
3.1	Learning objectives	24
3.2	Dataset	24
3.3	Multiple Linear Regression	25
3.3.1	Training with <code>statsmodels</code> (formula API)	25
3.3.2	Model Prediction and Error Metrics with <code>statsmodels</code>	26
3.3.3	Training with <code>sklearn</code>	27
3.3.4	Model Prediction and Error Metrics with <code>sklearn</code>	28
3.3.5	Diagnostics (test set)	28
3.4	Effect of adding noisy predictors on R^2	29
4	Extending LR: Interactions and Qualitative Features	31
4.1	Overview	31
4.2	Learning Objectives	32
4.2.1	Core Concepts	32
4.2.2	Technical Skills	32
4.2.3	Advanced Applications	32
4.3	Dataset	32
4.4	Part 1: Working with <code>statsmodels</code>	33
4.4.1	Interaction Terms (Continuous Variables)	33
4.4.2	Qualitative (Categorical) Predictors	36
4.4.3	Interactions Between Categorical and Continuous Variables	40
4.5	Part 2: Working with <code>scikit-learn</code>	45
4.5.1	Generating Interaction Terms with <code>PolynomialFeatures</code>	45

4.5.2	Full Polynomial Features (Including Squared Terms)	47
4.5.3	Limitations of PolynomialFeatures	47
4.6	Part 3: Categorical Variables in Scikit-Learn	51
4.6.1	Two Main Approaches:	51
4.6.2	Common Pitfalls:	51
4.6.3	Method 1: Using <code>pd.get_dummies()</code>	51
4.6.4	Method 2: Using <code>OneHotEncoder</code> (Recommended)	54
4.7	Summary and Best Practices	58
4.7.1	When to Use Each Approach:	58
4.7.2	Best Practices Checklist:	59
5	Extending LR: Feature Transformations	60
5.1	Learning objectives	60
5.2	Feature transformations in statsmodels	60
5.2.1	Why feature transformations matter	60
5.3	Polynomial Features in scikit-learn (same car dataset)	70
5.3.1	Handling Zero and Negative Values with Log Transformations	70
5.3.2	Train/test split	71
5.3.3	Baseline Model (original Features)	72
5.3.4	Transform Features with PolynomialFeatures (<code>degree = 2</code>)	73
5.3.5	Transform features with PolynomialFeatures (<code>degree = 3</code>)	74
5.3.6	<code>degree = 4</code>	76
5.3.7	Putting all together	78
5.3.8	Key takeaway:	80
5.4	Summary	80
6	Beyond Fit (implementation)	82
7	Beyond Fit (statistical theory)	97
7.1	Outliers	99
7.2	High leverage points	104
7.2.1	Identifying extrapolation using leverage	107
7.3	Influential points	108
7.3.1	Influence on single fitted value (DFFITS)	110
7.3.2	Influence on all fitted values (Cook's distance)	113
7.3.3	Influence on regression coefficients (DFBETAS)	114
7.4	Collinearity	117
7.4.1	Why and how is collinearity a problem	117
7.4.2	How to measure collinearity/multicollinearity	118
7.4.3	Manual computation of VIF	120
7.4.4	When can we overlook multicollinearity?	124

8	Introduction to Logistic Regression	125
8.1	Learning Objectives	125
8.2	Import Libraries and Load Data	125
8.3	Data Preparation	126
8.3.1	Imputing missing values	126
8.3.2	Check Target Distribution	127
8.3.3	Train-Test Splitting	128
8.4	Implementation 1: Statsmodels with Formula API	128
8.4.1	Interpreting the Coefficients	129
8.4.2	Prediction from Statsmodels	129
8.5	Implementation 2: Scikit-learn Integration	132
8.5.1	Understanding <code>.predict()</code> vs <code>.predict_proba()</code>	133
8.5.2	Turning off the default regularization	135
8.5.3	Pipeline Implementation	138
8.6	Statsmodels vs Sklearn Result Comparison	142
9	Logistic Regression Evaluation	145
9.1	Learning Objectives	145
9.2	Import Libraries and Load Data	145
9.3	Data Preparation	146
9.3.1	Imputing missing values	146
9.3.2	Check Target Distribution	147
9.4	Stratified Splitting	148
9.5	Scikit-learn Implementation	149
9.5.1	Pipeline Implementation	149
9.6	Performance Evaluation	152
9.6.1	Confusion Matrix	152
9.6.2	Precision, Recall, and F1 Score	154
9.6.3	Threshold Effects: Precision-Recall Tradeoff	156
9.6.4	ROC Curve and AUC Score	158
9.6.5	Summary: Choosing the Right Metric	160
10	Logistic Regression: Feature Engineering	162
10.1	Decision Boundaries in Classification	162
10.1.1	Linear Decision Boundary (Linearly Separable Data)	162
10.1.2	Nonlinear Decision Boundary (Non-Linearly Separable Data)	167
10.1.3	Step 2: Polynomial Feature Transformation (The Solution)	171
10.1.4	Analysis: Finding the Right Balance	175
10.1.5	Summary: Polynomial Feature Transformation	176
10.1.6	Beyond Polynomial Transformations	176
10.2	Practical Example: Feature Engineering with Titanic Dataset	177
10.2.1	Why This Example Is Valuable	177
10.2.2	Hypotheses to Test	177

10.2.3	Our Experimental Approach	178
10.2.4	The Real Lesson	178
10.2.5	Load and Explore the Titanic Dataset	178
10.2.6	Data Preprocessing and Train/Test Split	181
10.2.7	Model 1: Baseline Model (Raw Features Only)	183
10.2.8	Testing Feature Engineering Hypotheses	185
10.2.9	Step 2: Add Interaction Term ($\text{sex} \times \text{pclass}$)	186
10.2.10	Step 3: Add Binned Features (age_group , fare_group)	188
10.2.11	Step-by-Step Performance Comparison	191
10.3	Key Insights: What We Actually Learned	194
10.3.1	Why Didn't Our Features Help More?	194
10.3.2	Key Takeaway	194
10.3.3	Next Steps: Regularization	195
11	Cross-Validation	196
11.1	Review: Train-Test Split and Its Limitations	196
11.1.1	Train-Test Split Recap	196
11.1.2	Limitations of Train-Test Split	196
11.2	Cross-Validation: Key Concepts	197
11.2.1	Steps for K-fold cross-validation	197
11.3	Cross-Validation in Scikit-Learn	199
11.3.1	<code>cross_val_score</code>	199
11.3.2	<code>cross_validate</code> in Scikit-Learn	203
11.3.3	<code>cross_val_predict</code> in sklearn	204
11.3.4	Key Differences Between <code>cross_val_score</code> , <code>cross_validate</code> , and <code>cross_val_predict</code>	205
11.3.5	Advantages and Disadvantages of Cross-Validation	206
11.4	Cross-Validation for Hyperparameter Tuning	206
11.4.1	Finding the optimal Degree in Polynomial Regression	207
11.4.2	Tuning the classification threshold	222
12	Regularization	224
12.1	Why do we need regularization?	224
12.1.1	The Challenge of Overfitting and Underfitting	224
12.1.2	Understanding the Bias-Variance Tradeoff	225
12.1.3	Visualizing Overfitting vs. Underfitting	225
12.2	Simulating Data for an Overfitting Linear Model	225
12.2.1	Generating the data	225
12.2.2	Splitting the Data	227
12.2.3	Splitting the target and features	227
12.2.4	Building Models	228
12.2.5	Overfitting Indicated by Training and Test MRSS Trends	234

12.3	Regularization: Combating Overfitting	235
12.3.1	Regularized Loss Function	235
12.3.2	Regularization Does Not Penalize the Intercept	235
12.3.3	Types of Regularization	235
12.3.4	Why Is Feature Scaling Required in Regularization?	236
12.3.5	Ridge Regression: L2 Regularization	237
12.3.6	Lasso Regression: L1 Regularization	243
12.3.7	Elastic Net Regression: Combining L1 and L2 Regularization	249
12.3.8	RidgeCV, LassoCV, and ElasticNetCV in Scikit-Learn	254
12.4	Reference	256
13	Feature Selection	257
13.1	Filter Methods (Statistical approaches)	257
13.1.1	Characteristics:	258
13.1.2	Methods	258
13.2	Wrapper Methods	264
13.2.1	Characteristics:	264
13.2.2	Recursive Feature Elimination (RFE)	265
13.2.3	Recursive Feature Elimination with Cross-Validation (RFECV)	265
13.2.4	Sequential Feature Selection	267
13.3	Embedded methods	272
13.3.1	Characteristics:	272
13.4	Comparison of Feature Selection Methods	274
13.5	Conclusion	274
13.5.1	Reference:	274
	Appendices	276
A	Assignment 1: Statistical Learning and Simple Linear Regression	276
	Instructions	276
A.1	1) Case Studies: Regression vs Classification and Prediction vs Inference (16 points)	277
A.1.1	1a)	277
A.1.2	1b)	277
A.1.3	1c)	277
A.1.4	1d)	277
A.2	2) Examples for Different Regression Metrics: RMSE vs MAE (8 points)	278
A.2.1	2a)	278
A.2.2	2b)	278
A.3	3) Simple Linear Regression: Formulation (3 points)	278
A.4	4) Modeling the Petrol Consumption in U.S. States (58 points)	278
A.4.1	4a)	279

A.4.2	4b)	279
A.4.3	4c)	279
A.4.4	4d)	279
A.4.5	4e)	279
A.4.6	4f)	279
A.4.7	4g)	280
A.4.8	4h)	280
A.4.9	4i)	280
A.4.10	4j)	280
A.4.11	4k)	280
A.4.12	4l)	280
A.4.13	4m)	281
A.4.14	4n)	281
A.5	5) Reproducing the Results with Scikit-Learn (15 points)	281
A.5.1	5a)	281
A.5.2	5b)	281
A.6	6) Bonus Question (15 points)	282
A.6.1	6a)	282
A.6.2	6b)	282
A.6.3	6c)	282

B Assignment 2: MLR 283

	Instructions	283
B.1	1) Multiple Linear Regression (24 points)	284
B.1.1	1a)	284
B.1.2	1b)	284
B.1.3	1c)	284
B.1.4	1d)	284
B.1.5	1e)	284
B.1.6	1f)	285
B.1.7	1g)	285
B.1.8	1h)	285
B.2	2) Multiple Linear Regression with Variable Transformations (23 points)	285
B.2.1	2a)	286
B.2.2	2b)	286
B.2.3	2c)	286
B.2.4	2d)	286
B.2.5	2e)	287
B.3	3) Variable Transformations and Interactions (33 points)	287
B.3.1	3a)	287
B.3.2	3b)	287
B.3.3	3c)	287
B.3.4	3d)	288

B.3.5	3e)	288
B.3.6	3f)	288
B.3.7	3g)	289
B.4	4) Prediction with Sklearn (20 points)	289
B.4.1	Instructions	289
C	Datasets, assignment and project files	291

1

Preface {.unnumbered}

This book serves as the course notes for **STAT 303 (Sec 20), Winter 2026** at Northwestern University. It is the second course in the Python sequence and introduces linear models and other essentials in predictive modeling. Key topics include linear regression, logistic regression, cross-validation, regularization, and feature selection. To get the most out of these notes, please read the accompanying [textbook](#) before each class.

This edition builds on foundational materials developed by **Professor Arvind Krishna**, whose work has provided a strong framework for this resource. We are grateful for his contributions, which continue to shape the book's development.

Throughout the quarter, we will update and refine these notes to improve clarity, depth, and alignment with course objectives. Because this is a living document, feedback is always welcome. Contributions from students, instructors, and the broader academic community help make it stronger and more effective.

If you have suggestions, please share them using our dedicated [Textbook Improvement Form](#).

We hope these notes become a helpful companion as you build skills and confidence in data science with Python.

2 Simple Linear Regression

Read section 3.1 of the book before using these notes.

Before coding, create a dedicated Python environment for this course (use the same method as STAT303-1). If you see a `ModuleNotFoundError`, install the missing package, for example `pip install scikit-learn` or `pip install statsmodels`.

Note: these notes focus on implementation. The book provides the conceptual explanations.

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import pandas as pd
import seaborn as sns

sns.set_theme(style="whitegrid", font_scale=1.2)
```

2.1 Learning objectives

By the end of this notebook you should be able to:

- Fit a simple linear regression model ($Y = \beta_0 + \beta_1 X + \varepsilon$) using `statsmodels` and `sklearn`.
- Interpret the slope and intercept in context.
- Evaluate model performance on a held-out test set (RMSE, MAE, R^2).
- Compute and visualize confidence intervals (mean response) vs prediction intervals (new observation).

2.2 Dataset

- Training set: `Datasets/Car_features_train.csv`, `Datasets/Car_prices_train.csv`
- Test set: `Datasets/Car_features_test.csv`, `Datasets/Car_prices_test.csv`

```
# Reading training set
trainf = pd.read_csv('./Datasets/Car_features_train.csv') # Predictors
trainp = pd.read_csv('./Datasets/Car_prices_train.csv') # Response
train = pd.merge(trainf, trainp)
print(train.shape)
train.head()
```

(4960, 11)

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price
0	18473	bmw	6 Series	2020	Semi-Auto	11	Diesel	145	53.3282	3.0	37980
1	15064	bmw	6 Series	2019	Semi-Auto	10813	Diesel	145	53.0430	3.0	33980
2	18268	bmw	6 Series	2020	Semi-Auto	6	Diesel	145	53.4379	3.0	36850
3	18480	bmw	6 Series	2017	Semi-Auto	18895	Diesel	145	51.5140	3.0	25998
4	18492	bmw	6 Series	2015	Automatic	62953	Diesel	160	51.4903	3.0	18990

```
# Read the test data
testf = pd.read_csv('./Datasets/Car_features_test.csv') # Predictors
testp = pd.read_csv('./Datasets/Car_prices_test.csv') # Response
test = pd.merge(testf, testp)
print(test.shape)
test.head()
```

(2672, 11)

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price
0	12000	merc	GLS Class	2017	Automatic	12046	Diesel	150	37.1965	3.0	38000
1	12001	vw	Amarok	2017	Automatic	37683	Diesel	260	36.2442	3.0	23495
2	12004	merc	GLS Class	2019	Automatic	10000	Diesel	145	33.8061	3.0	59999
3	12013	skoda	Scala	2019	Manual	3257	Petrol	145	49.8767	1.0	16713
4	12017	audi	RS6	2015	Semi-Auto	20982	Petrol	325	29.4571	4.0	46000

2.3 Simple Linear Regression

We will build a simple linear regression model to predict car price based on engine size.

2.3.1 Training with statsmodels (formula API)

We will use `statsmodels.formula.api`. The formula interface lets you specify a model with a compact string (similar to R), which is convenient for statistical modeling.

```
import statsmodels.formula.api as smf
```

```
# Let's create the model_sm
```

```
# ols stands for Ordinary Least Squares - the name of the algorithm that optimizes Linear Regression
```

```
# data input needs the dataframe that has the predictor and the response
```

```
# formula input needs to:
```

```
    # be a string
```

```
    # have the following syntax: "response~predictor"
```

```
# Using engineSize to predict price
```

```
ols_object = smf.ols(formula = 'price~engineSize', data = train)
```

```
#Using the fit() function of the 'ols' class to fit the model_sm, i.e., train the model_sm
```

```
model_sm = ols_object.fit()
```

```
#Printing model_sm summary which contains among other things, the model_sm coefficients
```

```
model_sm.summary()
```

Dep. Variable:	price	R-squared:	0.390			
Model:	OLS	Adj. R-squared:	0.390			
Method:	Least Squares	F-statistic:	3177.			
Date:	Fri, 16 Jan 2026	Prob (F-statistic):	0.00			
Time:	09:29:29	Log-Likelihood:	-53949.			
No. Observations:	4960	AIC:	1.079e+05			
Df Residuals:	4958	BIC:	1.079e+05			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-4122.0357	522.260	-7.893	0.000	-5145.896	-3098.176
engineSize	1.299e+04	230.450	56.361	0.000	1.25e+04	1.34e+04
Omnibus:	1271.986	Durbin-Watson:	0.517			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	6490.719			
Skew:	1.137	Prob(JB):	0.00			
Kurtosis:	8.122	Cond. No.	7.64			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The fitted equation (from this dataset) is: $\hat{price} = -4122.0357 + 12990 * \text{engineSize}$

- R^2 is 39%. This is the proportion of variance in car price explained by **engineSize**.
- The coefficient of **engineSize** ($\hat{\beta}_1$) is statistically significant (p-value near 0), so there is evidence of a linear relationship.
- The 95% CI for $\hat{\beta}_1$ is [1.25e+04, 1.34e+04].
- Prediction intervals are not shown in the summary table.

The coefficient of **engineSize** is 1.299e+04 (dollars per liter).

- A one-unit increase in **engineSize** increases the expected price by about \$12,990.
- A three-unit increase increases the expected price by about \$38,970.

The coefficients can also be returned directly using the **params** attribute of the fitted **model_sm** object:

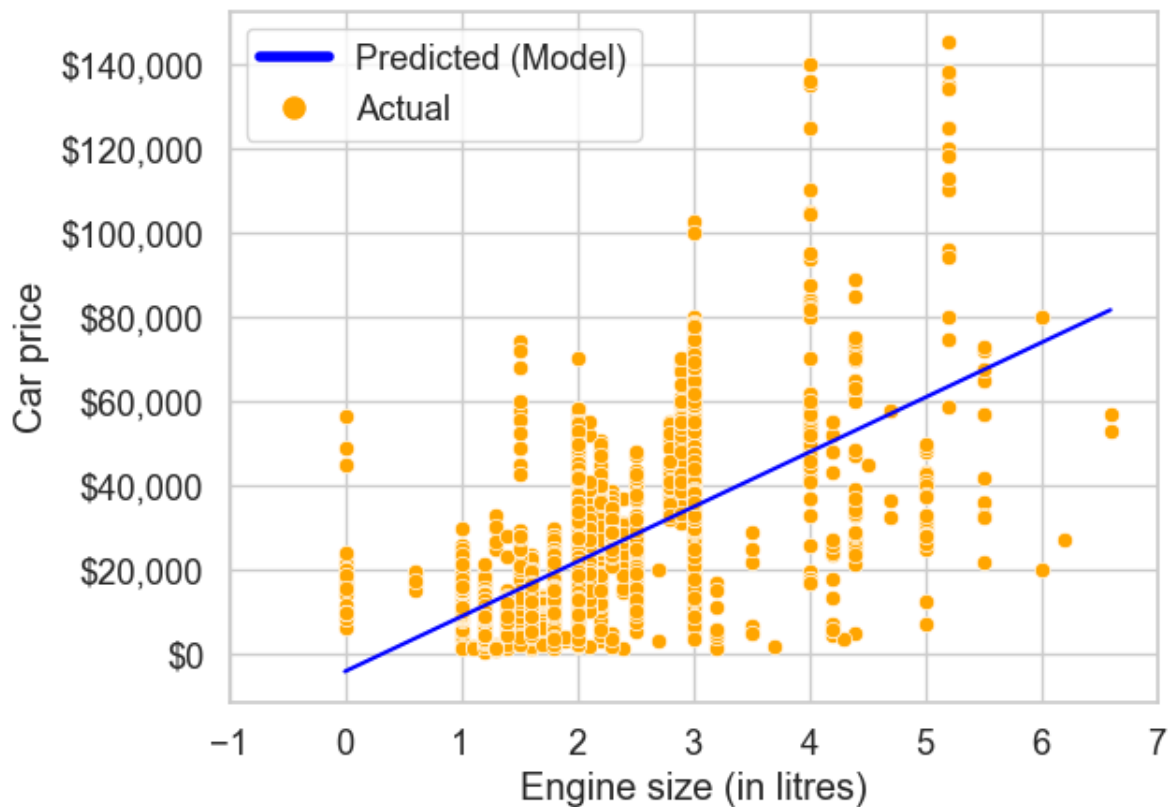
```
model_sm.params
```

```
Intercept      -4122.035744
engineSize      12988.281021
dtype: float64
```

Visualize the regression line

```
ax = sns.scatterplot(x=train.engineSize, y=train.price, color="orange")
sns.lineplot(x=train.engineSize, y=model_sm.fittedvalues, color="blue")
plt.xlim(-1, 7)
plt.xlabel("Engine size (in litres)")
plt.ylabel("Car price")
ax.yaxis.set_major_formatter("${x:,.0f}")

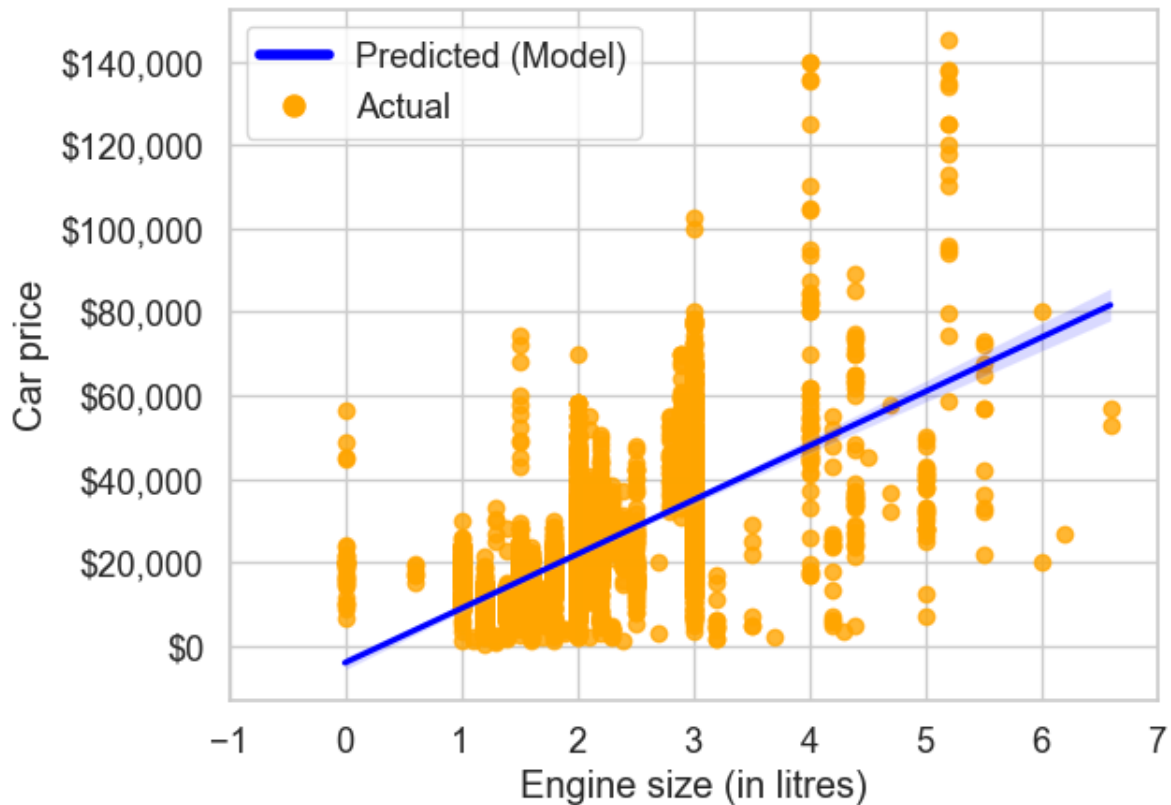
legend_elements = [
    Line2D([0], [0], color="blue", lw=4, label="Predicted (Model)"),
    Line2D([0], [0], marker="o", color="w", label="Actual", markerfacecolor="orange", marker
]
ax.legend(handles=legend_elements, loc="upper left");
```



The same plot can be made with `seaborn.regplot()`. It fits a simple linear regression with `y` as the response and `x` as the predictor, then overlays the fitted line on the scatterplot.

```
ax = sns.regplot(
    x="engineSize",
    y="price",
    data=train,
    color="orange",
    line_kws={"color": "blue"},
)
plt.xlim(-1, 7)
plt.xlabel("Engine size (in litres)")
plt.ylabel("Car price")
ax.yaxis.set_major_formatter("${x:,.0f}")
ax.legend(handles=legend_elements, loc="upper left");

# Note: some `engineSize` values appear to be 0; those are likely data issues
# and should ideally be handled (e.g., imputed/filtered) before modeling.
```



The light shaded region around the blue line is a 95% confidence interval for the mean response.

2.3.2 Model Prediction and Error Metrics

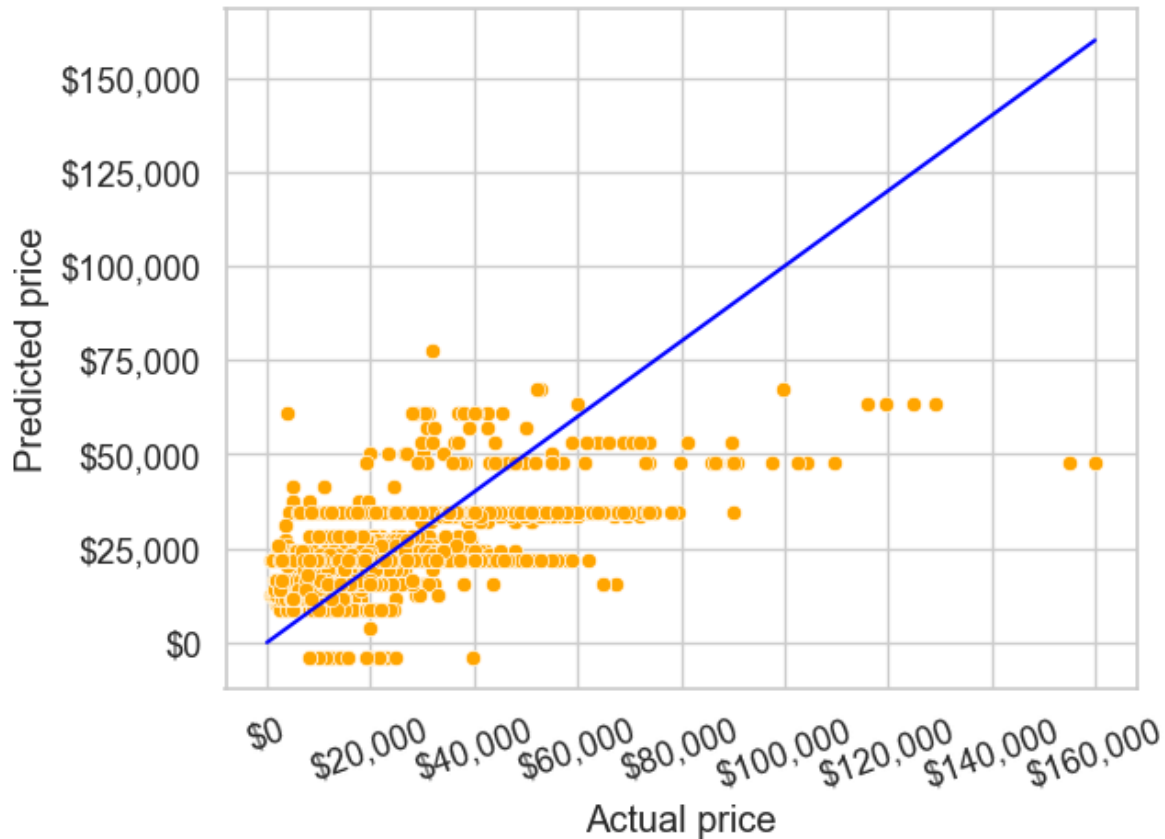
Now evaluate the fitted model on the test set.

```
# Predict car prices for the test set using the fitted statsmodels model
# (statsmodels will match the predictor name `engineSize` from `testf`).
pred_price = model_sm.predict(testf)
```

Visualize predicted prices against actual prices (perfect predictions would lie on the line $y = x$).

```
sns.scatterplot(x = testp.price, y = pred_price, color = 'orange')
#In case of a perfect prediction, all the points must lie on the line x = y.
ax = sns.lineplot(x = [0, testp.price.max()], y = [0, testp.price.max()], color='blue') #Plotting
plt.xlabel('Actual price')
```

```
plt.ylabel('Predicted price')
ax.yaxis.set_major_formatter('${x:,.0f}')
ax.xaxis.set_major_formatter('${x:,.0f}')
plt.xticks(rotation=20);
```



The predictions are limited because we are using only one predictor. We can likely improve performance by adding more predictors in multiple linear regression.

`statsmodels` does not include common error metrics, so we compute RMSE and MAE with NumPy. (The R^2 value is shown in the model summary above.)

```
sm_rmse = np.sqrt(np.mean((testp.price - pred_price)**2))
sm_mae = np.mean(np.abs(testp.price - pred_price))
print(f'Statsmodels Linear Regression Test RMSE: ${sm_rmse:,.2f}')
print(f'Statsmodels Linear Regression Test MAE: ${sm_mae:,.2f}')
```

```
Statsmodels Linear Regression Test RMSE: $12,995.11
Statsmodels Linear Regression Test MAE: $9,411.33
```


On this test set, the RMSE is around \$13k.

2.3.3 Confidence interval vs. prediction interval

Next we compute confidence and prediction intervals for the test-set predictions.

```
#Using the get_prediction() function associated with the 'model_sm' object to get the intervals
intervals = model_sm.get_prediction(testf)
```

```
#The function requires specifying alpha (probability of Type 1 error) instead of the confidence level
intervals.summary_frame(alpha=0.05)
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	34842.807319	271.666459	34310.220826	35375.393812	9723.677232	59961.937406
1	34842.807319	271.666459	34310.220826	35375.393812	9723.677232	59961.937406
2	34842.807319	271.666459	34310.220826	35375.393812	9723.677232	59961.937406
3	8866.245277	316.580850	8245.606701	9486.883853	-16254.905974	33987.396528
4	47831.088340	468.949360	46911.740050	48750.436631	22700.782946	72961.393735
...	...	...	...	...	...	...
2667	47831.088340	468.949360	46911.740050	48750.436631	22700.782946	72961.393735
2668	34842.807319	271.666459	34310.220826	35375.393812	9723.677232	59961.937406
2669	8866.245277	316.580850	8245.606701	9486.883853	-16254.905974	33987.396528
2670	21854.526298	184.135754	21493.538727	22215.513869	-3261.551421	46970.604017
2671	21854.526298	184.135754	21493.538727	22215.513869	-3261.551421	46970.604017

Show the confidence and prediction intervals for the test-set prices.

```
interval_table = intervals.summary_frame(alpha=0.05)
```

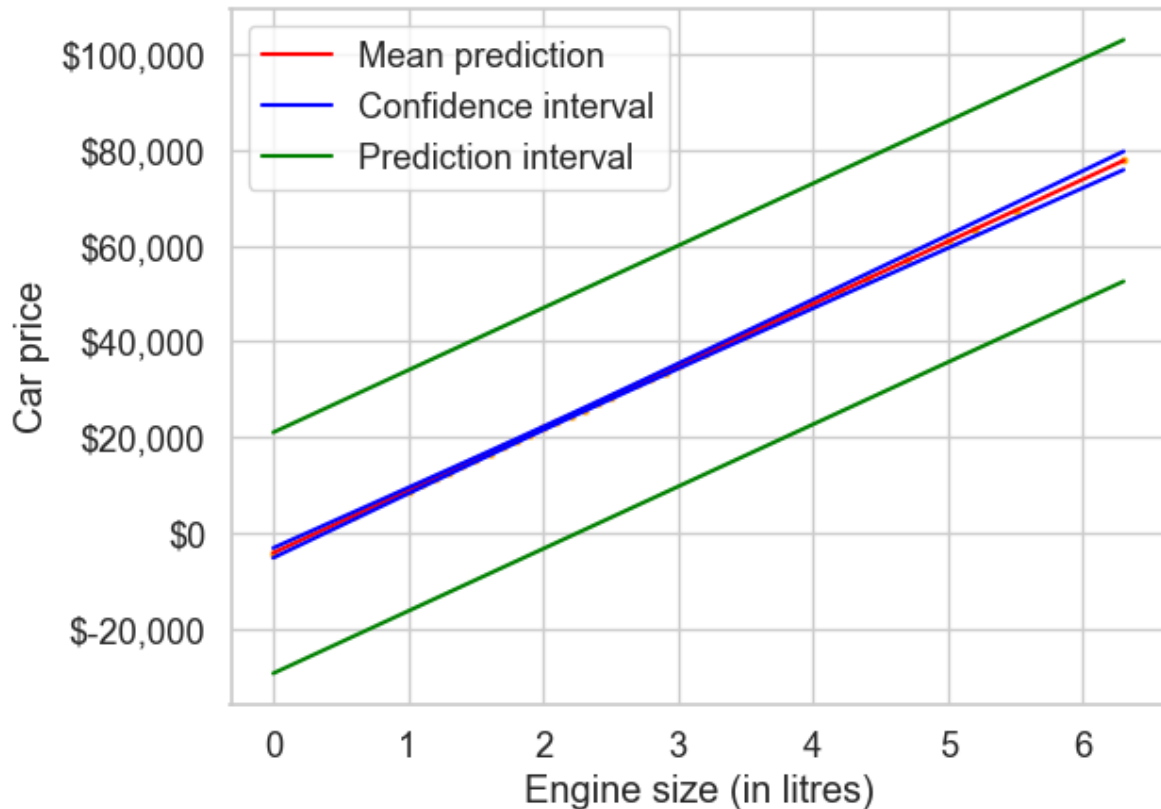
```
ax = sns.scatterplot(x = testf.engineSize, y = pred_price,color = 'orange', s = 10)
sns.lineplot(x = testf.engineSize, y = pred_price, color = 'red')
sns.lineplot(x = testf.engineSize, y = interval_table.mean_ci_lower, color = 'blue')
sns.lineplot(x = testf.engineSize, y = interval_table.mean_ci_upper, color = 'blue')
sns.lineplot(x = testf.engineSize, y = interval_table.obs_ci_lower, color = 'green')
sns.lineplot(x = testf.engineSize, y = interval_table.obs_ci_upper, color = 'green')

legend_elements = [Line2D([0], [0], color='red', label='Mean prediction'),
                    Line2D([0], [0], color='blue', label='Confidence interval'),
                    Line2D([0], [0], color='green', label='Prediction interval')]
```

```

ax.legend(handles=legend_elements, loc='upper left')
plt.xlabel('Engine size (in litres)')
plt.ylabel('Car price')
ax.yaxis.set_major_formatter('${x:,.0f}');

```



The prediction interval accounts for both noise and uncertainty in coefficient estimates, while the confidence interval only reflects coefficient uncertainty. The prediction interval is wider.

2.3.4 Training with `sklearn`

In `sklearn`, we typically work with:

- `X`: a feature matrix (`DataFrame` or `ndarray`)
- `y`: a 1D target vector (`Series` or 1D `ndarray`)

We can fit the model and evaluate it, but (unlike `statsmodels`) we do not get p-values or confidence intervals from the fitted object.

```
from sklearn.linear_model import LinearRegression
```

```
# Prepare training data (X must be 2D; y should be 1D for sklearn metrics)
X_train = train[["engineSize"]] # Note the double brackets to get a DataFrame (2D)
y_train = train["price"]

# Create the sklearn linear regression model
model_skl = LinearRegression()

# Fit the model
model_skl.fit(X_train, y_train)

# Inspect fitted parameters
slope = float(model_skl.coef_[0])
intercept = float(model_skl.intercept_)
print(f"Fitted model: price = {intercept:,.2f} + {slope:,.2f} * engineSize")
```

Fitted model: price = -4,122.04 + 12,988.28 * engineSize

Let's make prediction next

```
# Prepare test data
X_test = test[["engineSize"]]
y_test = test["price"]

# Predict on test
y_pred = model_skl.predict(X_test)

# Evaluate (train + test)
y_pred_train = model_skl.predict(X_train)
```

sklearn provides metrics, so we can import RMSE, MAE, and R^2 directly, the library will take care of computing.

```
from sklearn.metrics import root_mean_squared_error, mean_absolute_error, r2_score

metrics = pd.DataFrame(
    {
        "Split": ["Train", "Test"],
        "RMSE": [
```

```

        root_mean_squared_error(y_train, y_pred_train),
        root_mean_squared_error(y_test, y_pred),
    ],
    "MAE": [
        mean_absolute_error(y_train, y_pred_train),
        mean_absolute_error(y_test, y_pred),
    ],
    "R2": [
        r2_score(y_train, y_pred_train),
        r2_score(y_test, y_pred),
    ],
}
)
metrics

```

	Split	RMSE	MAE	R2
0	Train	12807.526232	9260.285302	0.390498
1	Test	12995.106452	9411.325913	0.386990

Test performance is worse than training performance, which is expected since the model was fit on the training set.

```

# Quick diagnostics on the test set
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Actual vs Predicted
sns.scatterplot(x=y_test, y=y_pred, color="orange", s=20, ax=axes[0])
min_price = float(min(y_test.min(), y_pred.min()))
max_price = float(max(y_test.max(), y_pred.max()))
axes[0].plot([min_price, max_price], [min_price, max_price], color="blue")
axes[0].set_title("Test: Actual vs Predicted")
axes[0].set_xlabel("Actual price")
axes[0].set_ylabel("Predicted price")
axes[0].xaxis.set_major_formatter("${x:,.0f}")
axes[0].yaxis.set_major_formatter("${x:,.0f}")

# Residuals vs Predicted
residuals = y_test - y_pred
sns.scatterplot(x=y_pred, y=residuals, color="orange", s=20, ax=axes[1])

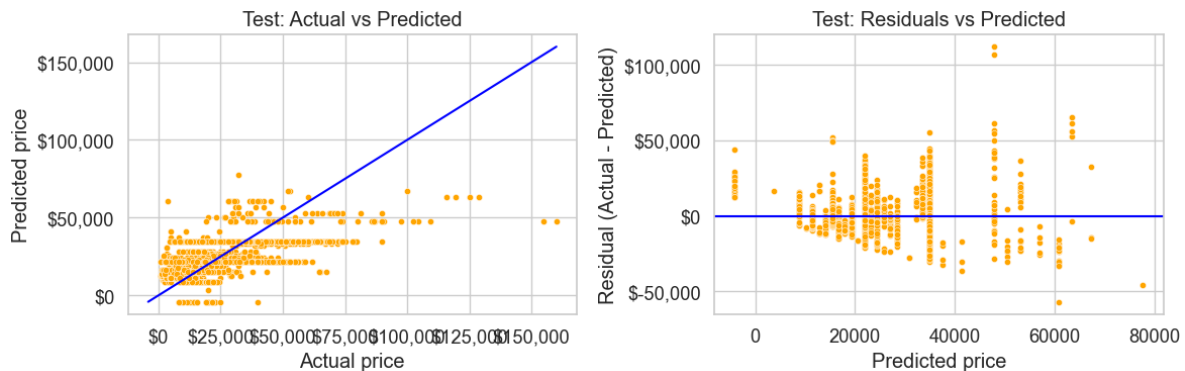
```

```

axes[1].axhline(0, color="blue")
axes[1].set_title("Test: Residuals vs Predicted")
axes[1].set_xlabel("Predicted price")
axes[1].set_ylabel("Residual (Actual - Predicted)")
axes[1].yaxis.set_major_formatter("${x:,.0f}")

plt.tight_layout()

```



2.3.5 Training with statsmodels (API)

`statsmodels` also provides a non-formula API (`statsmodels.api`) that works with matrices, similar to `sklearn`. This requires explicitly adding a constant term for the intercept but gives the same inference outputs (p-values, confidence intervals, etc.) as the formula API.

```

import statsmodels.api as sm

# Prepare the predictor matrix (need to add a constant term for the intercept)
X_train_sm = sm.add_constant(train[["engineSize"]])
y_train_sm = train["price"]

print("X_train with constant term:")
print(X_train_sm.head())

```

```

X_train with constant term:
   const  engineSize
0    1.0         3.0
1    1.0         3.0
2    1.0         3.0
3    1.0         3.0
4    1.0         3.0

```

```
# Create and fit the OLS model using the matrix API
model_sm_api = sm.OLS(y_train_sm, X_train_sm).fit()

# Display the model summary
model_sm_api.summary()
```

Dep. Variable:	price	R-squared:	0.390			
Model:	OLS	Adj. R-squared:	0.390			
Method:	Least Squares	F-statistic:	3177.			
Date:	Fri, 16 Jan 2026	Prob (F-statistic):	0.00			
Time:	09:29:33	Log-Likelihood:	-53949.			
No. Observations:	4960	AIC:	1.079e+05			
Df Residuals:	4958	BIC:	1.079e+05			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	-4122.0357	522.260	-7.893	0.000	-5145.896	-3098.176
engineSize	1.299e+04	230.450	56.361	0.000	1.25e+04	1.34e+04
Omnibus:	1271.986	Durbin-Watson:	0.517			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	6490.719			
Skew:	1.137	Prob(JB):	0.00			
Kurtosis:	8.122	Cond. No.	7.64			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Notice the coefficients match the formula API results exactly:

- Intercept (**const**): -4122.04
- Slope (**engineSize**): 1.299e+04

Both approaches give the same model, but the matrix API requires manually adding the constant term using `sm.add_constant()`.

```
# Make predictions on the test set (remember to add constant term)
X_test_sm = sm.add_constant(test[["engineSize"]])
pred_price_api = model_sm_api.predict(X_test_sm)

# Compare predictions from both statsmodels approaches
comparison = pd.DataFrame({
    "Formula API": pred_price[:5],
```

```

    "Matrix API": pred_price_api[:5],
    "Difference": pred_price[:5] - pred_price_api[:5]
})
print("First 5 predictions comparison:")
print(comparison)

```

First 5 predictions comparison:

	Formula API	Matrix API	Difference
0	34842.807319	34842.807319	7.275958e-12
1	34842.807319	34842.807319	7.275958e-12
2	34842.807319	34842.807319	7.275958e-12
3	8866.245277	8866.245277	0.000000e+00
4	47831.088340	47831.088340	0.000000e+00

2.4 Summary and Key Takeaways

- All three approaches estimate the same coefficients when the intercept is handled consistently.
- scikit-learn (`LinearRegression`) is simple and prediction-focused, with easy access to metrics, but no inference statistics.
- statsmodels formula API provides rich statistical output and automatic intercept handling.
- statsmodels matrix API provides the same inference, with more explicit control, but requires adding the intercept manually.
- Choose based on whether you need inference, prediction, or explicit model control.

3 Multiple Linear Regression

Read section 3.2 of the book before using these notes.

Note: these notes focus on implementation. The book provides the conceptual explanations.

```
# Import libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error, mean_absolute_error, r2_score

sns.set_theme(style="whitegrid", font_scale=1.2)
```

3.1 Learning objectives

By the end of this notebook you should be able to:

- Fit a multiple linear regression model using **statsmodels** and **sklearn**.
- Interpret coefficients and overall model fit.
- Evaluate model performance on a held-out test set (RMSE, MAE, R^2).
- Explain why R^2 can increase when adding irrelevant predictors.

3.2 Dataset

We will use the same dataset as SLR, specifically,

- Training set: `Datasets/Car_features_train.csv`, `Datasets/Car_prices_train.csv`
- Test set: `Datasets/Car_features_test.csv`, `Datasets/Car_prices_test.csv`

We will predict car price using `engineSize`, `age`, `mileage`, and `mpg`.

```
# Read training and test data
trainf = pd.read_csv('./Datasets/Car_features_train.csv')
trainp = pd.read_csv('./Datasets/Car_prices_train.csv')
train = pd.merge(trainf, trainp)

testf = pd.read_csv('./Datasets/Car_features_test.csv')
testp = pd.read_csv('./Datasets/Car_prices_test.csv')
test = pd.merge(testf, testp)

print('Train shape:', train.shape)
print('Test shape:', test.shape)
train.head()
# Create age feature using the most recent model year in the training data
reference_year = train["year"].max()
train["age"] = reference_year - train["year"]
test["age"] = reference_year - test["year"]
```

```
Train shape: (4960, 11)
```

```
Test shape: (2672, 11)
```

We convert model year to age using `reference_year = train["year"].max()`, then `age = reference_year - year`. This anchors age to the most recent model year in the training set and keeps ages non-negative.

3.3 Multiple Linear Regression

Instead of only using `engineSize`, we will use `age`, `mileage`, `mpg`, and `engineSize`.

3.3.1 Training with statsmodels (formula API)

We fit an OLS model with a formula string.

```
# Fit multiple linear regression with statsmodels (formula API)
model_sm = smf.ols(
    formula='price ~ age + mileage + mpg + engineSize',
    data=train,
).fit()
```

```
model_sm.summary()
```

Dep. Variable:	price	R-squared:	0.660			
Model:	OLS	Adj. R-squared:	0.660			
Method:	Least Squares	F-statistic:	2410.			
Date:	Fri, 16 Jan 2026	Prob (F-statistic):	0.00			
Time:	10:40:20	Log-Likelihood:	-52497.			
No. Observations:	4960	AIC:	1.050e+05			
Df Residuals:	4955	BIC:	1.050e+05			
Df Model:	4					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.102e+04	738.393	14.918	0.000	9567.676	1.25e+04
age	-1817.7366	73.751	-24.647	0.000	-1962.322	-1673.151
mileage	-0.1474	0.009	-16.817	0.000	-0.165	-0.130
mpg	-79.3126	9.338	-8.493	0.000	-97.620	-61.006
engineSize	1.218e+04	189.969	64.107	0.000	1.18e+04	1.26e+04
Omnibus:	2450.973	Durbin-Watson:	0.541			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	31060.548			
Skew:	2.045	Prob(JB):	0.00			
Kurtosis:	14.557	Cond. No.	1.94e+05			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.94e+05. This might indicate that there are strong multicollinearity or other numerical problems.

The fitted equation (from this dataset) is approximately:

$$\hat{price} = b_0 + b_1 * age + b_2 * mileage + b_3 * mpg + b_4 * engineSize$$

Interpretation: holding the other variables fixed, a one-unit increase in each predictor changes the expected price by its coefficient amount.

3.3.2 Model Prediction and Error Metrics with statsmodels

Predict car prices for the test set.

```

pred_price_sm = model_sm.predict(test)

#Compute RMSE and MAE using NumPy (since statsmodels does not provide these metrics directly)
sm_rmse = np.sqrt(np.mean((testp.price - pred_price_sm) ** 2))
sm_mae = np.mean(np.abs(testp.price - pred_price_sm))
print(f'Statsmodels Test RMSE: ${sm_rmse:,.2f}')
print(f'Statsmodels Test MAE: ${sm_mae:,.2f}')

```

```

Statsmodels Test RMSE: $9,956.82
Statsmodels Test MAE: $6,407.74

```

3.3.3 Training with sklearn

The procedure mirrors simple linear regression: prepare X and y, fit the model, and inspect coefficients.

```

# Prepare training data for sklearn
predictors = ["age", "mileage", "mpg", "engineSize"]
X_train = train[predictors]
y_train = train["price"]

model_skl = LinearRegression()
model_skl.fit(X_train, y_train)

coef_table = pd.Series(model_skl.coef_, index=predictors, name="coefficient")
print("Intercept:", model_skl.intercept_)
coef_table

```

```
Intercept: 11015.252952206762
```

```

age          -1817.736579
mileage      -0.147450
mpg          -79.312573
engineSize   12178.359942
Name: coefficient, dtype: float64

```

3.3.4 Model Prediction and Error Metrics with sklearn

Predict on the test set and prepare metrics. Compute RMSE, MAE, and R^2 for train and test.

```
X_test = test[predictors]
y_test = test["price"]

y_pred_skl = model_skl.predict(X_test)
y_pred_train = model_skl.predict(X_train)

metrics = pd.DataFrame(
    {
        "Split": ["Train", "Test"],
        "RMSE": [
            root_mean_squared_error(y_train, y_pred_train),
            root_mean_squared_error(y_test, y_pred_skl),
        ],
        "MAE": [
            mean_absolute_error(y_train, y_pred_train),
            mean_absolute_error(y_test, y_pred_skl),
        ],
        "R2": [
            r2_score(y_train, y_pred_train),
            r2_score(y_test, y_pred_skl),
        ],
    }
)
metrics
```

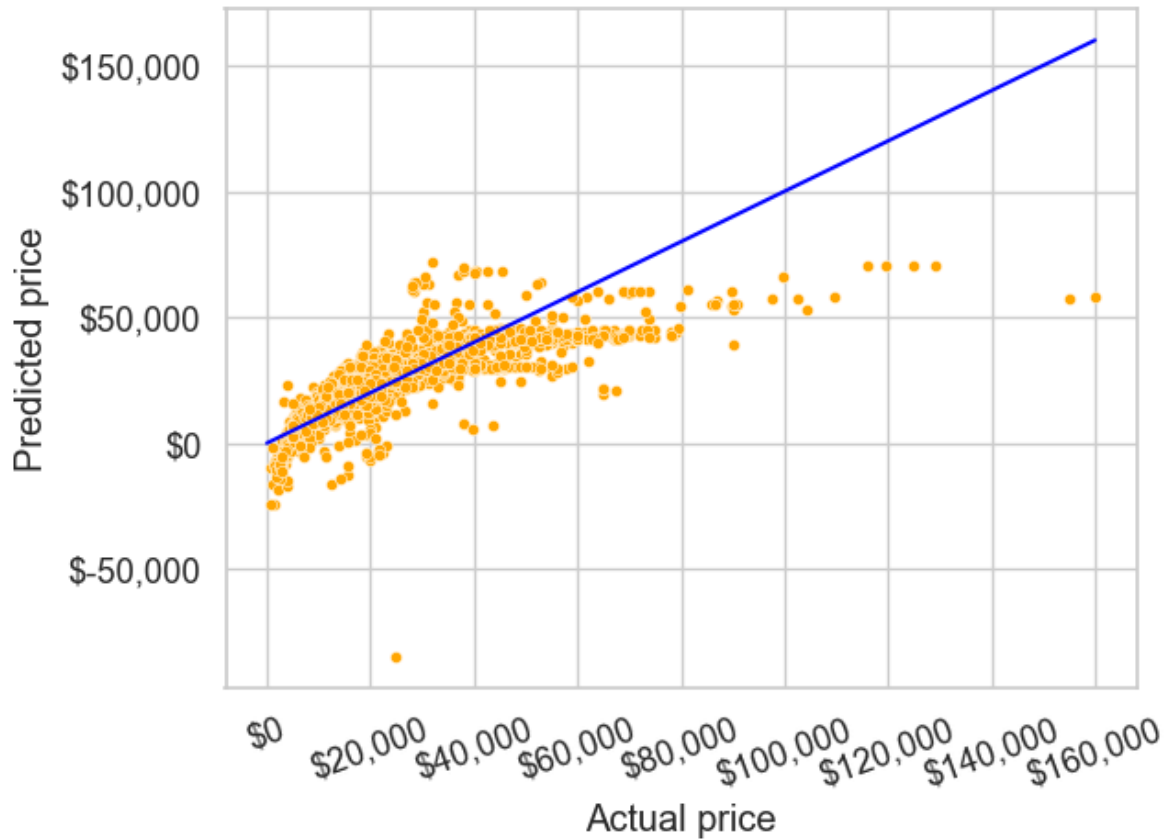
	Split	RMSE	MAE	R2
0	Train	9558.926176	6292.020876	0.660482
1	Test	9956.824980	6407.739919	0.640127

3.3.5 Diagnostics (test set)

Compare predicted vs actual prices. Perfect predictions lie on the line $y = x$.

```
sns.scatterplot(x=y_test, y=y_pred_skl, color="orange", s=20)
ax = sns.lineplot(x=[0, y_test.max()], y=[0, y_test.max()], color="blue")
```

```
plt.xlabel("Actual price")
plt.ylabel("Predicted price")
ax.yaxis.set_major_formatter('${x:,.0f}')
ax.xaxis.set_major_formatter('${x:,.0f}')
plt.xticks(rotation=20);
```



3.4 Effect of adding noisy predictors on R^2

Even a useless predictor can increase training R^2 because the model gains extra flexibility.

```
# Add a random predictor and refit with statsmodels
np.random.seed(1)
train = train.copy()
train["rand_col"] = np.random.rand(train.shape[0])
```

```

model_sm_noise = smf.ols(
    formula='price ~ age + mileage + mpg + engineSize + rand_col',
    data=train,
).fit()

print(f"Base model R2: {model_sm.rsquared:.4f}")
print(f"With random predictor R2: {model_sm_noise.rsquared:.4f}")

```

Base model R2: 0.6605
 With random predictor R2: 0.6605

3.4.0.1 Same idea with sklearn

Compare training R^2 with and without the random predictor.

```

X_train_base = train[predictors]
X_train_noise = train[predictors + ["rand_col"]]

model_skl_base = LinearRegression().fit(X_train_base, y_train)
model_skl_noise = LinearRegression().fit(X_train_noise, y_train)

r2_base = r2_score(y_train, model_skl_base.predict(X_train_base))
r2_noise = r2_score(y_train, model_skl_noise.predict(X_train_noise))

pd.DataFrame(
    {
        "Model": ["Base predictors", "With random predictor"],
        "Train R2": [r2_base, r2_noise],
    }
)

```

	Model	Train R2
0	Base predictors	0.660482
1	With random predictor	0.660545

Adding noise can increase training R^2 , but it does not guarantee better performance on new data. **Use validation or adjusted R^2 in MLR to guard against overfitting.**

4 Extending LR: Interactions and Qualitative Features

4.1 Overview

This notebook demonstrates advanced techniques for enhancing linear regression models by incorporating:

- **Interaction terms** between variables
- **Qualitative (categorical) predictors**
- **Polynomial transformations**
- **Mixed interactions** between categorical and continuous variables

Prerequisites: Read sections 3.3.1 and 3.3.2 of the textbook before using these notes.

Note: These notes focus on practical implementation. The textbook provides the theoretical foundation.

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import PolynomialFeatures, OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import root_mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import cross_val_score
import statsmodels.formula.api as smf
import warnings

# Configuration
pd.set_option('display.float_format', '{:.2f}'.format)
np.random.seed(42) # For reproducibility
sns.set_theme(style="whitegrid", font_scale=1.2)
```

```
plt.rcParams["figure.figsize"] = (10, 6)
warnings.filterwarnings('ignore')
```

4.2 Learning Objectives

By the end of this notebook, you should be able to:

4.2.1 Core Concepts

- **Understand** when and why to use interaction terms in regression models
- **Interpret** coefficients in models with interactions
- **Apply** different encoding strategies for categorical predictors

4.2.2 Technical Skills

- **Implement** interaction terms using both `statsmodels` and `sklearn`
- **Use** `PolynomialFeatures` to generate polynomial and interaction terms
- **Handle** categorical variables with `OneHotEncoder` and `get_dummies`
- **Create** robust preprocessing pipelines with `ColumnTransformer`

4.2.3 Advanced Applications

- **Model** complex relationships between qualitative and continuous predictors
- **Apply** polynomial transformations to capture non-linear patterns
- **Evaluate** and compare different model specifications
- **Avoid** common pitfalls like data leakage and multicollinearity

4.3 Dataset

- Training set: `Datasets/Car_features_train.csv`, `Datasets/Car_prices_train.csv`
- Test set: `Datasets/Car_features_test.csv`, `Datasets/Car_prices_test.csv`

We replace `year` with `age`, computed as `reference_year - year`, where `reference_year` is the most recent model year in the training data.


```
# Read data
trainf = pd.read_csv('./Datasets/Car_features_train.csv')
trainp = pd.read_csv('./Datasets/Car_prices_train.csv')
testf = pd.read_csv('./Datasets/Car_features_test.csv')
testp = pd.read_csv('./Datasets/Car_prices_test.csv')

train = pd.merge(trainf, trainp)
test = pd.merge(testf, testp)

# Create age feature using the most recent model year in the training data
reference_year = train["year"].max()
train["age"] = reference_year - train["year"]
test["age"] = reference_year - test["year"]

train.head()
```

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price	age
0	18473	bmw	6 Series	2020	Semi-Auto	11	Diesel	145	53.33	3.00	37980	0
1	15064	bmw	6 Series	2019	Semi-Auto	10813	Diesel	145	53.04	3.00	33980	1
2	18268	bmw	6 Series	2020	Semi-Auto	6	Diesel	145	53.44	3.00	36850	0
3	18480	bmw	6 Series	2017	Semi-Auto	18895	Diesel	145	51.51	3.00	25998	3
4	18492	bmw	6 Series	2015	Automatic	62953	Diesel	160	51.49	3.00	18990	5

4.4 Part 1: Working with statsmodels

4.4.1 Interaction Terms (Continuous Variables)

4.4.1.1 The Problem with Additive Models

In a standard linear model: $\text{price} = \beta_0 + \beta_1 \text{age} + \beta_2 \text{engineSize} + \epsilon$

This assumes the effect of `engineSize` on `price` is **constant** regardless of the car's `age`. But what if:

- Larger engines depreciate faster with age?
- The value premium for larger engines changes over time?

4.4.1.2 Solution: Add an Interaction Term

We relax the constant-association assumption by adding an interaction between `engineSize` and `age`:

`price =    +    *age +    *engineSize +    *(age * engineSize) +`

Now the effect of `engineSize` depends on `age`, and vice versa.

```
# Fit model with interaction between engineSize and age
# The '*' operator creates both main effects and their interaction
model_sm_interaction = smf.ols(
    formula='price ~ age*engineSize + mileage + mpg',
    data=train,
).fit()

model_sm_interaction.summary()
```

Dep. Variable:	price	R-squared:	0.682
Model:	OLS	Adj. R-squared:	0.681
Method:	Least Squares	F-statistic:	2121.
Date:	Fri, 23 Jan 2026	Prob (F-statistic):	0.00
Time:	08:15:24	Log-Likelihood:	-52338.
No. Observations:	4960	AIC:	1.047e+05
Df Residuals:	4954	BIC:	1.047e+05
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4313.7409	804.871	5.360	0.000	2735.838	5891.644
age	275.3833	135.695	2.029	0.042	9.361	541.405
engineSize	1.52e+04	248.298	61.231	0.000	1.47e+04	1.57e+04
age:engineSize	-896.7687	49.431	-18.142	0.000	-993.676	-799.861
mileage	-0.1525	0.008	-17.954	0.000	-0.169	-0.136
mpg	-84.3417	9.048	-9.322	0.000	-102.079	-66.604

Omnibus:	2330.413	Durbin-Watson:	0.524
Prob(Omnibus):	0.000	Jarque-Bera (JB):	29977.437
Skew:	1.908	Prob(JB):	0.00
Kurtosis:	14.423	Cond. No.	2.22e+05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.22e+05. This might indicate that there are strong multicollinearity or other numerical problems.

4.4.1.3 Understanding the Interaction Model

The complete model equation is:

$$\text{price} = \beta_0 + \beta_1 \cdot \text{age} + \beta_2 \cdot \text{engineSize} + \beta_3 \cdot (\text{age} \times \text{engineSize}) + \beta_4 \cdot \text{mileage} + \beta_5 \cdot \text{mpg}$$

4.4.1.4 Key Insights:

- 1. The effect of engineSize on price now depends on age:** - For a car of age t :
 $\frac{\partial \text{price}}{\partial \text{engineSize}} = \beta_2 + \beta_3 \cdot t$
- 2. The effect of age on price depends on engineSize:** - For a car with engine size s :
 $\frac{\partial \text{price}}{\partial \text{age}} = \beta_1 + \beta_3 \cdot s$
- 3. Interpretation of (interaction coefficient):** - How much the effect of engine size changes for each additional year of age - How much the effect of age changes for each additional liter of engine size

```
# Evaluate model performance on test data
pred_price = model_sm_interaction.predict(test)
rmse_interaction = np.sqrt(((test.price - pred_price) ** 2).mean())
mae_interaction = np.mean(np.abs(test.price - pred_price))
r2_interaction = 1 - np.sum((test.price - pred_price)**2) / np.sum((test.price - np.mean(test.price))**2)

print("Model Performance on Test Set:")
print("=" * 40)
print(f"RMSE:  ${rmse_interaction:,.2f}")
print(f"MAE:   ${mae_interaction:,.2f}")
print(f"R²:    {r2_interaction:.4f}")

# Store results for later comparison
interaction_results = {
    'RMSE': rmse_interaction,
    'MAE': mae_interaction,
    'R2': r2_interaction
}
```

Model Performance on Test Set:

=====

RMSE: \$9,423.60
MAE: \$6,140.26
 R^2 : 0.6776

4.4.1.5 Key Takeaway: The Bias-Variance Trade-off

Benefits of Interaction Terms:

- Capture complex, realistic relationships between variables
- Improve model flexibility and potentially better fit
- Allow effects to vary contextually

Potential Drawbacks:

- Increased model complexity $\rightarrow$ higher variance
- Risk of overfitting, especially with limited data
- More difficult to interpret
- Require larger sample sizes for reliable estimation

Best Practice: Always validate interaction terms using cross-validation or hold-out data! (will be covered later)

4.4.2 Qualitative (Categorical) Predictors

4.4.2.1 Understanding Categorical Variables in Regression

When we include a categorical variable in regression, we need to convert it to numerical form. The most common approach is **dummy coding** (also called one-hot encoding).

4.4.2.2 Example: Transmission Type

Let's develop a model predicting price based on `engineSize` and the categorical variable `transmission`.

```
# Explore the categorical variable
print("Transmission Type Distribution:")
print("=" * 40)
transmission_counts = train.transmission.value_counts()
print(transmission_counts)
```

Transmission Type Distribution:

```
=====
transmission
Manual      1948
Automatic   1660
Semi-Auto   1351
Other        1
Name: count, dtype: int64
```

4.4.2.3 Handling Rare Categories

The **Other** category has very few observations (< 1% of data). This can cause issues:

- **Unreliable estimates** due to small sample size
- **Overfitting** risk
- **Unstable predictions** on new data

Common Solutions:

1. **Remove** the rare category (what we'll do here)
2. **Combine** with the most similar category
3. **Create** a “Rare” category grouping multiple small categories

```
# Remove the rare 'Other' category
train_updated = train[train.transmission != 'Other'].copy()

print(f"\nUpdated distribution:")
print(train_updated.transmission.value_counts())
```

Updated distribution:

```
transmission
Manual      1948
Automatic   1660
Semi-Auto   1351
Name: count, dtype: int64
```

```
# Fit model with categorical predictor
# statsmodels automatically creates dummy variables
model_sm_trans = smf.ols(
    formula='price ~ engineSize + transmission',
    data=train_updated,
```

```

).fit()

print("Model with Categorical Predictor:")
print("=" * 50)
model_sm_trans.summary()

```

Model with Categorical Predictor:

=====

Dep. Variable:	price	R-squared:	0.459
Model:	OLS	Adj. R-squared:	0.458
Method:	Least Squares	F-statistic:	1400.
Date:	Fri, 23 Jan 2026	Prob (F-statistic):	0.00
Time:	08:16:23	Log-Likelihood:	-53644.
No. Observations:	4959	AIC:	1.073e+05
Df Residuals:	4955	BIC:	1.073e+05
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	3042.6765	661.190	4.602	0.000	1746.451	4338.902
transmission[T.Manual]	-6770.6165	442.116	-15.314	0.000	-7637.360	-5903.873
transmission[T.Semi-Auto]	4994.3112	442.989	11.274	0.000	4125.857	5862.765
engineSize	1.023e+04	247.485	41.323	0.000	9741.581	1.07e+04

Omnibus:	1575.518	Durbin-Watson:	0.579
Prob(Omnibus):	0.000	Jarque-Bera (JB):	11006.609
Skew:	1.334	Prob(JB):	0.00
Kurtosis:	9.793	Cond. No.	11.4

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

4.4.2.4 Understanding Dummy Variable Encoding

Key Concepts:

1. **Reference Category (Baseline):** The category that's omitted to avoid perfect multicollinearity
 - Here: "Automatic" (chosen alphabetically by default)
 - All other coefficients are interpreted **relative to this baseline**

2. Coefficient Interpretation:

- `transmission[T.Manual]`: How much Manual cars cost **compared to Automatic cars**, holding engine size constant
- `transmission[T.Semi-Auto]`: How much Semi-Auto cars cost **compared to Automatic cars**, holding engine size constant

3. Why Drop One Category?

- With k categories, we only need $k-1$ dummy variables
- Including all k would create perfect multicollinearity (dummy trap)

Q: Interpret the intercept term.

A: For a hypothetical car with zero engine size and *Automatic* transmission, the estimated mean price is the intercept.

Q: Interpret the coefficient of `transmission[T.Manual]`.

A: Holding engine size fixed, the estimated mean price of a manual car differs from an automatic car by that coefficient.

```
# Visualize the model with parallel lines
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: Model predictions
x = np.linspace(train_updated.engineSize.min(), train_updated.engineSize.max(), 100)
params = model_sm_trans.params

# Calculate predictions for each transmission type
auto_pred = params['engineSize'] * x + params['Intercept']
manual_pred = params['engineSize'] * x + params['Intercept'] + params['transmission[T.Manual]']
semi_pred = params['engineSize'] * x + params['Intercept'] + params['transmission[T.Semi-Auto]']

ax1.plot(x, auto_pred, 'r-', linewidth=2, label='Automatic')
ax1.plot(x, semi_pred, 'b-', linewidth=2, label='Semi-Automatic')
ax1.plot(x, manual_pred, 'g-', linewidth=2, label='Manual')

ax1.legend()
ax1.set_xlabel('Engine Size (liters)')
ax1.set_ylabel('Predicted Price ($)')
ax1.set_title('Parallel Lines Model\n(Constant Engine Size Effect)')
ax1.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:,.0f}'))
ax1.grid(True, alpha=0.3)

# Plot 2: Actual data points
```

```

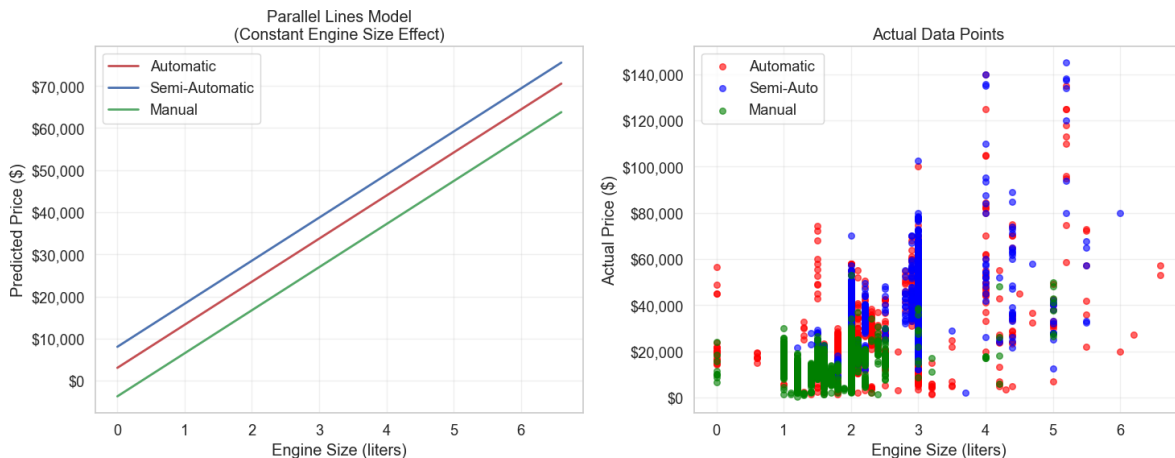
for i, transmission in enumerate(['Automatic', 'Semi-Auto', 'Manual']):
    data = train_updated[train_updated.transmission == transmission]
    colors = ['red', 'blue', 'green']
    ax2.scatter(data.engineSize, data.price,
                alpha=0.6, s=30, c=colors[i], label=transmission)

ax2.set_xlabel('Engine Size (liters)')
ax2.set_ylabel('Actual Price ($)')
ax2.set_title('Actual Data Points')
ax2.legend()
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:,.0f}'))
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(" Model Assumption: Parallel lines indicate that the effect of engine size")
print("   is the same across all transmission types (additive model).")

```



Model Assumption: Parallel lines indicate that the effect of engine size is the same across all transmission types (additive model).

4.4.3 Interactions Between Categorical and Continuous Variables

4.4.3.1 Problem with Parallel Lines

The model above assumes the effect of engine size is **identical** across transmission types (parallel lines). But what if:

- **Manual cars** lose value differently with engine size than automatics?
- **Sports cars** (often manual) command higher premiums for large engines?
- **Different transmission types** have different market dynamics?

4.4.3.2 Solution: Add Interaction Terms

We can relax the parallel lines assumption by adding interactions between `engineSize` and `transmission`.

```
# Fit model with interaction between continuous and categorical variables
# The '*' creates main effects AND interaction terms
model_sm_trans_inter = smf.ols(
    formula='price ~ engineSize*transmission',
    data=train_updated,
).fit()

print("Model with Categorical-Continuous Interaction:")
print("=" * 55)
model_sm_trans_inter.summary()
```

Model with Categorical-Continuous Interaction:

=====

Dep. Variable:	price	R-squared:	0.479
Model:	OLS	Adj. R-squared:	0.478
Method:	Least Squares	F-statistic:	909.9
Date:	Fri, 23 Jan 2026	Prob (F-statistic):	0.00
Time:	08:17:28	Log-Likelihood:	-53550.
No. Observations:	4959	AIC:	1.071e+05
Df Residuals:	4953	BIC:	1.072e+05
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	3754.7238	895.221	4.194	0.000	1999.695	5509.753
transmission[T.Manual]	1768.5856	1294.071	1.367	0.172	-768.366	4305.538
transmission[T.Semi-Auto]	-5282.7164	1416.472	-3.729	0.000	-8059.628	-2505.805
engineSize	9928.6082	354.511	28.006	0.000	9233.610	1.06e+04
engineSize:transmission[T.Manual]	-5285.9059	646.175	-8.180	0.000	-6552.695	-4019.117
engineSize:transmission[T.Semi-Auto]	4162.2428	552.597	7.532	0.000	3078.908	5245.578

Omnibus:	1379.846	Durbin-Watson:	0.622
Prob(Omnibus):	0.000	Jarque-Bera (JB):	9799.471
Skew:	1.139	Prob(JB):	0.00
Kurtosis:	9.499	Cond. No.	30.8

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Q: Interpret the coefficient of `transmission[T.Manual]`.

A: It shifts the intercept for manual cars relative to automatic cars.

Q: Interpret the coefficient of `engineSize:transmission[T.Manual]`.

A: It adjusts the slope of engine size for manual cars relative to automatic cars.

```
# Visualize the interaction model (non-parallel lines)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: Model with interactions (non-parallel lines)
x = np.linspace(train_updated.engineSize.min(), train_updated.engineSize.max(), 100)
params = model_sm_trans_inter.params

# Calculate predictions for each transmission type (now with different slopes!)
auto_pred = params['engineSize'] * x + params['Intercept']

manual_pred = ((params['engineSize'] + params['engineSize:transmission[T.Manual]']) * x +
               params['Intercept'] + params['transmission[T.Manual]'])

semi_pred = ((params['engineSize'] + params['engineSize:transmission[T.Semi-Auto]']) * x +
              params['Intercept'] + params['transmission[T.Semi-Auto]'])

ax1.plot(x, auto_pred, 'r-', linewidth=2, label='Automatic')
ax1.plot(x, semi_pred, 'b-', linewidth=2, label='Semi-Automatic')
ax1.plot(x, manual_pred, 'g-', linewidth=2, label='Manual')

ax1.legend()
ax1.set_xlabel('Engine Size (liters)')
ax1.set_ylabel('Predicted Price ($)')
ax1.set_title('Interaction Model\n(Different Engine Size Effects)')
ax1.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:,.0f}'))
ax1.grid(True, alpha=0.3)

# Plot 2: Comparison of models at specific engine sizes
```

```

engine_sizes = [1.0, 2.0, 3.0, 4.0]
transmission_types = ['Automatic', 'Manual', 'Semi-Auto']
colors = ['red', 'green', 'blue']

x_pos = np.arange(len(engine_sizes))
width = 0.25

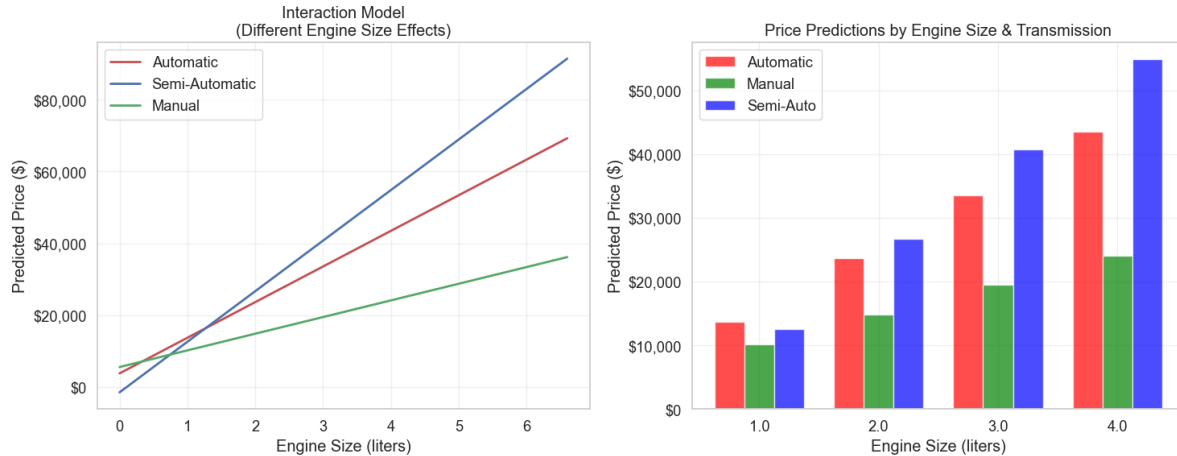
for i, trans_type in enumerate(['Automatic', 'Manual', 'Semi-Auto']):
    if trans_type == 'Automatic':
        pred_values = [params['engineSize'] * es + params['Intercept'] for es in engine_sizes]
    elif trans_type == 'Manual':
        pred_values = [(params['engineSize'] + params['engineSize:transmission[T.Manual]']) *
                        params['Intercept'] + params['transmission[T.Manual]']) for es in engine_sizes]
    else: # Semi-Auto
        pred_values = [(params['engineSize'] + params['engineSize:transmission[T.Semi-Auto]']) *
                        params['Intercept'] + params['transmission[T.Semi-Auto]']) for es in engine_sizes]

    ax2.bar(x_pos + i*width, pred_values, width, label=trans_type, color=colors[i], alpha=0.3)

ax2.set_xlabel('Engine Size (liters)')
ax2.set_ylabel('Predicted Price ($)')
ax2.set_title('Price Predictions by Engine Size & Transmission')
ax2.set_xticks(x_pos + width)
ax2.set_xticklabels(engine_sizes)
ax2.legend()
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:,.0f}'))
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```
# Calculate and display model performance
# Handle potential missing categories in test data first (before prediction)
test_filtered = test[test.transmission.isin(['Automatic', 'Manual', 'Semi-Auto'])].copy()
pred_interaction_filtered = model_sm_trans_inter.predict(test_filtered)

rmse_trans_inter = np.sqrt(((test_filtered.price - pred_interaction_filtered) ** 2).mean())
mae_trans_inter = np.mean(np.abs(test_filtered.price - pred_interaction_filtered))
r2_trans_inter = 1 - np.sum((test_filtered.price - pred_interaction_filtered)**2) / np.sum((test_filtered.price - test_filtered.price.mean())**2)

print(f"\n Model Performance:")
print("=" * 40)
print(f"RMSE:  ${rmse_trans_inter:,.2f}")
print(f"MAE:   ${mae_trans_inter:,.2f}")
print(f"R²:    {r2_trans_inter:.4f}")

# Note about filtered data
print(f"\nNote: Evaluated on {len(test_filtered)} test samples")
print(f"      (excluded {len(test) - len(test_filtered)} samples with 'Other' transmission)")
```

Model Performance:

```
=====
RMSE: $11,847.59
MAE:  $8,342.65
R²:   0.4906
```

```
Note: Evaluated on 2671 test samples
      (excluded 1 samples with 'Other' transmission)
```

4.5 Part 2: Working with scikit-learn

```
# Prepare data for scikit-learn
print(" Preparing Data for Scikit-Learn")

# Select features and target
feature_columns = ['mileage', 'engineSize', 'age', 'mpg']
X_train = train[feature_columns].copy()
y_train = train['price'].copy()
X_test = test[feature_columns].copy()
y_test = test['price'].copy()
```

Preparing Data for Scikit-Learn

4.5.1 Generating Interaction Terms with PolynomialFeatures

4.5.1.1 Understanding PolynomialFeatures

The PolynomialFeatures transformer is incredibly versatile:

Key Parameters:

- **degree:** Maximum degree of polynomial features
- **interaction_only:** If True, only interaction terms (no x^2)
- **include_bias:** Whether to include intercept column

Example with 2 features $[x, x]$:

Setting	Output Features
degree=1	$[x, x]$
degree=2, interaction_only=False	$[x, x, x^2, x x, x^2]$
degree=2, interaction_only=True	$[x, x, x x]$

```
# Generate interaction-only features (no squared terms)
print("Creating Interaction-Only Features")
print("=" * 45)

poly_inter = PolynomialFeatures(degree=2, include_bias=False, interaction_only=True)

# Fit on training data and transform both sets
```

```

X_train_inter = poly_inter.fit_transform(X_train)
X_test_inter = poly_inter.transform(X_test)

# Display the transformation
print(f"Original features: {X_train.shape[1]}")
print(f"After interaction features: {X_train_inter.shape[1]}")
print(f"Added interaction terms: {X_train_inter.shape[1] - X_train.shape[1]}")

print(f"\nFeature names:")
feature_names = poly_inter.get_feature_names_out(input_features=X_train.columns)
for i, name in enumerate(feature_names):
    marker = " (interaction)" if " " in name else " (original)"
    print(f" {i+1:2d}. {name}{marker}")

# Train model with interactions
model_inter = LinearRegression()
model_inter.fit(X_train_inter, y_train)

# Evaluate performance
pred_price_inter = model_inter.predict(X_test_inter)
rmse_inter = root_mean_squared_error(y_test, pred_price_inter)
mae_inter = mean_absolute_error(y_test, pred_price_inter)
r2_inter = r2_score(y_test, pred_price_inter)

print(f"\n Model Performance (Interaction-Only):")
print("=" * 45)
print(f"RMSE: ${rmse_inter:,.2f}")
print(f"MAE:  ${mae_inter:,.2f}")
print(f"R2:  {r2_inter:.4f}")

```

Creating Interaction-Only Features

=====

Original features: 4

After interaction features: 10

Added interaction terms: 6

Feature names:

1. mileage (original)
2. engineSize (original)
3. age (original)
4. mpg (original)
5. mileage engineSize (interaction)

6. mileage age (interaction)
7. mileage mpg (interaction)
8. engineSize age (interaction)
9. engineSize mpg (interaction)
10. age mpg (interaction)

Model Performance (Interaction-Only):

```
=====
RMSE: $8,729.91
MAE:  $5,400.07
R2:  0.7234
```

4.5.2 Full Polynomial Features (Including Squared Terms)

4.5.3 Limitations of PolynomialFeatures

Key Limitation: Unlike `statsmodels`, `PolynomialFeatures` applies transformations to **ALL** features:

- **Advantage:** Quick, systematic feature generation
- **Disadvantage:** No selective control over which interactions to include
- **Risk:** Can create too many features with high-dimensional data

Trade-off: Convenience vs. Control

Let's compare interaction-only vs. full polynomial features:

```
# Generate full polynomial features (includes squared terms)
print("Creating Full Polynomial Features (degree=2)")
print("=" * 55)

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

print(f"Original features: {X_train.shape[1]}")
print(f"After polynomial features: {X_train_poly.shape[1]}")
print(f"Added features: {X_train_poly.shape[1] - X_train.shape[1]}")

print(f"\nAll generated features:")
feature_names_poly = poly.get_feature_names_out(input_features=X_train.columns)
for i, name in enumerate(feature_names_poly):
    if "^2" in name:
```

```

        marker = " (squared)"
    elif " " in name:
        marker = " (interaction)"
    else:
        marker = " (original)"
    print(f" {i+1:2d}. {name}{marker}")

```

Creating Full Polynomial Features (degree=2)

=====

Original features: 4

After polynomial features: 14

Added features: 10

All generated features:

1. mileage (original)
2. engineSize (original)
3. age (original)
4. mpg (original)
5. mileage² (squared)
6. mileage engineSize (interaction)
7. mileage age (interaction)
8. mileage mpg (interaction)
9. engineSize² (squared)
10. engineSize age (interaction)
11. engineSize mpg (interaction)
12. age² (squared)
13. age mpg (interaction)
14. mpg² (squared)

```
# Train model with full polynomial features
```

```
print("Training Full Polynomial Model")
```

```
print("=" * 40)
```

```
model_poly = LinearRegression()
```

```
model_poly.fit(X_train_poly, y_train)
```

```
# Show the most important coefficients
```

```
coeffs = model_poly.coef_
```

```
feature_importance = pd.DataFrame({
```

```
    'Feature': feature_names_poly,
```

```
    'Coefficient': coeffs,
```



```

    'Abs_Coefficient': np.abs(coeffs)
}).sort_values('Abs_Coefficient', ascending=False)

print("Top 10 Most Important Features:")
display(feature_importance.head(10))

```

Training Full Polynomial Model

=====

Top 10 Most Important Features:

	Feature	Coefficient	Abs_Coefficient
1	engineSize	22746.56	22746.56
2	age	-3022.42	3022.42
9	engineSize age	-644.88	644.88
8	engineSize^2	-361.71	361.71
10	engineSize mpg	-136.77	136.77
11	age^2	104.62	104.62
3	mpg	39.42	39.42
12	age mpg	16.68	16.68
13	mpg^2	0.66	0.66
0	mileage	-0.10	0.10

```

# Evaluate and compare polynomial models
pred_poly = model_poly.predict(X_test_poly)

# Calculate metrics for full polynomial model
rmse_poly = root_mean_squared_error(y_test, pred_poly)
mae_poly = mean_absolute_error(y_test, pred_poly)
r2_poly = r2_score(y_test, pred_poly)

print(" Model Performance Comparison")
print("=" * 45)

# Cross-validation for full polynomial
cv_scores_poly = cross_val_score(
    LinearRegression(), X_train_poly, y_train,
    cv=5, scoring='neg_root_mean_squared_error'
)

# Create comprehensive comparison

```

```

results_df = pd.DataFrame({
    'Model': [
        'Baseline (Linear)',
        'Interaction-Only',
        'Full Polynomial (degree=2)'
    ],
    'Features': [
        X_train.shape[1],
        X_train_inter.shape[1],
        X_train_poly.shape[1]
    ],
    'Test_RMSE': [
        np.nan, # We'll calculate baseline
        rmse_inter,
        rmse_poly
    ],
    'Test_MAE': [
        np.nan,
        mae_inter,
        mae_poly
    ],
    'Test_R2': [
        np.nan,
        r2_inter,
        r2_poly
    ]
})

# Calculate baseline model for comparison
baseline_model = LinearRegression()
baseline_model.fit(X_train, y_train)
baseline_pred = baseline_model.predict(X_test)

# Update baseline results
results_df.loc[0, 'Test_RMSE'] = root_mean_squared_error(y_test, baseline_pred)
results_df.loc[0, 'Test_MAE'] = mean_absolute_error(y_test, baseline_pred)
results_df.loc[0, 'Test_R2'] = r2_score(y_test, baseline_pred)

display(results_df.round(4))

```

Model Performance Comparison

=====

	Model	Features	Test_RMSE	Test_MAE	Test_R2
0	Baseline (Linear)	4	9956.83	6407.74	0.64
1	Interaction-Only	10	8729.91	5400.07	0.72
2	Full Polynomial (degree=2)	14	8896.17	5335.30	0.71

4.6 Part 3: Categorical Variables in Scikit-Learn

Scikit-learn models require **numerical input**. We need to convert categorical variables to numbers.

4.6.1 Two Main Approaches:

Method	Pros	Cons	Best For
<code>pd.get_dummies()</code>	Simple, intuitive	Column mismatch risk, no pipeline integration	Quick prototyping
<code>OneHotEncoder</code>	Robust, handles unseen categories, pipeline-friendly	Slightly more setup	Production code

4.6.2 Common Pitfalls:

Data Leakage: Fitting encoder on full dataset

Column Mismatch: Train/test having different categories

Dummy Trap: Including all dummy variables → multicollinearity

4.6.3 Method 1: Using `pd.get_dummies()`

4.6.3.1 The Column Alignment Problem

When using `get_dummies()`, train and test sets might have different categorical values:

```
# Train has: ['A', 'B', 'C'] → columns: ['B', 'C'] (A is reference)
# Test has: ['A', 'B', 'D'] → columns: ['B', 'D'] (A is reference)
# Result: Mismatched feature matrices!
```

Solution: Use `reindex()` to align columns.

```
# Method 1: get_dummies approach
print(" Method 1: Using pd.get_dummies()")
print("=" * 45)

# Create dummy variables (watch for potential mismatched columns)
train_dum = pd.get_dummies(train_updated, columns=['transmission'], drop_first=True, prefix='trans')
test_dum = pd.get_dummies(test, columns=['transmission'], drop_first=True, prefix='trans')

print("Original column alignment:")
print(f"Train dummy columns: {[col for col in train_dum.columns if 'trans_' in col]}")
print(f"Test dummy columns: {[col for col in test_dum.columns if 'trans_' in col]}")

# Critical step: Align test columns to match training columns
test_dum = test_dum.reindex(columns=train_dum.columns, fill_value=0)

print(f"\nAfter reindexing:")
print(f"Train shape: {train_dum.shape}")
print(f"Test shape: {test_dum.shape}")
print(f"Columns match: {list(train_dum.columns) == list(test_dum.columns)}")

# Prepare feature matrices
feature_cols = ['engineSize', 'trans_Manual', 'trans_Semi-Auto']
X_train_dum = train_dum[feature_cols]
y_train_dum = train_dum['price']
X_test_dum = test_dum[feature_cols]
y_test_dum = test_dum['price']

print(f"\nFinal feature matrix:")
display(X_train_dum.head())

# Train and evaluate model
model_dum = LinearRegression()
model_dum.fit(X_train_dum, y_train_dum)
pred_dum = model_dum.predict(X_test_dum)
rmse_dum = root_mean_squared_error(y_test_dum, pred_dum)
```

```

print(f"\n Performance:")
print(f"RMSE: ${rmse_dum:,.2f}")

# Show coefficients with interpretation
coeffs_dum = pd.DataFrame({
    'Feature': X_train_dum.columns,
    'Coefficient': model_dum.coef_
})
print(f"\nModel Coefficients:")
display(coeffs_dum)

```

Method 1: Using `pd.get_dummies()`

=====

Original column alignment:

Train dummy columns: ['trans_Manual', 'trans_Semi-Auto']

Test dummy columns: ['trans_Manual', 'trans_Other', 'trans_Semi-Auto']

After reindexing:

Train shape: (4959, 13)

Test shape: (2672, 13)

Columns match: True

Final feature matrix:

	engineSize	trans_Manual	trans_Semi-Auto
0	3.00	False	True
1	3.00	False	True
2	3.00	False	True
3	3.00	False	True
4	3.00	False	False

Performance:

RMSE: \$12,192.23

Model Coefficients:

	Feature	Coefficient
0	engineSize	10226.76
1	trans_Manual	-6770.62
2	trans_Semi-Auto	4994.31

4.6.4 Method 2: Using OneHotEncoder (Recommended)

4.6.4.1 Why OneHotEncoder is Better:

1. **No Data Leakage:** Fit only on training data
2. **Handles Unknown Categories:** Use `handle_unknown='ignore'`
3. **Pipeline Integration:** Works seamlessly with sklearn workflows
4. **Consistent Behavior:** Same transformation guaranteed for train/test

4.6.4.2 Two Approaches:

- **With Pipeline** (recommended): Clean, maintainable code
- **Manual:** More control, better for understanding

```
# Method 2a: OneHotEncoder with Pipeline (RECOMMENDED)
print("Method 2a: OneHotEncoder with Pipeline")
print("=" * 50)

# Prepare data (mixing categorical and numerical)
X_train_mixed = train_updated[['engineSize', 'transmission']].copy()
y_train_mixed = train_updated['price'].copy()
X_test_mixed = test[['engineSize', 'transmission']].copy()
y_test_mixed = test['price'].copy()

print("Input data structure:")
print(f"Features: {list(X_train_mixed.columns)}")
print(f>Data types: {X_train_mixed.dtypes.to_dict()}")

# Create preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), ['transmission']),
        # Note: numerical features passed through unchanged via remainder='passthrough'
    ],
    remainder='passthrough' # Keep other columns as-is
```

```

)

# Create complete pipeline
pipeline_ohe = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

print(f"\n Pipeline Structure:")
print(f"Step 1: ColumnTransformer")
print(f" - OneHotEncoder for ['transmission']")
print(f" - PassThrough for ['engineSize']")
print(f"Step 2: LinearRegression")

# Fit pipeline
pipeline_ohe.fit(X_train_mixed, y_train_mixed)

# The beauty of pipelines: preprocessing happens automatically!
pred_ohe_pipeline = pipeline_ohe.predict(X_test_mixed)
rmse_ohe_pipeline = root_mean_squared_error(y_test_mixed, pred_ohe_pipeline)

print(f"\n Pipeline Performance:")
print(f"RMSE: ${rmse_ohe_pipeline:,.2f}")

# Inspect the transformation
X_transformed = preprocessor.fit_transform(X_train_mixed)
feature_names_ohe = (
    preprocessor.named_transformers_['cat'].get_feature_names_out(['transmission']).tolist()
    ['engineSize'] # remainder columns
)
print(f"\nTransformed features: {feature_names_ohe}")
print(f"Transformed shape: {X_transformed.shape}")

try:
    # Display coefficients (if accessible)
    final_model = pipeline_ohe.named_steps['regressor']
    coeff_df = pd.DataFrame({
        'Feature': feature_names_ohe,
        'Coefficient': final_model.coef_
    })
    print(f"\nPipeline Model Coefficients:")
    display(coeff_df)

```

```
except Exception as e:
    print(f"Note: Coefficient extraction failed: {e}")
```

Method 2a: OneHotEncoder with Pipeline

=====

Input data structure:

Features: ['engineSize', 'transmission']

Data types: {'engineSize': dtype('float64'), 'transmission': dtype('O')}

Pipeline Structure:

Step 1: ColumnTransformer

- OneHotEncoder for ['transmission']

- PassThrough for ['engineSize']

Step 2: LinearRegression

Pipeline Performance:

RMSE: \$12,192.23

Transformed features: ['transmission_Manual', 'transmission_Semi-Auto', 'engineSize']

Transformed shape: (4959, 3)

Pipeline Model Coefficients:

	Feature	Coefficient
0	transmission_Manual	-6770.62
1	transmission_Semi-Auto	4994.31
2	engineSize	10226.76

```
# Method 2b: OneHotEncoder without Pipeline (Manual Control)
print(" Method 2b: OneHotEncoder Manual Approach")
print("=" * 50)

# Separate numerical and categorical features
X_train_num = train_updated[['engineSize']].copy()
X_test_num = test[['engineSize']].copy()

print("Step 1: Initialize and fit OneHotEncoder")
ohe = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)

# Fit on training categorical data only (no data leakage!)
```



```

X_train_cat_encoded = ohe.fit_transform(train_updated[['transmission']])
X_test_cat_encoded = ohe.transform(test[['transmission']]) # Transform using fitted encoder

print(f"Categories found during fit: {ohe.categories_}")
print(f"Feature names: {ohe.get_feature_names_out(['transmission'])}")
print(f"Encoded categorical shape: {X_train_cat_encoded.shape}")

print("\nStep 2: Combine numerical and encoded categorical features")
# Stack numerical and categorical features
X_train_combined = np.column_stack([X_train_num.values, X_train_cat_encoded])
X_test_combined = np.column_stack([X_test_num.values, X_test_cat_encoded])

print(f"Combined feature matrix shape: {X_train_combined.shape}")

# Create feature names for clarity
numerical_features = X_train_num.columns.tolist()
categorical_features = ohe.get_feature_names_out(['transmission']).tolist()
all_feature_names = numerical_features + categorical_features

print(f"All features: {all_feature_names}")

print("\nStep 3: Train model")
model_ohe_manual = LinearRegression()
model_ohe_manual.fit(X_train_combined, y_train_mixed)

# Evaluate
pred_ohe_manual = model_ohe_manual.predict(X_test_combined)
rmse_ohe_manual = root_mean_squared_error(y_test_mixed, pred_ohe_manual)

print(f"\n Manual Approach Performance:")
print(f"RMSE: ${rmse_ohe_manual:,.2f}")

# Show coefficients
manual_coeffs = pd.DataFrame({
    'Feature': all_feature_names,
    'Coefficient': model_ohe_manual.coef_
})
print(f"\nManual Model Coefficients:")
display(manual_coeffs)

# Verify both approaches give same results
print(f"\n Verification:")

```

```
print(f"Pipeline RMSE: ${rmse_ohe_pipeline:,.2f}")
print(f"Manual RMSE:   ${rmse_ohe_manual:,.2f}")
print(f"Difference:    ${abs(rmse_ohe_pipeline - rmse_ohe_manual):,.6f}")
print(f"Results match: {np.isclose(rmse_ohe_pipeline, rmse_ohe_manual)}")
```

Method 2b: OneHotEncoder Manual Approach

=====

Step 1: Initialize and fit OneHotEncoder

Categories found during fit: [array(['Automatic', 'Manual', 'Semi-Auto'], dtype=object)]

Feature names: ['transmission_Manual' 'transmission_Semi-Auto']

Encoded categorical shape: (4959, 2)

Step 2: Combine numerical and encoded categorical features

Combined feature matrix shape: (4959, 3)

All features: ['engineSize', 'transmission_Manual', 'transmission_Semi-Auto']

Step 3: Train model

Manual Approach Performance:

RMSE: \$12,192.23

Manual Model Coefficients:

	Feature	Coefficient
0	engineSize	10226.76
1	transmission_Manual	-6770.62
2	transmission_Semi-Auto	4994.31

Verification:

Pipeline RMSE: \$12,192.23

Manual RMSE: \$12,192.23

Difference: \$0.000000

Results match: True

4.7 Summary and Best Practices

4.7.1 When to Use Each Approach:

Situation	Recommended Method	Why
Quick prototyping	<code>pd.get_dummies()</code>	Fast and simple
Production models	OneHotEncoder + Pipeline	Robust, scalable
Complex preprocessing	ColumnTransformer	Handles mixed data types
Statistical inference	statsmodels	Rich statistical output

4.7.2 Best Practices Checklist:

4.7.2.1 Data Preprocessing:

- Always fit transformers on **training data only**
- Handle rare categories before encoding
- Use `drop='first'` or `drop_first=True` to avoid multicollinearity
- Set `handle_unknown='ignore'` for robustness

4.7.2.2 Model Development:

- Use cross-validation for model selection
- Compare multiple interaction specifications
- Monitor for overfitting with polynomial features
- Keep pipelines simple and maintainable

4.7.2.3 Common Pitfalls to Avoid:

- **Data leakage:** Fitting on test data
- **Column mismatch:** Forgetting to align train/test features
- **Dummy trap:** Including all categorical levels
- **Overfitting:** Adding too many interaction terms without validation

5 Extending LR: Feature Transformations

5.1 Learning objectives

By the end of this notebook you should be able to:

- Explain why a log transform of the target (using `log(price)`) can be helpful, and correctly back-transform predictions with `exp()`.
- Fit polynomial regression models in `statsmodels` using the formula API (e.g., quadratic and cubic terms with `I(mileage**2) / I(mileage**3)`).
- Build polynomial feature models in scikit-learn with `PolynomialFeatures` and `LinearRegression`.
- Implement polynomial feature models using a scikit-learn `Pipeline` for a clean, reproducible workflow.

```
# Import libraries
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error, mean_absolute_error, r2_score

sns.set_theme(style="whitegrid", font_scale=1.2)
```

5.2 Feature transformations in statsmodels

5.2.1 Why feature transformations matter

A linear regression model is **linear in its coefficients**, not necessarily linear in the original variables. That means we can extend a “linear” model to capture many real-world patterns by transforming or expanding the predictors.

Feature transformations are important because they let us:

- **Capture nonlinearity while staying in the linear-model framework:** adding terms like *mileage*² or *mileage*³ can model curved relationships without switching to a completely different model class.
- **Improve model assumptions:** transforming variables (often the target) can reduce heteroscedasticity (non-constant variance), make residuals closer to normal, and improve linearity of the relationship.
- **Enable interpretable extensions:** polynomial terms and interactions are still interpretable, especially when you state clearly what was transformed and how predictions are back-transformed.

In this section, we use the same car dataset throughout for continuity.

```
# Read data
trainf = pd.read_csv('./Datasets/Car_features_train.csv')
trainp = pd.read_csv('./Datasets/Car_prices_train.csv')
testf = pd.read_csv('./Datasets/Car_features_test.csv')
testp = pd.read_csv('./Datasets/Car_prices_test.csv')

train = pd.merge(trainf, trainp)
test = pd.merge(testf, testp)

# Create age feature using the most recent model year in the training data
reference_year = train["year"].max()
train["age"] = reference_year - train["year"]
test["age"] = reference_year - test["year"]

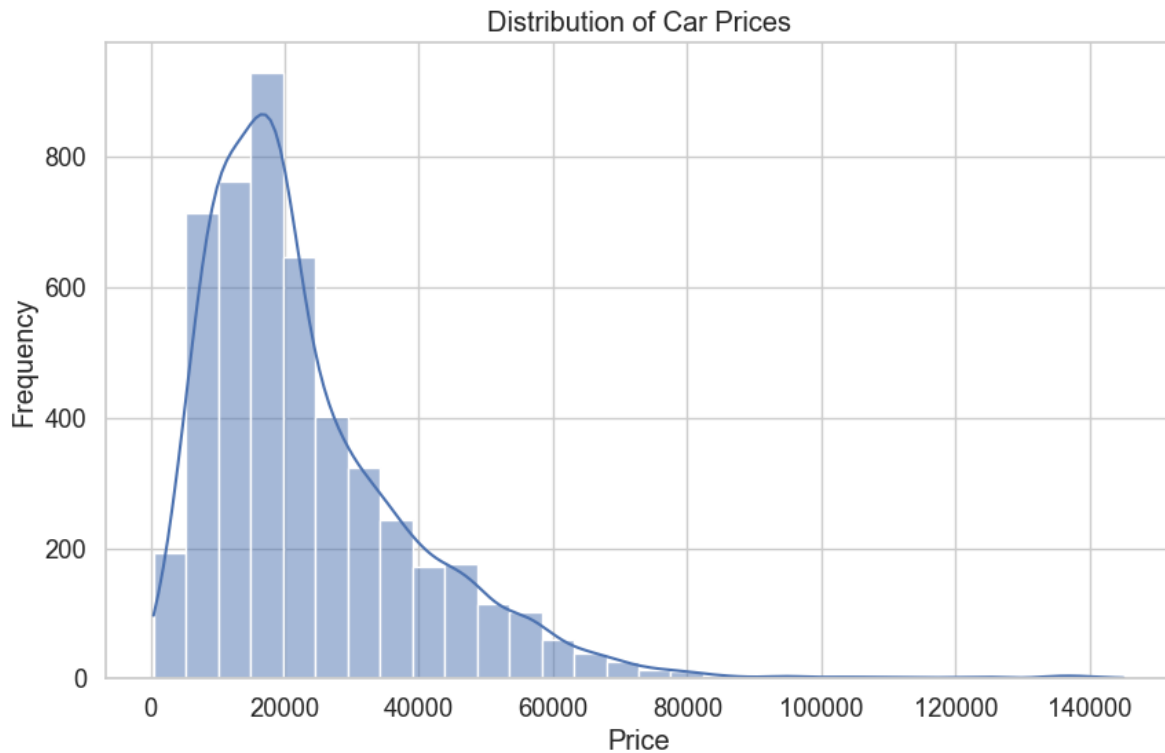
train.head()
```

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price
0	18473	bmw	6 Series	2020	Semi-Auto	11	Diesel	145	53.3282	3.0	37980
1	15064	bmw	6 Series	2019	Semi-Auto	10813	Diesel	145	53.0430	3.0	33980
2	18268	bmw	6 Series	2020	Semi-Auto	6	Diesel	145	53.4379	3.0	36850
3	18480	bmw	6 Series	2017	Semi-Auto	18895	Diesel	145	51.5140	3.0	25998
4	18492	bmw	6 Series	2015	Automatic	62953	Diesel	160	51.4903	3.0	18990

```
train_updated = train[train.transmission != 'Other']
```

Let's see the distribution of the target `price`

```
# draw the distribution of the price
plt.figure(figsize=(10, 6))
sns.histplot(train_updated['price'], bins=30, kde=True)
plt.title('Distribution of Car Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```



The price distribution is right-skewed, indicating that car prices have increasing variance at higher price levels. Linear models work best with normally distributed data and generally perform better when this assumption is met.

Benefits of using $\log(\text{price})$ as the dependent variable:

1. **Reduces Heteroscedasticity:** Car prices often have increasing variance at higher price levels; log transformation stabilizes this variance
2. **Handles Multiplicative Effects:** Captures percentage changes rather than absolute changes, which is more natural for price data
3. **Improves Model Fit:** Often leads to better R^2 values and improved residual behavior
4. **Interpretability:** Coefficients represent percentage changes in price, making economic interpretation more intuitive

Important: When using log-transformed dependent variables, predictions must be transformed back using `np.exp()` to get actual price predictions.

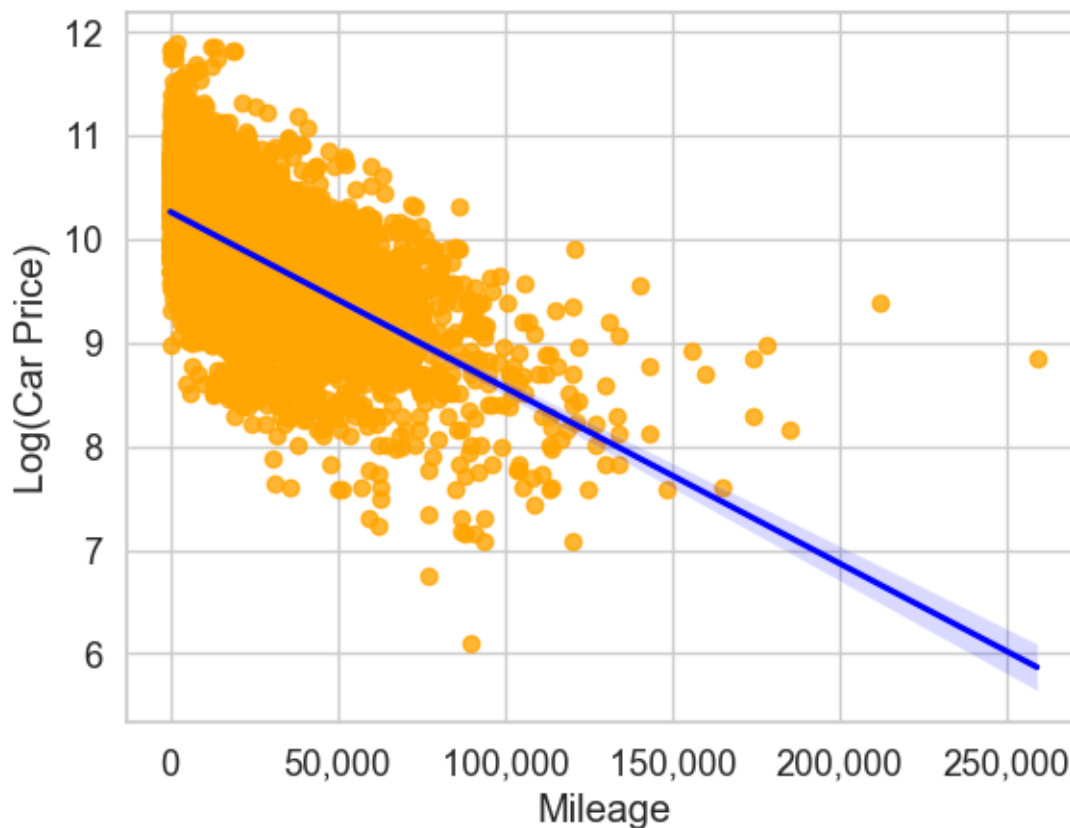
```
# R-squared of the model with just mileage (log-transformed price)
# Log transformation helps with heteroscedasticity and non-linear relationships
model_sm_mileage = smf.ols('np.log(price) ~ mileage', data=train_updated).fit()
print(f"R-squared (log price ~ mileage): {model_sm_mileage.rsquared:.4f}")

# Compare with original model without log transformation
model_sm_mileage_orig = smf.ols('price ~ mileage', data=train_updated).fit()
print(f"R-squared (price ~ mileage): {model_sm_mileage_orig.rsquared:.4f}")
```

```
R-squared (log price ~ mileage): 0.3601
R-squared (price ~ mileage): 0.2293
```

The model's R^2 increased from 22% to 36% with the log transformation, but 36% is still relatively low. Let's explore how we can improve it further by examining the relationship between log-transformed car price and mileage.

```
# Relationship between log(price) and mileage
ax = sns.regplot(x=train_updated.mileage, y=np.log(train_updated.price), color='orange', linecolor='blue')
plt.xlabel('Mileage')
plt.ylabel('Log(Car Price)')
ax.xaxis.set_major_formatter('{x:,.0f}')
```



The plot shows a reasonably linear relationship between mileage and $\log(\text{price})$, but the R^2 of 36% suggests there's still substantial unexplained variation. While the log transformation improved the model from 22% to 36%, we can explore polynomial terms to capture any remaining nonlinear patterns.

5.2.1.1 Quadratic transformation

Add a squared term for mileage using the `I()` operator to keep it as a separate predictor.

```
# Quadratic model with log-transformed price
model_sm_quad = smf.ols('np.log(price) ~ mileage + I(mileage**2)', data=train_updated).fit()
print("Log-transformed price with quadratic mileage term:")
print(f"R-squared: {model_sm_quad.rsquared:.4f}")
model_sm_quad.summary()
```

```
Log-transformed price with quadratic mileage term:
R-squared: 0.3825
```


Dep. Variable:	np.log(price)	R-squared:	0.382
Model:	OLS	Adj. R-squared:	0.382
Method:	Least Squares	F-statistic:	1535.
Date:	Fri, 23 Jan 2026	Prob (F-statistic):	0.00
Time:	09:30:37	Log-Likelihood:	-4007.5
No. Observations:	4959	AIC:	8021.
Df Residuals:	4956	BIC:	8040.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	10.3538	0.013	803.018	0.000	10.329	10.379
mileage	-2.452e-05	6.47e-07	-37.919	0.000	-2.58e-05	-2.32e-05
I(mileage ** 2)	8.112e-11	6.06e-12	13.385	0.000	6.92e-11	9.3e-11

Omnibus:	25.307	Durbin-Watson:	0.323
Prob(Omnibus):	0.000	Jarque-Bera (JB):	26.219
Skew:	-0.154	Prob(JB):	2.03e-06
Kurtosis:	3.178	Cond. No.	4.81e+09

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

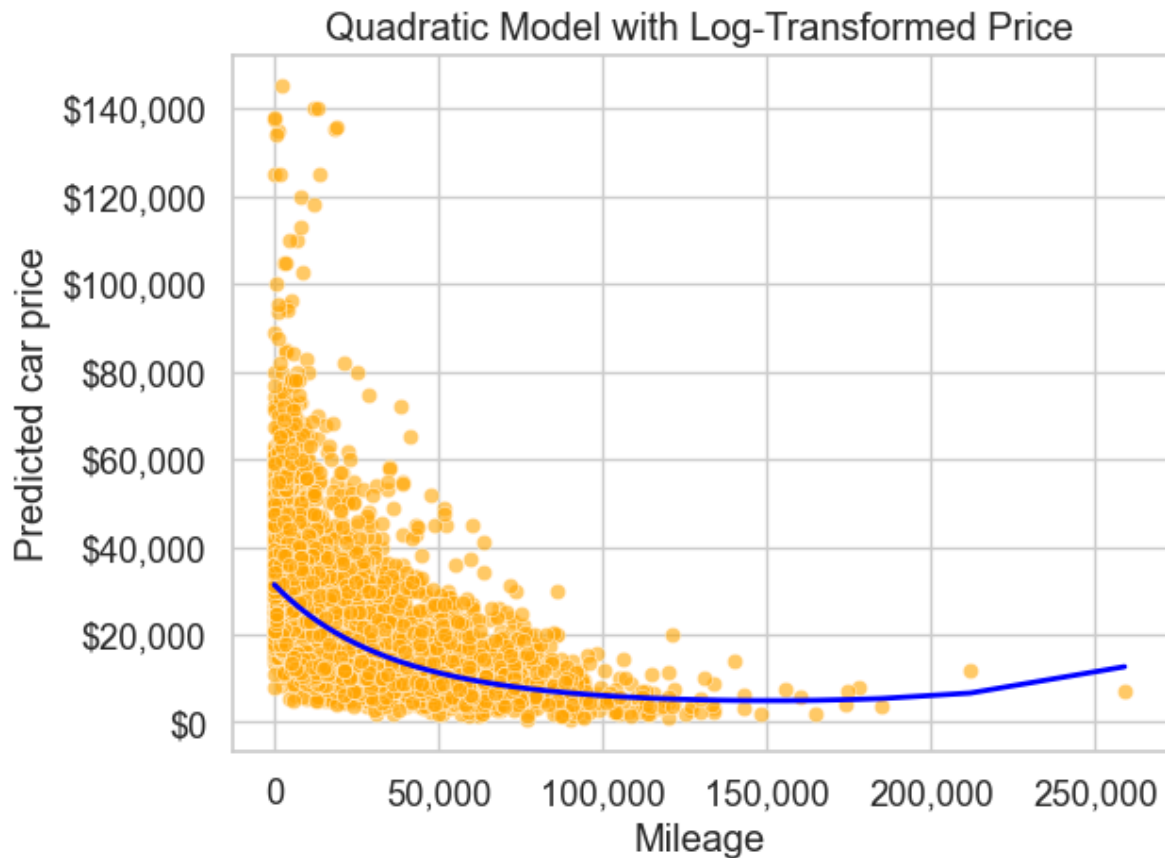
[2] The condition number is large, 4.81e+09. This might indicate that there are strong multicollinearity or other numerical problems.

```
# Visualize the quadratic fit (transform log predictions back to price scale)
pred_price_log = model_sm_quad.predict(train_updated)
pred_price = np.exp(pred_price_log) # Transform back from log scale

ax = sns.scatterplot(x='mileage', y='price', data=train_updated, color='orange', alpha=0.6)
sns.lineplot(x=train_updated.mileage, y=pred_price, color='blue', linewidth=2)
plt.xlabel('Mileage')
plt.ylabel('Predicted car price')
plt.title('Quadratic Model with Log-Transformed Price')

ax.yaxis.set_major_formatter('${x:,.0f}')

ax.xaxis.set_major_formatter('{x:,.0f}')
plt.show()
```



5.2.1.2 Cubic transformation

Add a cubic term for additional flexibility.

```
# Cubic model with log-transformed price
model_sm_cubic = smf.ols('np.log(price) ~ mileage + I(mileage**2) + I(mileage**3)', data=tra
print("Log-transformed price with cubic mileage terms:")
print(f"R-squared: {model_sm_cubic.rsquared:.4f}")
model_sm_cubic.summary()
```

```
Log-transformed price with cubic mileage terms:
R-squared: 0.3835
```

Dep. Variable:	np.log(price)	R-squared:	0.383
Model:	OLS	Adj. R-squared:	0.383
Method:	Least Squares	F-statistic:	1027.
Date:	Fri, 23 Jan 2026	Prob (F-statistic):	0.00
Time:	09:31:01	Log-Likelihood:	-4003.4
No. Observations:	4959	AIC:	8015.
Df Residuals:	4955	BIC:	8041.
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	10.3730	0.015	714.209	0.000	10.345	10.402
mileage	-2.705e-05	1.09e-06	-24.723	0.000	-2.92e-05	-2.49e-05
I(mileage ** 2)	1.328e-10	1.9e-11	6.985	0.000	9.55e-11	1.7e-10
I(mileage ** 3)	-2.22e-16	7.74e-17	-2.868	0.004	-3.74e-16	-7.02e-17

Omnibus:	28.207	Durbin-Watson:	0.325
Prob(Omnibus):	0.000	Jarque-Bera (JB):	29.444
Skew:	-0.161	Prob(JB):	4.04e-07
Kurtosis:	3.196	Cond. No.	7.73e+14

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 7.73e+14. This might indicate that there are strong multicollinearity or other numerical problems.

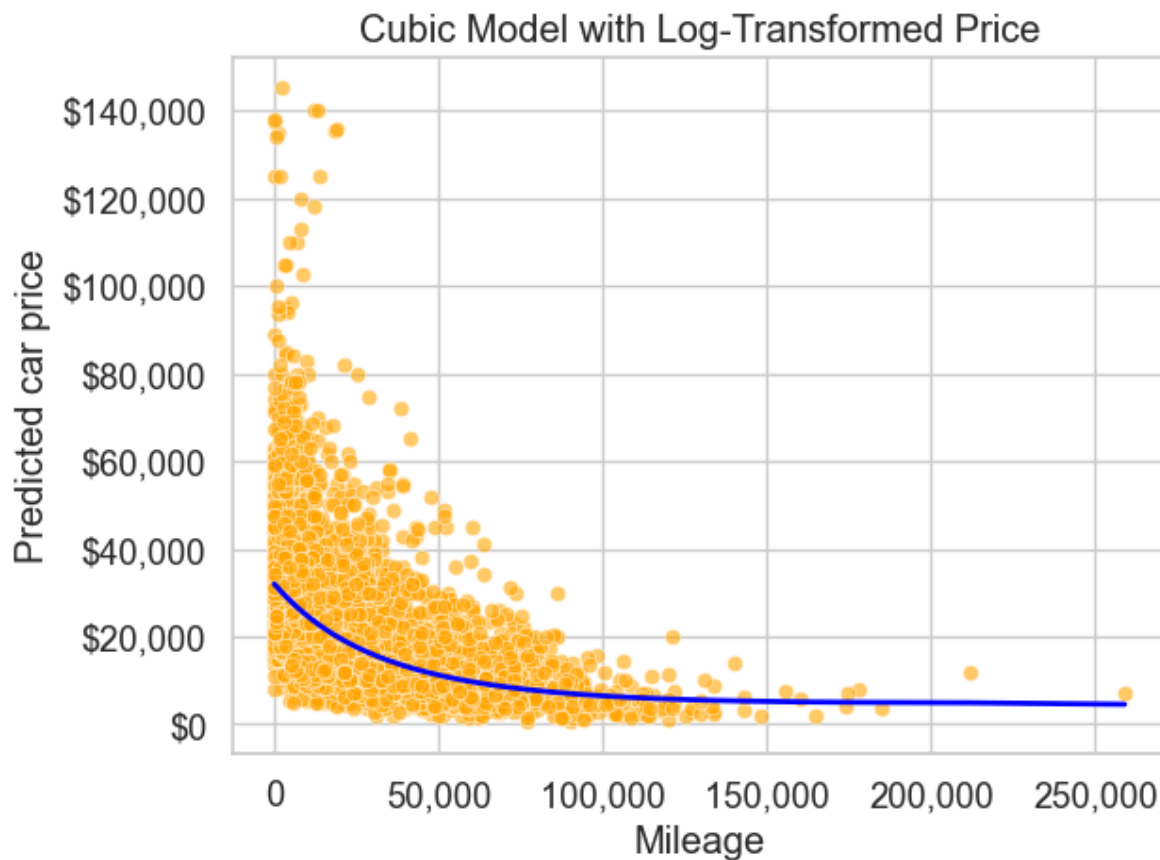
```
# Visualize the cubic fit (transform log predictions back to price scale)
pred_price_log = model_sm_cubic.predict(train_updated)
pred_price = np.exp(pred_price_log) # Transform back from log scale

ax = sns.scatterplot(x='mileage', y='price', data=train_updated, color='orange', alpha=0.6)
sns.lineplot(x=train_updated.mileage, y=pred_price, color='blue', linewidth=2)
plt.xlabel('Mileage')
plt.ylabel('Predicted car price')
plt.title('Cubic Model with Log-Transformed Price')

ax.yaxis.set_major_formatter('${x:,.0f}')

ax.xaxis.set_major_formatter('{x:,.0f}')

plt.show()
```



Do polynomial transformations work?

Adding polynomial terms yields: - **Quadratic model:** $R^2 = 38.2\%$ (improvement of 2.2 percentage points) - **Cubic model:** $R^2 = 38.3\%$ (improvement of 0.1 percentage points over quadratic)

Interpretation: The quadratic term provides a modest improvement (~2 percentage points), suggesting some nonlinearity exists in the mileage-price relationship. However, the cubic term adds almost nothing (0.1%), indicating diminishing returns.

While statistically these improvements may be significant, the practical gain is limited. The quadratic transformation is **moderately useful** but not transformative. The cubic term appears to be **overfitting** rather than capturing meaningful patterns. For this dataset, the log transformation itself provides the most substantial improvement, and a simple quadratic term may be sufficient if polynomial features are desired.

5.2.1.3 Putting interaction and transformation together

Combine the interaction term with a quadratic mileage term.

```
# Full model with interactions and log-transformed price
model_sm_full = smf.ols(
    formula='np.log(price) ~ age*engineSize + mileage + I(mileage**2)',
    data=train_updated,
).fit()

print("Full model with log-transformed price:")
print(f"R-squared: {model_sm_full.rsquared:.4f}")
model_sm_full.summary()
```

Full model with log-transformed price:
R-squared: 0.7993

Dep. Variable:	np.log(price)	R-squared:	0.799			
Model:	OLS	Adj. R-squared:	0.799			
Method:	Least Squares	F-statistic:	3944.			
Date:	Fri, 23 Jan 2026	Prob (F-statistic):	0.00			
Time:	11:37:33	Log-Likelihood:	-1221.1			
No. Observations:	4959	AIC:	2454.			
Df Residuals:	4953	BIC:	2493.			
Df Model:	5					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	9.4393	0.019	496.336	0.000	9.402	9.477
age	-0.1396	0.005	-30.210	0.000	-0.149	-0.131
engineSize	0.4634	0.008	58.287	0.000	0.448	0.479
age:engineSize	0.0075	0.002	4.487	0.000	0.004	0.011
mileage	-8.758e-06	4.6e-07	-19.044	0.000	-9.66e-06	-7.86e-06
I(mileage ** 2)	3.237e-11	3.53e-12	9.160	0.000	2.54e-11	3.93e-11
Omnibus:	456.087	Durbin-Watson:	0.491			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1602.553			
Skew:	0.433	Prob(JB):	0.00			
Kurtosis:	5.647	Cond. No.	1.35e+10			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, $1.35e+10$. This might indicate that there are strong multicollinearity or other numerical problems.

```
# RMSE on test data (transform log predictions back to price scale)
pred_price_log = model_sm_full.predict(test)
pred_price = np.exp(pred_price_log) # Transform back from log scale

rmse_full = np.sqrt(((test.price - pred_price) ** 2).mean())

print(f"RMSE on test data (log-transformed model): ${rmse_full:,.2f}")

rmse_log_scale = np.sqrt(((np.log(test.price) - pred_price_log) ** 2).mean())
# For comparison, calculate RMSE in log scale too
print(f"RMSE in log scale: {rmse_log_scale:.4f}")
```

```
RMSE on test data (log-transformed model): $9,646.67
RMSE in log scale: 0.3425
```

5.3 Polynomial Features in scikit-learn (same car dataset)

We keep the same train/test split and use `log(price)` to match the statsmodels section.

5.3.1 Handling Zero and Negative Values with Log Transformations

The Problem: `log(0)` is Undefined

In practice, Python returns `-inf` or throws an error when you try to compute `log(0)`. This is problematic when your data contains zero values.

Solution: `log1p()` - Best Solution for Most Cases

Use `np.log1p(x)` which computes `log(1 + x)`:

```
# Instead of:
y_transformed = np.log(y) # Fails if y contains 0

# Use:
y_transformed = np.log1p(y) # Works even when y = 0
```

Back-transformation:

```
# To reverse log1p, use expm1:  
y_original = np.expm1(y_transformed) # Computes exp(x) - 1
```

Benefits: - Handles zero values naturally: $\log(0) = \log(1) = 0$ - More numerically stable for small values - Common in financial and count data

Note: For this car price dataset, all prices are positive, so we can safely use `np.log()` directly.

```
# Create feature matrix and target vector (log price)  
feature_cols = ['mileage', 'engineSize', 'age', 'mpg']  
X_train = train_updated[feature_cols]  
X_test = test[feature_cols]  
y_train = np.log(train_updated['price'])  
y_test = np.log(test['price'])  
y_train_price = train_updated['price']  
y_test_price = test['price']  
  
X_train.head()
```

	mileage	engineSize	age	mpg
0	11	3.0	0	53.3282
1	10813	3.0	1	53.0430
2	6	3.0	0	53.4379
3	18895	3.0	3	51.5140
4	62953	3.0	5	51.4903

5.3.2 Train/test split

```
# We use the provided train/test split from the dataset.  
# Show train/test shapes  
print("Training set shape:", X_train.shape)  
print("Testing set shape:", X_test.shape)
```

Training set shape: (4000, 2)

Testing set shape: (1000, 2)

5.3.3 Baseline Model (original Features)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error, r2_score

# Fit a baseline linear regression model on log(price)
baseline_model = LinearRegression()
baseline_model.fit(X_train, y_train)

# Predict on train and test splits (log scale)
y_pred_baseline = baseline_model.predict(X_test)
y_train_pred_baseline = baseline_model.predict(X_train)

# Evaluate on test and training sets (log scale)
rmse_baseline = root_mean_squared_error(y_test, y_pred_baseline)
r2_baseline = r2_score(y_test, y_pred_baseline)

rmse_train_baseline = root_mean_squared_error(y_train, y_train_pred_baseline)
r2_train_baseline = r2_score(y_train, y_train_pred_baseline)

# Back-transform to price scale (see log-trap note above)
pred_price_baseline = np.exp(y_pred_baseline)
rmse_price_baseline = root_mean_squared_error(y_test_price, pred_price_baseline)

print()
print("Baseline Model Performance (log scale):")
print("Training Set:")
print("RMSE:", rmse_train_baseline)
print("R2:", r2_train_baseline)
print()
print("Testing Set:")
print("RMSE:", rmse_baseline)
print("R2:", r2_baseline)
print("RMSE (price scale):", f"${rmse_price_baseline:,.2f}")
```

```
Baseline Model Performance (log scale):
Training Set:
RMSE: 0.3126761090785014
R2: 0.7951679756008143
```


Testing Set:
RMSE: 0.34009843313619664
R2: 0.759217125194597
RMSE (price scale): \$9,802.82

5.3.4 Transform Features with PolynomialFeatures (degree = 2)

```
from sklearn.preprocessing import PolynomialFeatures

# Create PolynomialFeatures object with degree=2 (includes interaction terms)
poly_2 = PolynomialFeatures(degree=2, include_bias=False)

# Transform the training and testing features
X_train_poly_2 = poly_2.fit_transform(X_train)
X_test_poly_2 = poly_2.transform(X_test)

# Display the transformed feature names
print()
print("Transformed Feature Names:")
print(poly_2.get_feature_names_out())
```

Transformed Feature Names:
['mileage' 'engineSize' 'age' 'mpg' 'mileage^2' 'mileage engineSize'
 'mileage age' 'mileage mpg' 'engineSize^2' 'engineSize age'
 'engineSize mpg' 'age^2' 'age mpg' 'mpg^2']

5.3.4.1 Linear Model with transformed Features (degree = 2)

```
# Fit a linear regression model on degree-2 polynomial features (log price)
poly_2_model = LinearRegression()
poly_2_model.fit(X_train_poly_2, y_train)

# Predict on train and test splits (log scale)
y_pred_poly_2 = poly_2_model.predict(X_test_poly_2)
y_train_pred_poly_2 = poly_2_model.predict(X_train_poly_2)

# Evaluate on test and training sets (log scale)
```

```

rmse_poly_2 = root_mean_squared_error(y_test, y_pred_poly_2)
r2_poly_2 = r2_score(y_test, y_pred_poly_2)

rmse_train_poly_2 = root_mean_squared_error(y_train, y_train_pred_poly_2)
r2_train_poly_2 = r2_score(y_train, y_train_pred_poly_2)

# Back-transform to price scale (see log-trap note above)
pred_price_poly_2 = np.exp(y_pred_poly_2)
rmse_price_poly_2 = root_mean_squared_error(y_test_price, pred_price_poly_2)

print()
print("Polynomial Model Performance (degree=2, log scale):")
print("Training Set:")
print("RMSE:", rmse_train_poly_2)
print("R2:", r2_train_poly_2)
print()
print("Testing Set:")
print("RMSE:", rmse_poly_2)
print("R2:", r2_poly_2)
print("RMSE (price scale):", f"${rmse_price_poly_2:.2f}")

```

```

Polynomial Model Performance (degree=2, log scale):
Training Set:
RMSE: 0.3002407156431113
R2: 0.8111366745155597

```

```

Testing Set:
RMSE: 0.30894008525487293
R2: 0.8013151022053762
RMSE (price scale): $8,479.10

```

5.3.5 Transform features with PolynomialFeatures (degree = 3)

```

# Create PolynomialFeatures object with degree=3
poly_3 = PolynomialFeatures(degree=3, include_bias=False)

# Transform the training and testing features
X_train_poly_3 = poly_3.fit_transform(X_train)
X_test_poly_3 = poly_3.transform(X_test)

```

```
# Display the transformed feature names
print()
print("Transformed Feature Names:")
print(poly_3.get_feature_names_out())
```

Transformed Feature Names:

```
['mileage' 'engineSize' 'age' 'mpg' 'mileage^2' 'mileage engineSize'
 'mileage age' 'mileage mpg' 'engineSize^2' 'engineSize age'
 'engineSize mpg' 'age^2' 'age mpg' 'mpg^2' 'mileage^3'
 'mileage^2 engineSize' 'mileage^2 age' 'mileage^2 mpg'
 'mileage engineSize^2' 'mileage engineSize age' 'mileage engineSize mpg'
 'mileage age^2' 'mileage age mpg' 'mileage mpg^2' 'engineSize^3'
 'engineSize^2 age' 'engineSize^2 mpg' 'engineSize age^2'
 'engineSize age mpg' 'engineSize mpg^2' 'age^3' 'age^2 mpg' 'age mpg^2'
 'mpg^3']
```

5.3.5.1 Linear Model with transformed Features (degree = 3)

```
# Fit a linear regression model on degree-3 polynomial features (log price)
poly_3_model = LinearRegression()
poly_3_model.fit(X_train_poly_3, y_train)

# Predict on train and test splits (log scale)
y_pred_poly_3 = poly_3_model.predict(X_test_poly_3)
y_pred_train_poly_3 = poly_3_model.predict(X_train_poly_3)

# Evaluate on test and training sets (log scale)
rmse_poly_3 = root_mean_squared_error(y_test, y_pred_poly_3)
r2_poly_3 = r2_score(y_test, y_pred_poly_3)

rmse_poly_3_train = root_mean_squared_error(y_train, y_pred_train_poly_3)
r2_poly_3_train = r2_score(y_train, y_pred_train_poly_3)

# Back-transform to price scale (see log-trap note above)
pred_price_poly_3 = np.exp(y_pred_poly_3)
rmse_price_poly_3 = root_mean_squared_error(y_test_price, pred_price_poly_3)

print()
```

```

print("Polynomial Model Performance (degree=3, log scale):")
print("Training Set:")
print("RMSE:", rmse_poly_3_train)
print("R2:", r2_poly_3_train)
print()
print("Testing Set:")
print("RMSE:", rmse_poly_3)
print("R2:", r2_poly_3)
print("RMSE (price scale):", f"${rmse_price_poly_3:.2f}")

```

Polynomial Model Performance (degree=3, log scale):

Training Set:

RMSE: 0.3811542627298689

R2: 0.695624268973846

Testing Set:

RMSE: 0.3804705656720708

R2: 0.6986588360960538

RMSE (price scale): \$12,105.56

5.3.6 degree = 4

```

# Use polynomial degree of 4 to see if it improves the model
poly_4 = PolynomialFeatures(degree=4, include_bias=False)

# Transform the training and testing features
X_train_poly_4 = poly_4.fit_transform(X_train)
X_test_poly_4 = poly_4.transform(X_test)

# Create a linear regression model for the polynomial features
poly_4_model = LinearRegression()

# Train the model on the transformed features
poly_4_model.fit(X_train_poly_4, y_train)

# Make predictions on the test and training sets (log scale)
y_pred_poly_4 = poly_4_model.predict(X_test_poly_4)
y_pred_train_poly_4 = poly_4_model.predict(X_train_poly_4)

```

```

# Evaluate the polynomial model (log scale)
rmse_poly_4 = root_mean_squared_error(y_test, y_pred_poly_4)
r2_poly_4 = r2_score(y_test, y_pred_poly_4)

rmse_poly_4_train = root_mean_squared_error(y_train, y_pred_train_poly_4)
r2_poly_4_train = r2_score(y_train, y_pred_train_poly_4)

# Back-transform to price scale (see log-trap note above)
pred_price_poly_4 = np.exp(y_pred_poly_4)
rmse_price_poly_4 = root_mean_squared_error(y_test_price, pred_price_poly_4)

print()
print("Polynomial Model Performance (degree=4, log scale):")
print("Training Set:")
print("RMSE:", rmse_poly_4_train)
print("R2:", r2_poly_4_train)
print()
print("Testing Set:")
print("RMSE:", rmse_poly_4)
print("R2:", r2_poly_4)
print("RMSE (price scale):", f"${rmse_price_poly_4:,.2f}")

```

Polynomial Model Performance (degree=4, log scale):

Training Set:

RMSE: 0.4825146268287559

R2: 0.5122138946437124

Testing Set:

RMSE: 0.48497471912881407

R2: 0.5103852040335006

RMSE (price scale): \$14,132.89

```

# get the feature names for the polynomial degree 4 model
print("\nNumber of Features:", len(poly_4.get_feature_names_out()))
print("\nTransformed Feature Names:")
print(poly_4.get_feature_names_out())

```

Number of Features: 69

Transformed Feature Names:

```
['mileage' 'engineSize' 'age' 'mpg' 'mileage^2' 'mileage engineSize'
 'mileage age' 'mileage mpg' 'engineSize^2' 'engineSize age'
 'engineSize mpg' 'age^2' 'age mpg' 'mpg^2' 'mileage^3'
 'mileage^2 engineSize' 'mileage^2 age' 'mileage^2 mpg'
 'mileage engineSize^2' 'mileage engineSize age' 'mileage engineSize mpg'
 'mileage age^2' 'mileage age mpg' 'mileage mpg^2' 'engineSize^3'
 'engineSize^2 age' 'engineSize^2 mpg' 'engineSize age^2'
 'engineSize age mpg' 'engineSize mpg^2' 'age^3' 'age^2 mpg' 'age mpg^2'
 'mpg^3' 'mileage^4' 'mileage^3 engineSize' 'mileage^3 age'
 'mileage^3 mpg' 'mileage^2 engineSize^2' 'mileage^2 engineSize age'
 'mileage^2 engineSize mpg' 'mileage^2 age^2' 'mileage^2 age mpg'
 'mileage^2 mpg^2' 'mileage engineSize^3' 'mileage engineSize^2 age'
 'mileage engineSize^2 mpg' 'mileage engineSize age^2'
 'mileage engineSize age mpg' 'mileage engineSize mpg^2' 'mileage age^3'
 'mileage age^2 mpg' 'mileage age mpg^2' 'mileage mpg^3' 'engineSize^4'
 'engineSize^3 age' 'engineSize^3 mpg' 'engineSize^2 age^2'
 'engineSize^2 age mpg' 'engineSize^2 mpg^2' 'engineSize age^3'
 'engineSize age^2 mpg' 'engineSize age mpg^2' 'engineSize mpg^3' 'age^4'
 'age^3 mpg' 'age^2 mpg^2' 'age mpg^3' 'mpg^4']
```

5.3.7 Putting all together

```
# Summarize model performance side by side
models = ['Baseline', 'Polynomial Degree 2', 'Polynomial Degree 3', 'Polynomial Degree 4']
features = [
    X_train.shape[1],
    len(poly_2.get_feature_names_out()),
    len(poly_3.get_feature_names_out()),
    len(poly_4.get_feature_names_out()),
]
training_rmse = [rmse_train_baseline, rmse_train_poly_2, rmse_poly_3_train, rmse_poly_4_train]
testing_rmse = [rmse_baseline, rmse_poly_2, rmse_poly_3, rmse_poly_4]
testing_rmse_price = [rmse_price_baseline, rmse_price_poly_2, rmse_price_poly_3, rmse_price_poly_4]
training_r2 = [r2_train_baseline, r2_train_poly_2, r2_poly_3_train, r2_poly_4_train]
testing_r2 = [r2_baseline, r2_poly_2, r2_poly_3, r2_poly_4]

model_comparison = pd.DataFrame(
    {
        'Model': models,
        'Features': features,
```

```

        'Training RMSE (log)': training_rmse,
        'Testing RMSE (log)': testing_rmse,
        'Testing RMSE (price)': testing_rmse_price,
        'Training R2': training_r2,
        'Testing R2': testing_r2,
    }
)

model_comparison

```

	Model	Features	Training RMSE (log)	Testing RMSE (log)	Testing RMSE (price)	Th
0	Baseline	4	0.312676	0.340098	9802.816771	0.
1	Polynomial Degree 2	14	0.300241	0.308940	8479.095395	0.
2	Polynomial Degree 3	34	0.381154	0.380471	12105.564864	0.
3	Polynomial Degree 4	69	0.482515	0.484975	14132.893458	0.

When the polynomial degree increases, training error typically decreases because the model becomes more flexible. However, test performance may stop improving—or even get worse—which indicates overfitting. In our results, degree 3 and 4 appear to overfit, while degree 2 provides the best generalization.

Next, we'll implement the degree-2 model using a scikit-learn Pipeline.

```

# Pipeline for the best model (degree=2)
from sklearn.pipeline import Pipeline

poly2_pipeline = Pipeline(
    steps=[
        ('poly', PolynomialFeatures(degree=2, include_bias=False)),
        ('model', LinearRegression()),
    ]
)

poly2_pipeline.fit(X_train, y_train)
pred_log_pipe = poly2_pipeline.predict(X_test)
rmse_log_pipe = root_mean_squared_error(y_test, pred_log_pipe)
rmse_price_pipe = root_mean_squared_error(y_test_price, np.exp(pred_log_pipe))

print("Pipeline (degree=2) RMSE (log):", rmse_log_pipe)
print("Pipeline (degree=2) RMSE (price):", f"${rmse_price_pipe:,.2f}")

```

```
Pipeline (degree=2) RMSE (log): 0.30894008525487293
Pipeline (degree=2) RMSE (price): $8,479.10
```

5.3.8 Key takeaway:

In `scikit-learn`, the built-in `PolynomialFeatures` transformer is somewhat “all or nothing”: by default, it generates **all** polynomial terms (including interactions) up to a certain degree. You can toggle:

- `interaction_only=True` to generate only cross-terms
- `include_bias=False` to exclude the constant (bias) term,
- `degree` to control how high the polynomial powers go.

However, if you want **fine-grained control** over exactly which terms get generated (for example, only certain interaction terms, or only a subset of polynomial terms), you will need to create those features manually or write a custom transformer (skipped for beginner level)

And also when using `PolynomialFeatures` (or any other `scikit-learn` transformer), the fitting step is always done on the training data—not on the test data. This is a fundamental principle of machine learning pipelines: we do not use the test set for any part of model training (including feature encoding, feature generation, scaling, etc.).

5.4 Summary

Feature transformations extend the capabilities of linear regression by capturing nonlinear relationships while maintaining the interpretability and computational efficiency of linear models.

Key Takeaways:

1. **Exploratory Data Analysis First:** Use EDA to examine relationships between features and the target variable. Visualizations help identify nonlinear patterns that suggest transformation needs.
2. **Log Transformation Benefits:**
 - Applies to both the target variable and predictors
 - Reduces heteroscedasticity (non-constant variance)
 - Handles multiplicative effects and percentage changes
 - Improves model fit for skewed distributions
3. **Polynomial Features:**
 - Common approach for capturing nonlinear relationships

- Higher degrees increase model flexibility
- Can model curved patterns (quadratic, cubic, etc.)

4. **Bias-Variance Trade-off:**

- Adding predictors/features typically decreases bias (underfitting)
- But increases variance (risk of overfitting)
- More complex models fit training data better but may generalize poorly

5. **Model Evaluation is Critical:**

- Always compare training and test performance
- Large gaps indicate overfitting
- Use test metrics to guide model selection

Best Practices: - Start simple, add complexity gradually - Monitor both training and test performance - Choose models that generalize well over those that merely fit training data

6 Beyond Fit (implementation)

```
# Import necessary libraries
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
# Load the Boston Housing dataset (for demonstration purposes)
df = pd.read_csv('datasets/Housing.csv')
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating
0	13300000	7420	4	2	3	yes	no	no	no
1	12250000	8960	4	4	4	yes	no	no	no
2	12250000	9960	3	2	2	yes	no	yes	no
3	12215000	7500	4	2	2	yes	no	yes	no
4	11410000	7420	4	1	2	yes	yes	yes	no

```
# build a formular api model using price as the target, the rest of the variables as predictors
model = smf.ols('price ~ area + bedrooms + bathrooms + stories + mainroad + guestroom + basement', df)
model = model.fit()
print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          price    R-squared:                0.682
Model:                  OLS      Adj. R-squared:            0.674
Method:                 Least Squares    F-statistic:          87.52
Date:                   Wed, 05 Feb 2025    Prob (F-statistic):    9.07e-123
```

```

Time:                                08:46:02   Log-Likelihood:                -8331.5
No. Observations:                    545     AIC:                        1.669e+04
Df Residuals:                        531     BIC:                        1.675e+04
Df Model:                            13
Covariance Type:                    nonrobust

```

	coef	std err	t	P> t	[0.025
Intercept	4.277e+04	2.64e+05	0.162	0.872	-4.76e+05
mainroad[T.yes]	4.213e+05	1.42e+05	2.962	0.003	1.42e+05
guestroom[T.yes]	3.005e+05	1.32e+05	2.282	0.023	4.18e+04
basement[T.yes]	3.501e+05	1.1e+05	3.175	0.002	1.33e+05
hotwaterheating[T.yes]	8.554e+05	2.23e+05	3.833	0.000	4.17e+05
airconditioning[T.yes]	8.65e+05	1.08e+05	7.983	0.000	6.52e+05
prefarea[T.yes]	6.515e+05	1.16e+05	5.632	0.000	4.24e+05
furnishingstatus[T.semi-furnished]	-4.634e+04	1.17e+05	-0.398	0.691	-2.75e+05
furnishingstatus[T.unfurnished]	-4.112e+05	1.26e+05	-3.258	0.001	-6.59e+05
area	244.1394	24.289	10.052	0.000	196.425
bedrooms	1.148e+05	7.26e+04	1.581	0.114	-2.78e+04
bathrooms	9.877e+05	1.03e+05	9.555	0.000	7.85e+05
stories	4.508e+05	6.42e+04	7.026	0.000	3.25e+05
parking	2.771e+05	5.85e+04	4.735	0.000	1.62e+05
Omnibus:	97.909	Durbin-Watson:	1.209		
Prob(Omnibus):	0.000	Jarque-Bera (JB):	258.281		
Skew:	0.895	Prob(JB):	8.22e-57		
Kurtosis:	5.859	Cond. No.	3.49e+04		

Notes:

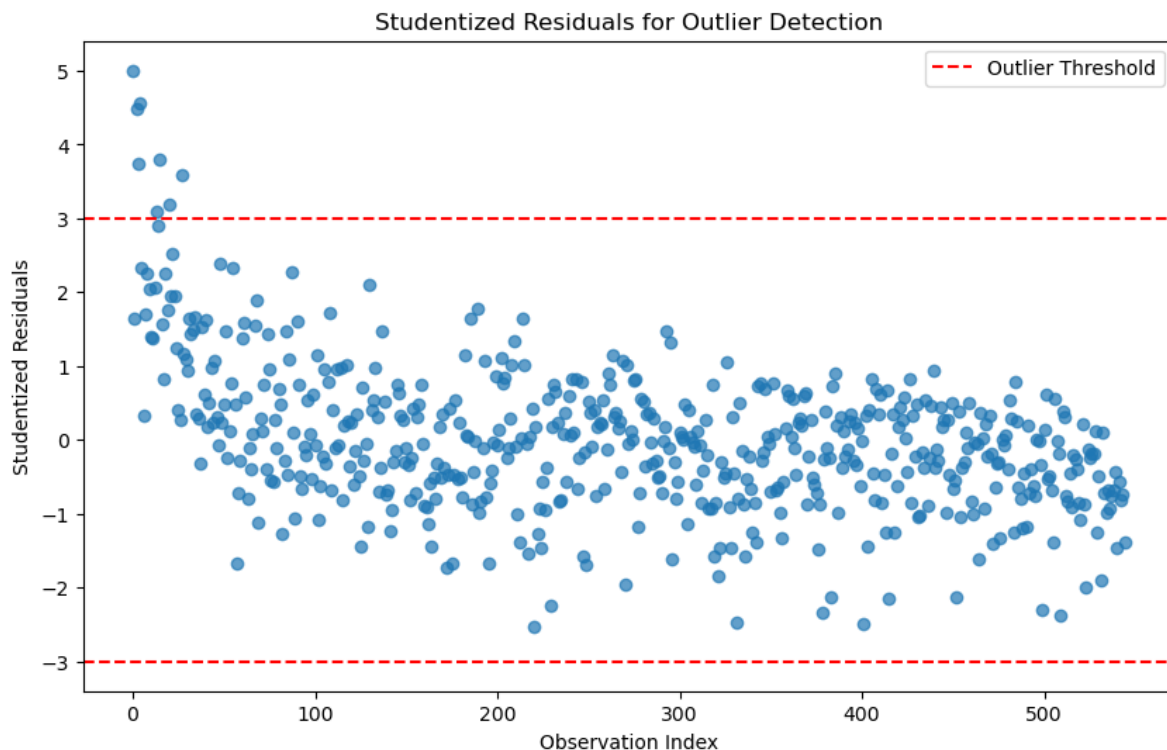
- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 3.49e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```

# -----
# 1. Identifying Outliers (using studentized residuals)
# -----
# Outliers can be detected using studentized residuals
outliers_studentized = model.get_influence().resid_studentized_external
outlier_threshold = 3 # Common threshold for studentized residuals

```

```
# Plot studentized residuals
plt.figure(figsize=(10, 6))
plt.scatter(range(len(outliers_studentized)), outliers_studentized, alpha=0.7)
plt.axhline(y=outlier_threshold, color='r', linestyle='--', label='Outlier Threshold')
plt.axhline(y=-outlier_threshold, color='r', linestyle='--')
plt.title('Studentized Residuals for Outlier Detection')
plt.xlabel('Observation Index')
plt.ylabel('Studentized Residuals')
plt.legend()
plt.show()
```



```
# Identify observations with high studentized residuals
outlier_indices_studentized = np.where(np.abs(outliers_studentized) > outlier_threshold)[0]
print(f"Outliers detected at indices: {outlier_indices_studentized}")
```

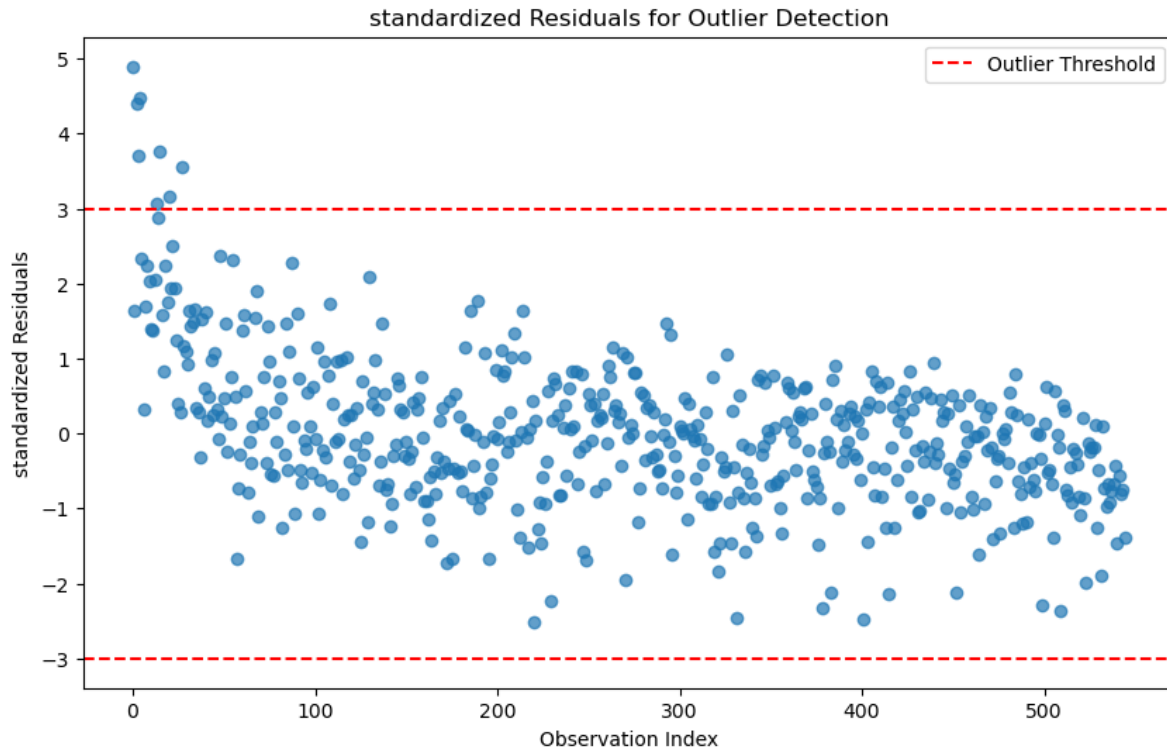
Outliers detected at indices: [0 2 3 4 13 15 20 27]

```
# -----
# 1. Identifying Outliers (using standardized residuals)
# -----
# Outliers can be detected using standardized residuals
outliers_standardized = model.get_influence().resid_studentized_internal
outlier_threshold = 3 # Common threshold for standardized residuals

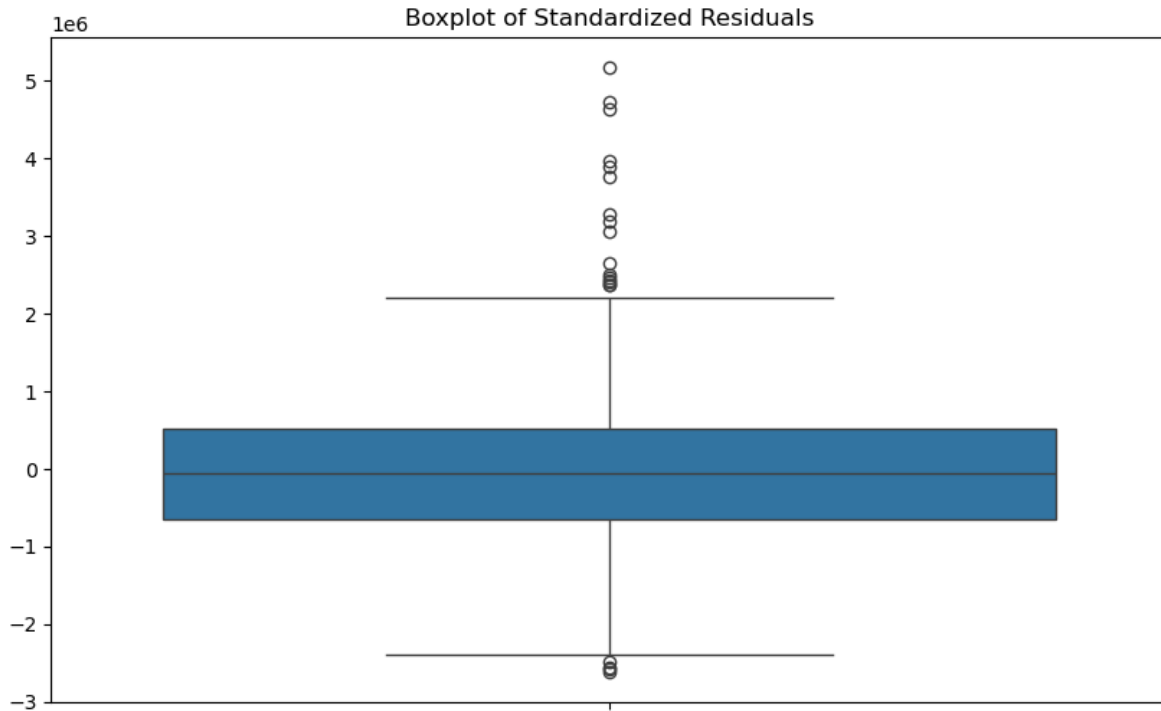
# Identify observations with high standardized residuals
outlier_indices_standardized = np.where(np.abs(outliers_standardized) > outlier_threshold)[0]
print(f"Outliers detected at indices: {outlier_indices_standardized}")
```

Outliers detected at indices: [0 2 3 4 13 15 20 27]

```
# Plot studentized residuals
plt.figure(figsize=(10, 6))
plt.scatter(range(len(outliers_standardized)), outliers_standardized, alpha=0.7)
plt.axhline(y=outlier_threshold, color='r', linestyle='--', label='Outlier Threshold')
plt.axhline(y=-outlier_threshold, color='r', linestyle='--')
plt.title('standardized Residuals for Outlier Detection')
plt.xlabel('Observation Index')
plt.ylabel('standardized Residuals')
plt.legend()
plt.show()
```



```
# -----  
# 1. Identifying Outliers (using boxplot)  
# -----  
# Outliers can be detected using boxplot of standardized residuals  
plt.figure(figsize=(10, 6))  
sns.boxplot(model.resid)  
plt.title('Boxplot of Standardized Residuals');
```



```
# use 3 standard deviation rule to identify outliers
outlier_indices = np.where(np.abs(model.resid) > 3 * model.resid.std())[0]
print(f"Outliers detected at indices: {outlier_indices}")
```

```
Outliers detected at indices: [ 0  2  3  4 13 15 20 27]
```

```
# -----
# 2. Identifying High Leverage Points
# -----
# High leverage points can be detected using the hat matrix (leverage values)
leverage = model.get_influence().hat_matrix_diag
leverage_threshold = 2 * (df.shape[1] / df.shape[0]) # Common threshold for leverage
```

6.0.0.1 Identifying High Leverage Points

A common threshold for identifying **high leverage points** in regression analysis is:

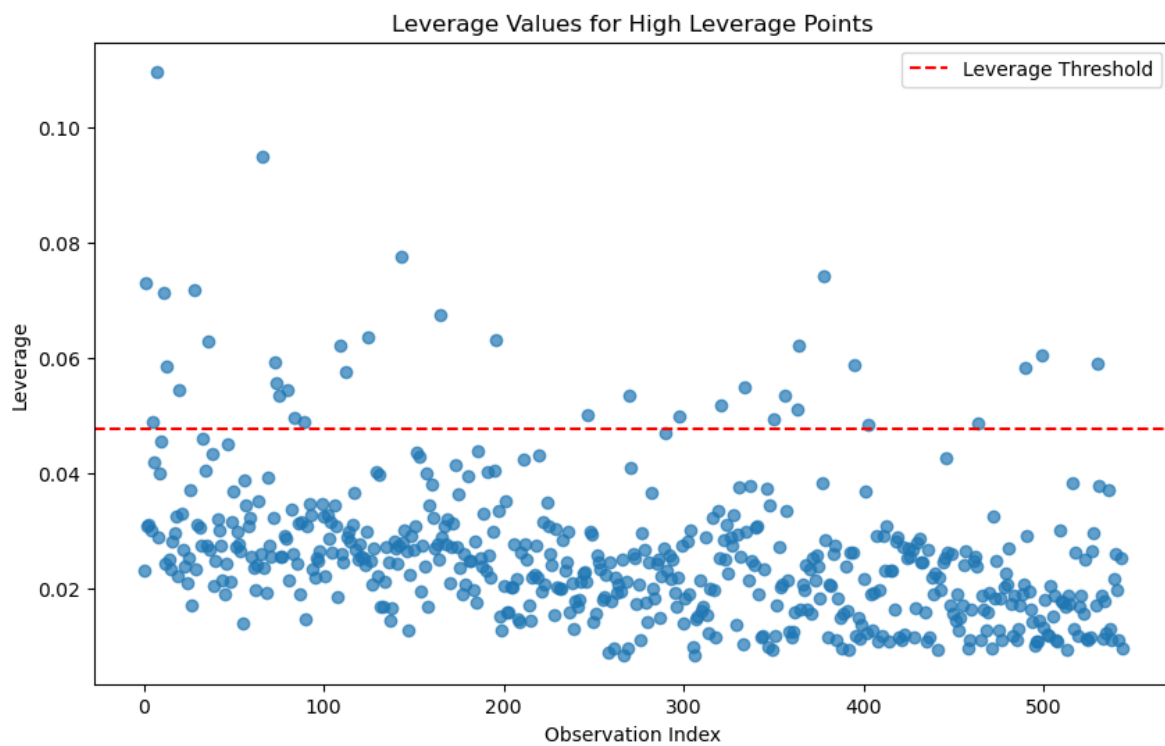
$$h_i > \frac{2p}{n}$$

where:

- h_i is the leverage value for the (i)-th observation,

- p is the number of predictors (including the intercept), and
- n is the total number of observations.

```
# Plot leverage values
plt.figure(figsize=(10, 6))
plt.scatter(range(len(leverage)), leverage, alpha=0.7)
plt.axhline(y=leverage_threshold, color='r', linestyle='--', label='Leverage Threshold')
plt.title('Leverage Values for High Leverage Points')
plt.xlabel('Observation Index')
plt.ylabel('Leverage')
plt.legend()
plt.show()
```



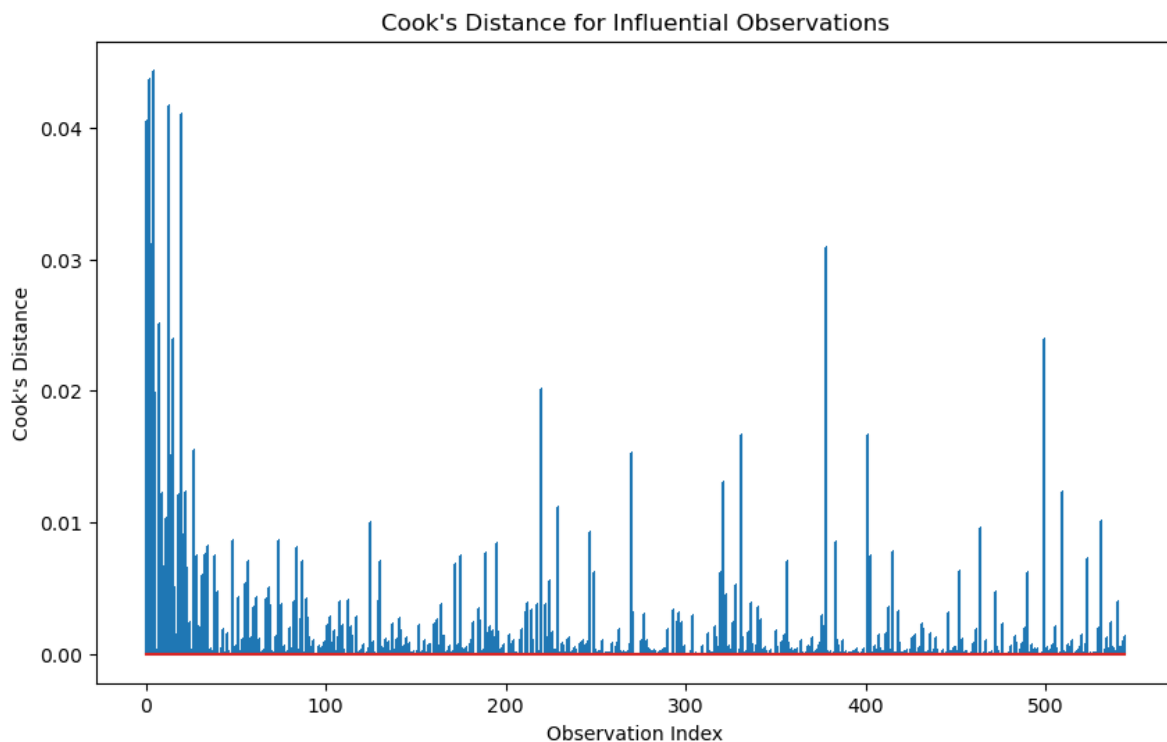
```
# Identify observations with high leverage
high_leverage_indices = np.where(leverage > leverage_threshold)[0]
print(f"High leverage points detected at indices: {high_leverage_indices}")
```

```
High leverage points detected at indices: [ 1  5  7 11 13 20 28 36 66 73 74 75 8
143 165 196 247 270 298 321 334 350 356 363 364 378 395 403 464 490 499
530]
```



```
# -----
# 3. Cook's Distance for Influential Observations
# -----
# Cook's distance measures the influence of each observation on the model
cooks_distance = model.get_influence().cooks_distance[0]

# Plot Cook's distance
plt.figure(figsize=(10, 6))
plt.stem(range(len(cooks_distance)), cooks_distance, markerfmt=",")
plt.title("Cook's Distance for Influential Observations")
plt.xlabel('Observation Index')
plt.ylabel("Cook's Distance")
plt.show()
```



Cook's distance is considered high if it is greater than 0.5 and extreme if it is greater than 1.

```
# Identify influential observations
influential_threshold = 4 / (df.shape[1] - 1) # Common threshold for Cook's distance
influential_indices = np.where(cooks_distance > influential_threshold)[0]
print(f"Influential observations detected at indices: {influential_indices}")
```

Influential observations detected at indices: []

```
# =====
# 4. Checking Multicollinearity (VIF)
# =====
# VIF calculation
from statsmodels.stats.outliers_influence import variance_inflation_factor

def calculate_vif(X):
    vif_data = pd.DataFrame()
    vif_data["Variable"] = X.columns
    vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
    return vif_data

X = df[['area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'parking', 'prefarea', 'furnishingstatus']]

# one-hot encoding for categorical variables
X = pd.get_dummies(X, drop_first=True, dtype=float)

vif_data = calculate_vif(X)
print("\nVariance Inflation Factors:")
print(vif_data.sort_values('VIF', ascending=False))
```

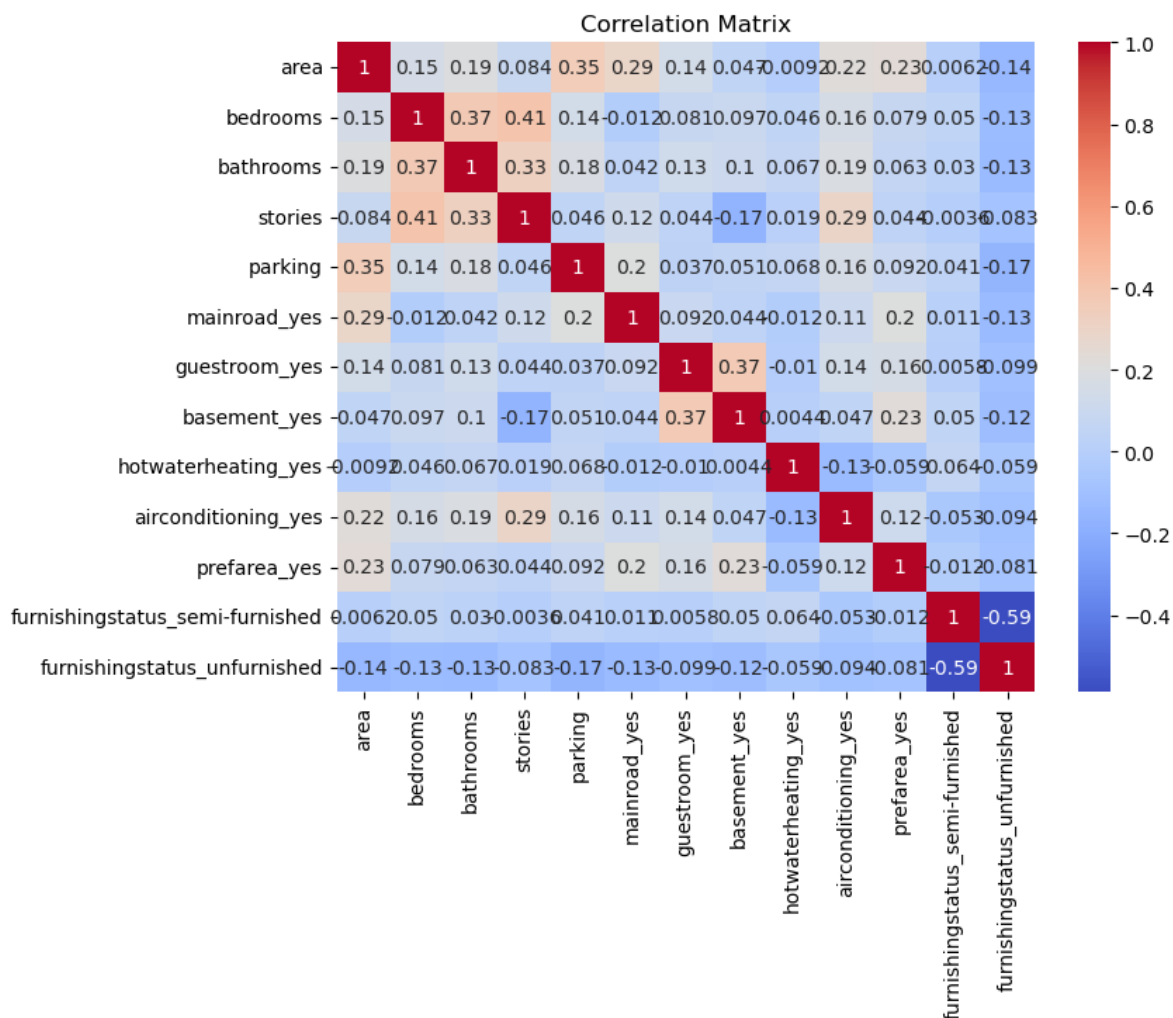
Variance Inflation Factors:

	Variable	VIF
1	bedrooms	16.652387
2	bathrooms	9.417643
0	area	8.276447
3	stories	7.880730
5	mainroad_yes	6.884806
11	furnishingstatus_semi-furnished	2.386831
7	basement_yes	2.019858
12	furnishingstatus_unfurnished	2.008632
4	parking	1.986400
9	airconditioning_yes	1.767753
10	prefarea_yes	1.494211
6	guestroom_yes	1.473234
8	hotwaterheating_yes	1.091568

```
# Rule of thumb: VIF > 10 indicates significant multicollinearity
multicollinear_features = vif_data[vif_data['VIF'] > 10]['Variable']
print(f"Features with significant multicollinearity: {multicollinear_features.tolist()}")
```

Features with significant multicollinearity: ['bedrooms']

```
# =====
# 4. Checking Multicollinearity (Correlation Matrix)
# =====
# Correlation matrix
correlation_matrix = X.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix');
```



```
#output the correlation of other predictors with the bedrooms
X.corr()['bedrooms'].abs().sort_values(ascending=False)
```

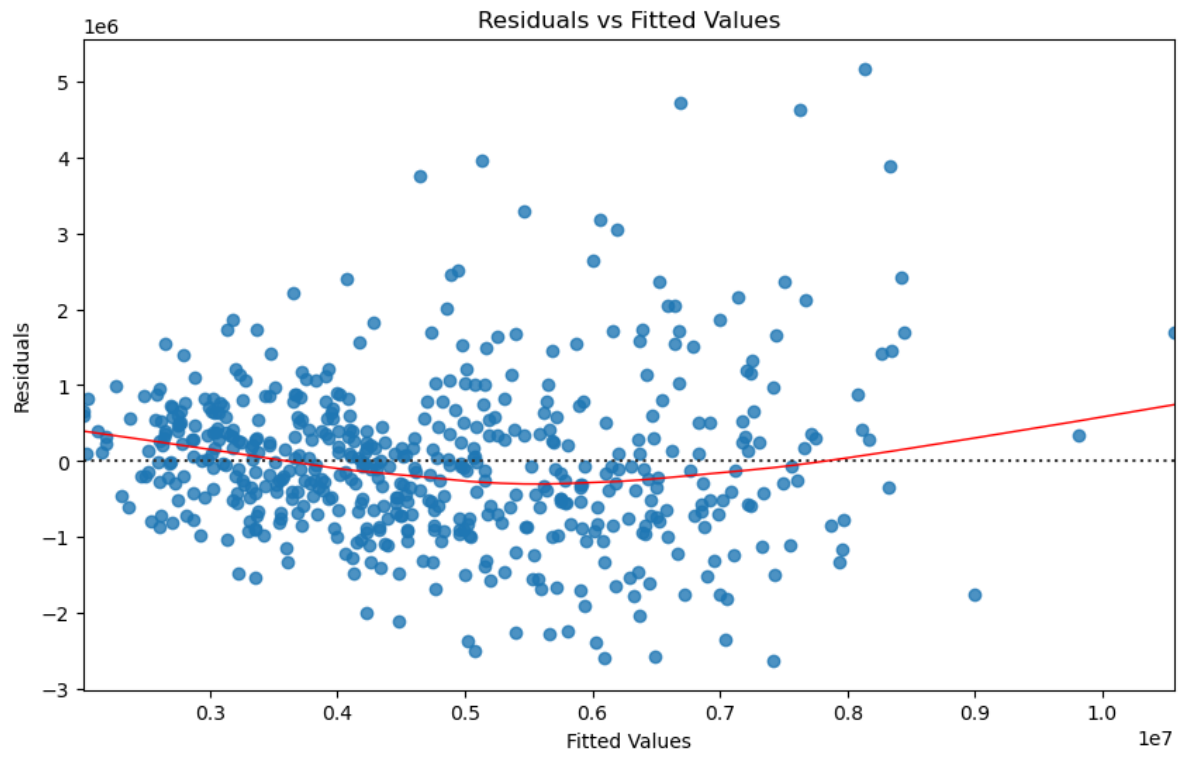
```
bedrooms          1.000000
stories           0.408564
bathrooms         0.373930
airconditioning_yes 0.160603
area              0.151858
parking           0.139270
furnishingstatus_unfurnished 0.126252
basement_yes      0.097312
guestroom_yes     0.080549
prefarea_yes      0.079023
```

```
furnishingstatus_semi-furnished    0.050040
hotwaterheating_yes                0.046049
mainroad_yes                       0.012033
Name: bedrooms, dtype: float64
```

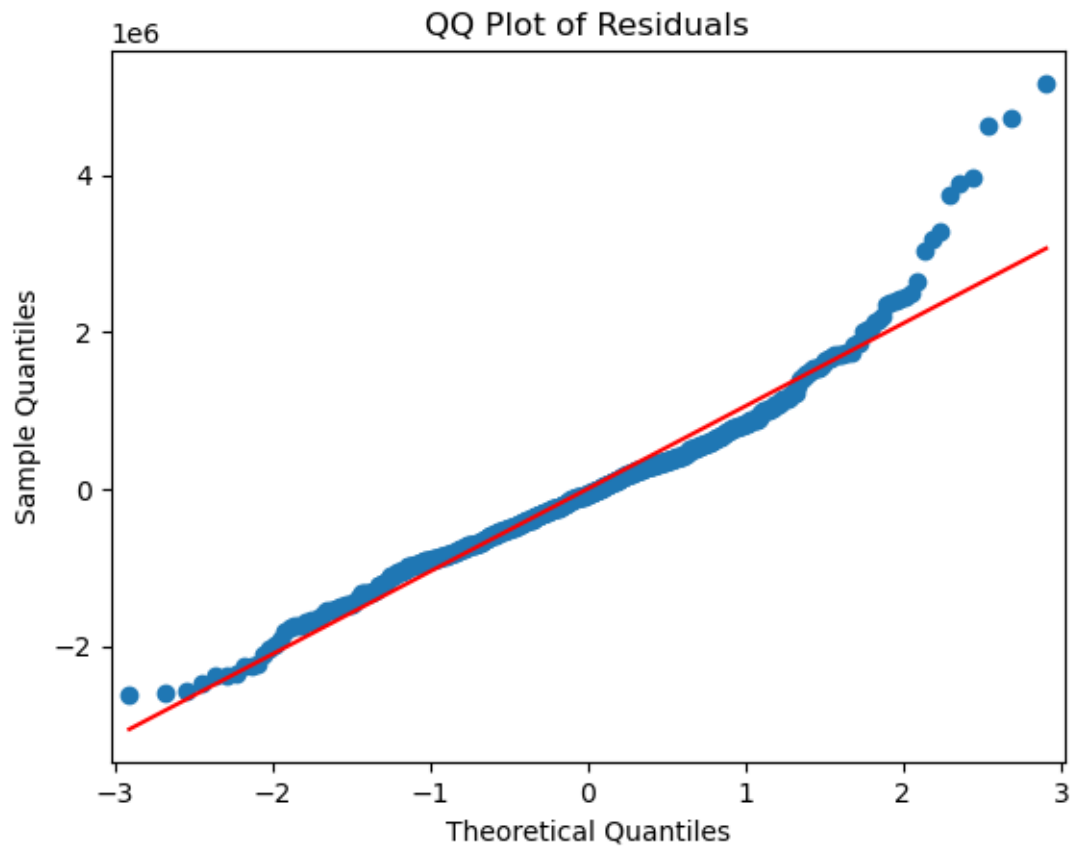
```
# -----
# 5. Analyzing Residual Patterns
# -----
# Residuals vs Fitted Values Plot
fitted_values = model.fittedvalues
residuals = model.resid

plt.figure(figsize=(10, 6))
sns.residplot(x=fitted_values, y=residuals, lowess=True, line_kws={'color': 'red', 'lw': 1})
plt.title('Residuals vs Fitted Values')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals');

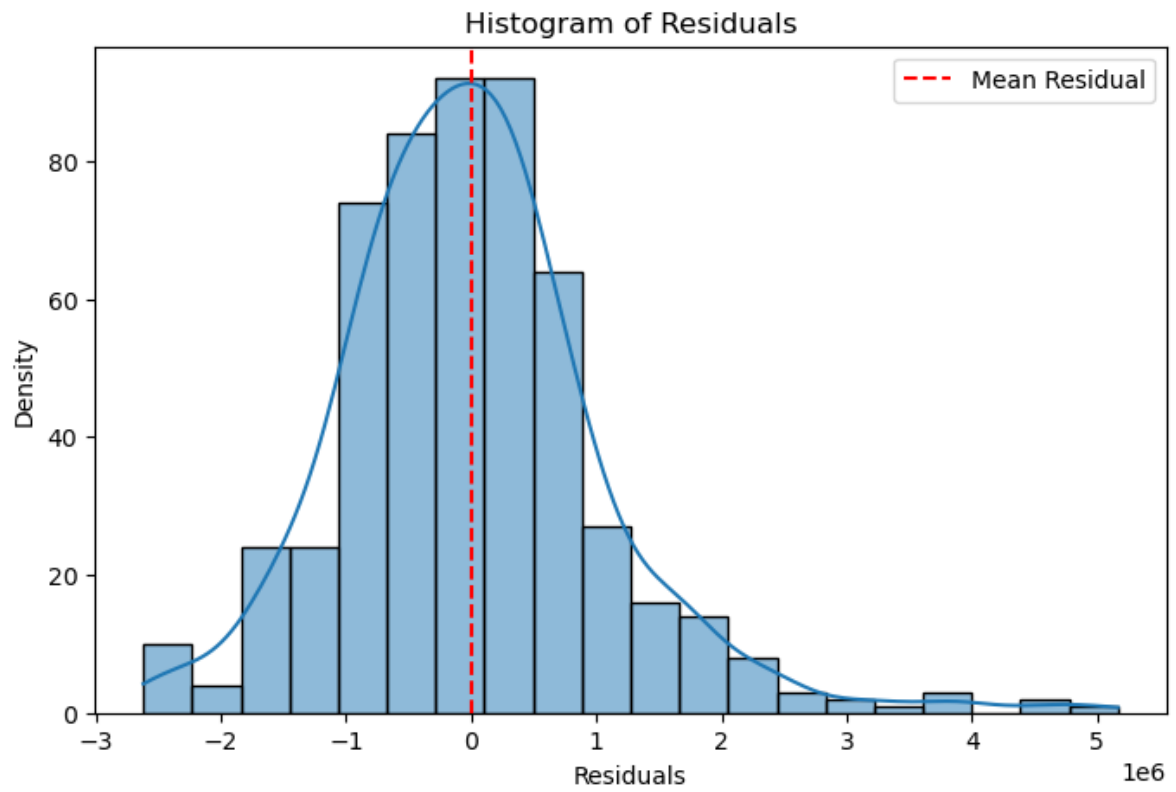
# QQ Plot for Normality of Residuals
plt.figure(figsize=(10, 6))
sm.qqplot(residuals, line='s')
plt.title('QQ Plot of Residuals');
```



<Figure size 1000x600 with 0 Axes>



```
plt.figure(figsize=(8, 5))
sns.histplot(residuals, kde=True, bins=20)
plt.axvline(residuals.mean(), color='red', linestyle='--', label="Mean Residual")
plt.xlabel("Residuals")
plt.ylabel("Density")
plt.title("Histogram of Residuals")
plt.legend()
plt.show()
```



7 Beyond Fit (statistical theory)

Read section 3.3.3 (4, 5, & 6) of the book before using these notes.

Note that in this course, lecture notes are not sufficient, you must read the book for better understanding. Lecture notes are just implementing the concepts of the book on a dataset, but not explaining the concepts elaborately.

Let us continue with the car price prediction example from the previous chapter.

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from scipy import stats
from sklearn.model_selection import cross_val_predict
from patsy import dmatrices
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
trainf = pd.read_csv('./Datasets/Car_features_train.csv')
trainp = pd.read_csv('./Datasets/Car_prices_train.csv')
testf = pd.read_csv('./Datasets/Car_features_test.csv')
testp = pd.read_csv('./Datasets/Car_prices_test.csv')
train = pd.merge(trainf, trainp)
train.head()
```

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price
0	18473	bmw	6 Series	2020	Semi-Auto	11	Diesel	145	53.3282	3.0	37980
1	15064	bmw	6 Series	2019	Semi-Auto	10813	Diesel	145	53.0430	3.0	33980
2	18268	bmw	6 Series	2020	Semi-Auto	6	Diesel	145	53.4379	3.0	36850
3	18480	bmw	6 Series	2017	Semi-Auto	18895	Diesel	145	51.5140	3.0	25998
4	18492	bmw	6 Series	2015	Automatic	62953	Diesel	160	51.4903	3.0	18990

```
# Considering the model developed to address assumptions in the previous chapter
# Model with an interaction term and a variable transformation term
ols_object = smf.ols(formula = 'np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)')
model_log = ols_object.fit()
model_log.summary()
```

Dep. Variable:	np.log(price)	R-squared:	0.803
Model:	OLS	Adj. R-squared:	0.803
Method:	Least Squares	F-statistic:	1834.
Date:	Sun, 10 Mar 2024	Prob (F-statistic):	0.00
Time:	16:51:01	Log-Likelihood:	-1173.8
No. Observations:	4960	AIC:	2372.
Df Residuals:	4948	BIC:	2450.
Df Model:	11		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-238.2125	25.790	-9.237	0.000	-288.773	-187.652
year	0.1227	0.013	9.608	0.000	0.098	0.148
engineSize	13.8349	5.795	2.387	0.017	2.475	25.195
mileage	0.0005	0.000	3.837	0.000	0.000	0.001
mpg	-1.2446	0.345	-3.610	0.000	-1.921	-0.569
year:engineSize	-0.0067	0.003	-2.324	0.020	-0.012	-0.001
year:mileage	-2.67e-07	6.8e-08	-3.923	0.000	-4e-07	-1.34e-07
year:mpg	0.0006	0.000	3.591	0.000	0.000	0.001
engineSize:mileage	-2.668e-07	4.08e-07	-0.654	0.513	-1.07e-06	5.33e-07
engineSize:mpg	0.0028	0.000	6.842	0.000	0.002	0.004
mileage:mpg	7.235e-08	1.79e-08	4.036	0.000	3.72e-08	1.08e-07
I(mileage ** 2)	1.828e-11	5.64e-12	3.240	0.001	7.22e-12	2.93e-11

Omnibus:	711.514	Durbin-Watson:	0.498
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2545.807
Skew:	0.699	Prob(JB):	0.00
Kurtosis:	6.220	Cond. No.	1.73e+13

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 1.73e+13. This might indicate that there are strong multicollinearity or other numerical problems.

```
#Computing RMSE on test data
pred_price_log = model_log.predict(testf)
np.sqrt(((testp.price - np.exp(pred_price_log))**2).mean())
```

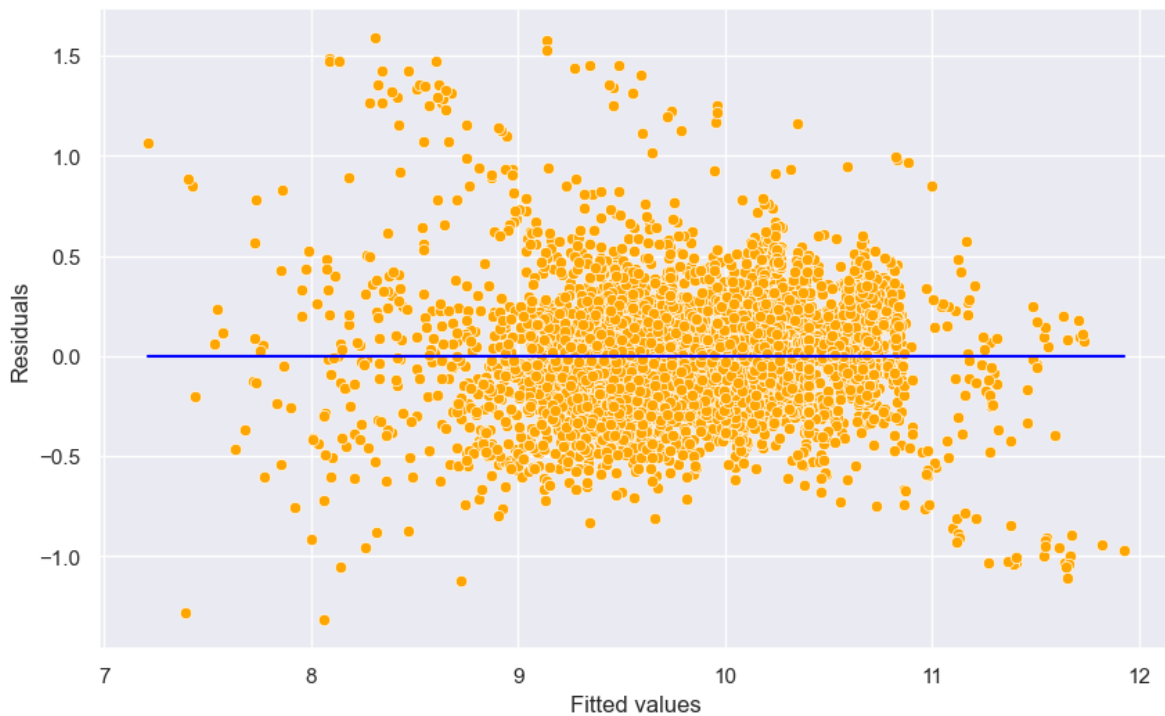
7.1 Outliers

An outlier is a point for which the true response (y_i) is far from the value predicted by the model. Residual plots can be used to identify outliers.

If the the response at the i^{th} observation is y_i , the prediction is $\hat{y}_i$, then the residual e_i is:

$$e_i = y_i - \hat{y}_i$$

```
#Plotting residuals vs fitted values
sns.set(rc={'figure.figsize':(10,6)})
sns.scatterplot(x = (model_log.fittedvalues), y=(model_log.resid),color = 'orange')
sns.lineplot(x = [model_log.fittedvalues.min(),model_log.fittedvalues.max()],y = [0,0],color
plt.xlabel('Fitted values')
plt.ylabel('Residuals');
```



Some of the errors may be high. However, it is difficult to decide how large a residual needs to be before we can consider a point to be an outlier. To address this problem, we have standardized residuals, which are defined as:

$$r_i = \frac{e_i}{RSE(\sqrt{1 - h_{ii}})},$$

where r_i is the standardized residual, RSE is the residual standard error, and h_{ii} is the leverage (introduced in the next section) of the i^{th} observation.

Standardized residuals, allow the residuals to be compared on a *standard scale*.

Issue with standardized residuals: If the observation corresponding to the standardized residual has a high leverage, then it will drag the regression line / plane / hyperplane towards it, thereby influencing the estimate of the residual itself.

Studentized residuals: To address the issue with standardized residuals, studentized residual for the i^{th} observation is computed as the standardized residual, but with the RSE (residual standard error) computed after removing the i^{th} observation from the data. Studentized residual, t_i for the i^{th} observation is given as:

$$t_i = \frac{e_i}{RSE_i(\sqrt{1 - h_{ii}})},$$

where RSE_i is the residual standard error of the model developed on the data without the i^{th} observation.

Distribution of studentized residuals: If the regression model is appropriate such that no case is outlying because of a change in the model, then each studentized residual will follow a t distribution with $(n-p-1)$ degrees of freedom.

As the studentized residuals follow a t distribution, we can conduct a hypothesis test to identify whether an observation is an outlier or not for a given significance level. Note that the test will be two-sided since we are not concerned with the sign of the residuals, but only their absolute values.

In the current example, for a significance level of 5%, the critical t -statistic is $t(1 - \frac{\alpha}{2}, n - p - 1)$, as calculated below.

```
n = train.shape[0]
p = model_log.df_model
alpha = 0.05

# Critical value
stats.t.ppf(1 - alpha/2, n - p - 1)
```

1.9604435402730618

If we were conducting the test for a single observation, we'll compare the studentized residual for that observation with the critical t -statistic, and if the residual is greater than the critical value, we'll consider that observation as an outlier.

However, typically, we'll be interested in conducting this test for all observations, and thus we'll need a more conservative critical value for the same significance level. This critical value is given by the Bonferroni correction as $t(1 - \frac{\alpha}{2n}, n - p - 1)$.

Thus, the minimum value of studentized residual for which the observation will be classified as an outlier is:

```
critical_value = stats.t.ppf(1-alpha/(2*n), n - p - 1)
critical_value
```

4.4200129981725365

The studentized residuals can be obtained using the `outlier_test()` method of the object returned by the `fit()` method of an OLS object. Let us find the studentized residuals in our car price prediction model.

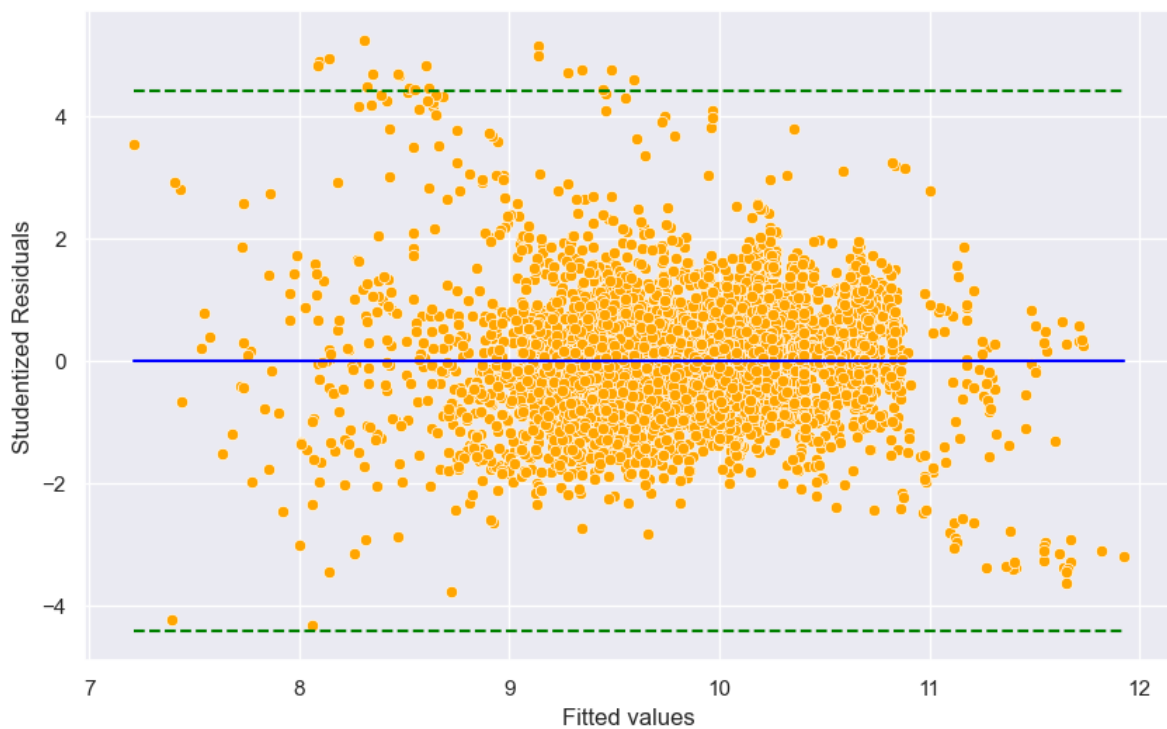
```
#Studentized residuals
out = model_log.outlier_test()
out
```

	student_resid	unadj_p	bonf(p)
0	-1.164204	0.244398	1.0
1	-0.801879	0.422661	1.0
2	-1.263820	0.206354	1.0
3	-0.614171	0.539131	1.0
4	0.027929	0.977720	1.0
...	...	...	...
4955	-0.523361	0.600747	1.0
4956	-0.509538	0.610398	1.0
4957	-1.718808	0.085712	1.0
4958	-0.077594	0.938154	1.0
4959	-0.482388	0.629551	1.0

Studentized residuals are in the first column of the above table. Let us plot the studentized residuals against fitted values. In the figure below, the studentized residuals above the top dotted green line and below the bottom dotted green line are outliers.

```
#Plotting studentized residuals vs fitted values
sns.scatterplot(x = (model_log.fittedvalues), y=(out.student_resid),color = 'orange')
sns.lineplot(x = [model_log.fittedvalues.min(),model_log.fittedvalues.max()],y = [0,0],color
ax = sns.lineplot(x = [model_log.fittedvalues.min(),model_log.fittedvalues.max()],y = [critical_
                color = 'green')
sns.lineplot(x = [model_log.fittedvalues.min(),model_log.fittedvalues.max()],y = [-critical_
                color = 'green')
ax.lines[1].set_linestyle("--")
ax.lines[2].set_linestyle("--")

plt.xlabel('Fitted values')
plt.ylabel('Studentized Residuals');
```



Outliers: Observations whose studentized residuals have a magnitude greater than $t(1 - \frac{\alpha}{2n}, n - p - 1)$.

Impact of outliers: Outliers do not have a large impact on the OLS line / plane / hyperplane as long as they don't have a high leverage (*discussed in the next section*). However, outliers do inflate the residual standard error (RSE). RSE in turn is used to compute the standard errors of regression coefficients. As a result, statistically significant variables may appear to be insignificant, and R^2 may appear to be lower.

Are there outliers in our example?

```
#Number of points with absolute studentized residuals greater than critical_value  
np.sum(np.abs(out.student_resid) > critical_value)
```

19

Let us analyze the outliers.

```
ind = (np.abs(out.student_resid) > critical_value)  
pd.concat([train.loc[ind,:], np.exp(model_log.fittedvalues[ind])], axis = 1)
```

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price
2042	18228	bmw	i3	2017	Automatic	24041	Hybrid	0	78.2726	0.0	2149
2046	17362	bmw	i3	2016	Automatic	68000	Hybrid	0	78.0258	0.0	1599
2050	19224	bmw	i3	2016	Automatic	20013	Hybrid	0	77.9310	0.0	1987
2051	13913	bmw	i3	2014	Automatic	34539	Hybrid	0	78.3838	0.0	1449
2055	16512	bmw	i3	2017	Automatic	28169	Hybrid	0	77.9799	0.0	2375
2059	15844	bmw	i3	2016	Automatic	19995	Hybrid	0	78.2825	0.0	1985
2060	12107	bmw	i3	2016	Automatic	8421	Hybrid	0	77.9125	0.0	1949
2061	18215	bmw	i3	2014	Automatic	37161	Hybrid	0	77.7505	0.0	1418
2063	15617	bmw	i3	2017	Automatic	41949	Hybrid	140	78.1907	0.0	1999
2064	18020	bmw	i3	2015	Automatic	9886	Hybrid	0	78.1810	0.0	1748
2143	12972	bmw	i8	2017	Automatic	9992	Hybrid	135	69.2767	1.5	5995
2144	13826	bmw	i8	2015	Automatic	43323	Hybrid	0	69.2683	1.5	4499
2150	18949	bmw	i8	2015	Automatic	43102	Hybrid	0	69.0922	1.5	4289
2151	18977	bmw	i8	2016	Automatic	10087	Hybrid	0	68.9279	1.5	4899
2744	18866	merc	M Class	2004	Automatic	121000	Diesel	325	29.3713	2.7	1995
3548	13149	audi	S4	2019	Automatic	4900	Diesel	145	40.7030	0.0	4500
4116	16420	audi	SQ5	2020	Automatic	1500	Diesel	145	34.7968	0.0	5645
4117	17611	audi	SQ5	2019	Automatic	1500	Diesel	145	34.5016	0.0	4880
4851	16577	bmw	Z3	2002	Automatic	16500	Petrol	325	29.7614	2.2	1499

Do you notice some unique characteristics of these observations due to which they may be outliers?

What methods you can propose to estimate the price of these outliers more accurately, which will also result in the overall reduction in RMSE?

7.2 High leverage points

High leverage points are those with an unusual value of the predictor(s). They have the potential to have a relatively higher impact on the OLS line / plane / hyperplane, as compared to the outliers.

Leverage statistic (page 99 of the book): In order to quantify an observation's leverage, we compute the leverage statistic. A large value of this statistic indicates an observation with high leverage. For simple linear regression,

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{i'=1}^n (x_{i'} - \bar{x})^2}. \quad (7.1)$$

It is clear from this equation that h_i increases with the distance of x_i from $\bar{x}$. A large value of h_i indicates that the i^{th} observation is distance from the center of all the other observations in terms of predictor values.

The leverage statistic h_i is always between $1/n$ and 1, and the average leverage for all the observations is always equal to $(p + 1)/n$:

$$\bar{h} = \frac{p + 1}{n} \quad (7.2)$$

So if a given observation has a leverage statistic that greatly exceeds $(p + 1)/n$, then we may suspect that the corresponding point has high leverage.

If the i^{th} observation has a large leverage h_i , it may exercise substantial leverage in determining the fitted value $\hat{Y}_i$, because:

- The fitted value $\hat{Y}_i$ is a linear combination of the observed Y values, and h_i is the weight of observation Y_i in determining this fitted value.
- The larger the h_i , the smaller is the variance of the residual e_i , and the closer the fitted value $\hat{Y}_i$ will tend to be the observed value Y_i .

Thumb rules:

- A leverage h_i is usually considered large if it is more than twice as large as the mean value $\bar{h}$.
- Another suggested guideline is that h_i values exceeding 0.5 indicate **very high leverage**, whereas those between 0.2 and 0.5 indicate moderate leverage.

Influential points: Note that if a high leverage point falls in line with the regression line, then it will not affect the regression line. However, it may inflate R -squared and increase the significance of predictors. If a high leverage point falls away from the regression line, then it is also an outlier, and will affect the regression line. The points whose presence significantly affects the regression line are called influential points. A point that is both a high leverage point and an outlier is likely to be an influential point. However, a high leverage point is not necessarily an influential point.

Source for influential points: <https://online.stat.psu.edu/stat501/book/export/html/973>

Let us see if there are any high leverage points in our regression model.

```
#Model with an interaction term and a variable transformation term
ols_object = smf.ols(formula = 'np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)')
model_log = ols_object.fit()
model_log.summary()
```

Dep. Variable:	np.log(price)	R-squared:	0.803
Model:	OLS	Adj. R-squared:	0.803
Method:	Least Squares	F-statistic:	1834.
Date:	Sun, 10 Mar 2024	Prob (F-statistic):	0.00
Time:	16:53:39	Log-Likelihood:	-1173.8
No. Observations:	4960	AIC:	2372.
Df Residuals:	4948	BIC:	2450.
Df Model:	11		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-238.2125	25.790	-9.237	0.000	-288.773	-187.652
year	0.1227	0.013	9.608	0.000	0.098	0.148
engineSize	13.8349	5.795	2.387	0.017	2.475	25.195
mileage	0.0005	0.000	3.837	0.000	0.000	0.001
mpg	-1.2446	0.345	-3.610	0.000	-1.921	-0.569
year:engineSize	-0.0067	0.003	-2.324	0.020	-0.012	-0.001
year:mileage	-2.67e-07	6.8e-08	-3.923	0.000	-4e-07	-1.34e-07
year:mpg	0.0006	0.000	3.591	0.000	0.000	0.001
engineSize:mileage	-2.668e-07	4.08e-07	-0.654	0.513	-1.07e-06	5.33e-07
engineSize:mpg	0.0028	0.000	6.842	0.000	0.002	0.004
mileage:mpg	7.235e-08	1.79e-08	4.036	0.000	3.72e-08	1.08e-07
I(mileage ** 2)	1.828e-11	5.64e-12	3.240	0.001	7.22e-12	2.93e-11


```
out = model_log.outlier_test()
```

```
#Average leverage of points  
average_leverage = (model_log.df_model+1)/model_log.nobs  
average_leverage
```

```
0.0024193548387096775
```

Let us consider points having four times the average leverage as high leverage points.

```
#We will remove all observations that have leverage higher than the threshold value.  
high_leverage_threshold = 3*average_leverage
```

```
#Number of high leverage points in the dataset  
np.sum(leverage>high_leverage_threshold)
```

```
269
```

7.2.1 Identifying extrapolation using leverage

Leverage can be used to check if prediction on a particular point will lead to extrapolation.

Below is the function that can be used to find the leverage at for a particular observation `xnew`. Note that `xnew` has to be a single-dimensional array, and `X` has to be the predictor matrix (also called the design matrix).

```
def leverage_compute(xnew, X):  
    return(xnew.reshape(-1, 1).T.dot(np.linalg.inv(X.T.dot(X))).dot(xnew.reshape(-1, 1)))[0]
```

As expected, the function will return the same leverage as provided by the `hat_matrix_diag` attribute of the object returned by the `get_influence()` method of `model_log` as shown below:

```
leverage[0]
```

```
0.0026426981240353694
```

As the observation for prediction is required we need to create the predictor matrix `X` to create all the observations with the interactions specified in the model.

```
y, X = dmatrices('np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)', data = train)
```

```
leverage_compute(X[0,:], X)
```

```
0.0026426973869101977
```

If the leverage for a new observation is higher than the maximum leverage among all the observations in the training dataset, then prediction at the new observation will be extrapolation.

7.3 Influential points

Observations that are both high leverage points and outliers are influential points that may affect the regression line. Let's remove these influential points from the data and see if it improves the model prediction accuracy on test data.

```
#Dropping influential points from data
train_filtered = train.drop(np.intersect1d(np.where(np.abs(out.student_resid) > critical_value),
                                           (np.where(leverage>high_leverage_threshold)[0])))
```

Note that as the Bonferroni's critical value is very conservative estimate, we have rounded off the critical value to 4, instead of 4.42.

```
train_filtered.shape
```

```
(4948, 11)
```

```
#Number of points removed as they were influential
train.shape[0]-train_filtered.shape[0]
```

```
12
```

We removed 12 influential data points from the training data.

```
#Model after removing the influential observations
ols_object = smf.ols(formula = 'np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)')
model_log = ols_object.fit()
model_log.summary()
```

Dep. Variable:	np.log(price)	R-squared:	0.815
Model:	OLS	Adj. R-squared:	0.814
Method:	Least Squares	F-statistic:	1971.
Date:	Sun, 10 Mar 2024	Prob (F-statistic):	0.00
Time:	16:54:08	Log-Likelihood:	-1027.9
No. Observations:	4948	AIC:	2080.
Df Residuals:	4936	BIC:	2158.
Df Model:	11		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-256.2339	25.421	-10.080	0.000	-306.070	-206.398
year	0.1317	0.013	10.462	0.000	0.107	0.156
engineSize	18.4650	5.663	3.261	0.001	7.364	29.566
mileage	0.0006	0.000	4.288	0.000	0.000	0.001
mpg	-1.1810	0.338	-3.489	0.000	-1.845	-0.517
year:engineSize	-0.0090	0.003	-3.208	0.001	-0.015	-0.004
year:mileage	-2.933e-07	6.7e-08	-4.374	0.000	-4.25e-07	-1.62e-07
year:mpg	0.0006	0.000	3.458	0.001	0.000	0.001
engineSize:mileage	-4.316e-07	4e-07	-1.080	0.280	-1.21e-06	3.52e-07
engineSize:mpg	0.0048	0.000	11.537	0.000	0.004	0.006
mileage:mpg	7.254e-08	1.75e-08	4.140	0.000	3.82e-08	1.07e-07
I(mileage ** 2)	1.668e-11	5.53e-12	3.017	0.003	5.84e-12	2.75e-11
Omnibus:	718.619		Durbin-Watson:	0.521		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	2512.509		
Skew:	0.714		Prob(JB):	0.00		
Kurtosis:	6.185		Cond. No.	1.75e+13		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.75e+13. This might indicate that there are strong multicollinearity or other numerical problems.

Let us compare the square root of 5-fold cross-validated mean squared error of the model with and without the influential points.

```
y, X = dmatrices('np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)', data = train
np.sqrt(mean_squared_error(np.exp(cross_val_predict(LinearRegression(), X, y)), np.exp(y)))
```

9811.74078331643

```
y, X = dmatrices('np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)', data = train)
np.sqrt(mean_squared_error(np.exp(cross_val_predict(LinearRegression(), X, y)), np.exp(y)))
```

9800.202063309154

Why can't we use `cross_val_score()` instead of `cross_val_predict()` here?

There seems to be a slight improvement in prediction error after removing influential points. Note that none of the points had “very high leverage”, and thus the change is not substantial.

Note that we obtain a higher R-squared value of 81.5% as compared to 80% with the complete data. Removing the influential points helped obtain a slightly better model fit. However, that may also happen just by reducing observations.

```
#Computing RMSE on test data
pred_price_log = model_log.predict(testf)
np.sqrt(((testp.price - np.exp(pred_price_log))**2).mean())
```

8922.977452912108

The RMSE on test data has also reduced. This shows that some of the influential points were impacting the regression line. With those points removed, the model better captures the general trend in the data.

7.3.1 Influence on single fitted value (DFFITS)

- A useful measure of the influence that the i^{th} observation has on the fitted value $\hat{Y}_i$ is:

$$(DFFITS)_i = \frac{\hat{Y}_i - \hat{Y}_{i(i)}}{\sqrt{MSE_i h_i}} \quad (7.3)$$

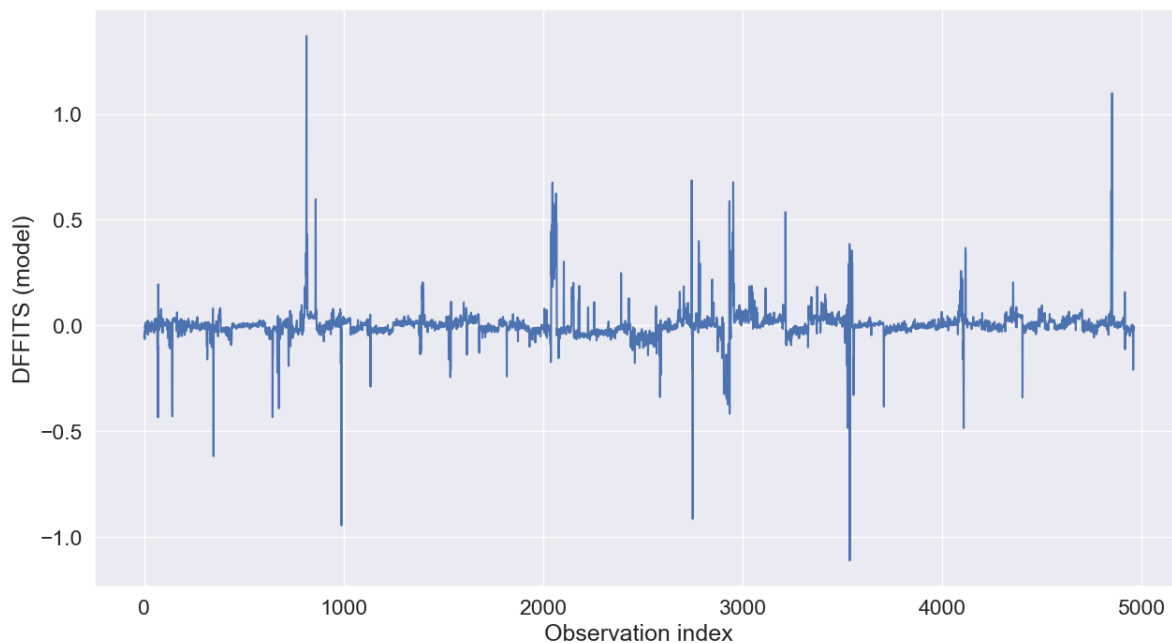
- Note that the denominator in the above fraction is the estimated standard deviation of $\hat{Y}_i$, but uses the error mean square when the i^{th} observation is omitted.
- $DFFITS$ for the i^{th} observation represents the number of estimated standard deviations of $\hat{Y}_i$ that the fitted value $\hat{Y}_i$ increases or decreases with the inclusion of the i^{th} observation in fitting the regression model.
- It can be shown that:

$$(DFFITS)_i = t_i \sqrt{\frac{h_i}{1 - h_i}} \quad (7.4)$$

where t_i is the studentized deleted residual for the i^{th} observation.

- We can see that if an observation has high leverage and is an outlier, it is likely to be influential
- For large datasets, an observation is considered influential if the magnitude of $DFFITS$ for it exceeds $2\sqrt{\frac{p}{n}}$

```
sns.set(font_scale = 1.5)
sns.lineplot(x = range(train.shape[0]), y = influence.dffits[0])
plt.xlabel('Observation index')
plt.ylabel('DFFITS (model)');
```



Let us analyze the point with the highest $DFFITS$.

```
np.where(influence.dffits[0]>1)
```

```
(array([ 813, 4851], dtype=int64),)
```

```
train.loc[813,:]
```

```
carID          12454
brand           vw
model          Caravelle
year           2012
transmission    Semi-Auto
mileage        212000
fuelType        Diesel
tax            325
mpg            34.4424
engineSize      2.0
price          11995
Name: 813, dtype: object
```

```
train.loc[train.model == ' Caravelle','mileage'].describe()
```

```
count      65.000000
mean       25638.692308
std        42954.135726
min         10.000000
25%        3252.000000
50%        6900.000000
75%       30414.000000
max       212000.000000
Name: mileage, dtype: float64
```

```
# Prediction with model developed based on all points
ols_object = smf.ols(formula = 'np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)
                      data = train)
model_log = ols_object.fit();
np.exp(model_log.predict(train.loc[[813],:]))
```

```
813    5502.647323
dtype: float64
```

```
# Prediction with model developed based on all points except the 813th point
ols_object = smf.ols(formula = 'np.log(price)~(year+engineSize+mileage+mpg)**2+I(mileage**2)
                      data = train.drop(index = 813))
model_log = ols_object.fit();
np.exp(model_log.predict(train.loc[[813],:]))
```



```
813      4581.374593
dtype: float64
```

Let us see the leverage and studentized residual for this observation.

```
# Leverage
leverage[813]
```

```
0.19038697461006687
```

```
# Studentized residual
out.student_resid[813]
```

```
2.823478041409651
```

Do you notice what may be contributing to the high influence of this point?

7.3.2 Influence on all fitted values (Cook's distance)

In contrast to *DFFITs*, which considers the influence of the i^{th} observation on the fitted value $\hat{Y}_i$, Cook's distance considers the influence of the i^{th} observation on all n the fitted values:

$$D_i = \frac{\sum_{j=1}^n (\hat{Y}_j - \hat{Y}_{j(i)})^2}{pMSE} \quad (7.5)$$

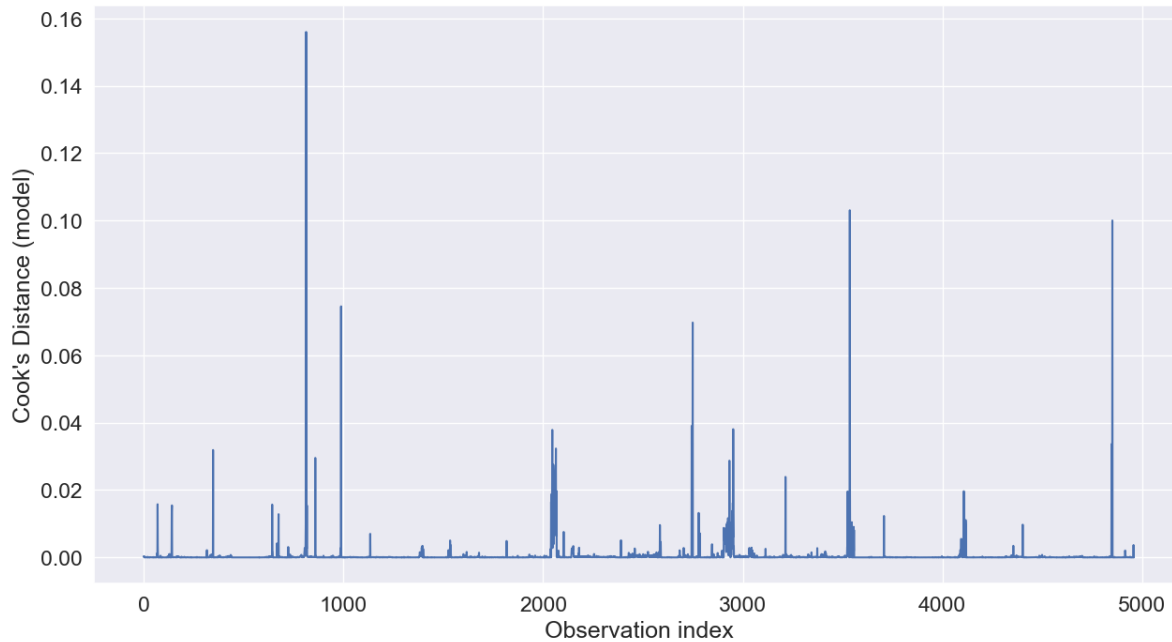
It can be shown that:

$$D_i = \frac{e_i^2}{pMSE} \left[\frac{h_i}{(1 - h_i)^2} \right] \quad (7.6)$$

The larger h_i or e_i , the larger is D_i . D_i can be related to the $F(p, n - p)$ distribution. If the percentile value is 50% or more, the observation is considered as highly influential.

Cook's distance is considered high if it is greater than 0.5 and extreme if it is greater than 1.

```
sns.set(font_scale = 1.5)
sns.lineplot(x = range(train.shape[0]), y = influence.cooks_distance[0])
plt.xlabel('Observation index')
plt.ylabel("Cook's Distance (model)");
```



```
# Point with the highest Cook's distance
np.where(influence.cooks_distance[0]>0.15)
```

```
(array([813], dtype=int64),)
```

The critical Cook's distance value for a point to be highly influential in this dataset is:

```
stats.f.ppf(0.5, 11, 4949)
```

```
0.9402181103263811
```

Thus, we don't have any highly influential points in the dataset.

7.3.3 Influence on regression coefficients (DFBETAS)

- *DFBETAS* measures the influence of the i^{th} observation on the regression coefficient.
- *DFBETAS* of the i^{th} observation on the k^{th} regression coefficient is:

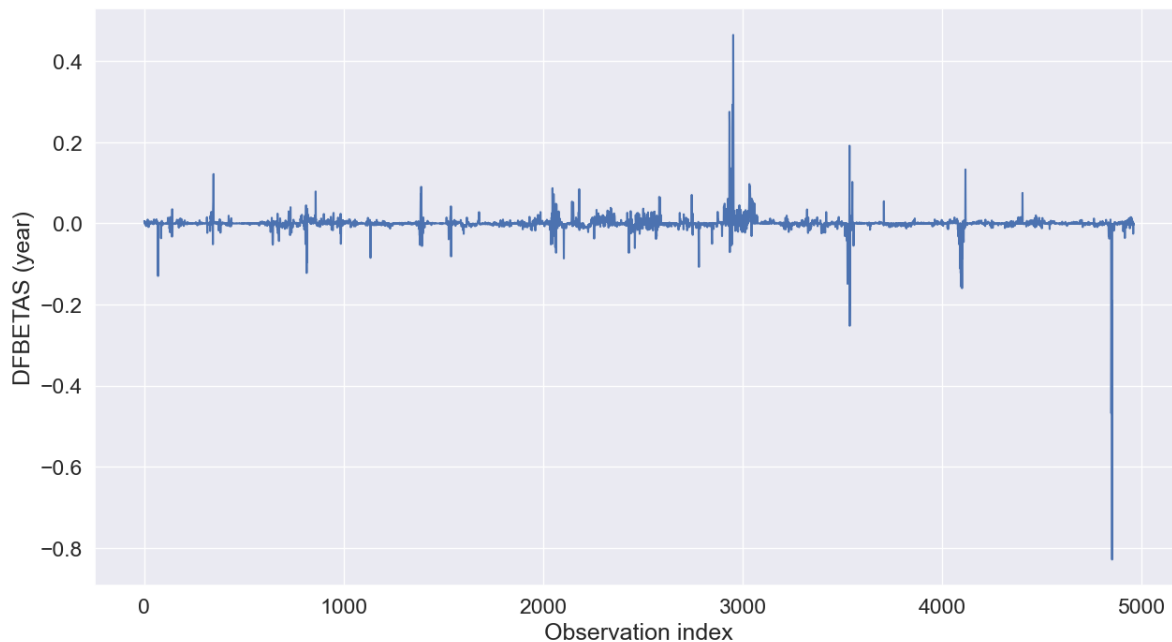
$$(DFBETAS)_{k(i)} = \frac{\hat{\beta}_k - \hat{\beta}_{k(i)}}{\sqrt{MSE_i c_k}} \quad (7.7)$$

where c_k is the k^{th} diagonal element of $(X^T X)^{-1}$.

For large datasets, an observation is considered influential if $DFBETAS$ exceeds $\frac{2}{\sqrt{n}}$.

Below is the plot of $DFBETAS$ for the `year` predictor against the observation index.

```
sns.set(font_scale = 1.5)
sns.lineplot(x = range(train.shape[0]), y = influence.dfbetas[:,1])
plt.xlabel('Observation index')
plt.ylabel("DFBETAS (year)");
```



Let us analyze the point with the highest magnitude of $DFBETAS$.

```
np.where(influence.dfbetas[:,1] < -0.8)
```

```
(array([4851], dtype=int64),)
```

```
train.year.describe()
```

```

count    4960.000000
mean     2016.737903
std       2.884035
min      1997.000000
25%      2016.000000
50%      2017.000000
75%      2019.000000
max      2020.000000
Name: year, dtype: float64

```

```
train.loc[train.year<=2002,:]
```

	carID	brand	model	year	transmission	mileage	fuelType	tax	mpg	engineSize	price
330	13200	audi	A8	1997	Automatic	122000	Petrol	265	19.3511	4.2	46500
732	13988	vw	Beetle	2001	Manual	47729	Petrol	330	32.5910	2.0	24900
3157	18794	ford	Puma	2002	Manual	108000	Petrol	230	38.5757	1.7	21900
3525	19395	merc	S Class	2001	Automatic	108800	Diesel	325	31.5473	3.2	16900
3532	17531	merc	S Class	1999	Automatic	34000	Petrol	145	24.8735	3.2	59900
3533	18761	merc	S Class	2001	Automatic	66000	Petrol	570	24.7744	3.2	44900
3535	18813	merc	S Class	1998	Automatic	43534	Petrol	265	23.2962	6.0	19900
3536	17891	merc	S Class	2002	Automatic	24000	Petrol	570	20.7968	5.0	69900
3707	18746	hyundi	Santa Fe	2002	Manual	94000	Petrol	325	30.2671	2.4	12000
4091	12995	merc	SLK	1998	Automatic	113557	Petrol	265	31.8368	2.3	19900
4094	19585	merc	SLK	2001	Automatic	69234	Petrol	325	30.8839	2.0	39900
4096	14265	merc	SLK	2001	Automatic	48172	Petrol	325	29.7058	2.3	39900
4097	15821	merc	SLK	2002	Automatic	61400	Petrol	325	29.6568	2.3	39900
4098	13021	merc	SLK	2001	Automatic	91000	Petrol	325	30.3248	2.3	39900
4099	12660	merc	SLK	2001	Automatic	42087	Petrol	325	29.9404	2.3	44900
4101	17521	merc	SLK	2002	Automatic	75034	Petrol	325	30.1380	2.3	49900
4107	13977	merc	SLK	2000	Automatic	87000	Petrol	265	27.2998	3.2	14900
4108	18679	merc	SLK	2000	Automatic	113237	Petrol	270	26.8765	3.2	39900
4109	14598	merc	SLK	2001	Automatic	64476	Petrol	325	27.4628	3.2	49900
4847	17268	bmw	Z3	1997	Manual	49000	Petrol	270	34.9548	1.9	39900
4848	12137	bmw	Z3	1999	Manual	58000	Petrol	270	35.3077	1.9	39900
4849	13288	bmw	Z3	1999	Manual	74282	Petrol	245	35.4143	1.9	39900
4850	19172	bmw	Z3	2001	Manual	60000	Petrol	325	30.7305	2.2	59900
4851	16577	bmw	Z3	2002	Automatic	16500	Petrol	325	29.7614	2.2	14900

Let us see the leverage and studentized residual for this observation.

```
# Leverage  
leverage[4851]
```

0.047120455781282225

```
# Studentized residual  
out.student_resid[4851]
```

4.938606329343604

Do you see what makes this point influential?

7.4 Collinearity

Collinearity refers to the situation when two or more predictor variables have a high linear association. Linear association between a pair of variables can be measured by the correlation coefficient. Thus the correlation matrix can indicate some potential collinearity problems.

7.4.1 Why and how is collinearity a problem

(Source: page 100-101 of book)

The presence of collinearity can pose problems in the regression context, since it can be difficult to separate out the individual effects of collinear variables on the response.

Since collinearity reduces the accuracy of the estimates of the regression coefficients, it causes the standard error for $\hat{\beta}_j$ to grow. Recall that the t -statistic for each predictor is calculated by dividing $\hat{\beta}_j$ by its standard error. Consequently, collinearity results in a decline in the t -statistic. As a result, **in the presence of collinearity, we may fail to reject $H_0 : \beta_j = 0$. This means that the power of the hypothesis test—the probability of correctly detecting a non-zero coefficient—is reduced by collinearity.**

7.4.2 How to measure collinearity/multicollinearity

(Source: page 102 of book)

Unfortunately, not all collinearity problems can be detected by inspection of the correlation matrix: it is possible for collinearity to exist between three or more variables even if no pair of variables has a particularly high correlation. We call this situation multicollinearity. Instead of inspecting the correlation matrix, a better way to assess multicollinearity is to compute the variance inflation factor (VIF). The VIF is variance inflation factor the ratio of the variance of $\hat{\beta}_j$ when fitting the full model divided by the variance of $\hat{\beta}_j$ if fit on its own. The smallest possible value for VIF is 1, which indicates the complete absence of collinearity. Typically in practice there is a small amount of collinearity among the predictors. As a rule of thumb, a **VIF value that exceeds 5 or 10 indicates a problematic amount of collinearity.**

The estimated variance of the coefficient β_j , of the j^{th} predictor X_j , can be expressed as:

$$\text{var}(\hat{\beta}_j) = \frac{(\hat{\sigma})^2}{(n-1)\text{var}(X_j)} \cdot \frac{1}{1 - R_{X_j|X_{-j}}^2},$$

where $R_{X_j|X_{-j}}^2$ is the R -squared for the regression of X_j on the other covariates (a regression that does not involve the response variable Y).

In case of simple linear regression, the variance expression in the equation above does not contain the term $\frac{1}{1 - R_{X_j|X_{-j}}^2}$, as there is only one predictor. However, in case of multiple linear regression, the variance of the estimate of the j^{th} coefficient ($\hat{\beta}_j$) gets inflated by a factor of $\frac{1}{1 - R_{X_j|X_{-j}}^2}$ (Note that in the complete absence of collinearity, $R_{X_j|X_{-j}}^2 = 0$, and the value of this factor will be 1).

Thus, the Variance inflation factor, or the VIF for the estimated coefficient of the j^{th} predictor X_j is:

$$VIF(\hat{\beta}_j) = \frac{1}{1 - R_{X_j|X_{-j}}^2} \quad (7.8)$$

```
#Correlation matrix  
train.corr()
```

	carID	year	mileage	tax	mpg	engineSize	price
carID	1.000000	0.006251	-0.001320	0.023806	-0.010774	0.011365	0.012129
year	0.006251	1.000000	-0.768058	-0.205902	-0.057093	0.014623	0.501296
mileage	-0.001320	-0.768058	1.000000	0.133744	0.125376	-0.006459	-0.478705

	carID	year	mileage	tax	mpg	engineSize	price
tax	0.023806	-0.205902	0.133744	1.000000	-0.488002	0.465282	0.144652
mpg	-0.010774	-0.057093	0.125376	-0.488002	1.000000	-0.419417	-0.369919
engineSize	0.011365	0.014623	-0.006459	0.465282	-0.419417	1.000000	0.624899
price	0.012129	0.501296	-0.478705	0.144652	-0.369919	0.624899	1.000000

Let us compute the Variance Inflation Factor (VIF) for the four predictors.

```
X = train[['mpg','year','mileage','engineSize']]
```

```
X.columns[1:]
```

```
Index(['year', 'mileage', 'engineSize'], dtype='object')
```

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant
X = add_constant(X)
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns

for i in range(len(X.columns)):
    vif_data.loc[i, 'VIF'] = variance_inflation_factor(X.values, i)

print(vif_data)
```

	feature	VIF
0	const	1.201579e+06
1	mpg	1.243040e+00
2	year	2.452891e+00
3	mileage	2.490210e+00
4	engineSize	1.219170e+00

As all the values of VIF are close to one, we do not have the problem of multicollinearity in the model. Note that the VIF of **year** and **mileage** is relatively high as they are the most correlated.

Q1: Why is the VIF of the constant so high?

Q2: Why do we need to include the constant while finding the VIF?

7.4.3 Manual computation of VIF

```
#Manually computing the VIF for year
ols_object = smf.ols(formula = 'price~mpg', data = train)
model_log = ols_object.fit()
model_log.summary()
```

Dep. Variable:	price	R-squared:	0.137			
Model:	OLS	Adj. R-squared:	0.137			
Method:	Least Squares	F-statistic:	786.0			
Date:	Wed, 06 Mar 2024	Prob (F-statistic):	1.14e-160			
Time:	17:04:39	Log-Likelihood:	-54812.			
No. Observations:	4960	AIC:	1.096e+05			
Df Residuals:	4958	BIC:	1.096e+05			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	4.144e+04	676.445	61.258	0.000	4.01e+04	4.28e+04
mpg	-374.2975	13.351	-28.036	0.000	-400.471	-348.124
Omnibus:	2132.208	Durbin-Watson:	0.320			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	13751.995			
Skew:	1.942	Prob(JB):	0.00			
Kurtosis:	10.174	Cond. No.	158.			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
(13.351/9.338)**2
```

```
2.044183378279958
```

```
#Manually computing the VIF for year
ols_object = smf.ols(formula = 'price~year+mpg+engineSize+mileage', data = train)
model_log = ols_object.fit()
model_log.summary()
```


Dep. Variable:	price	R-squared:	0.660
Model:	OLS	Adj. R-squared:	0.660
Method:	Least Squares	F-statistic:	2410.
Date:	Wed, 06 Mar 2024	Prob (F-statistic):	0.00
Time:	17:01:18	Log-Likelihood:	-52497.
No. Observations:	4960	AIC:	1.050e+05
Df Residuals:	4955	BIC:	1.050e+05
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-3.661e+06	1.49e+05	-24.593	0.000	-3.95e+06	-3.37e+06
year	1817.7366	73.751	24.647	0.000	1673.151	1962.322
mpg	-79.3126	9.338	-8.493	0.000	-97.620	-61.006
engineSize	1.218e+04	189.969	64.107	0.000	1.18e+04	1.26e+04
mileage	-0.1474	0.009	-16.817	0.000	-0.165	-0.130

Omnibus:	2450.973	Durbin-Watson:	0.541
Prob(Omnibus):	0.000	Jarque-Bera (JB):	31060.548
Skew:	2.045	Prob(JB):	0.00
Kurtosis:	14.557	Cond. No.	3.83e+07

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.83e+07. This might indicate that there are strong multicollinearity or other numerical problems.

```
#Manually computing the VIF for year
ols_object = smf.ols(formula = 'year~mpg+engineSize+mileage', data = train)
model_log = ols_object.fit()
model_log.summary()
```

Dep. Variable:	year	R-squared:	0.592
Model:	OLS	Adj. R-squared:	0.592
Method:	Least Squares	F-statistic:	2400.
Date:	Wed, 06 Mar 2024	Prob (F-statistic):	0.00
Time:	17:00:13	Log-Likelihood:	-10066.
No. Observations:	4960	AIC:	2.014e+04
Df Residuals:	4956	BIC:	2.017e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	2018.3135	0.140	1.44e+04	0.000	2018.039	2018.588
mpg	0.0095	0.002	5.301	0.000	0.006	0.013
engineSize	0.1171	0.037	3.203	0.001	0.045	0.189
mileage	-9.139e-05	1.08e-06	-84.615	0.000	-9.35e-05	-8.93e-05
Omnibus:		2949.664	Durbin-Watson:		1.161	
Prob(Omnibus):		0.000	Jarque-Bera (JB):		63773.271	
Skew:		-2.426	Prob(JB):		0.00	
Kurtosis:		19.883	Cond. No.		1.91e+05	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.91e+05. This might indicate that there are strong multicollinearity or other numerical problems.

```
#VIF for year
```

```
1/(1-0.592)
```

```
2.4509803921568625
```

Note that year and mileage have a high linear correlation. Removing one of them should decrease the standard error of the coefficient of the other, without significantly decrease R-squared.

```
ols_object = smf.ols(formula = 'price~mpg+engineSize+mileage+year', data = train)
model_log = ols_object.fit()
model_log.summary()
```

Table 7.6: OLS Regression Results

Dep. Variable:	price	R-squared:	0.660
Model:	OLS	Adj. R-squared:	0.660
Method:	Least Squares	F-statistic:	2410.
Date:	Tue, 07 Feb 2023	Prob (F-statistic):	0.00
Time:	21:39:45	Log-Likelihood:	-52497.
No. Observations:	4960	AIC:	1.050e+05
Df Residuals:	4955	BIC:	1.050e+05
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-3.661e+06	1.49e+05	-24.593	0.000	-3.95e+06	-3.37e+06
mpg	-79.3126	9.338	-8.493	0.000	-97.620	-61.006
engineSize	1.218e+04	189.969	64.107	0.000	1.18e+04	1.26e+04
mileage	-0.1474	0.009	-16.817	0.000	-0.165	-0.130
year	1817.7366	73.751	24.647	0.000	1673.151	1962.322

Omnibus:	2450.973	Durbin-Watson:	0.541
Prob(Omnibus):	0.000	Jarque-Bera (JB):	31060.548
Skew:	2.045	Prob(JB):	0.00
Kurtosis:	14.557	Cond. No.	3.83e+07

Removing mileage from the above regression.

```
ols_object = smf.ols(formula = 'price~mpg+engineSize+year', data = train)
model_log = ols_object.fit()
model_log.summary()
```

Table 7.9: OLS Regression Results

Dep. Variable:	price	R-squared:	0.641
Model:	OLS	Adj. R-squared:	0.641
Method:	Least Squares	F-statistic:	2951.
Date:	Tue, 07 Feb 2023	Prob (F-statistic):	0.00
Time:	21:40:00	Log-Likelihood:	-52635.
No. Observations:	4960	AIC:	1.053e+05
Df Residuals:	4956	BIC:	1.053e+05
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-5.586e+06	9.78e+04	-57.098	0.000	-5.78e+06	-5.39e+06
mpg	-101.9120	9.500	-10.727	0.000	-120.536	-83.288
engineSize	1.196e+04	194.848	61.392	0.000	1.16e+04	1.23e+04
year	2771.1844	48.492	57.147	0.000	2676.118	2866.251

Omnibus:	2389.075	Durbin-Watson:	0.528
Prob(Omnibus):	0.000	Jarque-Bera (JB):	26920.051
Skew:	2.018	Prob(JB):	0.00
Kurtosis:	13.675	Cond. No.	1.41e+06

Note that the standard error of the coefficient of *year* has reduced from 73 to 48, without any large reduction in R-squared.

7.4.4 When can we overlook multicollinearity?

- The severity of the problems increases with the degree of the multicollinearity. Therefore, if there is only moderate multicollinearity ($5 < VIF < 10$), we may overlook it.
- Multicollinearity affects only the standard errors of the coefficients of collinear predictors. Therefore, if multicollinearity is not present for the predictors that we are particularly interested in, we may not need to resolve it.
- Multicollinearity affects the standard error of the coefficients and thereby their p -values, but in general, it does not influence the prediction accuracy, except in the case that the coefficients are so unstable that the predictions are outside of the domain space of the response. If our sole aim is prediction, and we don't wish to infer the statistical significance of predictors, then we may avoid addressing multicollinearity. *"The fact that some or all predictor variables are correlated among themselves does not, in general, inhibit our ability to obtain a good fit nor does it tend to affect inferences about mean responses or predictions of new observations, provided these inferences are made within the region of observations"* - Neter, John, Michael H. Kutner, Christopher J. Nachtsheim, and William Wasserman. *"Applied linear statistical models."* (1996): 318.

8 Introduction to Logistic Regression

8.1 Learning Objectives

- Understand how to prepare data for logistic regression (handling missing values, stratified splitting)
- Learn to use `statsmodels` with formula notation and `C()` for categorical variables
- Interpret logistic regression coefficients and p-values generated from `statsmodels`
- Master sklearn's implementation with manual dummy variable creation
- Understand the difference between `.predict()` and `.predict_proba()` in sklearn
- Learn about regularization in logistic regression and the `C` parameter
- Compare coefficients with and without regularization
- Compare implementation differences between `statsmodels` and `sklearn`

8.2 Import Libraries and Load Data

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# For statsmodels implementation
import statsmodels.formula.api as smf

# For sklearn implementation
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Set display options
pd.set_option('display.max_columns', None)
plt.style.use('seaborn-v0_8-darkgrid')

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
# Load the Titanic dataset
titanic = sns.load_dataset('titanic')

print(f"Dataset shape: {titanic.shape}")
print(f"\nFirst few rows:")
titanic.head()
```

Dataset shape: (891, 15)

First few rows:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Na
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Na
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Na

8.3 Data Preparation

8.3.1 Imputing missing values

Like linear regression, logistic regression does not tolerate missing values

We'll focus on: - **Target variable:** survived (0 = died, 1 = survived) - **Predictors:** age, fare, sex, pclass

Let's check for missing values in these columns

```
# Check for missing values in our columns of interest
columns_of_interest = ['survived', 'age', 'fare', 'sex', 'pclass']
print("Missing values in columns of interest:")
print(titanic[columns_of_interest].isnull().sum())
```

```
Missing values in columns of interest:
survived      0
age           177
fare          0
sex           0
pclass        0
dtype: int64
```

There are 177 missing values in Age, which is a key predictor. Dropping all rows with missing values would result in a substantial loss of useful information. A simple and reasonable approach is group-wise median imputation: fill in missing ages using the median Age within each combination of **Sex** and **Pclass**.

```
# impute median age for each combination of sex and pclass
titanic['age'] = titanic.groupby(['sex', 'pclass'])['age'].transform(lambda x: x.fillna(x.median()))
```

Let's check again

```
columns_of_interest = ['survived', 'age', 'fare', 'sex', 'pclass']
print("Missing values in columns of interest:")
print(titanic[columns_of_interest].isnull().sum())
```

```
Missing values in columns of interest:
survived      0
age           0
fare          0
sex           0
pclass        0
dtype: int64
```

8.3.2 Check Target Distribution

Let's examine the distribution of the target variable to determine whether the dataset is balanced.

```
# Quick exploratory analysis
print("Target variable distribution:")
print(titanic['survived'].value_counts())
```

Target variable distribution:

survived

0 549

1 342

Name: count, dtype: int64

The dataset shows moderate class imbalance (about 62% non-survived vs 38% survived). While not extreme, this imbalance suggests we should look beyond accuracy and also consider metrics such as precision, recall, and ROC-AUC.

For such dataset, we need to do **stratified splitting** to make sure the proportion of classes is preserved in both train and test sets.

8.3.3 Train-Test Splitting

```
# Split into train and test sets
train_df, test_df = train_test_split(titanic, test_size=0.2, random_state=42, stratify=titanic['survived'])
```

8.4 Implementation 1: Statsmodels with Formula API

When using `statsmodels.formula.api`, we use the **C() notation** to indicate categorical variables:

```
formula = 'survived ~ age + fare + C(sex) + C(pclass)'
```

What does **C()** do?

- Automatically **dummy-codes** categorical variables
- Creates reference categories (first category alphabetically by default)
- For **sex**: female is reference, male gets a coefficient
- For **pclass**: First is reference, Second and Third get coefficients

This is much cleaner than manually creating dummy variables!

```
# Fit logistic regression using statsmodels
formula = 'survived ~ age + fare + C(sex) + pclass'
statsmodels_logit = smf.logit(formula=formula, data=train_df).fit()

# Display the summary
print(statsmodels_logit.summary())
```


Optimization terminated successfully.
 Current function value: 0.442700
 Iterations 6

Logit Regression Results

Dep. Variable:	survived	No. Observations:	712
Model:	Logit	Df Residuals:	707
Method:	MLE	Df Model:	4
Date:	Sat, 14 Feb 2026	Pseudo R-squ.:	0.3350
Time:	06:34:26	Log-Likelihood:	-315.20
converged:	True	LL-Null:	-473.99
Covariance Type:	nonrobust	LLR p-value:	1.749e-67

	coef	std err	z	P> z	[0.025	0.975]
Intercept	5.1033	0.612	8.343	0.000	3.904	6.302
C(sex)[T.male]	-2.6032	0.212	-12.280	0.000	-3.019	-2.188
age	-0.0400	0.009	-4.630	0.000	-0.057	-0.023
fare	0.0007	0.002	0.265	0.791	-0.004	0.006
pclass	-1.2734	0.163	-7.804	0.000	-1.593	-0.954

8.4.1 Interpreting the Coefficients

Key points to discuss (from the printed output):

1. **Age coefficient (negative, p=0.009):** Older passengers had lower survival odds.
2. **Fare coefficient (near 0, p=0.648):** Fare is not a statistically significant predictor here.
3. **C(sex)[T.male] (negative, p<0.001):** Being male significantly decreased survival odds.
4. **pclass (negative, p=0.077):** Lower passenger class is associated with lower survival odds, but it is not significant at the 0.05 level.

P-values ($P>|z|$): - Values < 0.05 indicate statistical significance at the 5% level. - Here, **sex** and **age** are significant; **fare** and **pclass** are not (at 0.05).

8.4.2 Prediction from Statsmodels

Now that we've fitted our logistic regression model, let's generate predictions.

In `statsmodels`, the `.predict()` method returns **predicted probabilities** for the positive class (survived = 1). These are continuous values between 0 and 1, representing the model's estimated probability that each observation belongs to the positive class.

Key points:

- Output values range from 0 to 1
- Values closer to 1 indicate higher probability of survival
- Values closer to 0 indicate higher probability of death
- Unlike `sklearn`, `statsmodels` does not have a built-in method to directly return class labels (0/1)—we'll need to apply a threshold manually if we want binary predictions

```
# Get predictions from statsmodels
train_probs_sm = statsmodels_logit.predict(train_df)
test_probs_sm = statsmodels_logit.predict(test_df)

print("Sample of predicted probabilities (statsmodels):")
test_probs_sm.head(10)
```

Sample of predicted probabilities (statsmodels):

```
565    0.094223
160    0.044466
553    0.100257
860    0.049784
241    0.606897
559    0.464106
387    0.755221
536    0.364969
698    0.341170
99     0.199713
dtype: float64
```

Why this design?

Statsmodels is more focused on:

- Statistical inference
- Coefficient interpretation
- Standard errors / p-values

It leaves classification decisions (thresholding, ROC analysis, etc.) to the user.

```

# Apply threshold to convert probabilities to class predictions
threshold = 0.5
train_pred_sm = (train_probs_sm >= threshold).astype(int)
test_pred_sm = (test_probs_sm >= threshold).astype(int)

# Calculate accuracy
train_accuracy_sm = accuracy_score(train_df['survived'], train_pred_sm)
test_accuracy_sm = accuracy_score(test_df['survived'], test_pred_sm)

print(f"Statsmodels Logistic Regression (threshold = {threshold}):")
print(f"  Training Accuracy:   {train_accuracy_sm:.4f} ({train_accuracy_sm:.2%})")
print(f"  Test Accuracy:        {test_accuracy_sm:.4f} ({test_accuracy_sm:.2%})")

# Show some example predictions
print(f"\nSample predictions (first 10 test observations):")
print(f"{'Index':<8} {'Probability':<12} {'Prediction':<12} {'Actual':<8}")
print("-" * 45)
for i in range(10):
    print(f"{i:<8} {test_probs_sm.iloc[i]:<12.4f} {test_pred_sm.iloc[i]:<12} {test_df['survived'].iloc[i]:<8}")

```

```

Statsmodels Logistic Regression (threshold = 0.5):
  Training Accuracy:   0.7992 (79.92%)
  Test Accuracy:      0.7821 (78.21%)

```

Sample predictions (first 10 test observations):

Index	Probability	Prediction	Actual
0	0.0942	0	0
1	0.0445	0	0
2	0.1003	0	1
3	0.0498	0	0
4	0.6069	1	1
5	0.4641	0	1
6	0.7552	1	1
7	0.3650	0	0
8	0.3412	0	0
9	0.1997	0	0

8.5 Implementation 2: Scikit-learn Integration

While `statsmodels` is excellent for **statistical inference** (p-values, confidence intervals), `sklearn` is optimized for **prediction and machine learning workflows**.

To use `sklearn.linear_model.LogisticRegression`, we have two approaches:

Approach 1: Manual Dummy Variable Creation 1. Manually create dummy variables using `pd.get_dummies()` (`sklearn` doesn't have formula notation) 2. Prepare feature matrix `X` and target vector `y`

Approach 2: Pipeline with ColumnTransformer - Use `sklearn`'s preprocessing pipeline to handle categorical encoding automatically - More scalable for production workflows

We'll start with the manual approach first, then demonstrate the pipeline method at the end.

```
# Prepare data for sklearn using the same predictors as statsmodels
predictors = ['age', 'fare', 'sex', 'pclass']

X_train = pd.get_dummies(train_df[predictors], columns=['sex'], drop_first=True)
X_test = pd.get_dummies(test_df[predictors], columns=['sex'], drop_first=True)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

y_train = train_df['survived']
y_test = test_df['survived']

print("Encoded training data:")
X_train.head()

print(f"\nColumn names: {list(X_train.columns)}")
```

Encoded training data:

Column names: ['age', 'fare', 'pclass', 'sex_male']

```
# Feature/target shapes
print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"\nFeature names: {list(X_train.columns)}")
```

X_train shape: (712, 4)

y_train shape: (712,)

Feature names: ['age', 'fare', 'pclass', 'sex_male']

```
# Fit logistic regression using sklearn
sklearn_logit = LogisticRegression(random_state=42)
sklearn_logit.fit(X_train, y_train)

print("Sklearn Logistic Regression model fitted successfully!")
print(f"\nCoefficients:")
for feature, coef in zip(X_train.columns, sklearn_logit.coef_[0]):
    print(f"    {feature:20s}: {coef:8.4f}")
print(f"\nIntercept: {sklearn_logit.intercept_[0]:.4f}")
```

Sklearn Logistic Regression model fitted successfully!

Coefficients:

age	:	-0.0389
fare	:	0.0010
pclass	:	-1.2189
sex_male	:	-2.4830

Intercept: 4.8699

8.5.1 Understanding `.predict()` vs `.predict_proba()`

Scikit-learn's logistic regression offers two prediction methods:

1. `.predict_proba()`: Returns probability estimates
2.
 - Output: Array with shape (n_samples, 2)
 - Column 0: Probability of class 0 (died)
 - Column 1: Probability of class 1 (survived)
 - Values are continuous between 0 and 1
3. `.predict()`: Returns class labels
4.
 - Output: Array with shape (n_samples,)
 - Values: 0 or 1 (binary classification)
 - Uses default threshold of 0.5 (if prob > 0.5, predict 1)

Key difference: `.predict_proba()` gives you the raw probabilities, while `.predict()` gives you the final classification decision.

```
# Get probability predictions
test_probs_sklearn = sklearn_logit.predict_proba(X_test)

print("Shape of predict_proba output:", test_probs_sklearn.shape)
```

```

print("\nFirst 10 probability predictions:")
print("Index | P(Died) | P(Survived)")
print("-" * 35)
for i in range(10):
    print(f"{i:5d} | {test_probs_sklearn[i, 0]:.4f} | {test_probs_sklearn[i, 1]:.4f}")

# Extract probabilities of survival (class 1)
test_probs_survival = test_probs_sklearn[:, 1]
print(f"\nProbabilities of survival (class 1): {test_probs_survival[:5]}")

```

Shape of predict_proba output: (179, 2)

First 10 probability predictions:

Index	P(Died)	P(Survived)
0	0.8983	0.1017
1	0.9510	0.0490
2	0.8927	0.1073
3	0.9454	0.0546
4	0.4031	0.5969
5	0.5424	0.4576
6	0.2603	0.7397
7	0.6355	0.3645
8	0.6512	0.3488
9	0.7937	0.2063

Probabilities of survival (class 1): [0.10166063 0.0490081 0.10730699 0.0546384 0.59687177]

```

# Get class predictions (using default 0.5 threshold)
test_pred_default = sklearn_logit.predict(X_test)

print("Shape of predict output:", test_pred_default.shape)
print("\nFirst 10 class predictions:")
print(test_pred_default[:10])

# Compare with probabilities
print("\nComparison of probabilities and predictions:")
print("Index | Probability | Prediction | Interpretation")
print("-" * 60)
for i in range(10):
    prob = test_probs_survival[i]

```

```

pred = test_pred_default[i]
interpretation = "Survived" if pred == 1 else "Died"
print(f"{i:5d} | {prob:11.4f} | {pred:10d} | {interpretation}")

```

Shape of predict output: (179,)

First 10 class predictions:

```
[0 0 0 0 1 0 1 0 0 0]
```

Comparison of probabilities and predictions:

Index	Probability	Prediction	Interpretation
-------	-------------	------------	----------------

0	0.1017		0 Died
1	0.0490		0 Died
2	0.1073		0 Died
3	0.0546		0 Died
4	0.5969		1 Survived
5	0.4576		0 Died
6	0.7397		1 Survived
7	0.3645		0 Died
8	0.3488		0 Died
9	0.2063		0 Died

Observation: Notice how `.predict()` converts probabilities to 0/1 using the 0.5 threshold: - Probability $\geq 0.5 \rightarrow$ Prediction = 1 (Survived) - Probability $< 0.5 \rightarrow$ Prediction = 0 (Died)

8.5.2 Turning off the default regularization

Important difference between statsmodels and sklearn:

By default, sklearn's `LogisticRegression` applies **L2 regularization** (ridge penalty) with `C=1.0`. This means: - The model penalizes large coefficients to prevent overfitting - Coefficients will be different from statsmodels (which uses no regularization by default)

Understanding the C parameter: - C is the **inverse of regularization strength** (higher C = less regularization) - Default: `C=1.0` applies moderate regularization - To turn off regularization: Set `C=np.inf` (infinite C = no penalty)

- Note: We'll cover regularization and tuning C in detail in a later lecture. Let's refit the model without regularization and compare the coefficients:

To match statsmodels' behavior and get coefficients that are directly comparable, we need to:-
Set `C=np.inf` to turn off regularization completely

```
# Fit logistic regression WITHOUT regularization (C=np.inf)
sklearn_logit_no_reg = LogisticRegression(C=np.inf, random_state=42)
sklearn_logit_no_reg.fit(X_train, y_train)

print("Sklearn Logistic Regression (NO regularization, C=np.inf):")
print(f"\nCoefficients:")
for feature, coef in zip(X_train.columns, sklearn_logit_no_reg.coef_[0]):
    print(f"    {feature:20s}: {coef:8.4f}")
print(f"\nIntercept: {sklearn_logit_no_reg.intercept_[0]:.4f}")
```

Sklearn Logistic Regression (NO regularization, C=np.inf):

Coefficients:

age	:	-0.0400
fare	:	0.0007
pclass	:	-1.2733
sex_male	:	-2.6032

Intercept: 5.1031

```
print("\n" + "="*60)
print("COMPARISON: With vs Without Regularization")
print("="*60)
print(f"{'Feature':<20} {'With Reg (C=1.0)':<18} {'No Reg (C=np.inf)':<18}")
print("-"*60)
for i, feature in enumerate(X_train.columns):
    coef_with = sklearn_logit.coef_[0][i]
    coef_without = sklearn_logit_no_reg.coef_[0][i]
    print(f"{feature:<20} {coef_with:17.4f} {coef_without:17.4f}")

print(f"\n{'Intercept':<20} {sklearn_logit.intercept_[0]:17.4f} {sklearn_logit_no_reg.intercept_[0]:17.4f}")
```

=====

COMPARISON: With vs Without Regularization

=====

Feature	With Reg (C=1.0)	No Reg (C=np.inf)
---------	------------------	-------------------

age	-0.0389	-0.0400
fare	0.0010	0.0007
pclass	-1.2189	-1.2733
sex_male	-2.4830	-2.6032
Intercept	4.8699	5.1031

Key Observations:

1. **With regularization (default $C=1.0$):** Coefficients are slightly shrunk toward zero
2. **Without regularization ($C=np.inf$):** Coefficients match statsmodels more closely

When to use which approach:

- **Use regularization (smaller C values):**
 - When you have many features or potential overfitting
 - For production models where generalization is key
 - Default sklearn behavior ($C=1.0$) is good for most machine learning workflows
- **Turn off regularization ($C=np.inf$):**
 - When you want to match statistical software like statsmodels or R
 - For small datasets with few predictors
 - When interpretability and hypothesis testing are priorities
 - When comparing results across different tools

Note: We'll explore how to choose optimal C values using cross-validation in the regularization lecture.

```
# the training and test accuracy for the model with and without regularization
train_pred_no_reg = sklearn_logit_no_reg.predict(X_train)
test_pred_no_reg = sklearn_logit_no_reg.predict(X_test)
train_accuracy_no_reg = accuracy_score(y_train, train_pred_no_reg)
test_accuracy_no_reg = accuracy_score(y_test, test_pred_no_reg)

train_pred_with_reg = sklearn_logit.predict(X_train)
test_pred_with_reg = sklearn_logit.predict(X_test)
train_accuracy_with_reg = accuracy_score(y_train, train_pred_with_reg)
test_accuracy_with_reg = accuracy_score(y_test, test_pred_with_reg)

print(f"\nSklearn Logistic Regression WITHOUT regularization (C=np.inf):")
print(f"  Training Accuracy:   {train_accuracy_no_reg:.4f} ({train_accuracy_no_reg:.2%})")
print(f"  Test Accuracy:         {test_accuracy_no_reg:.4f} ({test_accuracy_no_reg:.2%})")
```

```
print(f"\nSklern Logistic Regression WITH regularization:")
print(f"  Training Accuracy:   {train_accuracy_with_reg:.4f} ({train_accuracy_with_reg:.2%})")
print(f"  Test Accuracy:       {test_accuracy_with_reg:.4f} ({test_accuracy_with_reg:.2%})")
```

Sklern Logistic Regression WITHOUT regularization (C=np.inf):

```
Training Accuracy:   0.7992 (79.92%)
Test Accuracy:      0.7821 (78.21%)
```

Sklern Logistic Regression WITH regularization:

```
Training Accuracy:   0.8006 (80.06%)
Test Accuracy:      0.7821 (78.21%)
```

8.5.3 Pipeline Implementation

Why use Pipelines?

So far, we've manually created dummy variables using `pd.get_dummies()`. While this works, sklearn's **Pipeline** approach offers several advantages:

1. **Automated preprocessing:** Handles categorical encoding automatically
2. **Prevents data leakage:** Ensures transformations are fit only on training data
3. **Cleaner code:** Combines preprocessing and modeling in one object
4. **Production-ready:** Easier to deploy and maintain
5. **Reproducibility:** All steps are encapsulated in one pipeline

Key components: - **ColumnTransformer:** Applies different transformations to different columns - **OneHotEncoder:** Converts categorical variables to dummy variables - **Pipeline:** Chains preprocessing and model fitting together

Let's implement the same logistic regression model using a pipeline (without regularization to match statsmodels):

```
# Import required components for pipeline
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Define which columns are categorical and which are numeric
categorical_features = ['sex']
numeric_features = ['age', 'fare', 'pclass']
```

```

# Create the column transformer
# OneHotEncoder with drop='first' mimics the drop_first=True in pd.get_dummies
preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numeric_features), # Keep numeric features as-is
        ('cat', OneHotEncoder(drop='first'), categorical_features) # Encode categorical
    ])

# Create the pipeline: preprocessing + logistic regression
pipeline_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(C=np.inf, random_state=42))
])

# Fit the pipeline on training data
# Note: We pass the original dataframe with categorical variables
X_train_raw = train_df[['age', 'fare', 'sex', 'pclass']]
X_test_raw = test_df[['age', 'fare', 'sex', 'pclass']]

pipeline_model.fit(X_train_raw, y_train)

print("Pipeline model fitted successfully!")
print("\nPipeline structure:")
print(pipeline_model)

```

Pipeline model fitted successfully!

Pipeline structure:

```

Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('num', 'passthrough',
                                                  ['age', 'fare', 'pclass']),
                                                  ('cat',
                                                   OneHotEncoder(drop='first'),
                                                   ['sex'])])),
                 ('classifier', LogisticRegression(C=inf, random_state=42))])

```

```

# Extract the trained logistic regression model from the pipeline
pipeline_logit = pipeline_model.named_steps['classifier']

```

```

# Get feature names after transformation

```

```

feature_names = numeric_features + ['sex_male'] # OneHotEncoder creates sex_male (drops sex)

```

```

print("Pipeline Logistic Regression Coefficients (C=np.inf):")
print(f"\nCoefficients:")
for feature, coef in zip(feature_names, pipeline_logit.coef_[0]):
    print(f"    {feature:20s}: {coef:8.4f}")
print(f"\nIntercept: {pipeline_logit.intercept_[0]:.4f}")

# Make predictions
pipeline_train_pred = pipeline_model.predict(X_train_raw)
pipeline_test_pred = pipeline_model.predict(X_test_raw)

# Calculate accuracy
pipeline_train_accuracy = accuracy_score(y_train, pipeline_train_pred)
pipeline_test_accuracy = accuracy_score(y_test, pipeline_test_pred)

print(f"\nPipeline Model Performance:")
print(f"    Training Accuracy:    {pipeline_train_accuracy:.4f} ({pipeline_train_accuracy:.2%})")
print(f"    Test Accuracy:        {pipeline_test_accuracy:.4f} ({pipeline_test_accuracy:.2%})")

```

Pipeline Logistic Regression Coefficients (C=np.inf):

Coefficients:

age	:	-0.0400
fare	:	0.0007
pclass	:	-1.2733
sex_male	:	-2.6032

Intercept: 5.1031

Pipeline Model Performance:

Training Accuracy:	0.7992 (79.92%)
Test Accuracy:	0.7821 (78.21%)

```

# Verify that pipeline and manual approach give identical results
print("="*70)
print("COMPARISON: Pipeline vs Manual Approach (both with C=np.inf)")
print("="*70)
print(f"\n{'Feature':<20} {'Manual Approach':<18} {'Pipeline Approach':<18} {'Match?':<8}")
print("-"*70)

# Compare intercept
manual_intercept = sklearn_logit_no_reg.intercept_[0]

```

```

pipeline_intercept = pipeline_logit.intercept_[0]
match = " " if np.isclose(manual_intercept, pipeline_intercept) else " "
print(f"{'Intercept':<20} {manual_intercept:17.6f} {pipeline_intercept:17.6f} {match:<8}")

# Compare coefficients
for i, feature in enumerate(feature_names):
    manual_coef = sklearn_logit_no_reg.coef_[0][i]
    pipeline_coef = pipeline_logit.coef_[0][i]
    match = " " if np.isclose(manual_coef, pipeline_coef) else " "
    print(f"{feature:<20} {manual_coef:17.6f} {pipeline_coef:17.6f} {match:<8}")

print("\n" + "="*70)
print("RESULT: Both approaches produce IDENTICAL coefficients!")
print("="*70)

```

```

=====
COMPARISON: Pipeline vs Manual Approach (both with C=np.inf)
=====

```

Feature	Manual Approach	Pipeline Approach	Match?
Intercept	5.103138	5.103138	
age	-0.039957	-0.039957	
fare	0.000660	0.000660	
pclass	-1.273318	-1.273318	
sex_male	-2.603232	-2.603232	

```

=====
RESULT: Both approaches produce IDENTICAL coefficients!
=====

```

Key Advantages of Pipeline Approach:

1. **Identical Results:** Pipeline produces the exact same coefficients and predictions as the manual approach
2. **Cleaner Code:** All preprocessing and modeling in one object
3. **No Data Leakage:** The pipeline ensures that:
 - OneHotEncoder is fit only on training data
 - Categories seen only in test data are handled correctly
4. **Production Ready:** Can be easily saved and loaded for deployment

- 5. **Works with Cross-Validation:** Seamlessly integrates with GridSearchCV and other sklearn tools

When to use each approach:

- **Manual `pd.get_dummies()`:**
 - Quick exploratory analysis
 - When you need fine control over encoding
 - Simple, one-off models
- **Pipeline with `ColumnTransformer`:**
 - Production environments
 - When using cross-validation or hyperparameter tuning
 - Multiple preprocessing steps
 - When you need to ensure reproducibility and prevent data leakage

In the next 2 weeks, you will learn

- Use pipelines with cross-validation to tune hyperparameters (like `C`)
- Extend pipelines with additional preprocessing (scaling, imputation, feature engineering)
- Save trained pipelines for deployment using `joblib` or `pickle`

8.6 Statsmodels vs Sklearn Result Comparison

Why might coefficients differ?

Unlike linear regression which has a closed-form solution (matrix operations), logistic regression requires **iterative optimization algorithms** to find the best coefficients. Both statsmodels and sklearn use iterative methods, but:

- **Statsmodels** uses: Newton-Raphson/IRLS (Iteratively Reweighted Least Squares)
- **Sklearn** uses: LBFGS, liblinear, SAG, or SAGA (depending on settings)

Even with `C=np.inf` (no regularization), we might see **slight differences** due to: 1. Different optimization algorithms 2. Different convergence criteria 3. Different tolerance thresholds 4. Numerical precision differences

However, when both converge properly, the coefficients should be **very close** (differences typically in the 3rd-4th decimal place or smaller).

Let's compare the coefficients from statsmodels and sklearn (without regularization):

```

# Extract coefficients from statsmodels
# Note: statsmodels coefficients are in a different order due to formula notation
statsmodels_coefs = statsmodels_logit.params

print("="*70)
print("COEFFICIENT COMPARISON: Statsmodels vs Sklearn (C=np.inf)")
print("="*70)
print(f"\n{'Variable':<20} {'Statsmodels':<18} {'Sklearn (C=np.inf)':<18} {'Difference':<12}")
print("-"*70)

# Compare intercept
sklearn_intercept = sklearn_logit_no_reg.intercept_[0]
statsmodels_intercept = statsmodels_coefs['Intercept']
diff_intercept = abs(sklearn_intercept - statsmodels_intercept)
print(f"{'Intercept':<20} {statsmodels_intercept:17.6f} {sklearn_intercept:17.6f} {diff_intercept:11.6f}")

# Compare age coefficient
sklearn_age = sklearn_logit_no_reg.coef_[0][list(X_train.columns).index('age')]
statsmodels_age = statsmodels_coefs['age']
diff_age = abs(sklearn_age - statsmodels_age)
print(f"{'age':<20} {statsmodels_age:17.6f} {sklearn_age:17.6f} {diff_age:11.6f}")

# Compare fare coefficient
sklearn_fare = sklearn_logit_no_reg.coef_[0][list(X_train.columns).index('fare')]
statsmodels_fare = statsmodels_coefs['fare']
diff_fare = abs(sklearn_fare - statsmodels_fare)
print(f"{'fare':<20} {statsmodels_fare:17.6f} {sklearn_fare:17.6f} {diff_fare:11.6f}")

# Compare sex coefficient (male)
sklearn_sex_male = sklearn_logit_no_reg.coef_[0][list(X_train.columns).index('sex_male')]
statsmodels_sex_male = statsmodels_coefs['C(sex)[T.male]']
diff_sex = abs(sklearn_sex_male - statsmodels_sex_male)
print(f"{'sex_male':<20} {statsmodels_sex_male:17.6f} {sklearn_sex_male:17.6f} {diff_sex:11.6f}")

# Compare pclass coefficient
sklearn_pclass = sklearn_logit_no_reg.coef_[0][list(X_train.columns).index('pclass')]
statsmodels_pclass = statsmodels_coefs['pclass']
diff_pclass = abs(sklearn_pclass - statsmodels_pclass)
print(f"{'pclass':<20} {statsmodels_pclass:17.6f} {sklearn_pclass:17.6f} {diff_pclass:11.6f}")

print("\n" + "="*70)
print("INTERPRETATION:")

```

```

print("="*70)
print(" Coefficients are very close (differences < 0.001 typically)")
print(" Both models have converged to essentially the same solution")
print(" Minor differences are due to numerical optimization details")
print(" For practical purposes, both implementations give equivalent results")
print("="*70)

```

```

=====
COEFFICIENT COMPARISON: Statsmodels vs Sklearn (C=np.inf)
=====

```

Variable	Statsmodels	Sklearn (C=np.inf)	Difference
Intercept	5.103261	5.103138	0.000123
age	-0.039956	-0.039957	0.000001
fare	0.000659	0.000660	0.000001
sex_male	-2.603225	-2.603232	0.000007
pclass	-1.273379	-1.273318	0.000062

```

=====
INTERPRETATION:
=====
Coefficients are very close (differences < 0.001 typically)
Both models have converged to essentially the same solution
Minor differences are due to numerical optimization details
For practical purposes, both implementations give equivalent results
=====

```

When do these differences matter?

In most practical applications, these tiny differences are **negligible**: - Predictions will be virtually identical - Model performance metrics will be the same - Interpretation of coefficients remains consistent

Key takeaway: - When comparing results across tools, use `C=np.inf` in sklearn to match statsmodels - Both implementations are correct and reliable - Choose the tool based on your workflow needs (statsmodels for inference, sklearn for ML pipelines)

9 Logistic Regression Evaluation

9.1 Learning Objectives

- Understand the importance of stratified splitting for imbalanced datasets
- Build production-ready models using sklearn pipelines with `ColumnTransformer`
- Interpret confusion matrices and identify different types of prediction errors
- Calculate and understand precision, recall, and F1 score
- Analyze the precision-recall tradeoff when changing classification thresholds
- Evaluate models using ROC curves and AUC scores
- Choose appropriate evaluation metrics based on the problem context and costs of different error types

9.2 Import Libraries and Load Data

We will use the same titanic dataset and model preparation as the previous chapter for this chapter

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# For statsmodels implementation
import statsmodels.formula.api as smf

# For sklearn implementation
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Set display options
pd.set_option('display.max_columns', None)
plt.style.use('seaborn-v0_8-darkgrid')
```

```
print("Libraries imported successfully!")
```

Libraries imported successfully!

```
# Load the Titanic dataset
titanic = sns.load_dataset('titanic')

print(f"Dataset shape: {titanic.shape}")
print(f"\nFirst few rows:")
titanic.head()
```

Dataset shape: (891, 15)

First few rows:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Na
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Na
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Na

9.3 Data Preparation

9.3.1 Imputing missing values

Like linear regression, logistic regression does not tolerate missing values

We'll focus on: - **Target variable:** survived (0 = died, 1 = survived) - **Predictors:** age, fare, sex, pclass

Let's check for missing values in these columns

```
# Check for missing values in our columns of interest
columns_of_interest = ['survived', 'age', 'fare', 'sex', 'pclass']
print("Missing values in columns of interest:")
print(titanic[columns_of_interest].isnull().sum())
```

```
Missing values in columns of interest:
survived      0
age           177
fare          0
sex           0
pclass        0
dtype: int64
```

There are 177 missing values in Age, which is a key predictor. Dropping all rows with missing values would result in a substantial loss of useful information. A simple and reasonable approach is group-wise median imputation: fill in missing ages using the median Age within each combination of **Sex** and **Pclass**.

```
# impute median age for each combination of sex and pclass
titanic['age'] = titanic.groupby(['sex', 'pclass'])['age'].transform(lambda x: x.fillna(x.median()))
```

Let's check again

```
columns_of_interest = ['survived', 'age', 'fare', 'sex', 'pclass']
print("Missing values in columns of interest:")
print(titanic[columns_of_interest].isnull().sum())
```

```
Missing values in columns of interest:
survived      0
age           0
fare          0
sex           0
pclass        0
dtype: int64
```

9.3.2 Check Target Distribution

Let's examine the distribution of the target variable to determine whether the dataset is balanced.

```
# Quick exploratory analysis
print("Target variable distribution:")
print(titanic['survived'].value_counts())
```

```
Target variable distribution:
survived
0      549
1      342
Name: count, dtype: int64
```

9.4 Stratified Splitting

The dataset shows moderate class imbalance (about 62% non-survived vs 38% survived).

For such datasets, we need to do **stratified splitting** to make sure the proportion of classes is preserved in both train and test sets.

How to do stratified splitting:

Use the `stratify` parameter in `train_test_split()`:

```
train_df, test_df = train_test_split(
    titanic,
    test_size=0.2,
    random_state=42,
    stratify=titanic['survived'] # Pass the target variable here, or stratify=y
)
```

This ensures that both train and test sets have the same class distribution as the original dataset.

```
# Compare random vs stratified splitting
print("ORIGINAL DATASET:")
original_props = titanic['survived'].value_counts(normalize=True)
print(f"  Survived: {original_props[1]:.1%} | Died: {original_props[0]:.1%}")

# Random splitting (without stratify)
train_random, test_random = train_test_split(titanic, test_size=0.2, random_state=42)
print("\nRANDOM SPLIT (without stratify):")
print(f"  Train - Survived: {train_random['survived'].mean():.1%}")
print(f"  Test  - Survived: {test_random['survived'].mean():.1%}")

# Stratified splitting (with stratify)
train_strat, test_strat = train_test_split(titanic, test_size=0.2, random_state=42,
                                           stratify=titanic['survived'])
print("\nSTRATIFIED SPLIT (with stratify):")
```

```
print(f"  Train - Survived: {train_strat['survived'].mean():.1%}")
print(f"  Test  - Survived: {test_strat['survived'].mean():.1%}")

print("\n Stratified splitting maintains the same proportions in both sets")
print(" This ensures reliable model evaluation, especially for imbalanced data")
```

ORIGINAL DATASET:

Survived: 38.4% | Died: 61.6%

RANDOM SPLIT (without stratify):

Train - Survived: 37.6%

Test - Survived: 41.3%

STRATIFIED SPLIT (with stratify):

Train - Survived: 38.3%

Test - Survived: 38.5%

Stratified splitting maintains the same proportions in both sets

This ensures reliable model evaluation, especially for imbalanced data

```
# Split into train and test sets with stratification for the rest of the tasks
train_df, test_df = train_test_split(titanic, test_size=0.2, random_state=42, stratify=titanic['survived'])
```

9.5 Scikit-learn Implementation

Since `sklearn` is optimized for **prediction and machine learning workflows**, we will stick with `sklearn`

9.5.1 Pipeline Implementation

Why use Pipelines?

In the previous chapter, we've manually created dummy variables using `pd.get_dummies()`. While this works, `sklearn`'s **Pipeline** approach offers several advantages:

1. **Automated preprocessing:** Handles categorical encoding automatically
2. **Prevents data leakage:** Ensures transformations are fit only on training data
3. **Cleaner code:** Combines preprocessing and modeling in one object
4. **Production-ready:** Easier to deploy and maintain
5. **Reproducibility:** All steps are encapsulated in one pipeline

Key components: - **ColumnTransformer:** Applies different transformations to different columns - **OneHotEncoder:** Converts categorical variables to dummy variables - **Pipeline:** Chains preprocessing and model fitting together

Let's implement the same logistic regression model using a pipeline (with regularization to match statsmodels):

```
# Import required components for pipeline
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Define which columns are categorical and which are numeric
categorical_features = ['sex']
numeric_features = ['age', 'fare', 'pclass']

y_train = train_df['survived']
y_test = test_df['survived']

# Create the column transformer
# OneHotEncoder with drop='first' mimics the drop_first=True in pd.get_dummies
preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numeric_features), # Keep numeric features as-is
        ('cat', OneHotEncoder(drop='first'), categorical_features) # Encode categorical
    ])

# Create the pipeline: preprocessing + logistic regression
pipeline_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=42))
])

# Fit the pipeline on training data
# Note: We pass the original dataframe with categorical variables
X_train_raw = train_df[['age', 'fare', 'sex', 'pclass']]
X_test_raw = test_df[['age', 'fare', 'sex', 'pclass']]

pipeline_model.fit(X_train_raw, y_train)

print("Pipeline model fitted successfully!")
print("\nPipeline structure:")
print(pipeline_model)
```

Pipeline model fitted successfully!

Pipeline structure:

```
Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('num', 'passthrough',
                                                  ['age', 'fare', 'pclass']),
                                                  ('cat',
                                                   OneHotEncoder(drop='first'),
                                                   ['sex'])])),
                ('classifier', LogisticRegression(random_state=42))])
```

```
# Extract the trained logistic regression model from the pipeline
pipeline_logit = pipeline_model.named_steps['classifier']

# Get feature names after transformation
feature_names = numeric_features + ['sex_male'] # OneHotEncoder creates sex_male (drops sex)

print("Pipeline Logistic Regression Coefficients (C=np.inf):")
print(f"\nCoefficients:")
for feature, coef in zip(feature_names, pipeline_logit.coef_[0]):
    print(f" {feature:20s}: {coef:8.4f}")
print(f"\nIntercept: {pipeline_logit.intercept_[0]:.4f}")

# Make predictions
pipeline_train_pred = pipeline_model.predict(X_train_raw)
pipeline_test_pred = pipeline_model.predict(X_test_raw)

# Calculate accuracy
pipeline_train_accuracy = accuracy_score(y_train, pipeline_train_pred)
pipeline_test_accuracy = accuracy_score(y_test, pipeline_test_pred)

print(f"\nPipeline Model Performance:")
print(f" Training Accuracy: {pipeline_train_accuracy:.4f} ({pipeline_train_accuracy:.2%})")
print(f" Test Accuracy: {pipeline_test_accuracy:.4f} ({pipeline_test_accuracy:.2%})")
```

Pipeline Logistic Regression Coefficients (C=np.inf):

Coefficients:

age	:	-0.0389
fare	:	0.0010
pclass	:	-1.2189
sex_male	:	-2.4830

Intercept: 4.8699

Pipeline Model Performance:

Training Accuracy: 0.8006 (80.06%)
Test Accuracy: 0.7821 (78.21%)

9.6 Performance Evaluation

The dataset shows moderate class imbalance (about 62% non-survived vs 38% survived). While accuracy is a useful metric, for imbalanced datasets we should consider additional metrics:

- **Confusion Matrix:** Shows the breakdown of correct and incorrect predictions
- **Precision:** Of all positive predictions, how many were correct?
- **Recall (Sensitivity):** Of all actual positives, how many did we find?
- **F1 Score:** Harmonic mean of precision and recall
- **ROC-AUC:** Overall measure of model discrimination ability

Let's evaluate our model using these metrics.

9.6.1 Confusion Matrix

A confusion matrix shows the breakdown of predictions:

	Predicted: 0 (Died)	Predicted: 1 (Survived)
Actual: 0	True Negative (TN)	False Positive (FP)
Actual: 1	False Negative (FN)	True Positive (TP)

- **True Positives (TP):** Correctly predicted survivors
- **True Negatives (TN):** Correctly predicted deaths
- **False Positives (FP):** Predicted survival but actually died (Type I error)
- **False Negatives (FN):** Predicted death but actually survived (Type II error)

```
# Generate predictions on test set
y_pred = pipeline_model.predict(X_test_raw)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
```



```

print("="*50)
print(f"                Predicted")
print(f"                Died | Survived")
print(f"Actual Died      {cm[0,0]:4d} | {cm[0,1]:4d}")
print(f"Actual Survived  {cm[1,0]:4d} | {cm[1,1]:4d}")
print("="*50)

# Visualize confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Died', 'Survived'],
            yticklabels=['Died', 'Survived'],
            cbar_kws={'label': 'Count'})
plt.title('Confusion Matrix', fontsize=14, fontweight='bold')
plt.ylabel('Actual', fontsize=12)
plt.xlabel('Predicted', fontsize=12)
plt.tight_layout()
plt.show()

print(f"\nBreakdown:")
print(f" True Negatives (TN): {cm[0,0]} - Correctly predicted deaths")
print(f" False Positives (FP): {cm[0,1]} - Incorrectly predicted survival")
print(f" False Negatives (FN): {cm[1,0]} - Incorrectly predicted death")
print(f" True Positives (TP): {cm[1,1]} - Correctly predicted survival")

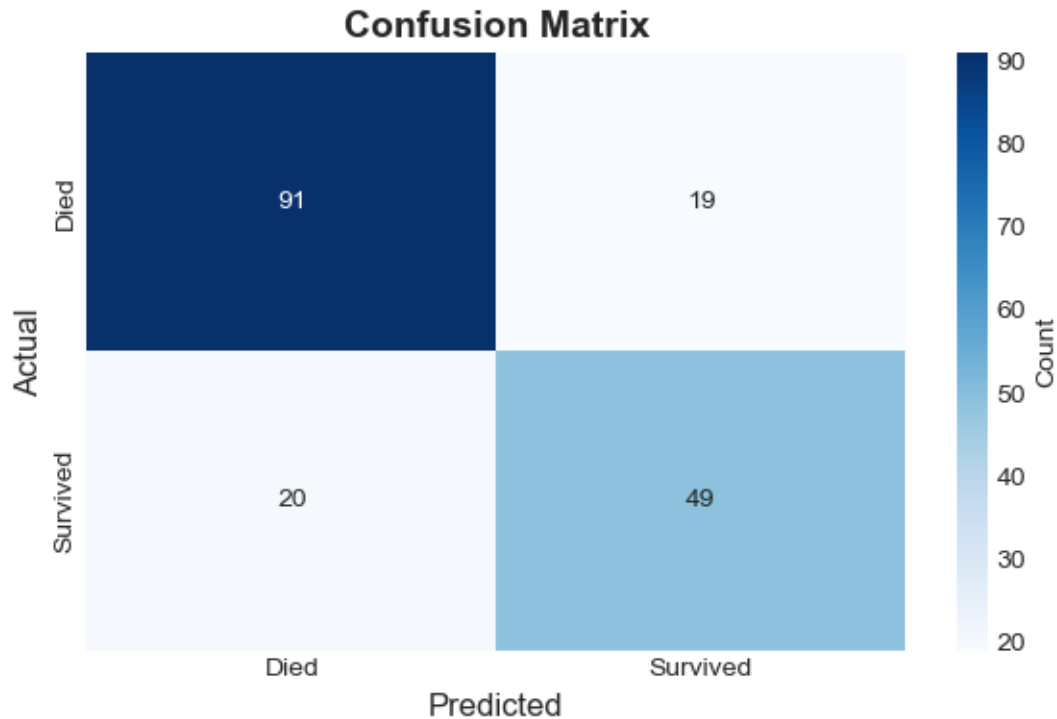
```

Confusion Matrix:

```

=====
                Predicted
                Died | Survived
Actual Died      91 |   19
Actual Survived  20 |   49
=====

```



Breakdown:

True Negatives (TN): 91 - Correctly predicted deaths
 False Positives (FP): 19 - Incorrectly predicted survival
 False Negatives (FN): 20 - Incorrectly predicted death
 True Positives (TP): 49 - Correctly predicted survival

9.6.2 Precision, Recall, and F1 Score

Precision = $\frac{TP}{TP+FP}$ = “Of all predicted survivors, how many actually survived?” - Focuses on the quality of positive predictions - High precision = few false alarms

Recall (Sensitivity) = $\frac{TP}{TP+FN}$ = “Of all actual survivors, how many did we correctly identify?” - Focuses on finding all positive cases - High recall = few missed cases

F1 Score = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ = Harmonic mean of precision and recall - Balances precision and recall - Useful when you need a single metric

```
# Calculate metrics from sklearn
from sklearn.metrics import f1_score
```

```

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Model Performance Metrics:")
print("="*50)
print(f"Accuracy:  {accuracy:.4f} ({accuracy:.2%})")
print(f"Precision: {precision:.4f} ({precision:.2%})")
print(f"Recall:    {recall:.4f} ({recall:.2%})")
print(f"F1 Score:   {f1:.4f}")
print("="*50)

# Manual calculation to verify
TP = cm[1, 1]
TN = cm[0, 0]
FP = cm[0, 1]
FN = cm[1, 0]

manual_precision = TP / (TP + FP)
manual_recall = TP / (TP + FN)
manual_f1 = 2 * (manual_precision * manual_recall) / (manual_precision + manual_recall)

print("\nManual Calculation (verification):")
print(f"Precision = TP / (TP + FP) = {TP} / ({TP} + {FP}) = {manual_precision:.4f}")
print(f"Recall    = TP / (TP + FN) = {TP} / ({TP} + {FN}) = {manual_recall:.4f}")
print(f"F1 Score  = 2 * (P * R) / (P + R) = {manual_f1:.4f}")

print("\n Metrics match!")

```

Model Performance Metrics:

```

=====
Accuracy:  0.7821 (78.21%)
Precision: 0.7206 (72.06%)
Recall:    0.7101 (71.01%)
F1 Score:  0.7153
=====

```

Manual Calculation (verification):

```

Precision = TP / (TP + FP) = 49 / (49 + 19) = 0.7206
Recall    = TP / (TP + FN) = 49 / (49 + 20) = 0.7101
F1 Score  = 2 * (P * R) / (P + R) = 0.7153

```

Metrics match!

9.6.3 Threshold Effects: Precision-Recall Tradeoff

By default, sklearn uses a **threshold of 0.5** to classify predictions: - If probability $\geq 0.5 \rightarrow$ Predict 1 (Survived) - If probability $< 0.5 \rightarrow$ Predict 0 (Died)

What happens when we change the threshold?

- **Higher threshold (e.g., 0.7):** More conservative predictions
 - Fewer positive predictions $\rightarrow$ Higher precision, Lower recall
 - We only predict survival when we're very confident
- **Lower threshold (e.g., 0.3):** More liberal predictions
 - More positive predictions $\rightarrow$ Lower precision, Higher recall
 - We predict survival more readily

Let's demonstrate this tradeoff:

```
# Get predicted probabilities
y_pred_proba = pipeline_model.predict_proba(X_test_raw)[: , 1]

# Test different thresholds
thresholds = [0.3, 0.5, 0.7]
results = []

print("Impact of Different Thresholds:")
print("="*70)
print(f"{'Threshold':<12} {'Precision':<12} {'Recall':<12} {'F1 Score':<12} {'Predictions'}")
print("="*70)

for threshold in thresholds:
    # Apply threshold
    y_pred_threshold = (y_pred_proba >= threshold).astype(int)

    # Calculate metrics
    prec = precision_score(y_test, y_pred_threshold, zero_division=0)
    rec = recall_score(y_test, y_pred_threshold)
    f1 = f1_score(y_test, y_pred_threshold, zero_division=0)
    n_pos = y_pred_threshold.sum()
```

```

print(f"{threshold:<12.1f} {prec:<12.4f} {rec:<12.4f} {f1:<12.4f} {n_pos:>4d}/{len(y_test)}")
results.append({'threshold': threshold, 'precision': prec, 'recall': rec, 'f1': f1})

print("="*70)
print("\nKey Observations:")
print("  → Higher threshold: Higher precision, Lower recall (fewer false positives)")
print("  → Lower threshold: Lower precision, Higher recall (fewer false negatives)")
print("  → F1 score balances both metrics")

```

Impact of Different Thresholds:

Threshold	Precision	Recall	F1 Score	Predictions
0.3	0.6512	0.8116	0.7226	86/179
0.5	0.7206	0.7101	0.7153	68/179
0.7	0.8889	0.4638	0.6095	36/179

Key Observations:

- Higher threshold: Higher precision, Lower recall (fewer false positives)
- Lower threshold: Lower precision, Higher recall (fewer false negatives)
- F1 score balances both metrics

```

# Visualize the precision-recall tradeoff across many thresholds
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds_pr = precision_recall_curve(y_test, y_pred_proba)

plt.figure(figsize=(12, 5))

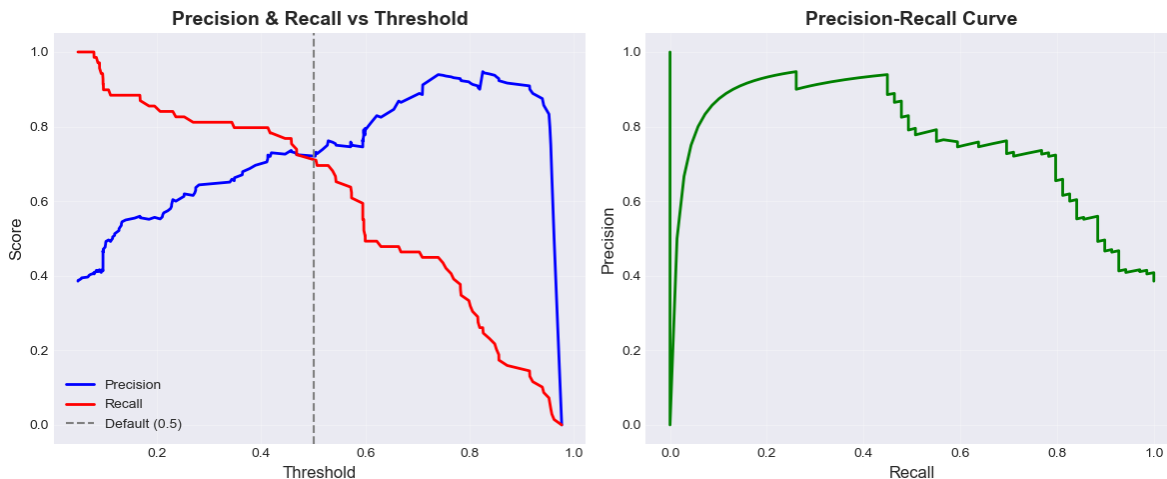
# Plot 1: Precision and Recall vs Threshold
plt.subplot(1, 2, 1)
plt.plot(thresholds_pr, precisions[:-1], 'b-', label='Precision', linewidth=2)
plt.plot(thresholds_pr, recalls[:-1], 'r-', label='Recall', linewidth=2)
plt.axvline(x=0.5, color='gray', linestyle='--', label='Default (0.5)')
plt.xlabel('Threshold', fontsize=12)
plt.ylabel('Score', fontsize=12)
plt.title('Precision & Recall vs Threshold', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(alpha=0.3)

```

```
# Plot 2: Precision-Recall Curve
plt.subplot(1, 2, 2)
plt.plot(recalls, precisions, 'g-', linewidth=2)
plt.xlabel('Recall', fontsize=12)
plt.ylabel('Precision', fontsize=12)
plt.title('Precision-Recall Curve', fontsize=14, fontweight='bold')
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

print("Interpretation:")
print(" • Left plot: Shows how precision and recall change with different thresholds")
print(" • Right plot: Precision-Recall curve - higher is better")
print(" • The curves show the fundamental tradeoff between precision and recall")
```



Interpretation:

- Left plot: Shows how precision and recall change with different thresholds
- Right plot: Precision-Recall curve - higher is better
- The curves show the fundamental tradeoff between precision and recall

9.6.4 ROC Curve and AUC Score

ROC (Receiver Operating Characteristic) Curve plots: - **X-axis:** False Positive Rate (FPR) = $\frac{FP}{FP+TN}$ = “Of all actual negatives, how many did we incorrectly predict as positive?”
- **Y-axis:** True Positive Rate (TPR) = Recall = $\frac{TP}{TP+FN}$

AUC (Area Under Curve): Measures the overall ability of the model to discriminate between classes - AUC = 1.0: Perfect classifier - AUC = 0.5: Random classifier (no better than flipping a coin) - AUC > 0.8: Generally considered good

The ROC curve is **threshold-independent** - it shows model performance across all possible thresholds.

```
# Calculate ROC curve and AUC
from sklearn.metrics import roc_curve, roc_auc_score

# Calculate ROC curve
fpr, tpr, thresholds_roc = roc_curve(y_test, y_pred_proba)

# Calculate AUC
auc_score = roc_auc_score(y_test, y_pred_proba)

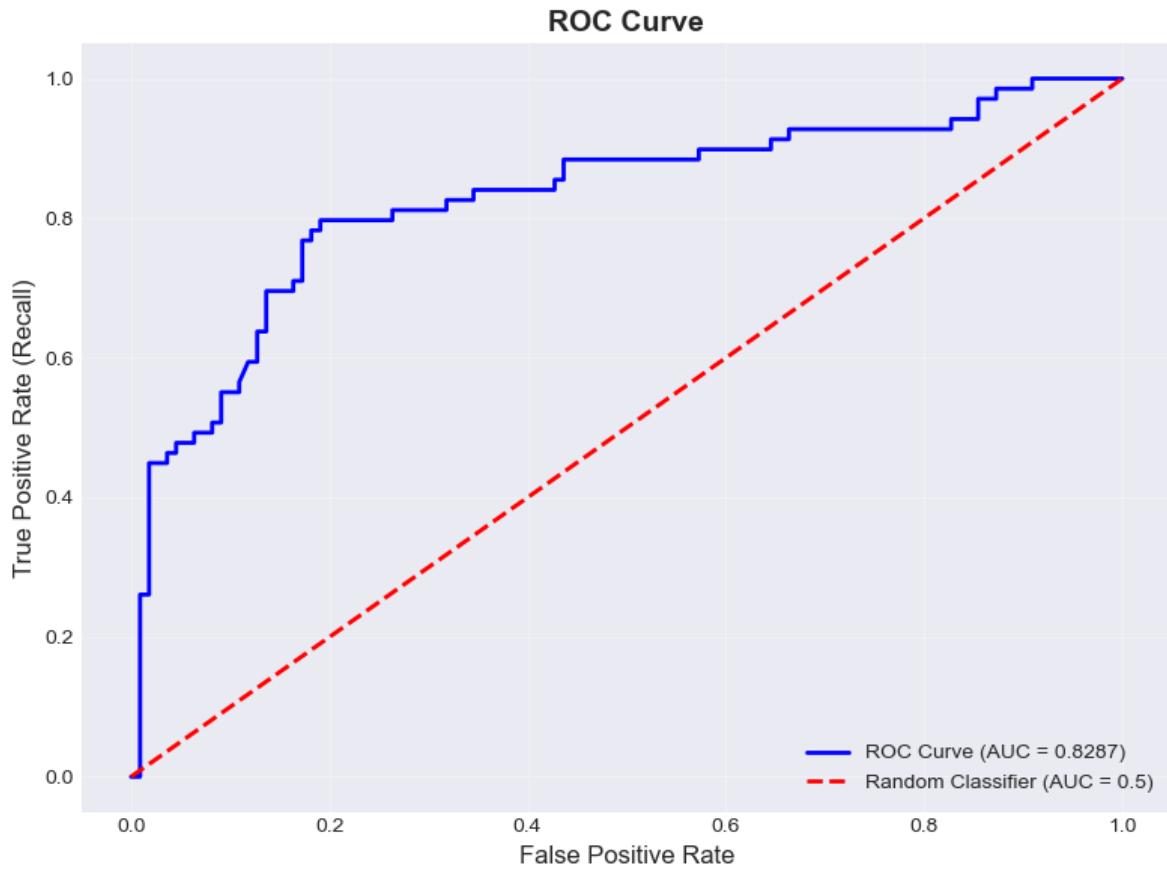
print(f"ROC-AUC Score: {auc_score:.4f}")
print(f"\nInterpretation: The model has a {auc_score:.1%} probability of ranking")
print(f"a randomly chosen positive example higher than a randomly chosen negative example.")

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, 'b-', linewidth=2, label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], 'r--', linewidth=2, label='Random Classifier (AUC = 0.5)')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate (Recall)', fontsize=12)
plt.title('ROC Curve', fontsize=14, fontweight='bold')
plt.legend(loc='lower right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print("\nKey Points:")
print(f"    AUC = {auc_score:.4f} indicates {'good' if auc_score > 0.8 else 'moderate'} discriminative power")
print(f"    Our model performs {'much ' if auc_score > 0.8 else ''}better than random guessing")
print(f"    ROC-AUC is useful for comparing models regardless of threshold choice")
```

ROC-AUC Score: 0.8287

Interpretation: The model has a 82.9% probability of ranking
a randomly chosen positive example higher than a randomly chosen negative example.



Key Points:

- AUC = 0.8287 indicates good discrimination ability
- Our model performs much better than random guessing
- ROC-AUC is useful for comparing models regardless of threshold choice

9.6.5 Summary: Choosing the Right Metric

Different metrics are appropriate for different situations:

Metric	When to Use	What It Tells You
Accuracy	Balanced datasets	Overall correctness
Precision	False positives are costly (e.g., spam detection)	Quality of positive predictions

Metric	When to Use	What It Tells You
Recall	False negatives are costly (e.g., disease diagnosis)	Coverage of actual positives
F1 Score	Need balance between precision and recall	Harmonic mean of both
ROC-AUC	Comparing models, threshold-independent evaluation	Overall discrimination ability

For imbalanced datasets: - Don't rely solely on accuracy - Consider precision, recall, and F1 score - Use ROC-AUC for model comparison - Choose your threshold based on the cost of different error types

Next steps: - Experiment with different thresholds based on your use case - Try different models and compare their ROC-AUC scores - Consider techniques like class weighting or resampling for severe imbalance

10 Logistic Regression: Feature Engineering

```
# Importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

sns.set()
```

10.1 Decision Boundaries in Classification

Logistic regression with raw features creates **linear decision boundaries**. To create nonlinear boundaries, we need **feature transformation**.

In classification problems, a **decision boundary** is the surface that separates different classes in the feature space. Understanding decision boundaries is crucial for feature engineering:

- **Nonlinear Decision Boundary:** A curved boundary required when classes cannot be separated by a straight line.
- **Linear Decision Boundary:** A straight line (in 2D) or hyperplane (in higher dimensions) that separates classes. Works when data is linearly separable.

10.1.1 Linear Decision Boundary (Linearly Separable Data)

Let's start with a simple case where two classes can be perfectly separated by a straight line.

```
# Load and prepare the apples and oranges dataset
apples_oranges = pd.read_csv('datasets/apples_and_oranges.csv')

# Encode the target variable (Apple=0, Orange=1)
```

```

le = LabelEncoder()
apples_oranges['Class'] = le.fit_transform(apples_oranges['Class'])

print(f"Dataset shape: {apples_oranges.shape}")
print(f"\nClass distribution:")
print(apples_oranges['Class'].value_counts())
print(f"\nClass labels: {le.classes_}")
apples_oranges.head()

```

Dataset shape: (40, 3)

Class distribution:

Class

1 20

0 20

Name: count, dtype: int64

Class labels: ['apple' 'orange']

	Weight	Size	Class
0	69	4.39	1
1	69	4.21	1
2	65	4.09	1
3	72	5.85	0
4	67	4.70	1

```

# Visualize the data
plt.figure(figsize=(7, 4))
plt.scatter(apples_oranges[apples_oranges['Class']==0]['Weight'],
            apples_oranges[apples_oranges['Class']==0]['Size'],
            c='purple', marker='o', s=100, alpha=0.7, label='Apple (Class 0)', edgecolors='b')
plt.scatter(apples_oranges[apples_oranges['Class']==1]['Weight'],
            apples_oranges[apples_oranges['Class']==1]['Size'],
            c='gold', marker='^', s=100, alpha=0.7, label='Orange (Class 1)', edgecolors='b')
plt.xlabel('Weight', fontsize=12)
plt.ylabel('Size', fontsize=12)
plt.title('Apples vs Oranges: Linearly Separable Data', fontweight='bold', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```



Observation: The two classes are clearly separated. We can draw a straight line between apples and oranges - this is **linearly separable data**.

10.1.1.1 Building Logistic Regression with Raw Features

For linearly separable data, the standard logistic regression hypothesis works perfectly:

$$h_w(x) = \sigma(w_0 + w_1x_1 + w_2x_2)$$

where:

- σ is the sigmoid function
- w_0 is the intercept
- w_1, w_2 are coefficients for Weight and Size
- The decision boundary occurs where $w_0 + w_1x_1 + w_2x_2 = 0$

```

# Prepare features and target
X_apples = apples_oranges[['Weight', 'Size']]
y_apples = apples_oranges['Class']

# Train logistic regression model
linear_model = LogisticRegression()
linear_model.fit(X_apples, y_apples)

# Make predictions
y_pred_apples = linear_model.predict(X_apples)

# Evaluate performance
accuracy = accuracy_score(y_apples, y_pred_apples)
print(f"Model Accuracy: {accuracy:.4f} ({accuracy*100:.1f}%)")
print(f"\nModel Parameters:")
print(f"  Intercept (w): {linear_model.intercept_[0]:.4f}")
print(f"  Weight coefficient (w): {linear_model.coef_[0][0]:.4f}")
print(f"  Size coefficient (w): {linear_model.coef_[0][1]:.4f}")

```

Model Accuracy: 1.0000 (100.0%)

Model Parameters:

Intercept (w): 106.6029
Weight coefficient (w): -1.4283
Size coefficient (w): -1.3129

10.1.1.2 Visualizing the Linear Decision Boundary

The decision boundary is a straight line defined by: $w_0 + w_1 \cdot \text{Weight} + w_2 \cdot \text{Size} = 0$

Rearranging: $\text{Size} = -\frac{w_0}{w_2} - \frac{w_1}{w_2} \cdot \text{Weight}$

```

# Extract model parameters
w0 = linear_model.intercept_[0]
w1 = linear_model.coef_[0][0]
w2 = linear_model.coef_[0][1]

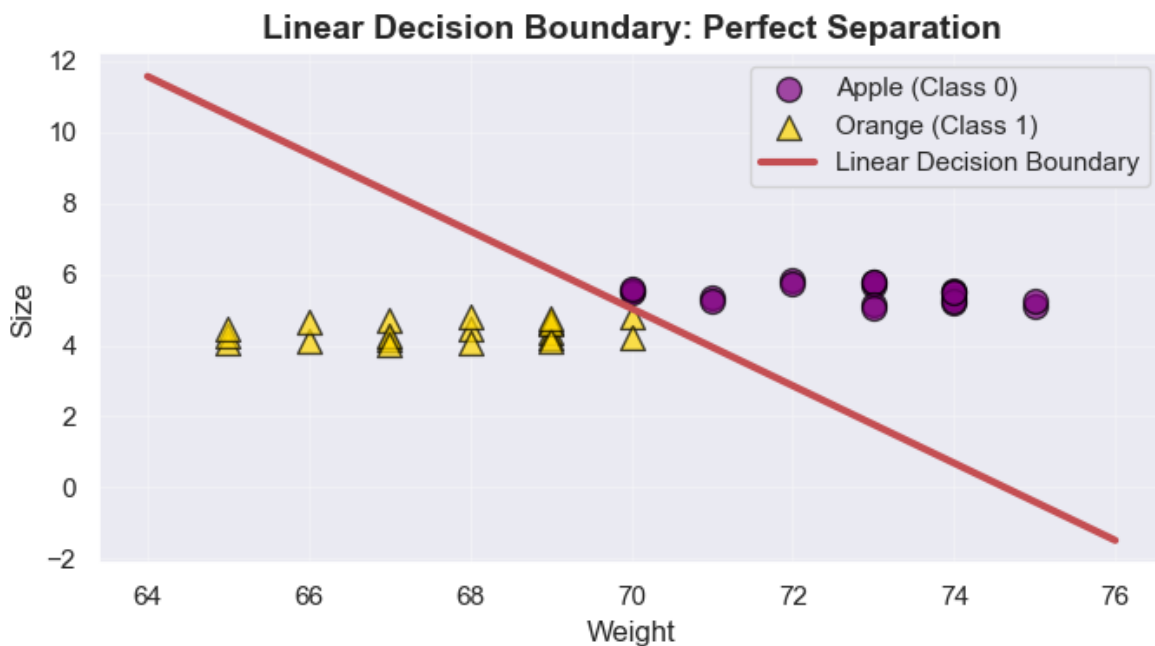
# Calculate decision boundary line
weight_range = np.array([apples_oranges['Weight'].min() - 1, apples_oranges['Weight'].max() - 1])
size_boundary = -(w0 / w2) - (w1 / w2) * weight_range

```

```

# Plot data with decision boundary
plt.figure(figsize=(7, 4))
plt.scatter(apples_oranges[apples_oranges['Class']==0]['Weight'],
            apples_oranges[apples_oranges['Class']==0]['Size'],
            c='purple', marker='o', s=100, alpha=0.7, label='Apple (Class 0)', edgecolors='b')
plt.scatter(apples_oranges[apples_oranges['Class']==1]['Weight'],
            apples_oranges[apples_oranges['Class']==1]['Size'],
            c='gold', marker='^', s=100, alpha=0.7, label='Orange (Class 1)', edgecolors='b')
plt.plot(weight_range, size_boundary, 'r-', linewidth=3, label='Linear Decision Boundary')
plt.xlabel('Weight', fontsize=12)
plt.ylabel('Size', fontsize=12)
plt.title('Linear Decision Boundary: Perfect Separation', fontweight='bold', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



Key Takeaway: When data is **linearly separable**, a straight-line decision boundary (using raw features) achieves perfect classification. No feature transformation is needed.

10.1.2 Nonlinear Decision Boundary (Non-Linearly Separable Data)

In some cases, we encounter data where classes cannot be separated by a straight line. Consider the training data distribution illustrated below:

When confronted with such a circular or curved pattern, it becomes imperative to generate polynomial features (such as x_1^2, x_2^2) in order to enhance the delineation between the two classes.

Let's delve into a concrete example using a microchip quality control dataset. For the purpose of illustrating the decision boundary clearly, I chose not to split the data into training and test sets.

```
# Load the microchip quality control dataset
microchip_test = pd.read_csv('datasets/ex2data2.txt', header=None,
                             names=['test_1', 'test_2', 'faulty'])

print(f"Dataset shape: {microchip_test.shape}")
print(f"\nClass distribution:")
print(microchip_test['faulty'].value_counts())
print(f"\n0 = Pass (Non-Faulty), 1 = Fail (Faulty)")
microchip_test.head()
```

Dataset shape: (118, 3)

Class distribution:

faulty

0 60

1 58

Name: faulty, dtype: int64

0 = Pass (Non-Faulty), 1 = Fail (Faulty)

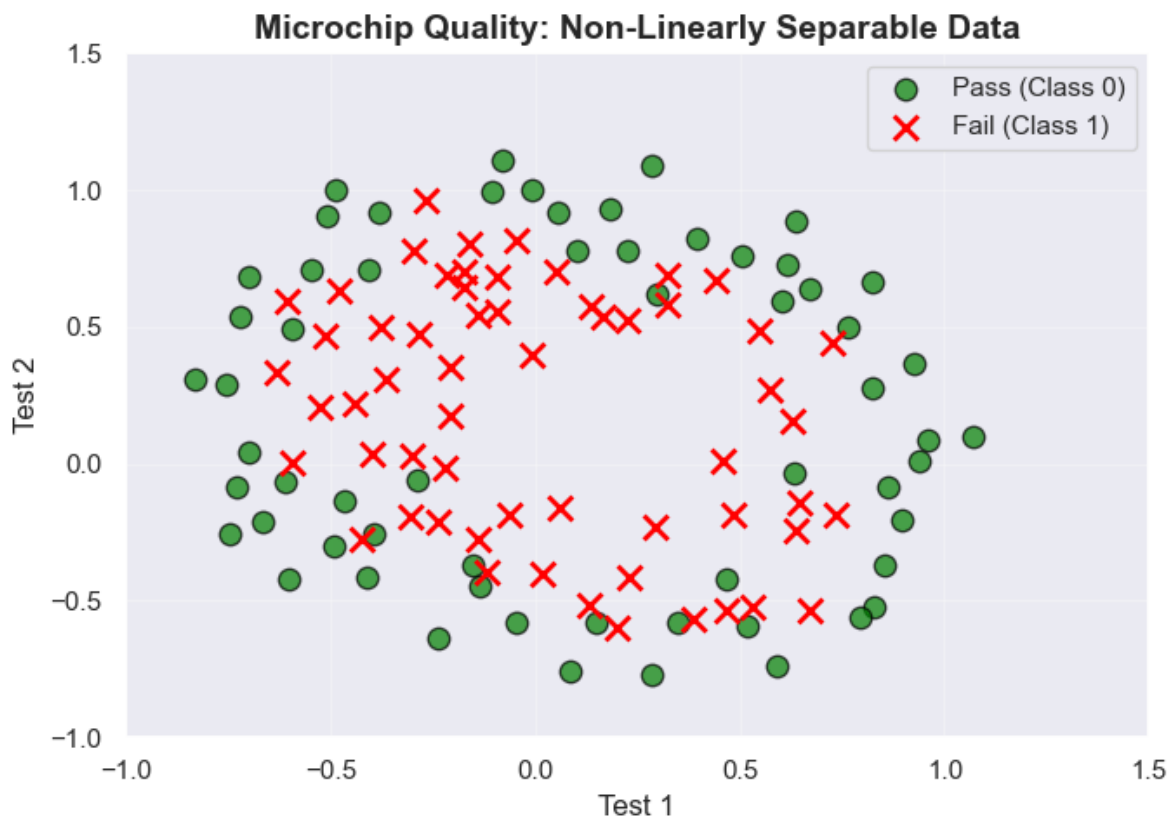
	test_1	test_2	faulty
0	0.051267	0.69956	1
1	-0.092742	0.68494	1
2	-0.213710	0.69225	1
3	-0.375000	0.50219	1
4	-0.513250	0.46564	1

```

# Visualize the data
plt.figure(figsize=(7, 5))
pos = microchip_test['faulty'] == 1
neg = microchip_test['faulty'] == 0

plt.scatter(microchip_test.loc[neg, 'test_1'], microchip_test.loc[neg, 'test_2'],
            c='green', marker='o', s=80, alpha=0.7, label='Pass (Class 0)', edgecolors='black')
plt.scatter(microchip_test.loc[pos, 'test_1'], microchip_test.loc[pos, 'test_2'],
            c='red', marker='x', s=100, linewidths=2, label='Fail (Class 1)')
plt.xlabel('Test 1', fontsize=12)
plt.ylabel('Test 2', fontsize=12)
plt.title('Microchip Quality: Non-Linearly Separable Data', fontweight='bold', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.xlim(-1, 1.5)
plt.ylim(-1, 1.5)
plt.tight_layout()
plt.show()

```



Observation: The faulty microchips (red X's) are surrounded by passing microchips (green circles). **No straight line can separate these two classes** - the data exhibits a circular pattern and is not linearly separable.

10.1.2.1 Step 1: Attempt with Raw Features (Linear Model - Will Fail)

Let's first try logistic regression with only the raw features to demonstrate why transformation is necessary.

```
# Prepare features and target
X_raw = microchip_test[['test_1', 'test_2']]
y = microchip_test['faulty']

# Train linear logistic regression with raw features
linear_model = LogisticRegression()
linear_model.fit(X_raw, y)

# Evaluate performance
y_pred_linear = linear_model.predict(X_raw)
accuracy_linear = accuracy_score(y, y_pred_linear)

print(f"Linear Model (Raw Features) Performance:")
print(f"  Accuracy: {accuracy_linear:.4f} ({accuracy_linear*100:.1f}%)")
print(f"\n Poor performance! Linear model cannot capture the circular pattern.")
```

Linear Model (Raw Features) Performance:

Accuracy: 0.5424 (54.2%)

Poor performance! Linear model cannot capture the circular pattern.

```
# Visualize the linear decision boundary (inadequate for circular pattern)
fig, ax = plt.subplots(figsize=(8, 6))

# Plot data points
ax.scatter(microchip_test.loc[neg, 'test_1'], microchip_test.loc[neg, 'test_2'],
           c='green', marker='o', s=80, alpha=0.7, label='Pass', edgecolors='black')
ax.scatter(microchip_test.loc[pos, 'test_1'], microchip_test.loc[pos, 'test_2'],
           c='red', marker='x', s=100, linewidths=2, label='Fail')

# Create mesh for linear decision boundary
xx, yy = np.meshgrid(np.linspace(-1, 1.5, 200), np.linspace(-1, 1.5, 200))
```

```

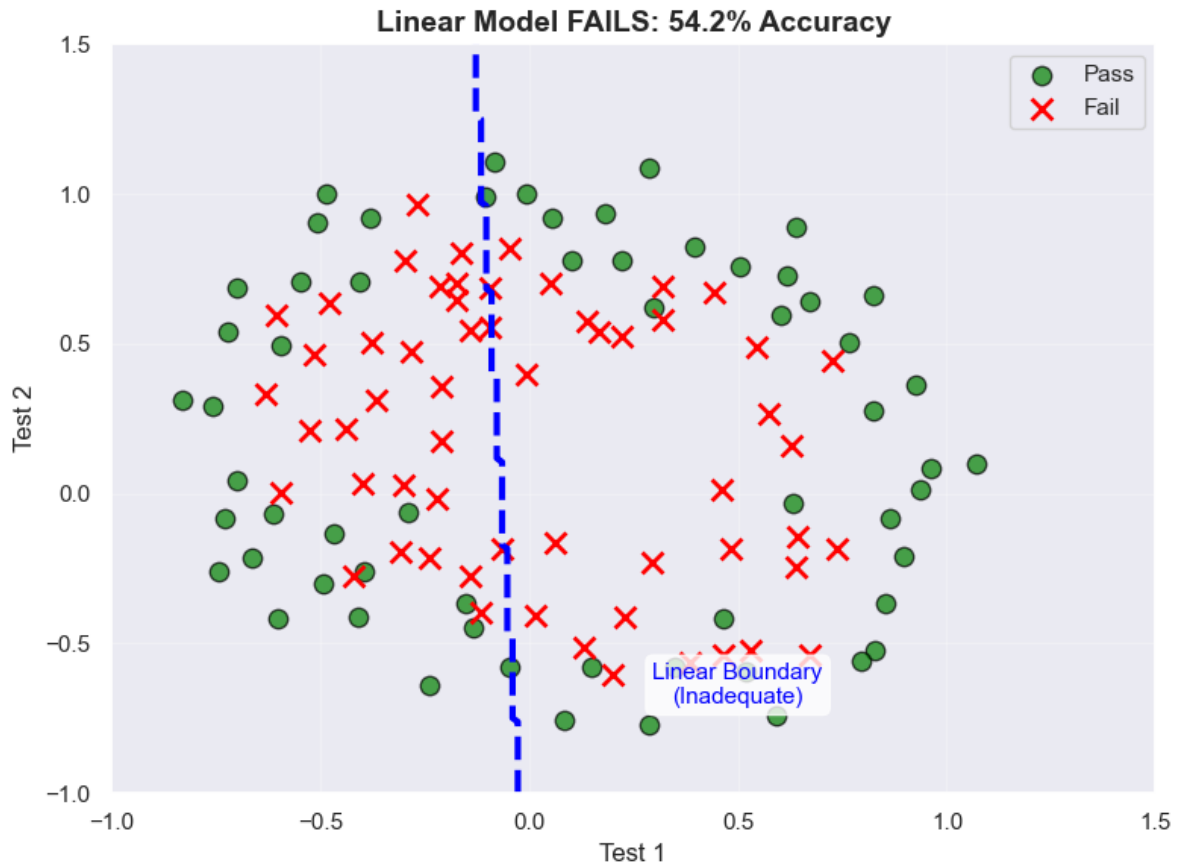
mesh_data = pd.DataFrame(np.c_[xx.ravel(), yy.ravel()], columns=['test_1', 'test_2'])
Z_linear = linear_model.predict(mesh_data)
Z_linear = Z_linear.reshape(xx.shape)

# Plot linear decision boundary
ax.contour(xx, yy, Z_linear, levels=[0.5], colors='blue', linewidths=3, linestyle='dashed')
ax.text(0.5, -0.7, 'Linear Boundary\n(Inadequate)', fontsize=11, color='blue',
        ha='center', bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

ax.set_xlabel('Test 1', fontsize=12)
ax.set_ylabel('Test 2', fontsize=12)
ax.set_title(f'Linear Model FAILS: {accuracy_linear*100:.1f}% Accuracy', fontweight='bold', )
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_xlim(-1, 1.5)
ax.set_ylim(-1, 1.5)
plt.tight_layout()
plt.show()

print(" The straight-line boundary cannot capture the circular separation pattern!")

```



The straight-line boundary cannot capture the circular separation pattern!

10.1.3 Step 2: Polynomial Feature Transformation (The Solution)

Key Insight: Transform the original features into higher-order polynomial features to enable nonlinear decision boundaries.

Original features: x_1, x_2 (Test 1, Test 2)

Polynomial features (degree 2): $1, x_1, x_2, x_1^2, x_1x_2, x_2^2$ (6 features)

Polynomial features (degree 6): 28 features including all combinations up to degree 6

This allows logistic regression to learn curves and circles while remaining a linear model in the transformed feature space.

```

from scipy.optimize import minimize

# Helper functions for polynomial feature transformation
def mapFeature(X1, X2, degree):
    """Map features to polynomial features up to given degree"""
    res = np.ones(X1.shape[0])
    for i in range(1, degree + 1):
        for j in range(0, i + 1):
            res = np.column_stack((res, (X1 ** (i-j)) * (X2 ** j)))
    return res

def sigmoid(z):
    """Sigmoid activation function"""
    return 1 / (1 + np.exp(-z))

def costFunc(theta, X, y):
    """Logistic regression cost function"""
    m = y.shape[0]
    z = X.dot(theta)
    h = sigmoid(z)
    term1 = y * np.log(h)
    term2 = (1 - y) * np.log(1 - h)
    J = -np.sum(term1 + term2, axis=0) / m
    return J

def plotDecisionBoundary(theta, degree, ax):
    """Plot nonlinear decision boundary"""
    u = np.linspace(-1, 1.5, 100)
    v = np.linspace(-1, 1.5, 100)
    U, V = np.meshgrid(u, v)
    U_flat = np.ravel(U)
    V_flat = np.ravel(V)

    X_poly = mapFeature(U_flat, V_flat, degree)
    Z = X_poly.dot(theta)
    Z = Z.reshape(U.shape)

    ax.contour(U, V, Z, levels=[0], colors='purple', linewidths=3)
    return ax

# Train polynomial models with degrees 2, 3, 4, 5, and 6
results = {}

```

```

for degree in [2, 3, 4, 5, 6]:
    # Transform features to polynomial
    X_poly = mapFeature(microchip_test['test_1'], microchip_test['test_2'], degree)

    # Initialize parameters
    initial_theta = np.zeros(X_poly.shape[1])

    # Optimize using BFGS
    res = minimize(costFunc, initial_theta, args=(X_poly, y), method='BFGS')
    theta = res.x

    # Evaluate
    predictions = sigmoid(X_poly.dot(theta)) >= 0.5
    accuracy = accuracy_score(y, predictions)

    results[degree] = {'theta': theta, 'accuracy': accuracy, 'n_features': X_poly.shape[1]}

    print(f"Degree {degree} Polynomial:")
    print(f"  Number of features: {X_poly.shape[1]} (from 2 original features)")
    print(f"  Accuracy: {accuracy:.4f} ({accuracy*100:.1f}%)")
    print()

```

Degree 2 Polynomial:

Number of features: 6 (from 2 original features)
Accuracy: 0.8559 (85.6%)

Degree 3 Polynomial:

Number of features: 10 (from 2 original features)
Accuracy: 0.8475 (84.7%)

Degree 4 Polynomial:

Number of features: 15 (from 2 original features)
Accuracy: 0.8475 (84.7%)

Degree 5 Polynomial:

Number of features: 21 (from 2 original features)
Accuracy: 0.8814 (88.1%)

Degree 6 Polynomial:

Number of features: 28 (from 2 original features)
Accuracy: 0.9153 (91.5%)

```

# Visualize all decision boundaries in a grid layout
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.flatten()

for idx, degree in enumerate([2, 3, 4, 5, 6]):
    ax = axes[idx]

    # Plot data points
    ax.scatter(microchip_test.loc[neg, 'test_1'], microchip_test.loc[neg, 'test_2'],
               c='green', marker='o', s=60, alpha=0.7, label='Pass', edgecolors='black')
    ax.scatter(microchip_test.loc[pos, 'test_1'], microchip_test.loc[pos, 'test_2'],
               c='red', marker='x', s=80, linewidths=2, label='Fail')

    # Plot decision boundary
    theta = results[degree]['theta']
    plotDecisionBoundary(theta, degree, ax)

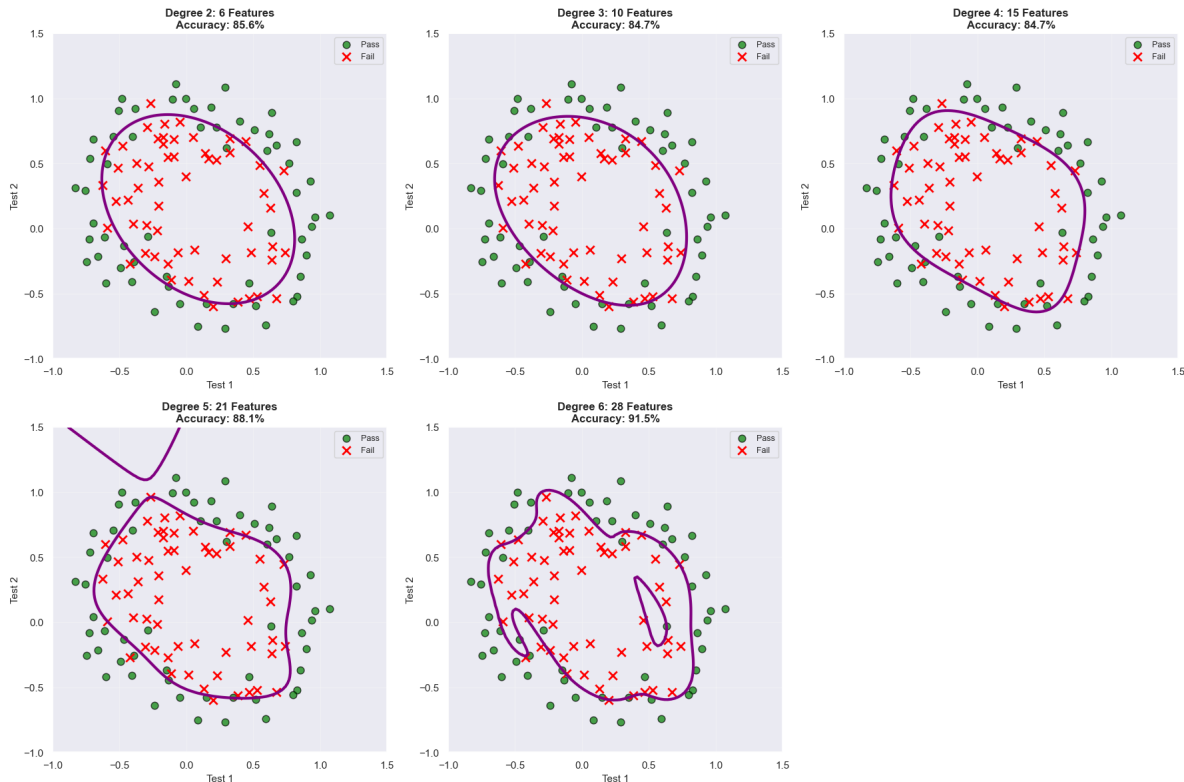
    accuracy = results[degree]['accuracy']
    n_features = results[degree]['n_features']

    ax.set_xlabel('Test 1', fontsize=11)
    ax.set_ylabel('Test 2', fontsize=11)
    ax.set_title(f'Degree {degree}: {n_features} Features\nAccuracy: {accuracy*100:.1f}%',
                 fontweight='bold', fontsize=12)
    ax.legend(fontsize=9, loc='upper right')
    ax.grid(True, alpha=0.3)
    ax.set_xlim(-1, 1.5)
    ax.set_ylim(-1, 1.5)

# Hide the 6th subplot (bottom right)
axes[5].axis('off')

plt.tight_layout()
plt.show()

```



10.1.4 Analysis: Finding the Right Balance

Degree 2 Polynomial (6 features): - **Good fit:** Creates a smooth circular boundary that captures the general pattern - **Good generalization:** The boundary is not overly complex - **Interpretable:** Simple polynomial terms like $x_1^2 + x_2^2$ represent circles

Degree 3 Polynomial (10 features): - **Better fit:** Captures more nuances in the data - **Still reasonable:** Adds some flexibility without excessive complexity - **Slight increase in complexity:** Boundary becomes slightly more irregular

Degree 4 Polynomial (15 features): - **Starting to overfit:** Boundary begins to follow training data too closely - **Increased complexity:** More parameters to learn - **Diminishing returns:** Accuracy gains become smaller

Degree 5 Polynomial (21 features): - **Overfitting evident:** Boundary shows noticeable irregularities - **High complexity:** Many parameters relative to data size - **Poor generalization potential:** Likely to fail on new data

Degree 6 Polynomial (28 features): - **Severe overfitting:** Tries to perfectly classify every training point - **Too complex:** Creates very wiggly, irregular boundaries - **Poor**

generalization: Will likely fail on new data - **Uninterpretable:** 28 features with high-order interactions

Key Lesson: As polynomial degree increases, we see a progression from underfitting → good fit → overfitting. The optimal degree (typically 2-3 for this data) balances model complexity with generalization. **Too many features cause overfitting** - the model memorizes training data rather than learning the underlying pattern.

10.1.5 Summary: Polynomial Feature Transformation

We've seen how **polynomial feature transformation** enables logistic regression to learn nonlinear decision boundaries: Polynomial features work well for capturing geometric nonlinear patterns (circles, ellipses, curves), but they come with tradeoffs: - **Pros:** Captures complex boundaries, mathematically elegant - **Cons:** Feature explosion (grows rapidly with degree), overfitting risk, computational cost

10.1.6 Beyond Polynomial Transformations

While polynomial features are powerful for capturing certain types of nonlinear relationships, they are **not the only tool** for feature engineering in classification problems. Other transformation techniques can be equally or more effective depending on the data characteristics:

1. Log Transformation - Purpose: Handles skewed distributions, reduces impact of outliers - **When to use:** Features with exponential growth patterns or heavy right skew - **Example:** Transforming fare prices, income, or counts

2. Binning Continuous Variables - Purpose: Converts continuous features into categorical groups - **When to use:** Non-smooth relationships, domain-based groupings (age groups, price ranges) - **Benefit:** Captures threshold effects and categorical patterns

3. Interaction Terms - Purpose: Captures combined effects of multiple features - **When to use:** When one feature's impact depends on another (e.g., age effect varies by passenger class)

4. Manually Creating New Features - Purpose: Derive meaningful features by combining or transforming existing ones - **When to use:** Domain knowledge suggests useful feature combinations - **Example:** Creating `family_size` by adding `siblings/spouses` + `parents/children` counts - **Benefit:** Can capture domain-specific patterns more effectively than automatic transformations

In the following sections, we'll demonstrate these transformations using a real-world dataset where **polynomial features alone may not be the best choice**. The Titanic dataset provides an excellent case study because: - Survival patterns involve **categorical groupings**

(e.g., children vs. adults) - Features like fare are **heavily skewed** - Effects are **context-dependent** (e.g., gender's impact varies by class)

Let's explore how these alternative transformations can be more effective than polynomials for real-world classification problems.

10.2 Practical Example: Feature Engineering with Titanic Dataset

Important Learning Objective: This example demonstrates that **not all feature engineering attempts succeed**. Unlike the microchip example where polynomial features dramatically improved performance, we'll see that real-world feature engineering is often a process of trial, testing, and honest evaluation.

10.2.1 Why This Example Is Valuable

The Titanic dataset presents fundamentally different challenges than the microchip example:

Microchip Dataset	Titanic Dataset
Geometric circular pattern	Complex social and demographic patterns
Polynomial features: Dramatic improvement	Feature engineering: Mixed results
2 continuous features	Mix of continuous and categorical features
Clear nonlinear relationship	Already well-captured by raw features

10.2.2 Hypotheses to Test

Based on domain knowledge about Titanic survival, we might expect:

- **Threshold Effects:** Children (age < 12) had higher survival rates → suggests age binning might help
- **Skewed Distributions:** Fare prices are right-skewed → suggests fare grouping might help
- **Interaction Effects:** Gender's impact varied by class → suggests sex × pclass interaction might help
- **Combined Features:** Family size mattered → suggests combining sibsp + parch might help

10.2.3 Our Experimental Approach

We'll systematically test **three feature engineering strategies** and evaluate whether they actually improve performance:

1. **Baseline Model** - Raw features only (establishes performance floor)
2. **Step 1: Combined features** - `family_size = sibsp + parch`
3. **Step 2: Interaction terms** - `sex × pclass`
4. **Step 3: Binned features** - `age_group, fare_group`

Starting features:

```
['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare']
```

Where `sibsp` = siblings/spouses aboard, `parch` = parents/children aboard

10.2.4 The Real Lesson

This example will demonstrate: - **Feature engineering is trial-and-error** - not every logical idea improves performance - **Testing is essential** - domain knowledge suggests features, but data decides if they help - **Sometimes the baseline is hard to beat** - raw features may already capture the patterns well - **Be willing to discard features** - if engineered features don't help, stick with the simpler model

Let's see what actually happens!

10.2.5 Load and Explore the Titanic Dataset

```
# Load the Titanic dataset from seaborn
titanic = sns.load_dataset('titanic')

print(f"\nSurvival distribution:")
print(titanic['survived'].value_counts())
print(f"\nSurvival rate: {titanic['survived'].mean():.2%}")

print(f"Dataset shape: {titanic.shape}")
print(f"\nFirst few rows:")
titanic.head()
```

Survival distribution:
survived
0 549
1 342
Name: count, dtype: int64

Survival rate: 38.38%
Dataset shape: (891, 15)

First few rows:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Na
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Na
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Na

```
# Visualize survival rates by key features
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Survival by Sex
sex_survival = titanic.groupby('sex')['survived'].mean()
axes[0, 0].bar(sex_survival.index, sex_survival.values, color=['steelblue', 'coral'], alpha=0.3)
axes[0, 0].set_xlabel('Sex', fontsize=11)
axes[0, 0].set_ylabel('Survival Rate', fontsize=11)
axes[0, 0].set_title('Survival Rate by Sex', fontweight='bold', fontsize=12)
axes[0, 0].grid(axis='y', alpha=0.3)
for i, v in enumerate(sex_survival.values):
    axes[0, 0].text(i, v + 0.02, f'{v:.2%}', ha='center', va='bottom', fontsize=10)

# Survival by Passenger Class
class_survival = titanic.groupby('pclass')['survived'].mean()
axes[0, 1].bar(class_survival.index, class_survival.values, color=['gold', 'silver', 'green'], alpha=0.3)
axes[0, 1].set_xlabel('Passenger Class', fontsize=11)
axes[0, 1].set_ylabel('Survival Rate', fontsize=11)
axes[0, 1].set_title('Survival Rate by Passenger Class', fontweight='bold', fontsize=12)
axes[0, 1].grid(axis='y', alpha=0.3)
for i, v in enumerate(class_survival.values):
    axes[0, 1].text(i, v + 0.02, f'{v:.2%}', ha='center', va='bottom', fontsize=10)
```

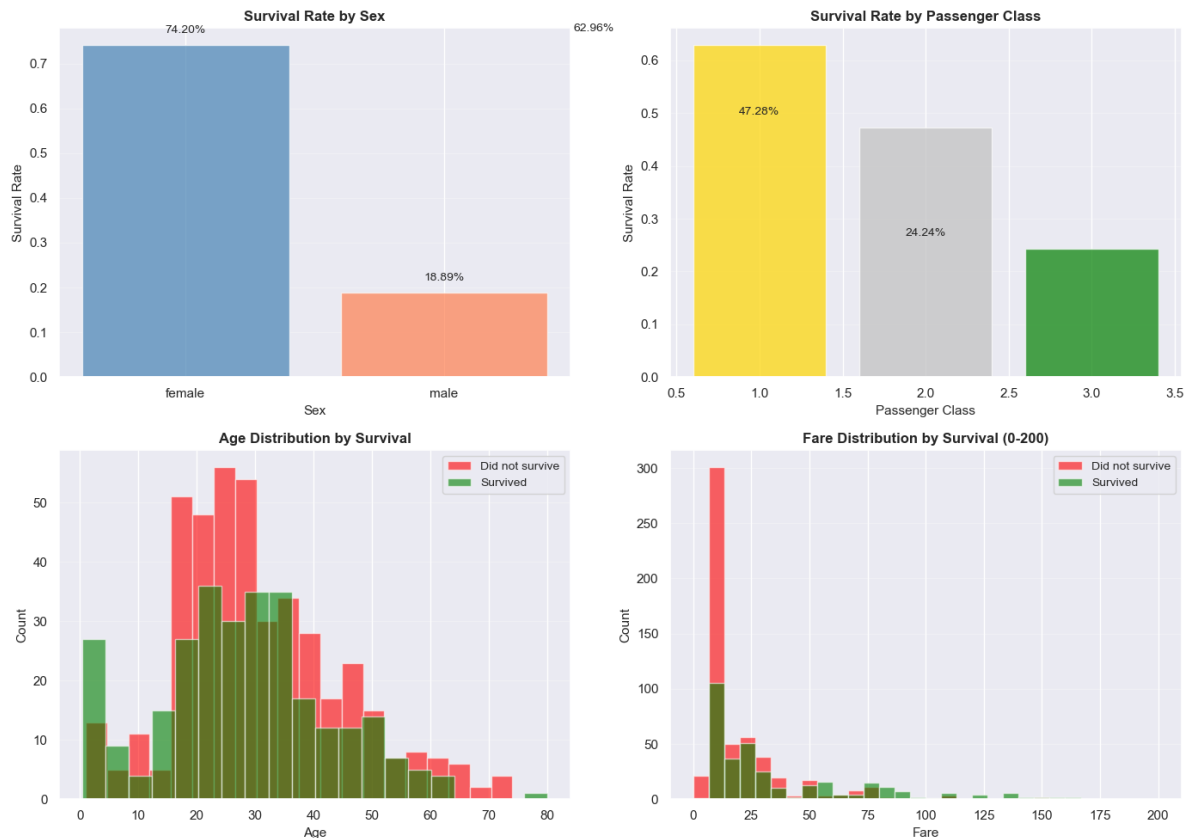
```

# Survival by Age Distribution
axes[1, 0].hist(titanic[titanic['survived']==0]['age'].dropna(), bins=20, alpha=0.6, label='Died')
axes[1, 0].hist(titanic[titanic['survived']==1]['age'].dropna(), bins=20, alpha=0.6, label='Survived')
axes[1, 0].set_xlabel('Age', fontsize=11)
axes[1, 0].set_ylabel('Count', fontsize=11)
axes[1, 0].set_title('Age Distribution by Survival', fontweight='bold', fontsize=12)
axes[1, 0].legend(fontsize=10)
axes[1, 0].grid(axis='y', alpha=0.3)

# Survival by Fare Distribution
axes[1, 1].hist(titanic[titanic['survived']==0]['fare'].dropna(), bins=30, alpha=0.6, label='Died')
axes[1, 1].hist(titanic[titanic['survived']==1]['fare'].dropna(), bins=30, alpha=0.6, label='Survived')
axes[1, 1].set_xlabel('Fare', fontsize=11)
axes[1, 1].set_ylabel('Count', fontsize=11)
axes[1, 1].set_title('Fare Distribution by Survival (0-200)', fontweight='bold', fontsize=12)
axes[1, 1].legend(fontsize=10)
axes[1, 1].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```



Key Observations from Exploration: * Women had much higher survival rates than men (74.20% vs 18.89%) * First class passengers had higher survival rates (62.96%) than third class (24.24%) * Children appear to have higher survival rates (visible in age distribution) * Fare prices are heavily right-skewed (most passengers paid low fares)

10.2.6 Data Preprocessing and Train/Test Split

Before building models, we need to: 1. Select relevant features 2. Handle missing values 3. Encode categorical variables 4. Split into training and test sets

```
from sklearn.model_selection import train_test_split

# Select relevant features for modeling
# We'll use: pclass, sex, age, sibsp (siblings/spouses), parch (parents/children), fare
features_to_use = ['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare']

# Create a clean dataset
titanic_clean = titanic[features_to_use + ['survived']].copy()
```

```

# Handle missing values
print("Missing values before cleaning:")
print(titanic_clean.isnull().sum())

# Fill missing age with median age
titanic_clean['age'] = titanic_clean['age'].fillna(titanic_clean['age'].median())

# Fill missing fare with median fare
titanic_clean['fare'] = titanic_clean['fare'].fillna(titanic_clean['fare'].median())

# Encode categorical variables (sex: male=1, female=0)
titanic_clean['sex'] = (titanic_clean['sex'] == 'male').astype(int)

print("\nMissing values after cleaning:")
print(titanic_clean.isnull().sum())
print(f"\nCleaned dataset shape: {titanic_clean.shape}")
print(f"\nFirst few rows after preprocessing:")
print(titanic_clean.head())

```

Missing values before cleaning:

```

pclass      0
sex          0
age        177
sibsp       0
parch       0
fare        0
survived    0
dtype: int64

```

Missing values after cleaning:

```

pclass      0
sex          0
age         0
sibsp       0
parch       0
fare        0
survived    0
dtype: int64

```

Cleaned dataset shape: (891, 7)

First few rows after preprocessing:

	pclass	sex	age	sibsp	parch	fare	survived
0	3	1	22.0	1	0	7.2500	0
1	1	0	38.0	1	0	71.2833	1
2	3	0	26.0	0	0	7.9250	1
3	1	0	35.0	1	0	53.1000	1
4	3	1	35.0	0	0	8.0500	0

```
# Split into features (X) and target (y)
X = titanic_clean.drop('survived', axis=1)
y = titanic_clean['survived']

# Split into training and test sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")
print(f"\nTraining set survival rate: {y_train.mean():.2%}")
print(f"Test set survival rate: {y_test.mean():.2%}")
print(f"\nFeatures: {list(X.columns)}")
```

Training set size: 712 samples

Test set size: 179 samples

Training set survival rate: 38.34%

Test set survival rate: 38.55%

Features: ['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare']

10.2.7 Model 1: Baseline Model (Raw Features Only)

Let's start with a simple logistic regression model using only the raw features (no transformations). This serves as our baseline for comparison.

```
# Train baseline model with raw features
baseline_model = LogisticRegression(max_iter=1000, random_state=42)
baseline_model.fit(X_train, y_train)

# Make predictions
y_train_pred_baseline = baseline_model.predict(X_train)
y_test_pred_baseline = baseline_model.predict(X_test)
```

```

# Evaluate performance
train_acc_baseline = accuracy_score(y_train, y_train_pred_baseline)
test_acc_baseline = accuracy_score(y_test, y_test_pred_baseline)

print(f"Baseline Model (Raw Features) Performance:")
print(f"  Training Accuracy: {train_acc_baseline:.4f} ({train_acc_baseline*100:.1f}%)" )
print(f"  Test Accuracy: {test_acc_baseline:.4f} ({test_acc_baseline*100:.1f}%)" )

# Display feature coefficients
print(f"\nFeature Coefficients:")
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': baseline_model.coef_[0]
}).sort_values('Coefficient', ascending=False)
print(coef_df.to_string(index=False))

print(f"\nInterpretation:")
print(f"  • Negative coefficient for 'sex': Being male (sex=1) decreases survival probability")
print(f"  • Negative coefficient for 'pclass': Higher class number (3rd class) decreases survival probability")
print(f"  • Features with positive coefficients increase survival probability")

```

Baseline Model (Raw Features) Performance:

Training Accuracy: 0.7921 (79.2%)

Test Accuracy: 0.8045 (80.4%)

Feature Coefficients:

Feature	Coefficient
fare	0.003219
age	-0.038486
parch	-0.100630
sibsp	-0.267350
pclass	-1.051361
sex	-2.612639

Interpretation:

- Negative coefficient for 'sex': Being male (sex=1) decreases survival probability
- Negative coefficient for 'pclass': Higher class number (3rd class) decreases survival probability
- Features with positive coefficients increase survival probability

10.2.8 Testing Feature Engineering Hypotheses

Now we'll **incrementally test** different types of engineered features, one at a time, to see if they actually improve model performance. This is **real-world feature engineering** - we propose features based on domain knowledge, then let the data tell us if they help.

Starting point: 6 raw features $\rightarrow$ ['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare']

Critical mindset: Each step we'll ask: "Does this engineered feature actually help, or should we discard it?"

We'll test: 1. **Combined features** (family_size) - Does total family size matter more than separate counts? 2. **Interaction terms** (sex $\times$ pclass) - Does gender's effect truly vary by class? 3. **Binned features** (age_group, fare_group) - Do categorical bins beat continuous values?

Let's find out what actually works!

10.2.8.1 Step 1: Add Combined Feature (family_size)

Feature Engineering Technique: Combining related features to capture domain-specific patterns

Why: sibsp (siblings/spouses) and parch (parents/children) separately may not be as predictive as knowing the **total family size**. Families traveling together had different survival patterns.

New feature: family_size = sibsp + parch

```
# Step 1: Add family_size feature
X_train_step1 = X_train.copy()
X_test_step1 = X_test.copy()

# Create family_size by combining sibsp and parch
X_train_step1['family_size'] = X_train_step1['sibsp'] + X_train_step1['parch']
X_test_step1['family_size'] = X_test_step1['sibsp'] + X_test_step1['parch']

print(f"Feature Engineering Step 1:")
print(f"    Original features: {X_train.shape[1]}")
print(f"    After adding family_size: {X_train_step1.shape[1]}")
print(f"    New feature: family_size = sibsp + parch")
print(f"\n    Sample values:")
print(X_train_step1[['sibsp', 'parch', 'family_size']].head())
```

```
# Train model
model_step1 = LogisticRegression(max_iter=1000, random_state=42)
model_step1.fit(X_train_step1, y_train)

# Evaluate
y_pred_step1 = model_step1.predict(X_test_step1)
acc_step1 = accuracy_score(y_test, y_pred_step1)

print(f"\n Model Performance:")
print(f"    Test Accuracy: {acc_step1:.4f} ({acc_step1*100:.1f}%)")
print(f"    Improvement over baseline: {(acc_step1 - test_acc_baseline)*100:+.2f}%")
```

Feature Engineering Step 1:

Original features: 6

After adding family_size: 7

New feature: family_size = sibsp + parch

Sample values:

	sibsp	parch	family_size
692	0	0	0
481	0	0	0
527	0	0	0
855	0	1	1
801	1	1	2

Model Performance:

Test Accuracy: 0.8101 (81.0%)

Improvement over baseline: +0.56%

10.2.9 Step 2: Add Interaction Term (sex × pclass)

Feature Engineering Technique: Creating interaction terms to capture context-dependent effects

Why: The “women and children first” policy wasn’t uniform across all classes. First-class women had much higher survival rates than third-class women. The effect of gender **depends on** passenger class.

New feature: sex_pclass = sex × pclass

```

# Step 2: Add interaction term (sex × pclass)
X_train_step2 = X_train_step1.copy()
X_test_step2 = X_test_step1.copy()

# Create interaction between sex and pclass
X_train_step2['sex_pclass'] = X_train_step2['sex'] * X_train_step2['pclass']
X_test_step2['sex_pclass'] = X_test_step2['sex'] * X_test_step2['pclass']

print(f"Feature Engineering Step 2:")
print(f"    Previous features: {X_train_step1.shape[1]}")
print(f"    After adding sex_pclass interaction: {X_train_step2.shape[1]}")
print(f"    New feature: sex_pclass = sex × pclass")
print(f"\n    How it works:")
print(f"    • Male (1) in 3rd class (3): 1 × 3 = 3")
print(f"    • Male (1) in 1st class (1): 1 × 1 = 1")
print(f"    • Female (0) in any class: 0 × pclass = 0")

# Train model
model_step2 = LogisticRegression(max_iter=1000, random_state=42)
model_step2.fit(X_train_step2, y_train)

# Evaluate
y_pred_step2 = model_step2.predict(X_test_step2)
acc_step2 = accuracy_score(y_test, y_pred_step2)

print(f"\n Model Performance:")
print(f"    Test Accuracy: {acc_step2:.4f} ({acc_step2*100:.1f}%)")
print(f"    Improvement over Step 1: {(acc_step2 - acc_step1)*100:+.2f}%")
print(f"    Improvement over baseline: {(acc_step2 - test_acc_baseline)*100:+.2f}%")

```

Feature Engineering Step 2:

Previous features: 7

After adding sex_pclass interaction: 8

New feature: sex_pclass = sex × pclass

How it works:

- Male (1) in 3rd class (3): 1 × 3 = 3
- Male (1) in 1st class (1): 1 × 1 = 1
- Female (0) in any class: 0 × pclass = 0

Model Performance:

Test Accuracy: 0.8101 (81.0%)

Improvement over Step 1: +0.00%
Improvement over baseline: +0.56%

10.2.10 Step 3: Add Binned Features (age_group, fare_group)

Feature Engineering Technique: Converting continuous variables into categorical bins

Why: - **Age:** Children (< 12) had distinctly higher survival rates - a threshold effect that's better captured by categories than smooth curves - **Fare:** Heavily right-skewed with outliers. Grouping into price ranges (low/medium/high/premium) can be more robust than raw values

New features: age_group (5 categories), fare_group (4 categories)

```
# First, let's visualize why binning makes sense for age
age_bins_analysis = [0, 12, 18, 35, 60, 100]
age_labels = ['Child', 'Teen', 'Young Adult', 'Adult', 'Senior']
titanic_clean['age_group'] = pd.cut(titanic_clean['age'], bins=age_bins_analysis, labels=age_labels)

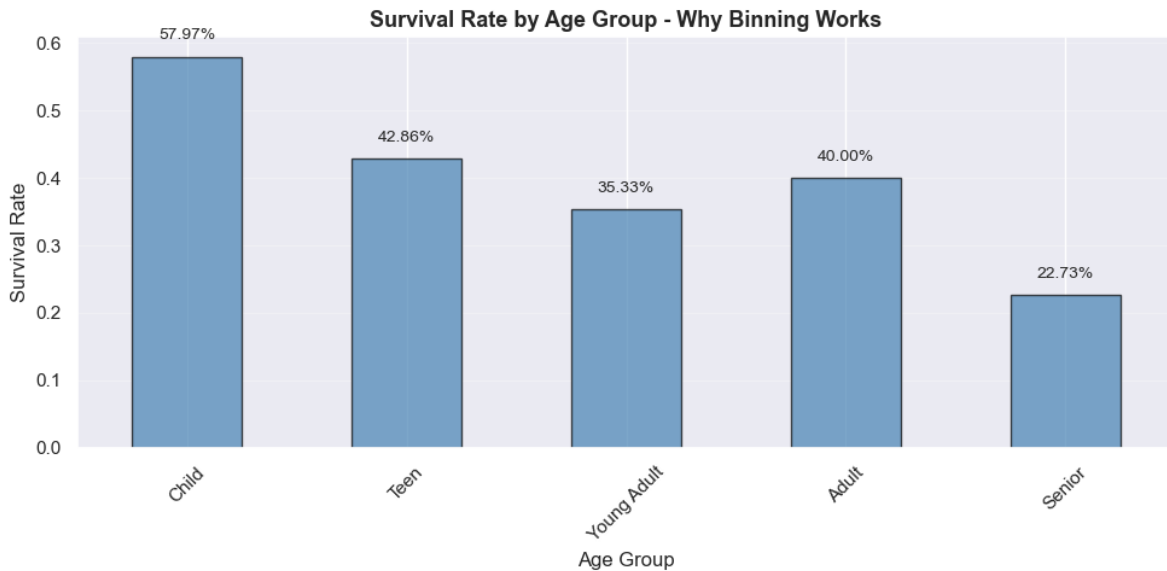
survival_by_group = titanic_clean.groupby('age_group')['survived'].mean()
plt.figure(figsize=(10, 5))
survival_by_group.plot(kind='bar', color='steelblue', alpha=0.7, edgecolor='black')
plt.xlabel('Age Group', fontsize=12)
plt.ylabel('Survival Rate', fontsize=12)
plt.title('Survival Rate by Age Group - Why Binning Works', fontweight='bold', fontsize=13)
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)

# Add value labels on bars
for i, v in enumerate(survival_by_group.values):
    plt.text(i, v + 0.02, f'{v:.2%}', ha='center', va='bottom', fontsize=10)

plt.tight_layout()
plt.show()

print("Survival rates by age group:")
for group, rate in survival_by_group.items():
    print(f"    • {group}: {rate:.2%}")
print("\nNotice the clear threshold: Children have distinctly higher survival!")
```

```
C:\Users\lsi8012\AppData\Local\Temp\ipykernel_11776\3460317799.py:6: FutureWarning: The default
survival_by_group = titanic_clean.groupby('age_group')['survived'].mean()
```



Survival rates by age group:

- Child: 57.97%
- Teen: 42.86%
- Young Adult: 35.33%
- Adult: 40.00%
- Senior: 22.73%

Notice the clear threshold: Children have distinctly higher survival!

```
# Step 4: Add binned features
X_train_step3 = X_train_step2.copy()
X_test_step3 = X_test_step2.copy()

# Create age bins
age_bins = [0, 12, 18, 35, 60, 100]
X_train_step3['age_group'] = pd.cut(X_train_step3['age'], bins=age_bins, labels=[0, 1, 2, 3, 4])
X_test_step3['age_group'] = pd.cut(X_test_step3['age'], bins=age_bins, labels=[0, 1, 2, 3, 4])
X_train_step3['age_group'] = X_train_step3['age_group'].astype(int)
X_test_step3['age_group'] = X_test_step3['age_group'].astype(int)

# Create fare bins
fare_bins = [0, 10, 30, 100, 1000]
X_train_step3['fare_group'] = pd.cut(X_train_step3['fare'], bins=fare_bins, labels=[0, 1, 2, 3, 4])
X_test_step3['fare_group'] = pd.cut(X_test_step3['fare'], bins=fare_bins, labels=[0, 1, 2, 3, 4])
X_train_step3['fare_group'] = X_train_step3['fare_group'].astype(int)
```

```

X_test_step3['fare_group'] = X_test_step3['fare_group'].astype(int)

print(f"Feature Engineering Step 4:")
print(f"    Previous features: {X_train_step2.shape[1]}")
print(f"    After adding binned features: {X_train_step3.shape[1]}")
print(f"    New features: age_group, fare_group")
print(f"\n    Age groups: 0=Child, 1=Teen, 2=Young Adult, 3=Adult, 4=Senior")
print(f"    Fare groups: 0=Low(0-10), 1=Medium(10-30), 2=High(30-100), 3=Premium(100+)")

# Train model
model_step3 = LogisticRegression(max_iter=1000, random_state=42)
model_step3.fit(X_train_step3, y_train)

# Evaluate
y_pred_step3 = model_step3.predict(X_test_step3)
acc_step3 = accuracy_score(y_test, y_pred_step3)

# Train model with binned features
model_step4 = LogisticRegression(max_iter=1000, random_state=42)
model_step4.fit(X_train_step3, y_train)

# Evaluate
y_pred_step4 = model_step4.predict(X_test_step3)
acc_step4 = accuracy_score(y_test, y_pred_step4)

print(f"\n Model Performance:")
print(f"    Test Accuracy: {acc_step4:.4f} ({acc_step4*100:.1f}%)")
print(f"    Improvement over Step 3: {(acc_step4 - acc_step2)*100:+.2f}%")
print(f"    Improvement over baseline: {(acc_step4 - test_acc_baseline)*100:+.2f}%")

print(f"\nFinal feature list ({X_train_step3.shape[1]} features):")
print(f"    {list(X_train_step3.columns)}")

```

Feature Engineering Step 4:

Previous features: 8

After adding binned features: 10

New features: age_group, fare_group

Age groups: 0=Child, 1=Teen, 2=Young Adult, 3=Adult, 4=Senior

Fare groups: 0=Low(0-10), 1=Medium(10-30), 2=High(30-100), 3=Premium(100+)

Model Performance:

Test Accuracy: 0.8045 (80.4%)
Improvement over Step 3: -0.56%
Improvement over baseline: +0.00%

Final feature list (10 features):

['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'family_size', 'sex_pclass', 'age_group']

10.2.11 Step-by-Step Performance Comparison

Now let's honestly evaluate whether each feature engineering technique actually improved performance. **Spoiler:** Not all of them did! This is the reality of feature engineering.

```
# Create comparison dataframe showing step-by-step improvements
steps_comparison = pd.DataFrame({
    'Step': ['Baseline\n(6 raw features)',
            'Step 1\n+ family_size',
            'Step 2\n+ sex×pclass',
            'Step 3\n+ age/fare bins'],
    'Features': [X_train.shape[1],
                 X_train_step1.shape[1],
                 X_train_step2.shape[1],
                 X_train_step3.shape[1]],
    'Test Accuracy': [test_acc_baseline, acc_step1, acc_step2, acc_step4]
})

print("="*80)
print("STEP-BY-STEP FEATURE ENGINEERING RESULTS")
print("="*80)
print(steps_comparison.to_string(index=False))
print("="*80)

# Calculate incremental improvements
print("\nINCREMENTAL IMPROVEMENTS:")
for i in range(1, len(steps_comparison)):
    prev_acc = steps_comparison.loc[i-1, 'Test Accuracy']
    curr_acc = steps_comparison.loc[i, 'Test Accuracy']
    improvement = (curr_acc - prev_acc) * 100
    step_name = steps_comparison.loc[i, 'Step'].replace('\n', ' ')
    print(f"    {step_name}: {improvement:+.2f}%")

print(f"\nTOTAL IMPROVEMENT FROM BASELINE:")
total_improvement = (acc_step4 - test_acc_baseline) * 100
```

```

print(f"    {total_improvement:+.2f}% (from {test_acc_baseline:.1%} to {acc_step4:.1%})")

# Visualize the progression
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Left plot: Accuracy progression
x = np.arange(len(steps_comparison))
bars = ax1.bar(x, steps_comparison['Test Accuracy'], alpha=0.8,
              color=['gray', 'lightblue', 'skyblue', 'steelblue', 'darkblue'],
              edgecolor='black', linewidth=1.5)

ax1.set_xlabel('Feature Engineering Step', fontsize=12, fontweight='bold')
ax1.set_ylabel('Test Accuracy', fontsize=12, fontweight='bold')
ax1.set_title('Step-by-Step Performance Improvement', fontweight='bold', fontsize=14)
ax1.set_xticks(x)
ax1.set_xticklabels(steps_comparison['Step'], fontsize=10)
ax1.set_ylim(0.75, 0.85)
ax1.grid(axis='y', alpha=0.3)

# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height + 0.003,
            f'{height:.3f}',
            ha='center', va='bottom', fontsize=10, fontweight='bold')

# Add arrows showing improvements
for i in range(len(steps_comparison) - 1):
    x_pos = i + 0.5
    y_start = steps_comparison.loc[i, 'Test Accuracy']
    y_end = steps_comparison.loc[i+1, 'Test Accuracy']
    improvement = (y_end - y_start) * 100

    # Show both positive and negative changes
    if abs(improvement) > 0.01: # Only show if change is noticeable
        arrow_color = 'green' if improvement > 0 else 'red'
        text_color = 'darkgreen' if improvement > 0 else 'darkred'
        sign = '+' if improvement > 0 else '-'

    ax1.annotate('', xy=(x_pos + 0.3, y_end - 0.002), xytext=(x_pos, y_start + 0.002),
                arrowprops=dict(arrowstyle='->', color=arrow_color, lw=2))
    ax1.text(x_pos + 0.15, (y_start + y_end) / 2, f'{sign}{improvement:.2f}%',

```



```

        fontsize=9, color=text_color, fontweight='bold', ha='center',
        bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.8))

# Right plot: Feature count progression
bars2 = ax2.bar(x, steps_comparison['Features'], alpha=0.8,
               color=['gray', 'lightblue', 'skyblue', 'steelblue', 'darkblue'],
               edgecolor='black', linewidth=1.5)

ax2.set_xlabel('Feature Engineering Step', fontsize=12, fontweight='bold')
ax2.set_ylabel('Number of Features', fontsize=12, fontweight='bold')
ax2.set_title('Feature Count Growth', fontweight='bold', fontsize=14)
ax2.set_xticks(x)
ax2.set_xticklabels(steps_comparison['Step'], fontsize=10)
ax2.grid(axis='y', alpha=0.3)

# Add value labels on bars
for bar in bars2:
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height + 0.2,
             f'{int(height)}',

             ha='center', va='bottom', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.show()

```

```
=====
```

STEP-BY-STEP FEATURE ENGINEERING RESULTS

```
=====
```

	Step	Features	Test Accuracy
Baseline\n(6 raw features)		6	0.804469
Step 1\n+ family_size		7	0.810056
Step 2\n+ sex×pclass		8	0.810056
Step 3\n+ age/fare bins		10	0.804469

```
=====
```

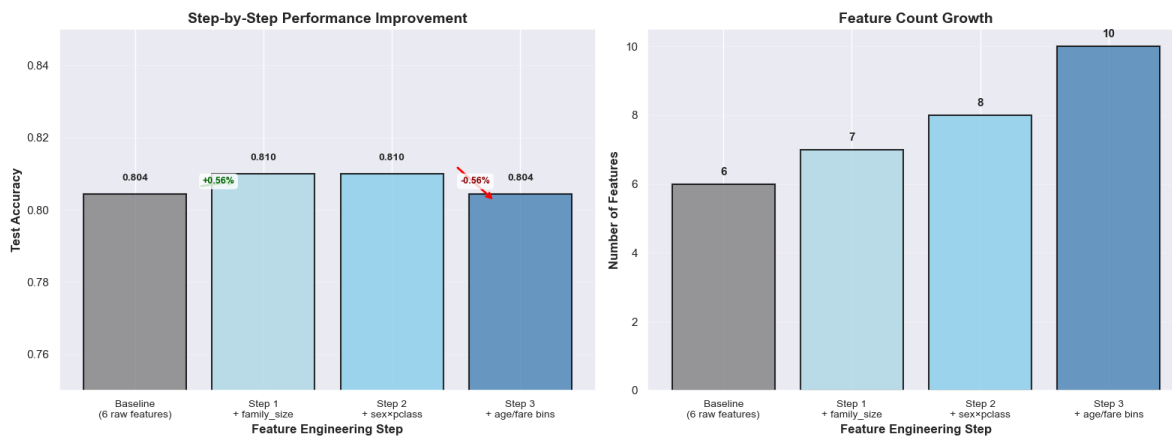
INCREMENTAL IMPROVEMENTS:

```

Step 1 + family_size: +0.56%
Step 2 + sex×pclass: +0.00%
Step 3 + age/fare bins: -0.56%

```

TOTAL IMPROVEMENT FROM BASELINE:
+0.00% (from 80.4% to 80.4%)



10.3 Key Insights: What We Actually Learned

10.3.1 Why Didn't Our Features Help More?

- 1. Strong Baseline** - The 6 raw features already captured survival patterns well (~80% accuracy). Titanic survival was driven by clear factors that didn't require complex transformations.
- 2. Feature Engineering Isn't Always Necessary** - Unlike the microchip dataset (where polynomial features gave huge gains), these patterns were already well-represented. Adding complexity doesn't help when simpler features work.
- 3. Binning Can Lose Information** - Converting continuous values to discrete bins loses granularity. Only bin when there's strong evidence of threshold effects.
- 4. Model Limitations** - Logistic regression assumes linear relationships between features and log-odds. Tree-based models (Random Forest, XGBoost) might capture interaction patterns better.

10.3.2 Key Takeaway

Feature engineering is **trial-and-error**, not magic. The most important skill is **honestly evaluating** whether engineered features actually improve performance. In this case, the baseline was hard to beat, and that's okay! Real data science means reporting what actually happened, not what we hoped would happen.

10.3.3 Next Steps: Regularization

One technique to prevent overfitting (especially when using many features) is **regularization**. The idea is to penalize the algorithm when it tries to overfit by adding a regularization term to the cost function.

The New Cost function with regularization is specified as:

$$J(w) = \frac{1}{m} \sum_{i=1}^m [-y_i \log(h_w(z_i)) - (1 - y_i) \log(1 - h_w(z_i))] + \frac{\lambda}{2m} \sum_{j=1}^n [w_j^2]$$

where: - λ = regularization factor (controls how much to penalize large weights) - n = number of features - **Note:** The regularization term does NOT include the intercept term w_0

Regularization will be covered in the next section!

11 Cross-Validation

<IPython.core.display.Image object>

11.1 Review: Train-Test Split and Its Limitations

11.1.1 Train-Test Split Recap

Throughout this course, we have used the train-test split approach to evaluate models:

- We split the dataset into training and testing sets.
- The training set is used to fit the model.
- The testing set is used to evaluate performance on unseen data.

This approach provides a simple yet effective way to estimate out-of-sample performance. However, it has limitations:

11.1.2 Limitations of Train-Test Split

- **High Variance:** The model's performance depends on which specific observations end up in the training and testing sets. A different split might lead to different results.
- **Data Efficiency:** A portion of the dataset is reserved for testing, meaning the model is not trained on all available data, which can be problematic for small datasets.
- **Instability:** A single split does not always provide a robust estimate of performance, especially when data is imbalanced or noisy.

```
import numpy as np
import pandas as pd
import random
import seaborn as sns
import statsmodels.api as sm
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```

from sklearn.metrics import roc_auc_score, f1_score, accuracy_score

from sklearn.datasets import make_classification

%matplotlib inline
plt.style.use('ggplot')
from matplotlib.pyplot import rcParams
rcParams['figure.figsize'] = 5, 4

# Generate synthetic dataset for binary classification
X, y = make_classification(n_samples=1000, n_features=10, random_state=42)

```

Please execute the cell below multiple times. It will be evident that the accuracy score varies with each run due to different observations in both the training and test sets.

```

# use train/test split
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(X, y)

# Define the model
logit_model = LogisticRegression()
logit_model.fit(X_train_class, y_train_class)
y_pred = logit_model.predict(X_test_class)
print(accuracy_score(y_test_class, y_pred))

```

0.856

11.2 Cross-Validation: Key Concepts

To address the limitations of train-test split, we introduce cross-validation, which provides a more reliable estimate of model performance by using multiple train-test splits.

11.2.1 Steps for K-fold cross-validation

1. Split the dataset into K **equal** partitions (or “folds”).
2. Use fold 1 as the **testing set** and the union of the other folds as the **training set**.
3. Calculate **testing accuracy**.
4. Repeat steps 2 and 3 K times, using a **different fold** as the testing set each time.
5. Use the **average testing accuracy** as the estimate of out-of-sample accuracy.

Diagram of **5-fold cross-validation**:

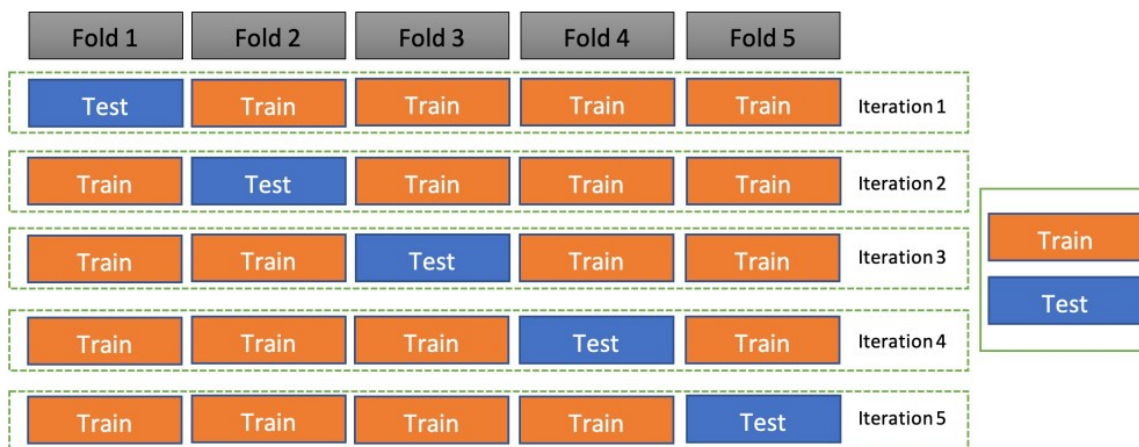


Figure 11.1: 5-fold cross-validation

```
# simulate splitting a dataset of 25 observations into 5 folds
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=False).split(range(25))

# print the contents of each training and testing set
print('{ } {:~61} {}'.format('Iteration', 'Training set observations', 'Testing set observations'))
for iteration, data in enumerate(kf, start=1):
    print('{:~9} {} {:~25}'.format(iteration, data[0], str(data[1])))
```

Iteration	Training set observations	Testing set observations
1	[5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]	[0 1 2 3 4]
2	[0 1 2 3 4 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]	[5 6 7 8 9]
3	[0 1 2 3 4 5 6 7 8 9 15 16 17 18 19 20 21 22 23 24]	[10 11 12 13 14]
4	[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 20 21 22 23 24]	[15 16 17 18 19]
5	[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19]	[20 21 22 23 24]

Key takeaways

- The dataset consists of **25 observations** (indexed from 0 to 24).
- We use **5-fold cross-validation**, meaning the process runs for **5 iterations** (one per fold).
- In each iteration:

- The dataset is split into a **training set** and a **testing set**.
- Each observation is included in either the training set or the testing set, **but never both simultaneously**.
- Over the entire cross-validation process:
 - Each observation appears in the **testing set exactly once**.
 - Each observation is included in the **training set for $(K-1) = 4$ iterations**.

This ensures that every data point contributes to both model training and evaluation, improving the robustness of performance estimates.

11.3 Cross-Validation in Scikit-Learn

11.3.1 `cross_val_score`

`cross_val_score` is a function in **Scikit-Learn** that simplifies **k-fold cross-validation** for model evaluation. It automates the process of splitting the dataset, training the model, and computing performance metrics across multiple folds.

- By default, it uses **5-fold cross-validation** (`cv=5`).
- For **classification models**, it evaluates performance using **accuracy** as the default scoring metric.
- For **regression models**, it uses **R^2 (coefficient of determination)** by default.
- The function returns an array of scores, one for each fold, providing a more reliable estimate of model performance than a single train-test split.

Using `cross_val_score` ensures a more robust evaluation by reducing variance and making better use of available data.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(logit_model, X, y)

print(scores)
```

```
[0.87  0.855 0.85  0.83  0.875]
```

Finally, we compute the **mean performance score** across all folds to obtain a robust evaluation.

```
# get the mean score
scores.mean()
```

```
0.8560000000000001
```

By default, `cross_val_score` in **Scikit-Learn** performs **k-fold cross-validation** using the default cross-validator for the given estimator type.

11.3.1.1 Default k-fold cross-validation Settings

`cv=5` unless specified

Model Type	Default Cross-Validator	Shuffling	Stratification
Classification	<code>StratifiedKFold(n_splits=5)</code>	No	Yes
Regression	<code>KFold(n_splits=5)</code>	No	No

11.3.1.2 How to Modify the Behavior

If you need shuffling or a different cross-validation strategy, specify a custom cross-validator.

11.3.1.2.1 Enable Shuffling

```
# Turn on the shuffle
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(logit_model, X, y, cv=kf, scoring='accuracy')
scores
```

```
array([0.83 , 0.875, 0.87 , 0.85 , 0.865])
```

11.3.1.2.2 Stratified Splitting for Classification

For classification problems, **stratified sampling** is recommended for creating the folds

- Each response class should be represented with equal proportions in each of the K folds


```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(logit_model, X, y, cv=skf, scoring='accuracy')
scores
```

```
array([0.83 , 0.84 , 0.875, 0.89 , 0.845])
```

11.3.1.2.3 Use Leave-One-Out (LOO) Cross-Validation

If you don't have much data, so any split from the full set to the training and validation set is going to result in really very few observations on which you can train. Leave-on-out cross validation (LOOCV) that might work better for cross-validation. Say you have 16 observations. Train on 15 and validate on the other one. Repeat this until you have trained on every set of 15 with the 16th sample left out.

```
from sklearn.model_selection import LeaveOneOut

loo = LeaveOneOut()
scores = cross_val_score(logit_model, X, y, cv=loo, scoring='accuracy')
print(len(scores))
scores[:10]
```

```
1000
```

```
array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

Types of Cross-Validation

Method	Description	Use Case
k-Fold CV	Splits data into k folds, each used for training/testing	Most common approach (e.g., k=5 or k=10)
Stratified k-Fold	Like k-Fold, but preserves class proportions	Useful for imbalanced classification
Leave-One-Out (LOO-CV)	Uses each sample as a test set once	For very small datasets
Leave-P-Out (LPO-CV)	Leaves out p samples for testing in each iteration	Computationally expensive

Method	Description	Use Case
Time Series CV	Ensures training data precedes test data (rolling windows)	Time-series forecasting problems

11.3.1.3 Changing the Scoring Metric in `cross_val_score`

we can specify a different evaluation metric using the `scoring` parameter. Scikit-Learn provides various built-in metrics, including **F1-score**, precision, recall, and more.

You can specify different metrics based on the type of model:

Task Type	Metric Name (for <code>scoring</code>)	Description
Classification	'accuracy'	Default, ratio of correct predictions
Classification	'precision', 'recall'	Measure of correctness for positive class
Classification	'f1', 'f1_macro'	Harmonic mean of precision & recall
Regression	'neg_mean_squared_error'	MSE (lower is better)
Regression	'r2'	Default, measures variance explained

You can refer to the scikit-learn [documentation](#). If a built-in metric doesn't fit your needs, you can define a custom scoring function

For classification problems, especially imbalanced datasets, **F1-score** is a better metric than accuracy as it considers both **precision and recall**. Let's use it as our metric next

```
# create a KFold object with 5 splits
folds = KFold(n_splits = 5, shuffle = True, random_state = 100)
scores = cross_val_score(logit_model, X, y, cv=folds, scoring= "f1")
scores
```

```
array([0.875      , 0.81632653, 0.88151659, 0.87958115, 0.85869565])
```

11.3.2 cross_validate in Scikit-Learn

The `cross_validate` function provides a **more comprehensive evaluation** of a model compared to `cross_val_score`.

Its Key Feature includes:

- Allows **multiple evaluation metrics** to be specified at once.
- Returns a **detailed dictionary** containing:
 - **Training scores** and **test scores** across different folds.
 - **Fit times** (time taken to train the model for each fold).
 - **Score times** (time taken to evaluate the model for each fold).
- Useful for:
 - **Analyzing training/testing time variability** across folds.
 - **Comparing multiple performance metrics simultaneously** to get a more complete picture of model performance.

This function is ideal when you need **deeper insights** into how your model behaves across different folds, beyond just performance scores.

```
from sklearn.model_selection import cross_validate

scores = cross_validate(logit_model, X, y, scoring="accuracy", return_train_score=True)

df_scores = pd.DataFrame(scores)
df_scores
```

	fit_time	score_time	test_score	train_score
0	0.007137	0.001002	0.870	0.86125
1	0.006532	0.000997	0.855	0.86750
2	0.004417	0.000998	0.850	0.87375
3	0.001992	0.001005	0.830	0.87000
4	0.001035	0.000000	0.875	0.85750

Let's use multiple metrics

```
# Define scoring metrics explicitly for multiclass
scoring = ['accuracy', 'recall', 'precision', 'f1', 'roc_auc']

# Perform cross-validation
```

```
scores = cross_validate(logit_model, X, y, scoring=scoring, return_train_score=True)

# Convert to DataFrame for better readability
df_scores = pd.DataFrame(scores)
df_scores
```

	fit_time	score_time	test_accuracy	train_accuracy	test_recall	train_recall	test_precision	train_precision
0	0.003802	0.000000	0.870	0.86125	0.868687	0.852500	0.868687	0.868687
1	0.004556	0.004991	0.855	0.86750	0.810000	0.862155	0.890110	0.871429
2	0.003989	0.004989	0.850	0.87375	0.840000	0.867168	0.857143	0.871429
3	0.003987	0.004063	0.830	0.87000	0.840000	0.862155	0.823529	0.871429
4	0.000000	0.011978	0.875	0.85750	0.890000	0.847118	0.864078	0.868687

11.3.3 cross_val_predict in sklearn

`cross_val_predict` is a function in Scikit-Learn that performs cross-validation but instead of returning evaluation scores, it returns predicted values for each instance in the dataset as if they were unseen.

11.3.3.1 Process:

1. The dataset is split into **k folds**.
2. The model is trained on **k-1 folds**.
3. Predictions are made on the **remaining fold**.
4. This process repeats **until every instance has been predicted once**, ensuring that each prediction is made on unseen data.

This approach **mimics real-world predictions**, making it useful for evaluating model performance with **classification reports, confusion matrices, and ROC curves**.

```
from sklearn.model_selection import cross_val_predict

# output the predicted probabilities
y_pred_prob = cross_val_predict(logit_model, X, y, cv=5, method='predict_proba')[:,1]
print(y_pred_prob.shape)
print(y_pred_prob[:10])
```

```
(1000,)
[0.04211674 0.97024937 0.96688462 0.01087727 0.65634726 0.02306063
 0.14247751 0.92610262 0.8783967 0.2408313 ]
```

11.3.4 Key Differences Between `cross_val_score`, `cross_validate`, and `cross_val_predict`

Function	Returns	Requires <code>scoring</code> ?	Supports Multiple Metrics?	Can Return Train Scores?	Purpose
<code>cross_val_score</code>	Array of scores	No (defaults to accuracy for classification)	No	No	Evaluates model performance using cross-validation
<code>cross_validate</code>	Dictionary	Yes (must specify)	Yes (<code>scoring={'accuracy', 'precision'}</code>)	Yes (<code>return_train_score=True</code>)	Provides tailored evaluation metrics, including training times
<code>cross_val_predict</code>	Array of predictions	No (uses <code>predict</code> or <code>predict_proba</code>)	No	No	Generates out-of-sample predictions for each instance

11.3.5 Advantages and Disadvantages of Cross-Validation

11.3.5.1 Advantages:

- **Reduces Variance:** By averaging results over multiple folds, cross-validation provides a more stable estimate of model performance.
- **Better Use of Data:** Every observation gets a chance to be in both the training and testing sets, improving data efficiency.
- **More Reliable Performance Metrics:** The results are less dependent on a single random split, making the evaluation more robust.

11.3.5.2 Disadvantages Compared to Train-Test Split:

- **Higher Computational Cost:** Instead of training the model once (as in a train-test split), cross-validation requires training the model **k times**, making it computationally expensive for large datasets.
- **Potential Overfitting to Small Data:** If **k** is too large (e.g., Leave-One-Out Cross-Validation), it can lead to high variance and make the model too sensitive to small changes in data.
- **Choosing k:** Typically, **k=5** or **k=10** is recommended as a balance between bias and variance.

Cross-validation is **preferable** for small-to-moderate datasets where stable performance estimates are important, while a simple **train-test split** is often sufficient for large datasets when computational efficiency is a priority.

11.4 Cross-Validation for Hyperparameter Tuning

Cross-validation is a powerful technique that can be used for:

- **Model Performance Evaluation:** Provides a more reliable estimate of how well a model generalizes to unseen data.
- **Hyperparameter Tuning:** Helps find the optimal model parameters by evaluating different configurations.

- **Model Selection:** Compares multiple models to choose the one that performs best across different folds.
- **Feature Selection:** Assesses the impact of different feature subsets on model performance.

In this notebook, we focus exclusively on **Hyperparameter Tuning with Cross-Validation**.

We demonstrate how cross-validation can be used to systematically search for the best hyperparameters, ensuring **better generalization** and **optimized model performance**.

11.4.1 Finding the optimal Degree in Polynomial Regression

11.4.1.1 Background: Polynomial Regression

You already know simple linear regression:

$$y = \beta_0 + \beta_1 x_1$$

In polynomial regression of degree n , we fit a curve of the form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \beta_3 x_1^3 \dots + \beta_n x_1^n$$

In the experiment below, we fit polynomial models of varying degrees on simulated data to analyze their performance.

We build a **linear regression model** and use **cross-validation** to tune the polynomial degree (p).

By selecting the optimal degree, we aim to **balance the trade-off between underfitting and overfitting**, ensuring the model generalizes well to unseen data.

Specifically, we Will Cover:

- Using `cross_val_score` for hyperparameter tuning to evaluate model performance across folds.
- Using `cross_validate` to obtain detailed metrics, including training scores and fit times.
- Applying `GridSearchCV` to systematically search for the optimal polynomial degree.

11.4.1.2 Finding the Optimal Degree with cross_val_score

11.4.1.2.1 Step 1: Let's begin by importing the required libraries.

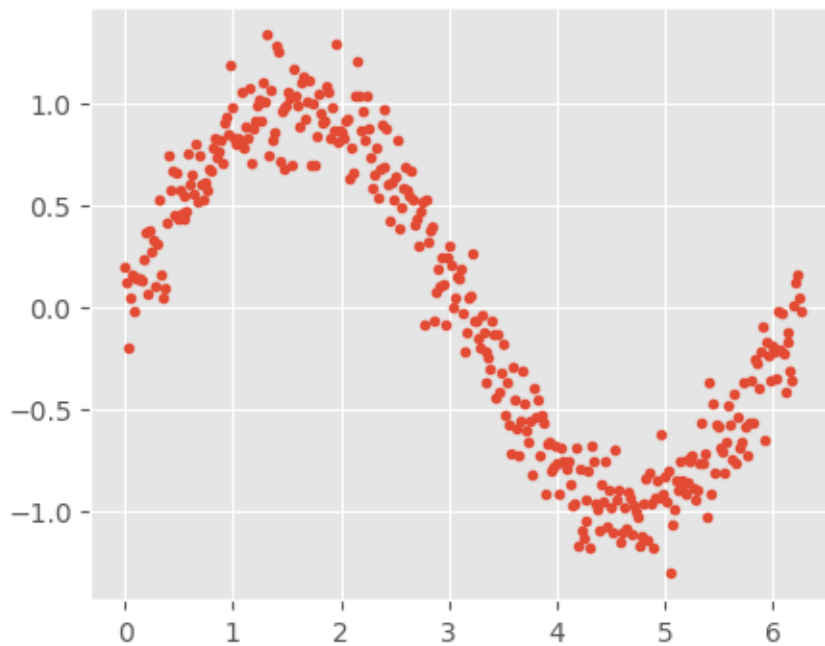
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, PolynomialFeatures, OneHotEncoder
```

11.4.1.2.2 Step 2: Let's generate input data

```
# Simulate input data

#Define input array with angles from 60deg to 300deg converted to radians
x = np.array([i*np.pi/180 for i in range(360)])
np.random.seed(10) #Setting seed for reproducibility
y = np.sin(x) + np.random.normal(0,0.15,len(x))
data = pd.DataFrame(np.column_stack([x,y]),columns=['x','y'])
plt.plot(data['x'],data['y'],'.');
```



11.4.1.2.3 Step 3: Train-Test Split

```
# Split the data
X = data['x'].values.reshape(-1, 1)
y = data['y'].values

# train test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

11.4.1.2.4 Step 4: Using cross_val_score to tune the p in polynomialFeatures

In sklearn, polynomial features can be generated using the `PolynomialFeatures` class. Also, to perform `LinearRegression` and `PolynomialFeatures` in tandem, we will use the module `sklearn.pipeline` - it basically creates the features and feeds the output to the model (in that sequence).

```
def cross_validation_score(X_train, y_train, max_degree=10, scoring='r2'):
    """
    Perform cross-validation for polynomial regression models with different degrees.

    Parameters:
        X_train (array-like): Training feature data.
        y_train (array-like): Training target data.
        max_degree (int): Maximum polynomial degree to evaluate.
        scoring (str): Scoring metric ('r2' for R2, 'rmse' for root mean squared error).

    Returns:
        degrees (list): List of polynomial degrees evaluated.
        scores_df (DataFrame): DataFrame of cross-validation scores across different degrees
    """
    degrees = range(1, max_degree + 1)
    cv_scores = []

    for degree in degrees:
        # Create polynomial regression model
        model = make_pipeline(
            PolynomialFeatures(degree),
            LinearRegression()
        )

        # Compute cross-validation scores
        if scoring == 'rmse':
```

```

        raw_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_sq
        cv_score = np.sqrt(-raw_scores) # Convert negative MSE to RMSE
        score_label = "RMSE (lower is better)"
    else: # Default to R2
        cv_score = cross_val_score(model, X_train, y_train, cv=5, scoring='r2')
        score_label = "R2 (higher is better)"

    cv_scores.append(cv_score)

# Convert scores to a DataFrame
scores_df = pd.DataFrame(np.array(cv_scores), index=degrees, columns=[f'Fold {i+1}' for i in range(5)])

print(f"Cross-validation scores ({score_label}):")
return degrees, scores_df

```

```

# Example Usage:
scoring_metric = 'rmse' # Change to 'r2' if needed
degrees, scores = cross_validation_score(X_train, y_train, scoring="rmse")

# Print the formatted score matrix
print(scores)

```

Cross-validation scores (RMSE (lower is better)):

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	0.500794	0.456810	0.472232	0.484231	0.461889
2	0.500255	0.455897	0.475378	0.485450	0.461456
3	0.164968	0.158555	0.156423	0.152829	0.161931
4	0.165521	0.158619	0.156413	0.153359	0.162379
5	0.145887	0.138576	0.143649	0.141445	0.152968
6	0.145995	0.139056	0.143390	0.144567	0.153870
7	0.151113	0.138186	0.144122	0.144211	0.153438
8	0.150406	0.137350	0.143297	0.146944	0.154557
9	0.151095	0.138084	0.143219	0.146828	0.154599
10	0.152122	0.139420	0.145089	0.147192	0.154581

```

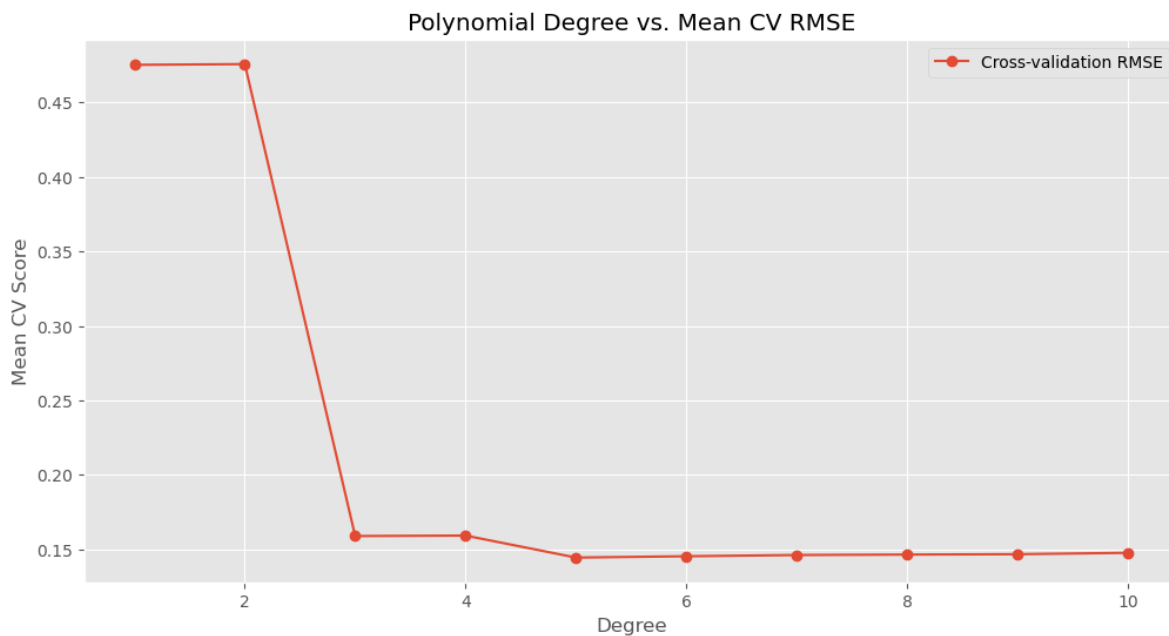
# take mean of each row and add it as the last column of the scores dataframe
scores['mean'] = scores.mean(axis=1)
print(scores)

```

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	mean
1	0.500794	0.456810	0.472232	0.484231	0.461889	0.475191

2	0.500255	0.455897	0.475378	0.485450	0.461456	0.475687
3	0.164968	0.158555	0.156423	0.152829	0.161931	0.158941
4	0.165521	0.158619	0.156413	0.153359	0.162379	0.159258
5	0.145887	0.138576	0.143649	0.141445	0.152968	0.144505
6	0.145995	0.139056	0.143390	0.144567	0.153870	0.145376
7	0.151113	0.138186	0.144122	0.144211	0.153438	0.146214
8	0.150406	0.137350	0.143297	0.146944	0.154557	0.146511
9	0.151095	0.138084	0.143219	0.146828	0.154599	0.146765
10	0.152122	0.139420	0.145089	0.147192	0.154581	0.147681

```
# plot the mean scores for each degree
plt.figure(figsize=(12, 6))
plt.plot(degrees, scores['mean'], 'o-', label='Cross-validation RMSE')
plt.xlabel('Degree')
plt.ylabel('Mean CV Score')
plt.title('Polynomial Degree vs. Mean CV RMSE')
plt.grid(True)
plt.legend();
```



```
# Find optimal degree based on the mean cross-validation scores
optimal_degree = scores['mean'].idxmin()
print(f'Optimal polynomial degree: {optimal_degree}')
```

Optimal polynomial degree: 5

```
# use r2 as performance metric, note that r2 is default metric in the function definition
degrees, scores = cross_validation_score(X_train, y_train)

# Print the formatted score matrix
print(scores)
```

Cross-validation scores (R^2 (higher is better)):

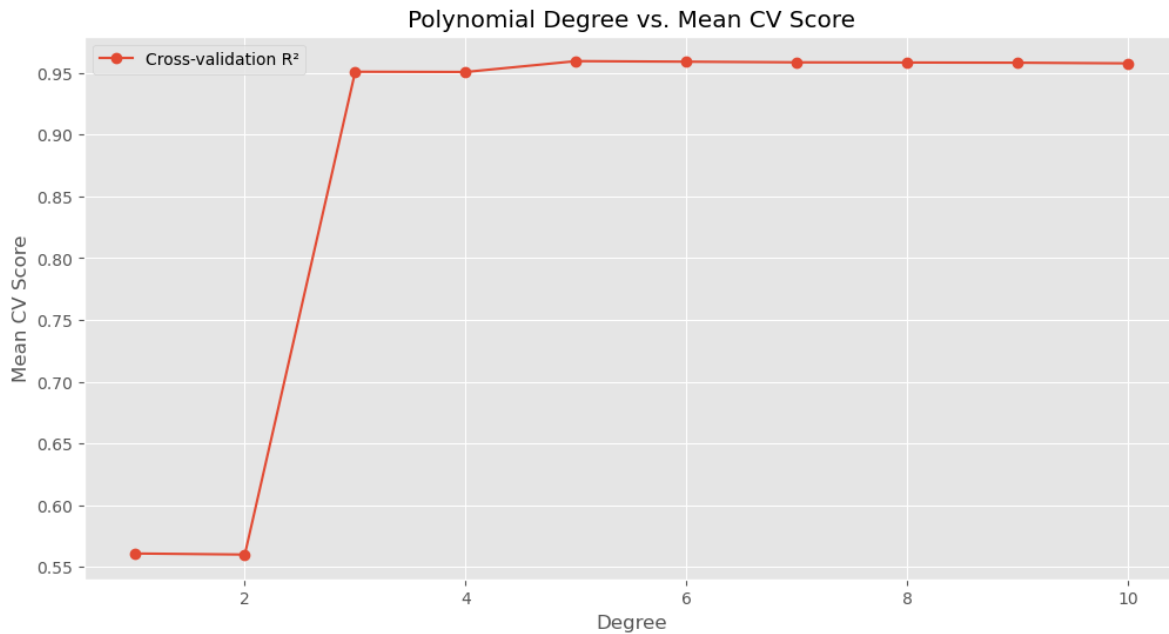
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	0.474906	0.587650	0.567349	0.584471	0.590389
2	0.476036	0.589297	0.561564	0.582378	0.591156
3	0.943020	0.950323	0.952529	0.958609	0.949655
4	0.942638	0.950283	0.952535	0.958321	0.949376
5	0.955439	0.962054	0.959965	0.964546	0.955074
6	0.955373	0.961790	0.960110	0.962963	0.954543
7	0.952190	0.962267	0.959701	0.963145	0.954797
8	0.952636	0.962722	0.960162	0.961735	0.954136
9	0.952201	0.962322	0.960205	0.961795	0.954111
10	0.951549	0.961590	0.959159	0.961606	0.954121

```
# take mean of each row and add it as the last column of the scores dataframe
scores['mean'] = scores.mean(axis=1)
print(scores)
```

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	mean
1	0.474906	0.587650	0.567349	0.584471	0.590389	0.560953
2	0.476036	0.589297	0.561564	0.582378	0.591156	0.560086
3	0.943020	0.950323	0.952529	0.958609	0.949655	0.950827
4	0.942638	0.950283	0.952535	0.958321	0.949376	0.950631
5	0.955439	0.962054	0.959965	0.964546	0.955074	0.959416
6	0.955373	0.961790	0.960110	0.962963	0.954543	0.958956
7	0.952190	0.962267	0.959701	0.963145	0.954797	0.958420
8	0.952636	0.962722	0.960162	0.961735	0.954136	0.958278
9	0.952201	0.962322	0.960205	0.961795	0.954111	0.958127
10	0.951549	0.961590	0.959159	0.961606	0.954121	0.957605

```
# plot the mean scores for each degree
plt.figure(figsize=(12, 6))
plt.plot(degrees, scores['mean'], 'o-', label='Cross-validation  $R^2$ ')
plt.xlabel('Degree')
```

```
plt.ylabel('Mean CV Score')
plt.title('Polynomial Degree vs. Mean CV Score')
plt.grid(True)
plt.legend();
```



```
# Find optimal degree based on the mean cross-validation scores
optimal_degree = scores['mean'].idxmax()
print(f'Optimal polynomial degree: {optimal_degree}')
```

Optimal polynomial degree: 5

11.4.1.2.5 Step 5: Fitting the Final Model with the Optimal Polynomial Degree

After determining the **optimal polynomial degree** ($p=5$) using cross-validation, we now **refit the model** on the entire training dataset.

This ensures that the model fully utilizes all available training data for the best possible fit.

Steps: 1. **Build the final model** using a polynomial transformation with the optimal degree ($p=5$). 2. **Train the model** on the full training dataset. 3. **Generate predictions** and visualize how well the polynomial regression fits the data.

The plot below shows the **final polynomial regression fit**, highlighting how the model captures patterns in the dataset.

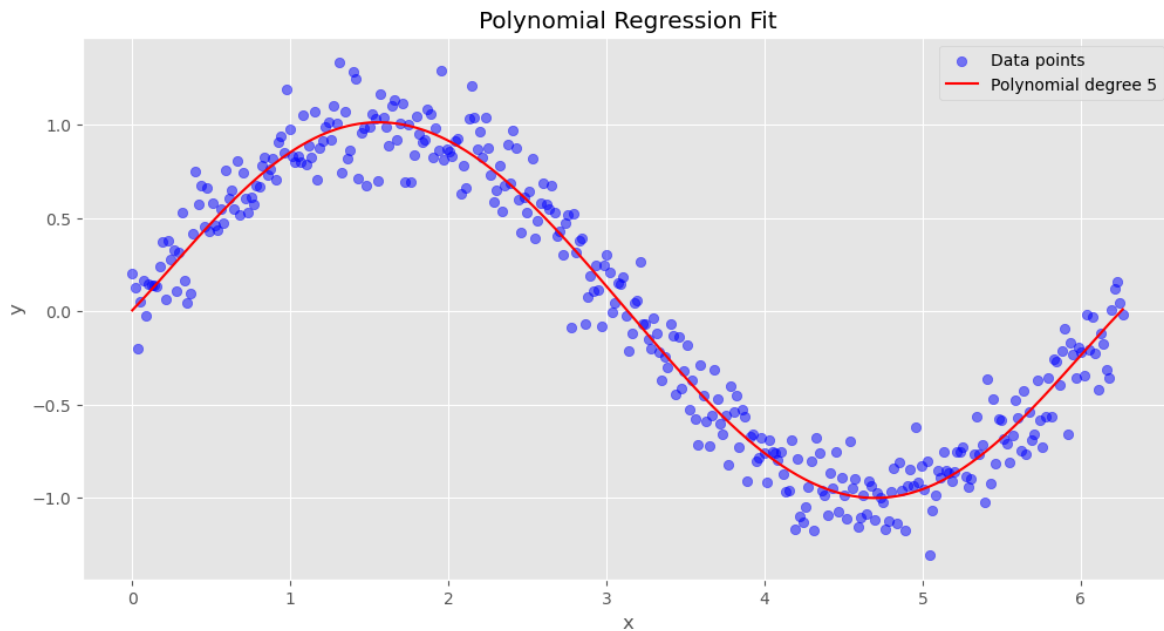
```

# Fit final model with optimal degree
final_model = make_pipeline(
    PolynomialFeatures(optimal_degree),
    LinearRegression()
)
final_model.fit(X_train, y_train)

# Plot final model predictions
plt.figure(figsize=(12, 6))
X_sorted = np.sort(X)
y_pred = final_model.predict(X_sorted)

plt.scatter(X, y, color='blue', alpha=0.5, label='Data points')
plt.plot(X_sorted, y_pred, color='red', label=f'Polynomial degree {optimal_degree}')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Polynomial Regression Fit')
plt.legend()
plt.grid(True)
plt.show()

```



11.4.1.2.6 Step 6: Output the final model performance

```
# Print final model performance metrics
print("\nFinal Model Performance:")
print(f"Training R2: {final_model.score(X_train, y_train):.4f}")
print(f"Test R2: {final_model.score(X_test, y_test):.4f}")
```

```
Final Model Performance:
Training R2: 0.9613
Test R2: 0.9521
```

11.4.1.3 Hyperparameter Tuning with `cross_validate`

The `cross_validate` function differs from `cross_val_score` in two ways:

- It allows specifying multiple metrics for evaluation.
- It returns a dict containing fit-times, score-times (and optionally training scores, fitted estimators, train-test split indices) in addition to the test score.

```
from sklearn.model_selection import cross_validate

def cross_validation_full_results(X_train, y_train, max_degree=10):
    """
    Perform cross-validation for polynomial regression models with different degrees.
    Captures all cross-validation results including R2, RMSE, fit time, score time, etc.

    Parameters:
        X_train (array-like): Training feature data.
        y_train (array-like): Training target data.
        max_degree (int): Maximum polynomial degree to evaluate.

    Returns:
        full_results_df (DataFrame): DataFrame containing all cross-validation metrics for each degree.
    """
    degrees = []
    results_dict = {
        "Fold": [],
        "Fit Time": [],
        "Score Time": [],
        "R2": [],
        "RMSE": []
    }
```

```

for degree in range(1, max_degree + 1):
    # Create polynomial regression model
    model = make_pipeline(
        PolynomialFeatures(degree),
        LinearRegression()
    )

    # Perform cross-validation for both R2 and RMSE, capturing additional metrics
    cv_results = cross_validate(model, X_train, y_train, cv=5,
                                scoring=['r2', 'neg_root_mean_squared_error'], return_train_score=False)

    # Store each fold's results separately
    for fold in range(5): # 5 folds
        degrees.append(degree)
        results_dict["Fold"].append(fold + 1)
        results_dict["Fit Time"].append(cv_results["fit_time"][fold])
        results_dict["Score Time"].append(cv_results["score_time"][fold])
        results_dict["R2"].append(cv_results["test_r2"][fold])
        results_dict["RMSE"].append(-cv_results["test_neg_root_mean_squared_error"][fold])

# Convert to DataFrame
full_results_df = pd.DataFrame(results_dict)
full_results_df.insert(0, "Degree", degrees) # Add Degree column at the front

print("Complete Cross-Validation Results (Higher R2 is better, Lower RMSE is better, Fit Time & Score Time are lower is better)")
return full_results_df

scores_df = cross_validation_full_results(X_train, y_train, max_degree=10)

scores_df

```

Complete Cross-Validation Results (Higher R² is better, Lower RMSE is better, Fit Time & Score Time are lower is better)

	Degree	Fold	Fit Time	Score Time	R ²	RMSE
0	1	1	0.001993	0.002523	0.474906	0.500794
1	1	2	0.001994	0.002033	0.587650	0.456810
2	1	3	0.000000	0.000000	0.567349	0.472232
3	1	4	0.000996	0.000996	0.584471	0.484231
4	1	5	0.000997	0.001993	0.590389	0.461889
5	2	1	0.001482	0.000000	0.476036	0.500255

	Degree	Fold	Fit Time	Score Time	R ²	RMSE
6	2	2	0.006236	0.002004	0.589297	0.455897
7	2	3	0.000996	0.000997	0.561564	0.475378
8	2	4	0.000997	0.000997	0.582378	0.485450
9	2	5	0.000997	0.000997	0.591156	0.461456
10	3	1	0.000996	0.000997	0.943020	0.164968
11	3	2	0.000997	0.001250	0.950323	0.158555
12	3	3	0.001089	0.000000	0.952529	0.156423
13	3	4	0.000000	0.000000	0.958609	0.152829
14	3	5	0.000000	0.000000	0.949655	0.161931
15	4	1	0.000000	0.000000	0.942638	0.165521
16	4	2	0.000000	0.000000	0.950283	0.158619
17	4	3	0.000996	0.000996	0.952535	0.156413
18	4	4	0.000997	0.000996	0.958321	0.153359
19	4	5	0.000997	0.000997	0.949376	0.162379
20	5	1	0.000996	0.000992	0.955439	0.145887
21	5	2	0.000997	0.001001	0.962054	0.138576
22	5	3	0.000997	0.000993	0.959965	0.143649
23	5	4	0.000996	0.000997	0.964546	0.141445
24	5	5	0.000000	0.000000	0.955074	0.152968
25	6	1	0.000000	0.000000	0.955373	0.145995
26	6	2	0.000000	0.000000	0.961790	0.139056
27	6	3	0.000000	0.000000	0.960110	0.143390
28	6	4	0.000000	0.000996	0.962963	0.144567
29	6	5	0.000000	0.000997	0.954543	0.153870
30	7	1	0.000998	0.001304	0.952190	0.151113
31	7	2	0.000997	0.001003	0.962267	0.138186
32	7	3	0.000000	0.000997	0.959701	0.144122
33	7	4	0.000000	0.000993	0.963145	0.144211
34	7	5	0.000997	0.000997	0.954797	0.153438
35	8	1	0.000997	0.000842	0.952636	0.150406
36	8	2	0.000000	0.000000	0.962722	0.137350
37	8	3	0.000000	0.000000	0.960162	0.143297
38	8	4	0.010667	0.000995	0.961735	0.146944
39	8	5	0.000000	0.000000	0.954136	0.154557
40	9	1	0.000000	0.000000	0.952201	0.151095
41	9	2	0.000000	0.000000	0.962322	0.138084
42	9	3	0.000000	0.000000	0.960205	0.143219
43	9	4	0.000996	0.001993	0.961795	0.146828
44	9	5	0.000170	0.000000	0.954111	0.154599
45	10	1	0.000000	0.000000	0.951549	0.152122
46	10	2	0.000000	0.000000	0.961590	0.139420

	Degree	Fold	Fit Time	Score Time	R ²	RMSE
47	10	3	0.000000	0.012748	0.959159	0.145089
48	10	4	0.000996	0.001614	0.961606	0.147192
49	10	5	0.000000	0.000000	0.954121	0.154581

```
mean_results_df = scores_df.groupby("Degree")[["R2", "RMSE"]].mean().reset_index()
mean_results_df
```

	Degree	R ²	RMSE
0	1	0.560953	0.475191
1	2	0.560086	0.475687
2	3	0.950827	0.158941
3	4	0.950631	0.159258
4	5	0.959416	0.144505
5	6	0.958956	0.145376
6	7	0.958420	0.146214
7	8	0.958278	0.146511
8	9	0.958127	0.146765
9	10	0.957605	0.147681

11.4.1.4 Grid Search Cross-Validation for Hyperparameter Tuning

A common use of **cross-validation** is tuning the hyperparameters of a model. One of the most widely used techniques for this is **grid search cross-validation**.

How Grid Search Cross-Validation Works

1. **Define a Grid of Hyperparameters:**
We specify a set of hyperparameters and the possible values we want to evaluate for each.
2. **Evaluate All Combinations:**
Every possible combination of hyperparameter values in the grid is systematically tested.
3. **Cross-Validation for Model Evaluation:**
For each combination, the model is trained and evaluated using cross-validation to assess performance.
4. **Select the Best Hyperparameter Setting:**
The combination that yields the best validation performance is chosen as the optimal set of hyperparameters.

Grid search ensures that we explore multiple hyperparameter settings in a structured way, improving the model's performance without manually adjusting parameters.

```

from sklearn.model_selection import GridSearchCV

# Define the pipeline
model = make_pipeline(PolynomialFeatures(), LinearRegression())

# Define the hyperparameter grid
param_grid = {'polynomialfeatures__degree': np.arange(1, 11)}

# define a KFold cross-validation with shuffling
cv = KFold(n_splits=5, shuffle=True, random_state=42)

# Perform Grid Search with Cross-Validation
grid_search = GridSearchCV(model, param_grid, cv=cv, scoring= 'r2', return_train_score=True,

# Fit the model
grid_search.fit(X_train, y_train)

```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

```

GridSearchCV(cv=KFold(n_splits=5, random_state=42, shuffle=True),
             estimator=Pipeline(steps=[('polynomialfeatures',
                                         PolynomialFeatures()),
                                         ('linearregression',
                                          LinearRegression())]),
             n_jobs=-1,
             param_grid={'polynomialfeatures__degree': array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])},
             return_train_score=True, scoring='r2', verbose=1)

```

```

cv_results = pd.DataFrame(grid_search.cv_results_)
cv_results

```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_polynomialfeatures__degr
0	0.014676	0.001534	0.002452	0.000697	1
1	0.014464	0.001952	0.002246	0.000386	2
2	0.010744	0.001704	0.001196	0.000399	3
3	0.003352	0.001858	0.001900	0.000197	4
4	0.003586	0.002212	0.001844	0.000487	5
5	0.002876	0.001588	0.001994	0.000631	6
6	0.003966	0.003833	0.001792	0.000374	7

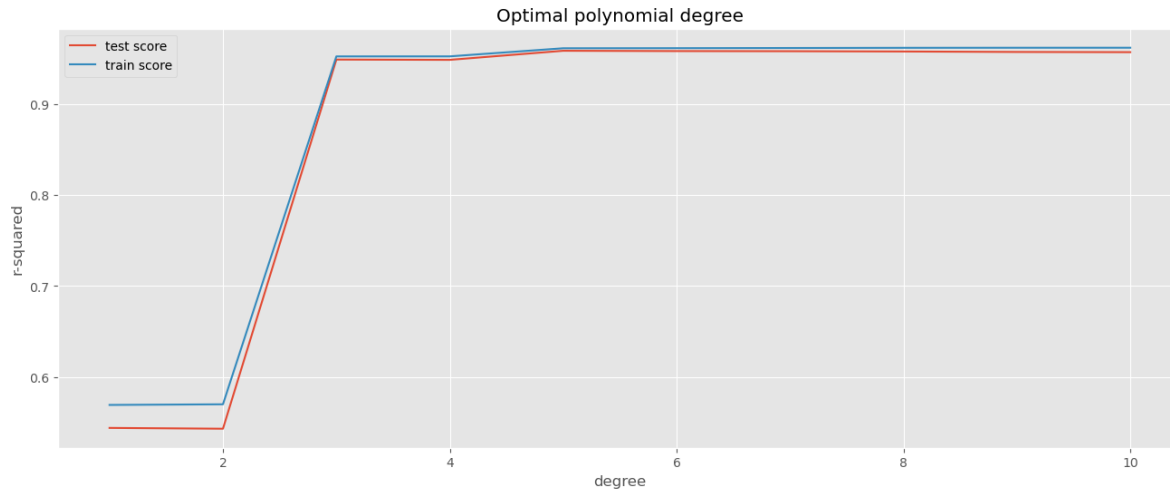
	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_polynomialfeatures__degr
7	0.002766	0.000773	0.001784	0.000394	8
8	0.002394	0.000487	0.001599	0.000487	9
9	0.002936	0.000066	0.001993	0.000036	10

```
pd.set_option('display.float_format', '{:.6f}'.format)
cv_results[["mean_test_score", "mean_train_score"]]
```

	mean_test_score	mean_train_score
0	0.543991	0.569256
1	0.543109	0.569957
2	0.948949	0.952430
3	0.948634	0.952467
4	0.958620	0.961378
5	0.958278	0.961477
6	0.958136	0.961659
7	0.957822	0.961856
8	0.957306	0.961933
9	0.957090	0.961955

```
# plotting cv results
plt.figure(figsize=(16,6))

plt.plot(cv_results["param_polynomialfeatures__degree"], cv_results["mean_test_score"])
plt.plot(cv_results["param_polynomialfeatures__degree"], cv_results["mean_train_score"])
plt.xlabel('degree')
plt.ylabel('r-squared')
plt.title("Optimal polynomial degree")
plt.legend(['test score', 'train score'], loc='upper left');
```



```
# print out the best hyperparameters and the best score
print(grid_search.best_params_)
print(grid_search.best_score_)
print(grid_search.best_estimator_)
```

```
{'polynomialfeatures__degree': 5}
0.9586200484884486
Pipeline(steps=[('polynomialfeatures', PolynomialFeatures(degree=5)),
                 ('linearregression', LinearRegression())])
```

Benefits of GridSearchCV

- No need for manually writing nested for loops for hyperparameter tuning.
- Allows parallel processing using **multiple CPU cores** (`n_jobs=-1`), speeding up the search.
- **GridSearchCV** guarantees finding the best combination.

However, **GridSearchCV** performs an **exhaustive search** over all possible hyperparameter combinations, ensuring that the best parameters are found. This can be slow, especially when dealing with multiple hyperparameters.

11.4.1.5 GridSearchCV vs. Faster Alternatives for Hyperparameter Tuning

Two Ways to Speed Up Hyperparameter Tuning

1. RandomizedSearchCV

- Instead of checking every combination, it **randomly samples** a set number of hyperparameter values, significantly reducing computation time.
- Suitable when a rough estimate of the best hyperparameters is sufficient.

2. ShuffleSplit

- Unlike KFold, which ensures each sample is used exactly once as a test set, **ShuffleSplit randomly selects train-test splits** in each iteration.
- Reduces redundant computations, making the process faster while maintaining good model performance.

Both approaches can be combined with `n_jobs=-1` to leverage parallel processing for even faster results.

11.4.2 Tuning the classification threshold

By default, classifiers use 0.5 as the threshold for classification. However, adjusting this threshold can improve **precision**, **recall**, or **F1-score** depending on the application. Let's use `cross_val_predict` to tune the Classification Threshold

11.4.2.1 Using TunedThresholdClassifierCV from sklearn (>= Version 1.5)

```
from sklearn.model_selection import TunedThresholdClassifierCV

# Define cross-validation strategy
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Define TunedThresholdClassifierCV
tuned_clf = TunedThresholdClassifierCV(
    estimator=logit_model,
    scoring="f1", # Optimize for F1-score
    cv=cv
)

# Fit the model
tuned_clf.fit(X_train_class, y_train_class)
```

```
# Print the best threshold and the corresponding score
print(f"Best threshold: {tuned_clf.best_threshold:.2f}")
print(f"Best F1-score: {tuned_clf.best_score:.4f}")
```

```
Best threshold: 0.39
Best F1-score: 0.8757
```

11.4.2.2 Using cross_val_predict

```
# use the knn to tune the threshold for classification, the threshold is among np.arange(0.1
thresholds = np.arange(0.1, 0.9, 0.1)
scores = []
for threshold in thresholds:
    y_pred_class_prob = cross_val_predict(logit_model, X_train_class, y_train_class, cv=5, m
    y_pred_class = (y_pred_class_prob > threshold).astype(int)
    scores.append(accuracy_score(y_train_class, y_pred_class))
scores = pd.Series(scores, index=thresholds)

df_scores = pd.DataFrame({'threshold': thresholds, 'accuracy': scores}, )
print(df_scores.to_string(index=False))
```

threshold	accuracy
0.100000	0.716000
0.200000	0.832000
0.300000	0.864000
0.400000	0.868000
0.500000	0.865333
0.600000	0.861333
0.700000	0.841333
0.800000	0.810667

```
# print out the best threshold and the cv score for the best threshold
best_threshold = scores.idxmax()
print(best_threshold)
print(scores.loc[best_threshold])
```

```
0.4
0.868
```

12 Regularization

<IPython.core.display.Image object>

```
import numpy as np
import pandas as pd
import random
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

%matplotlib inline
plt.style.use('ggplot')
from matplotlib.pylab import rcParams
rcParams['figure.figsize'] = 5, 4
```

12.1 Why do we need regularization?

12.1.1 The Challenge of Overfitting and Underfitting

When building machine learning models, we aim to find patterns in data that generalize well to unseen samples. However, models can suffer from two key issues:

- **Underfitting:** The model is too simple to capture the underlying pattern in the data.
- **Overfitting:** The model is too complex and captures noise rather than generalizable patterns.

Regularization is a technique used to address overfitting by penalizing overly complex models.

12.1.2 Understanding the Bias-Variance Tradeoff

A well-performing model balances two competing sources of error:

- **Bias:** Error due to overly simplistic assumptions (e.g., underfitting).
- **Variance:** Error due to excessive sensitivity to training data (e.g., overfitting).

A **high-bias** model (e.g., a simple linear regression) may not capture the underlying trend, while a **high-variance** model (e.g., a deep neural network with many parameters) may memorize noise instead of learning meaningful patterns.

Regularization helps **reduce variance** while maintaining an appropriate level of model complexity.

12.1.3 Visualizing Overfitting vs. Underfitting

To better understand this concept, consider three different models:

1. **Underfitting (High Bias):** The model is too simple and fails to capture important trends.
2. **Good Fit (Balanced Bias & Variance):** The model generalizes well to unseen data.
3. **Overfitting (High Variance):** The model is too complex and captures noise, leading to poor generalization.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Regularization helps control **variance** by penalizing large coefficients, leading to a model that generalizes better.

12.2 Simulating Data for an Overfitting Linear Model

12.2.1 Generating the data

```
#Define input array with angles from 60deg to 300deg converted to radians
x = np.array([i*np.pi/180 for i in range(360)])
np.random.seed(10) #Setting seed for reproducibility
y = np.sin(x) + np.random.normal(0,0.15,len(x))
data = pd.DataFrame(np.column_stack([x,y]),columns=['x','y'])
plt.plot(data['x'],data['y'],'.');
```



```
# check how the data looks like
data.head()
```

	x	y
0	0.000000	0.199738
1	0.017453	0.124744
2	0.034907	-0.196911
3	0.052360	0.051078
4	0.069813	0.162957

Polynomial features allow linear regression to model non-linear relationships. Higher-degree terms capture more complex patterns in the data. Let's manually expand features, similar to `PolynomialFeatures` in `sklearn.preprocessing`. Using polynomial regression, we can evaluate different polynomial degrees and analyze the balance between underfitting and overfitting.

```
for i in range(2,16): #power of 1 is already there
    colname = 'x_%d'%i #new var will be x_power
    data[colname] = data['x']**i
data.head()
```

	x	y	x_2	x_3	x_4	x_5	x_6	x_7
0	0.000000	0.199738	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
1	0.017453	0.124744	0.000305	0.000005	9.279177e-08	1.619522e-09	2.826599e-11	4.933346e-13
2	0.034907	-0.196911	0.001218	0.000043	1.484668e-06	5.182470e-08	1.809023e-09	6.314683e-11
3	0.052360	0.051078	0.002742	0.000144	7.516134e-06	3.935438e-07	2.060591e-08	1.078923e-09
4	0.069813	0.162957	0.004874	0.000340	2.375469e-05	1.658390e-06	1.157775e-07	8.082794e-09

What This Code Does

- **Generates Higher-Degree Polynomial Features:** Iterates over the range **2 to 15**, computing polynomial terms (x^2 , x^3 , ..., x^{15}).
- **Dynamically Creates Column Names:** New feature names are automatically generated in the format **x_2**, **x_3**, ..., **x_15**.
- **Expands the Dataset:** Each polynomial-transformed feature is stored

12.2.2 Splitting the Data

Next, we will split the data into **training and testing sets**. As we've learned, models tend to **overfit** when trained on a small dataset.

To intentionally create an **overfitting scenario**, we will: - Use **only 10% of the data for training**. - Reserve **90% of the data for testing**.

This is **not a typical train-test split** but is deliberately done to **demonstrate overfitting**, where the model performs well on the training data but generalizes poorly to unseen data.

```
from sklearn.model_selection import train_test_split
train, test = train_test_split(data, test_size=0.9)
```

```
print('Number of observations in the training data:', len(train))
print('Number of observations in the test data:', len(test))
```

Number of observations in the training data: 36

Number of observations in the test data: 324

12.2.3 Splitting the target and features

```
X_train = train.drop('y', axis=1).values
y_train = train.y.values
X_test = test.drop('y', axis=1).values
y_test = test.y.values
```

12.2.4 Building Models

12.2.4.1 Building a linear model with only 1 predictor x

```
# Linear regression with one feature
independent_variable_train = X_train[:, 0:1]

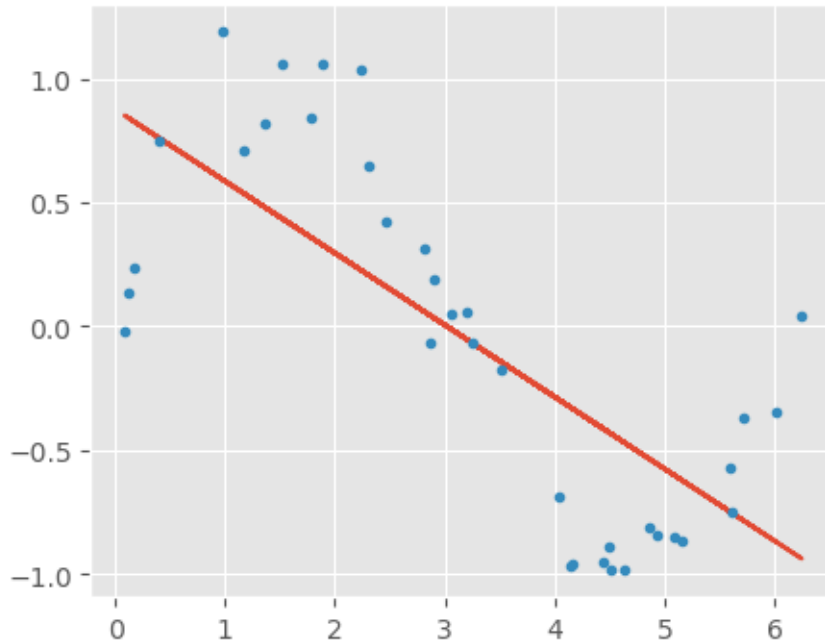
linreg = LinearRegression()
linreg.fit(independent_variable_train, y_train)
y_train_pred = linreg.predict(independent_variable_train)
rss_train = sum((y_train_pred-y_train)**2)/X_train.shape[0]

independent_variable_test = X_test[:, 0:1]
y_test_pred = linreg.predict(independent_variable_test)
rss_test = sum((y_test_pred-y_test)**2)/X_test.shape[0]

print("Training Error", rss_train)
print("Testing Error", rss_test)

plt.plot(X_train[:, 0:1], y_train_pred)
plt.plot(X_train[:, 0:1], y_train, '.');
```

```
Training Error 0.22398220582126424
Testing Error 0.22151086120574928
```



12.2.4.2 Building a linear regression model with three features x , x_2 , x_3

```
independent_variable_train = X_train[:, 0:3]
independent_variable_train[:3]
```

```
array([[ 1.36135682,  1.85329238,  2.52299222],
       [ 2.30383461,  5.30765392, 12.22795682],
       [ 1.51843645,  2.30564925,  3.50098186]])
```

```
def sort_xy(x,y):
    idx = np.argsort(x)
    x2,y2= x[idx] ,y[idx]
    return x2,y2
```

```
# Linear regression with 3 features
```

```
linreg = LinearRegression()
```

```
linreg.fit(independent_variable_train, y_train)
y_train_pred = linreg.predict(independent_variable_train)
```

```

rss_train = sum((y_train_pred-y_train)**2)/X_train.shape[0]

independent_variable_test = X_test[:, 0:3]
y_test_pred = linreg.predict(independent_variable_test)
rss_test = sum((y_test_pred-y_test)**2)/X_test.shape[0]

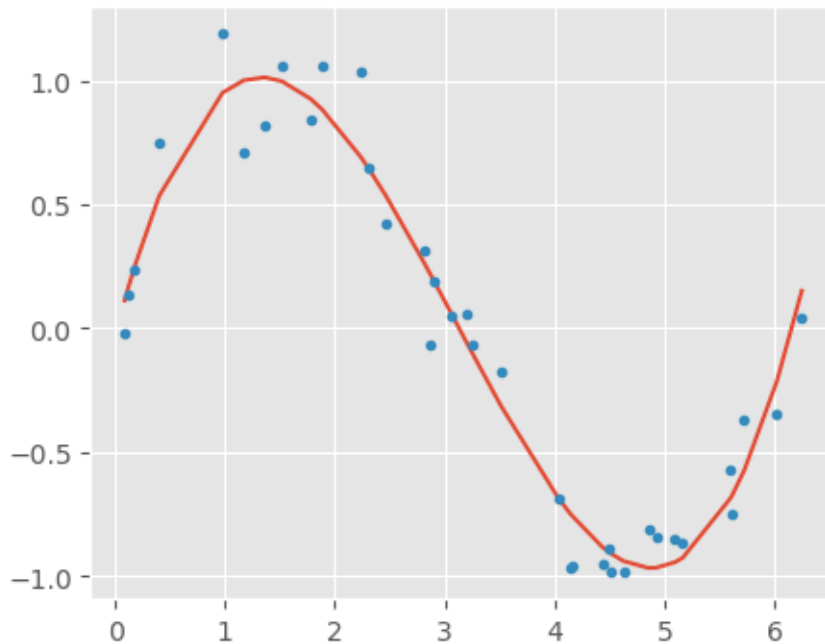
print("Training Error", rss_train)
print("Testing Error", rss_test)

plt.plot(*sort_xy(X_train[:, 0], y_train_pred))
plt.plot(X_train[:, 0], y_train, '.');

```

Training Error 0.02167114498970705

Testing Error 0.028159311299747036



Let's define a helper function that dynamically builds and trains a linear regression model based on a specified number of features. It allows for flexibility in selecting features and automates the process for multiple models.

```

# Define a function which will fit linear vregression model, plot the results, and return the
def linear_regression(train_x, train_y, test_x, test_y, features, models_to_plot):

```

```

#fit the model
linreg = LinearRegression()
linreg.fit(train_x, train_y)
train_y_pred = linreg.predict(train_x)
test_y_pred = linreg.predict(test_x)

#check if a plot is to be made for the entered features
if features in models_to_plot:
    plt.subplot(models_to_plot[features])
    # plt.tight_layout()
    plt.plot(*sort_xy(train_x[:, 0], train_y_pred))
    plt.plot(train_x[:, 0], train_y, '.')

    plt.title('Number of Predictors: %d'%features)

#return the result in pre-defined format
rss_train = sum((train_y_pred - train_y)**2)/train_x.shape[0]
ret = [rss_train]

rss_test = sum((test_y_pred - test_y)**2)/test_x.shape[0]
ret.extend([rss_test])

ret.extend([linreg.intercept_])
ret.extend(linreg.coef_)

return ret

```

```

#initialize a dataframe to store the results:
col = ['mrss_train', 'mrss_test', 'intercept'] + ['coef_VaR_%d'%i for i in range(1, 16)]
ind = ['Number_of_variable_%d'%i for i in range(1, 16)]
coef_matrix_simple = pd.DataFrame(index=ind, columns=col)

```

```

# Define the number of features for which a plot is required:
models_to_plot = {1:231, 3:232, 6:233, 9:234, 12:235, 15:236}

```

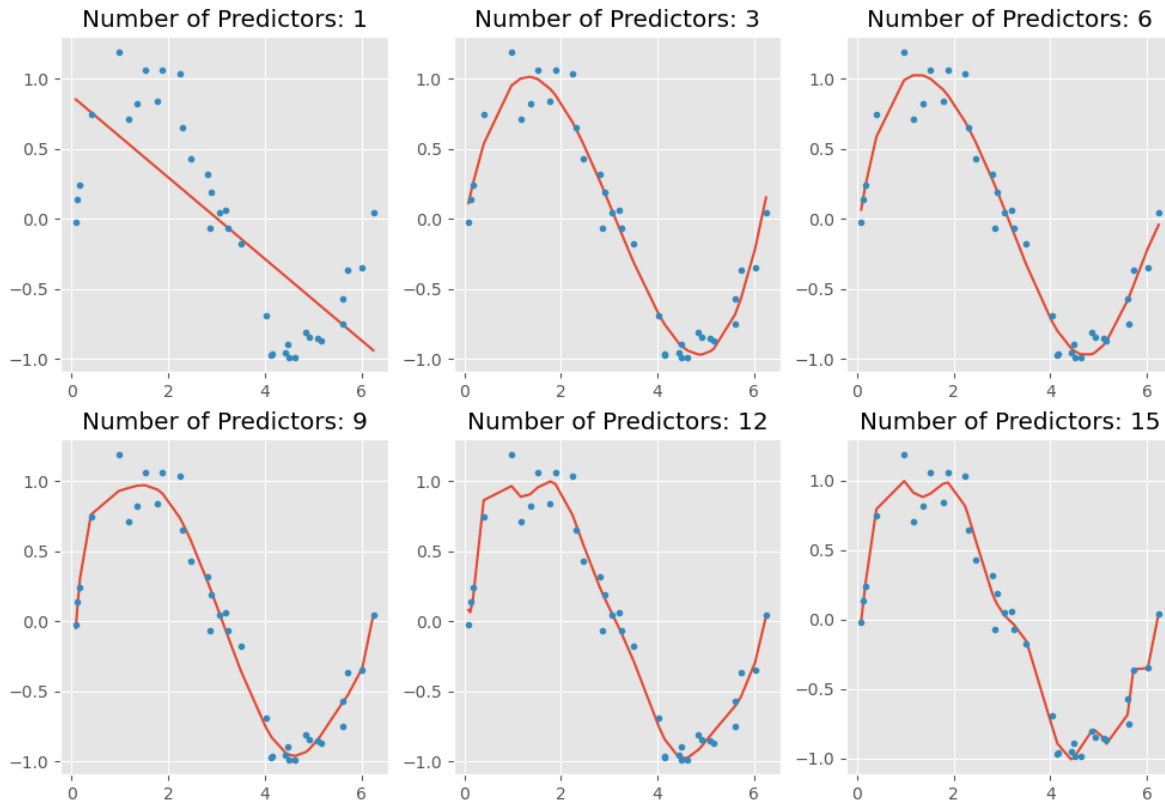
```

import matplotlib.pyplot as plt
# Iterate through all powers and store the results in a matrix form
plt.figure(figsize = (12, 8))
for i in range(1, 16):
    train_x = X_train[:, 0:i]
    train_y = y_train
    test_x = X_test[:, 0:i]

```

```
test_y = y_test
```

```
coef_matrix_simple.iloc[i-1, 0:i+3] = linear_regression(train_x, train_y, test_x, test_y
```



Key Takeaways:

As we increase the number of features (higher-degree polynomial terms), we observe the following:

- The model becomes **more flexible**, capturing intricate patterns in the training data.
- The curve becomes **increasingly wavy and complex**, adapting too closely to the data points.
- This results in **overfitting**, where the model performs well on the training set but fails to generalize to unseen data.

Overfitting occurs because the model learns **noise** instead of the true underlying pattern, leading to poor performance on new data.

To better understand this phenomenon, let's:

- **Evaluate model performance** on both the training and test sets.

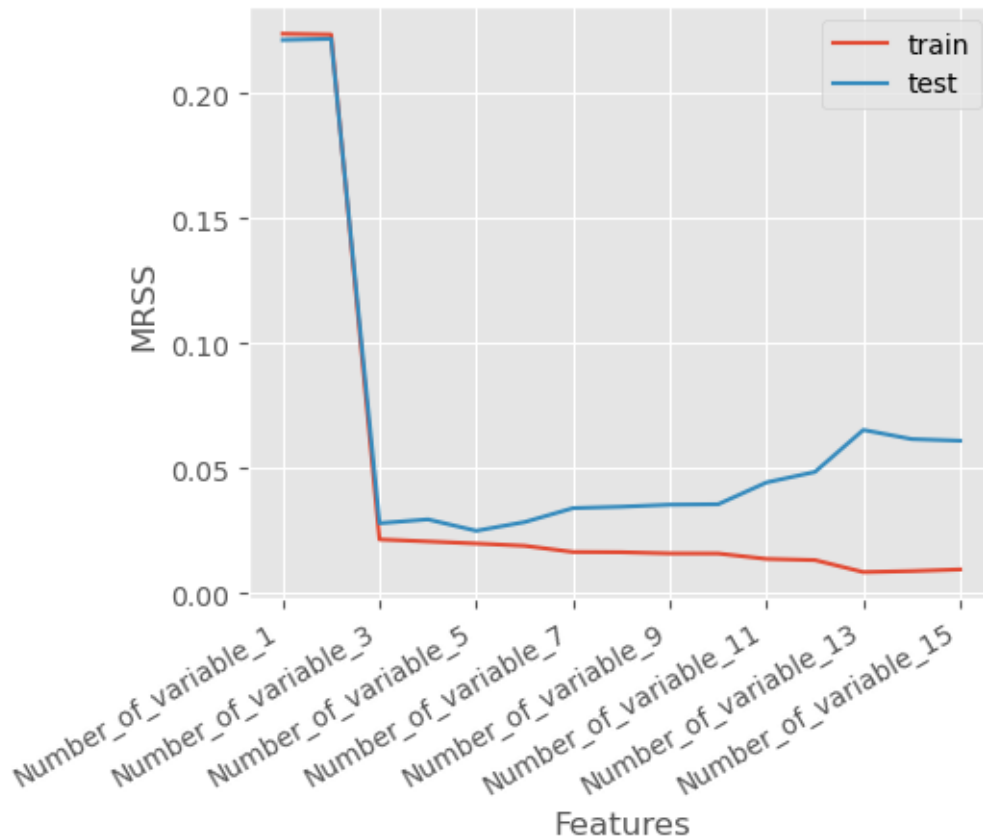
- **Output the model coefficients** to analyze how feature coefficients changes with increasing complexity.

```
# Set the display format to be scientific for ease of analysis
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_simple
```

	mrss_train	mrss_test	intercept	coef_VaR_1	coef_VaR_2	coef_VaR_3
Number_of_variable_1	0.22	0.22	0.88	-0.29	NaN	NaN
Number_of_variable_2	0.22	0.22	0.84	-0.25	-0.0057	NaN
Number_of_variable_3	0.022	0.028	-0.032	1.7	-0.83	0.089
Number_of_variable_4	0.021	0.03	-0.09	2	-1	0.14
Number_of_variable_5	0.02	0.025	-0.019	1.5	-0.48	-0.092
Number_of_variable_6	0.019	0.029	-0.13	2.4	-1.9	0.77
Number_of_variable_7	0.017	0.034	-0.37	4.7	-6.5	4.7
Number_of_variable_8	0.017	0.035	-0.42	5.3	-8	6.4
Number_of_variable_9	0.016	0.036	-0.57	7.1	-14	16
Number_of_variable_10	0.016	0.036	-0.51	6.2	-11	9.7
Number_of_variable_11	0.014	0.044	0.17	-4	36	-86
Number_of_variable_12	0.013	0.049	0.54	-10	67	-1.6e+02
Number_of_variable_13	0.0086	0.065	-0.56	9.9	-56	2e+02
Number_of_variable_14	0.009	0.062	-0.075	0.62	7	-5.1
Number_of_variable_15	0.0097	0.061	-0.3	3.4	-0.93	-2

Let's plot the training error versus the test error below and identify the overfitting

```
coef_matrix_simple[['mrss_train', 'mrss_test']].plot()
ax = plt.gca()
plt.xlabel('Features')
plt.ylabel('MRSS')
plt.setp(ax.get_xticklabels(), rotation=30, horizontalalignment='right')
plt.legend(['train', 'test']);
```



12.2.5 Overfitting Indicated by Training and Test MRSS Trends

As observed in the plot:

- The **Training Mean Residual Sum of Squares (MRSS)** consistently decreases as the number of features increases.
- However, after a certain point, the **Test MRSS starts to rise**, indicating that the model is no longer generalizing well to unseen data.

This trend suggests that while adding more features helps the model fit the **training data** better, it also causes the model to **memorize noise**, leading to poor performance on the test set.

This is a classic sign of **overfitting**, where the model captures excessive complexity in the data rather than the true underlying pattern.

Next, let's mitigate the overfitting issue using regularization

12.3 Regularization: Combating Overfitting

Regularization is a technique that **modifies the loss function** by adding a penalty term to control model complexity.

This helps **prevent overfitting** by discouraging large coefficients in the model.

12.3.1 Regularized Loss Function

The regularized loss function is given by:

$$L_{reg}(\beta) = L(\beta) + \alpha R(\beta)$$

where: - $L(\beta)$ is the original loss function (e.g., Mean Squared Error in linear regression).

- $R(\beta)$ is the **regularization term**, which penalizes large coefficients.

- α is a **hyperparameter** that controls the strength of regularization.

12.3.2 Regularization Does Not Penalize the Intercept

- The **intercept (bias term)** captures the **baseline mean** of the target variable.
- Penalizing the intercept would **shift predictions** incorrectly instead of controlling complexity.
- Thus, regularization **only applies to feature coefficients**, not the intercept.

12.3.3 Types of Regularization

1. **L1 Regularization (Lasso Regression)**: Encourages sparsity by driving some coefficients to zero.
2. **L2 Regularization (Ridge Regression)**: Shrinks coefficients but keeps all of them nonzero.
3. **Elastic Net**: A combination of both L1 and L2 regularization.

By applying **regularization**, we obtain models that balance **bias-variance tradeoff**, leading to **better generalization**.

12.3.4 Why Is Feature Scaling Required in Regularization?

12.3.4.1 The Effect of Feature Magnitudes on Regularization

Regularization techniques such as **Lasso (L1)**, **Ridge (L2)**, and **Elastic Net** apply penalties to the model's coefficients. However, when features have vastly different scales, **regularization disproportionately affects certain features**, leading to:

- **Uneven shrinkage** of coefficients, causing instability in the model.
- **Incorrect feature importance interpretation**, as some features dominate due to their larger numerical scale.
- **Suboptimal performance**, since regularization penalizes large coefficients more, even if they belong to more informative features.

12.3.4.2 Example: The Need for Feature Scaling

Imagine a dataset with two features:

- **Feature 1:** Number of bedrooms (range: 1-5).
- **Feature 2:** House area in square feet (range: 500-5000).

Since house area has much larger values, the model assigns smaller coefficients to compensate, making regularization unfairly **biased toward smaller-scale features**.

12.3.4.3 How to Scale Features for Regularization

To ensure fair treatment of all features, apply **feature scaling** before training a regularized model:

12.3.4.3.1 Standardization (Recommended)

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

- Centers the data around **zero** with unit variance. - Used in **Lasso, Ridge, and Elastic Net**.

12.3.4.3.2 Min-Max Scaling

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Scales features to a fixed **[0, 1]** range. - Common for **neural networks** but less effective for regularization.

Let's use **StandardScaler** to scale the features

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Feature scaling on X_train
X_train_std = scaler.fit_transform(X_train)
columns = data.drop('y', axis=1).columns
X_train_std = pd.DataFrame(X_train_std, columns = columns)
X_train_std.head()

# Feature scaling on X_test
X_test_std = scaler.transform(X_test)
X_test_std = pd.DataFrame(X_test_std, columns = columns)
X_test_std.head()
```

	x	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_10	x_11	x_12	x_13	x_14	x_15
0	-0.96	-1	-0.89	-0.78	-0.69	-0.62	-0.57	-0.52	-0.48	-0.45	-0.43	-0.4	-0.38	-0.37	-0.36
1	-0.061	-0.34	-0.49	-0.55	-0.57	-0.56	-0.53	-0.5	-0.47	-0.45	-0.42	-0.4	-0.38	-0.37	-0.36
2	-1.8	-1.2	-0.95	-0.8	-0.7	-0.62	-0.57	-0.52	-0.48	-0.45	-0.43	-0.4	-0.38	-0.37	-0.36
3	0.21	-0.051	-0.24	-0.36	-0.43	-0.46	-0.47	-0.46	-0.45	-0.43	-0.41	-0.4	-0.38	-0.36	-0.35
4	-1.7	-1.2	-0.95	-0.8	-0.7	-0.62	-0.57	-0.52	-0.48	-0.45	-0.43	-0.4	-0.38	-0.37	-0.36

In the next section, we will explore **different types of regularization techniques** and see how they help in preventing overfitting.

12.3.5 Ridge Regression: L2 Regularization

Ridge regression is a type of **linear regression** that incorporates **L2 regularization** to prevent overfitting by **penalizing large coefficients**.

12.3.5.1 Ridge Regression Loss Function

The regularized loss function for Ridge regression is given by:

$$L_{\text{Ridge}}(\beta) = \frac{1}{n} \sum_{i=1}^n |y_i - \beta^\top x_i|^2 + \alpha \sum_{j=1}^J \beta_j^2$$

where:

- y_i is the true target value for observation i .
- x_i is the feature vector for observation i .
- β is the vector of model coefficients.
- α is the regularization parameter, which controls the penalty strength.
- $\sum_{j=1}^J \beta_j^2$ is the L2 norm (sum of squared coefficients).

Note that j starts from 1, excluding the intercept from regularization.

The penalty term in Ridge regression,

$$\sum_{j=1}^J \beta_j^2 = \|\beta\|_2^2$$

is the **squared L2 norm** of the coefficient vector β .

Minimizing this norm ensures that the model coefficients remain **small and stable**, reducing sensitivity to variations in the data.

Let's build a Ridge Regression model using scikit-learn, The **alpha** parameter controls the strength of the regularization:

```
from sklearn.linear_model import Ridge
# defining a function which will fit ridge regression model, plot the results, and return the results
def ridge_regression(train_x, train_y, test_x, test_y, alpha, models_to_plot={}):

    #fit the model
    ridgereg = Ridge(alpha=alpha)
    ridgereg.fit(train_x, train_y)
    train_y_pred = ridgereg.predict(train_x)
    test_y_pred = ridgereg.predict(test_x)

    #check if a plot is to be made for the entered alpha
    if alpha in models_to_plot:
        plt.subplot(models_to_plot[alpha])
        # plt.tight_layout()
```

```

plt.plot(*sort_xy(train_x.values[:, 0], train_y_pred))
plt.plot(train_x.values[:, 0], train_y, '.')

plt.title('Plot for alpha: %.3g'%alpha)

#return the result in pre-defined format
mrss_train = sum((train_y_pred - train_y)**2)/train_x.shape[0]
ret = [mrss_train]

mrss_test = sum((test_y_pred - test_y)**2)/test_x.shape[0]
ret.extend([mrss_test])

ret.extend([ridgereg.intercept_])
ret.extend(ridgereg.coef_)

return ret

```

Let's experiment with different values of alpha in Ridge Regression and observe how it affects the model's coefficients and performance.

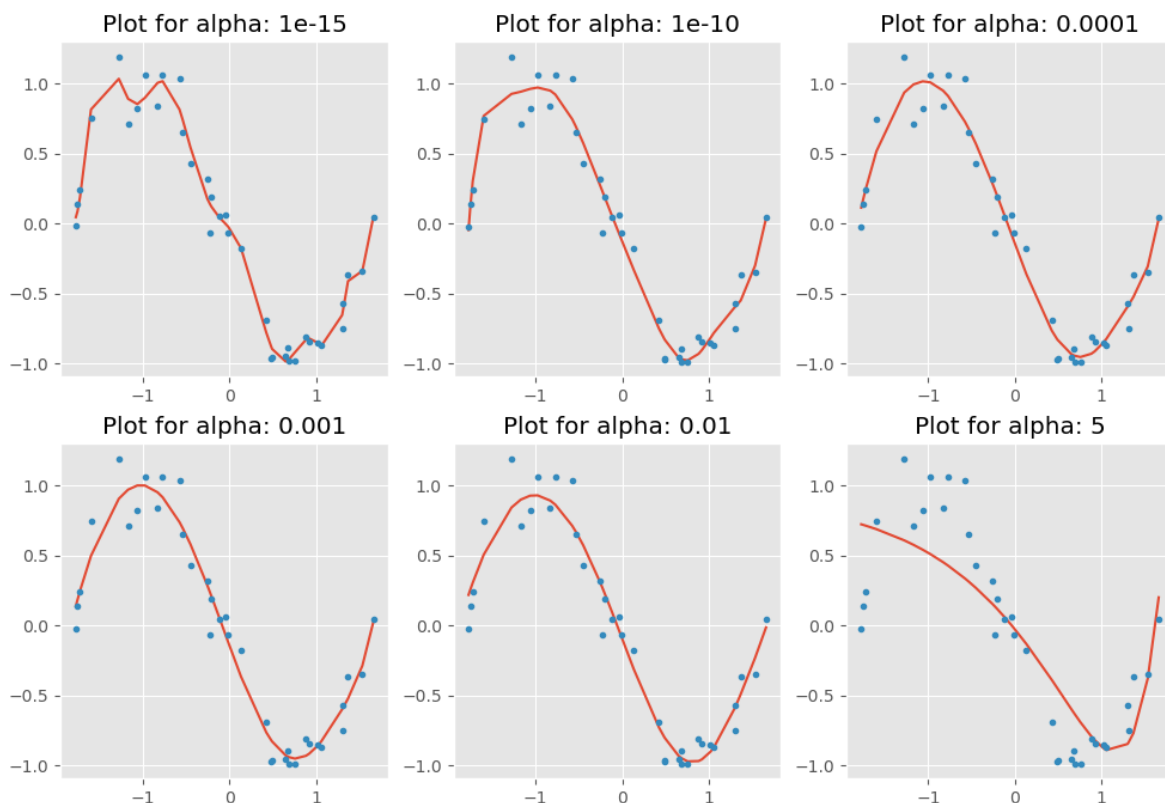
```

#initialize a dataframe to store the coefficient:
alpha_ridge = [1e-15, 1e-10, 1e-8, 1e-4, 1e-3, 1e-2, 1, 5, 10, 20]
col = ['mrss_train', 'mrss_test', 'intercept'] + ['coef_VaR_%d'%i for i in range(1, 16)]
ind = ['alpha_%.2g'%alpha_ridge[i] for i in range(0, 10)]
coef_matrix_ridge = pd.DataFrame(index=ind, columns=col)

# Define the number of features for which a plot is required:
models_to_plot = {1e-15:231, 1e-10:232, 1e-4:233, 1e-3:234, 1e-2:235, 5:236}

#Iterate over the 10 alpha values:
plt.figure(figsize=(12, 8))
for i in range(10):
    coef_matrix_ridge.iloc[i,] = ridge_regression(X_train_std, train_y, X_test_std, test_y, a

```



As we can observe, when increasing alpha, the model becomes simpler, with coefficients shrinking more aggressively due to stronger regularization. This reduces the risk of overfitting but may lead to underfitting if alpha is set too high.

Next, let's output the training error versus the test error and examine how the feature coefficients change with different alpha values.

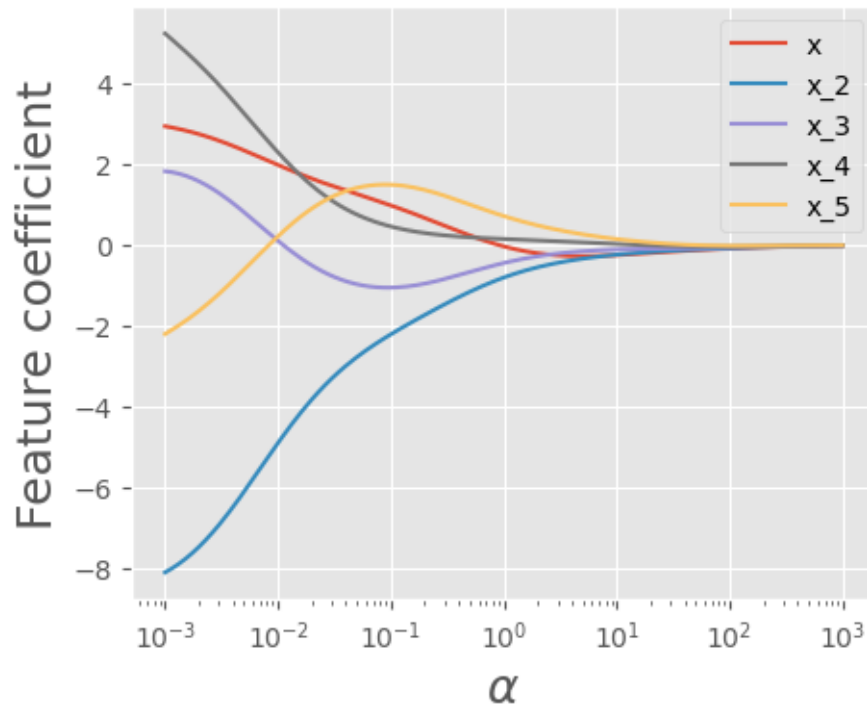
```
#Set the display format to be scientific for ease of analysis
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_ridge
```

	mrss_train	mrss_test	intercept	coef_VaR_1	coef_VaR_2	coef_VaR_3	coef_VaR_4
alpha_1e-15	0.0092	0.059	-0.072	-2.8	1.9e+02	-1.3e+03	-6e+03
alpha_1e-10	0.016	0.036	-0.072	12	-1.3e+02	7.1e+02	-1.8e+03
alpha_1e-08	0.017	0.034	-0.072	8.1	-70	2.8e+02	-5.6e+02
alpha_0.0001	0.019	0.024	-0.072	2.9	-8.2	4.5	-0.022
alpha_0.001	0.019	0.023	-0.072	2.5	-5.6	-0.5	1.3
alpha_0.01	0.022	0.024	-0.072	1.9	-4	-1.3	0.73

	mrss_train	mrss_test	intercept	coef_VaR_1	coef_VaR_2	coef_VaR_3	coef_VaR_4
alpha_1	0.089	0.093	-0.072	0.08	-0.61	-0.48	-0.22
alpha_5	0.12	0.13	-0.072	-0.17	-0.3	-0.24	-0.14
alpha_10	0.14	0.14	-0.072	-0.19	-0.24	-0.18	-0.12
alpha_20	0.16	0.17	-0.072	-0.18	-0.19	-0.15	-0.097

To better observe the pattern, let's visualize how the coefficients change as we increase α

```
def plot_ridge_reg_coeff(train_x):
    alphas = np.logspace(3,-3,200)
    coefs = []
    #X_train_std, train_y
    for a in alphas:
        ridge = Ridge(alpha = a)
        ridge.fit(train_x, train_y)
        coefs.append(ridge.coef_)
    #Visualizing the shrinkage in ridge regression coefficients with increasing values of the
    plt.xlabel('xlabel', fontsize=18)
    plt.ylabel('ylabel', fontsize=18)
    plt.plot(alphas, coefs)
    plt.xscale('log')
    plt.xlabel(r'$\alpha$')
    plt.ylabel('Feature coefficient')
    plt.legend(train_x.columns );
plot_ridge_reg_coeff(X_train_std.iloc[:,5])
plt.savefig("test.png")
```



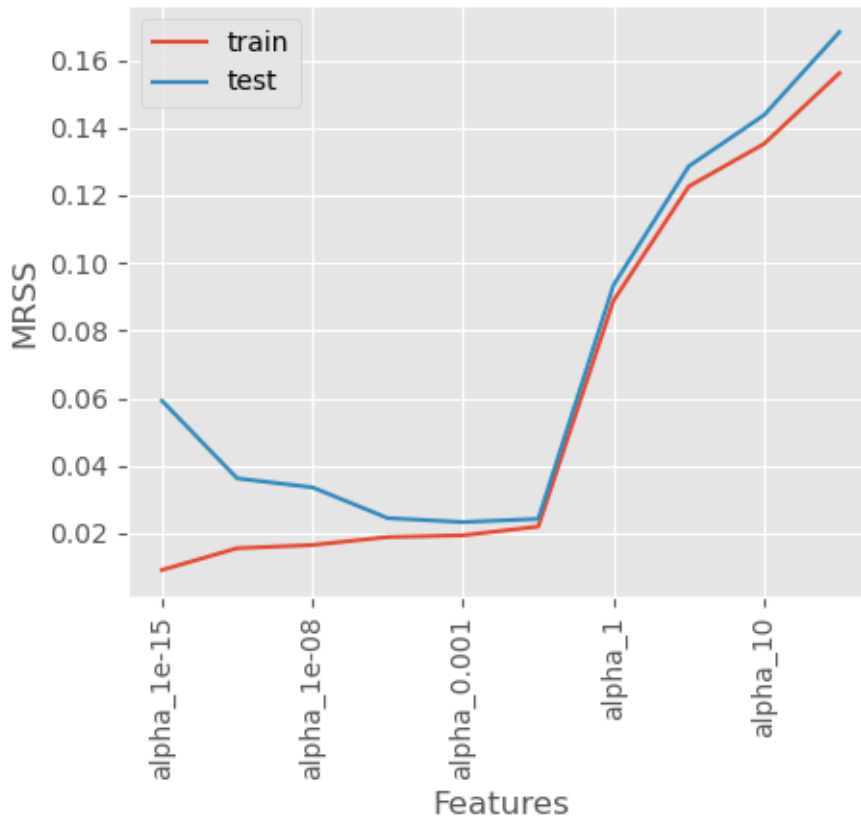
As we can see, as α increases, the coefficients become smaller and approach zero. Now, let's examine the number of zero coefficients.

```
coef_matrix_ridge.apply(lambda x: sum(x.values==0),axis=1)
```

```
alpha_1e-15      0
alpha_1e-10      0
alpha_1e-08      0
alpha_0.0001     0
alpha_0.001      0
alpha_0.01       0
alpha_1          0
alpha_5          0
alpha_10         0
alpha_20         0
dtype: int32
```

Let's plot how the test error and training error change as we increase α

```
coef_matrix_ridge[['mrss_train', 'mrss_test']].plot()
plt.xlabel('Features')
plt.ylabel('MRSS')
plt.xticks(rotation=90)
plt.legend(['train', 'test']);
```



As we can observe, as λ increases beyond a certain value, both the training MRSS and test MRSS begin to rise, indicating that the model starts underfitting.

12.3.6 Lasso Regression: L1 Regularization

LASSO stands for **Least Absolute Shrinkage and Selection Operator**. There are two key aspects in this name:

- “**Absolute**” refers to the use of the absolute values of the coefficients in the penalty term.
- “**Selection**” highlights LASSO’s ability to shrink some coefficients to exactly zero, effectively performing **feature selection**.

Lasso regression performs **L1 regularization**, which helps prevent overfitting by penalizing large coefficients and enforcing sparsity in the model.

12.3.6.1 Lasso Regression Loss Function

In standard **linear regression**, the loss function is the **Mean Squared Error (MSE)**:

$$L_{\text{MSE}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2$$

LASSO modifies this by adding an **L1 regularization** penalty, leading to the following **regularized loss function**:

$$L_{\text{Lasso}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \alpha \sum_{j=1}^J |\beta_j|$$

where:

- y_i is the true target value for observation i .
- x_i is the feature vector for observation i .
- β is the vector of model coefficients.
- α is the regularization parameter, which controls the penalty strength.
- $\sum_{j=1}^J |\beta_j|$ is the **L1** norm (sum of absolute values of coefficients).

The penalty term in Lasso regression,

$$\sum_{j=1}^J |\beta_j| = \|\beta\|_1$$

is the **L1 norm** of the coefficient vector (β).

Minimizing this norm encourages **sparsity**, meaning some coefficients become exactly zero, leading to an automatically selected subset of features.

Next, let's build a Lasso Regression model. Similar to Ridge regression, we will explore a range of values for the regularization parameter alpha.

```
from sklearn.linear_model import Lasso
alpha_lasso = [1e-15, 1e-10, 1e-8, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1, 5]
```

```
# Defining a function which will fit lasso regression model, plot the results, and return the result
def lasso_regression(train_x, train_y, test_x, test_y, alpha, models_to_plot={}):
```

```
    #fit the model
    if alpha == 0:
        lassoreg = LinearRegression()
    else:
        lassoreg = Lasso(alpha=alpha, max_iter=200000000, tol=0.01)
    lassoreg.fit(train_x, train_y)
    train_y_pred = lassoreg.predict(train_x)
    test_y_pred = lassoreg.predict(test_x)

    #check if a plot is to be made for the entered alpha
    if alpha in models_to_plot:
        plt.subplot(models_to_plot[alpha])
        # plt.tight_layout()
        plt.plot(*sort_xy(train_x.values[:, 0], train_y_pred))
        plt.plot(train_x.values[:, 0:1], train_y, '.')

        plt.title('Plot for alpha: %.3g'%alpha)

    #return the result in pre-defined format
    mrss_train = sum((train_y_pred - train_y)**2)/train_x.shape[0]
    ret = [mrss_train]

    mrss_test = sum((test_y_pred - test_y)**2)/test_x.shape[0]
    ret.extend([mrss_test])

    ret.extend([lassoreg.intercept_])
    ret.extend(lassoreg.coef_)

    return ret
```

```
#initialize a dataframe to store the coefficient:
col = ['mrss_train', 'mrss_test', 'intercept'] + ['coef_VaR_%d'%i for i in range(1, 16)]
ind = ['alpha_%.2g'%alpha_lasso[i] for i in range(0, 10)]
coef_matrix_lasso = pd.DataFrame(index=ind, columns=col)
```

```
# Define the number of features for which a plot is required:
models_to_plot = {1e-10:231, 1e-5:232, 1e-4:233, 1e-3:234, 1e-2:235, 0.1:236}
```

```
models_to_plot
```

```
{1e-10: 231, 1e-05: 232, 0.0001: 233, 0.001: 234, 0.01: 235, 0.1: 236}
```

```
#Iterate over the 10 alpha values:
plt.figure(figsize=(12, 8))
for i in range(10):
    coef_matrix_lasso.iloc[i,] = lasso_regression(X_train_std, train_y, X_test_std, test_y, a
```

```
#Set the display format to be scientific for ease of analysis
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_lasso
```

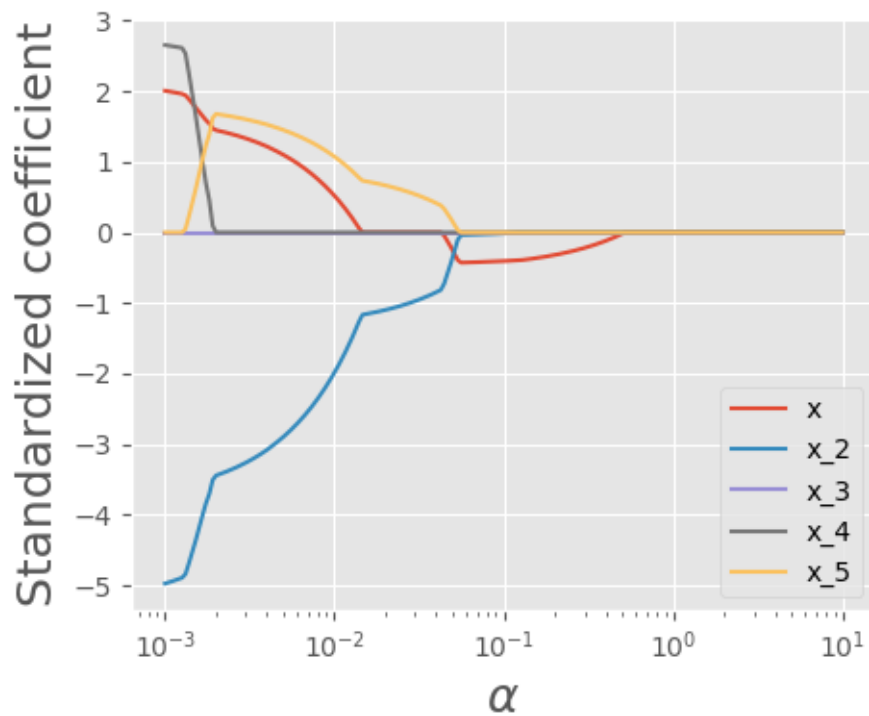
	mrss_train	mrss_test	intercept	coef_VaR_1	coef_VaR_2	coef_VaR_3	coef_VaR_4
alpha_1e-15	0.019	0.024	-0.072	2.8	-6.9	0.83	1.5
alpha_1e-10	0.019	0.024	-0.072	2.8	-6.9	0.83	1.5
alpha_1e-08	0.019	0.024	-0.072	2.8	-6.9	0.83	1.5
alpha_1e-05	0.019	0.024	-0.072	2.8	-6.8	0.73	1.7
alpha_0.0001	0.02	0.024	-0.072	2.7	-6.5	0	2.8
alpha_0.001	0.024	0.026	-0.072	2	-4.5	-0	0
alpha_0.01	0.084	0.088	-0.072	0.27	-1.3	-0	-0
alpha_0.1	0.19	0.2	-0.072	-0.22	-0.27	-0	-0
alpha_1	0.49	0.53	-0.072	-0	-0	-0	-0
alpha_5	0.49	0.53	-0.072	-0	-0	-0	-0

```
def plot_lasso_reg_coeff(train_x):
    alphas = np.logspace(1,-3,200)
    coefs = []
    #X_train_std, train_y
    for a in alphas:
        laso = Lasso(alpha=a, max_iter = 100000)
        laso.fit(train_x, train_y)
        coefs.append(lasso.coef_)
    #Visualizing the shrinkage in ridge regression coefficients with increasing values of the
    plt.xlabel('xlabel', fontsize=18)
    plt.ylabel('ylabel', fontsize=18)
```

```

plt.plot(alphas, coefs)
plt.xscale('log')
plt.xlabel(r'$\alpha$')
plt.ylabel('Standardized coefficient')
plt.legend(train_x.columns );
plot_lasso_reg_coeff(X_train_std.iloc[:,5])
plt.savefig("test1.png")

```

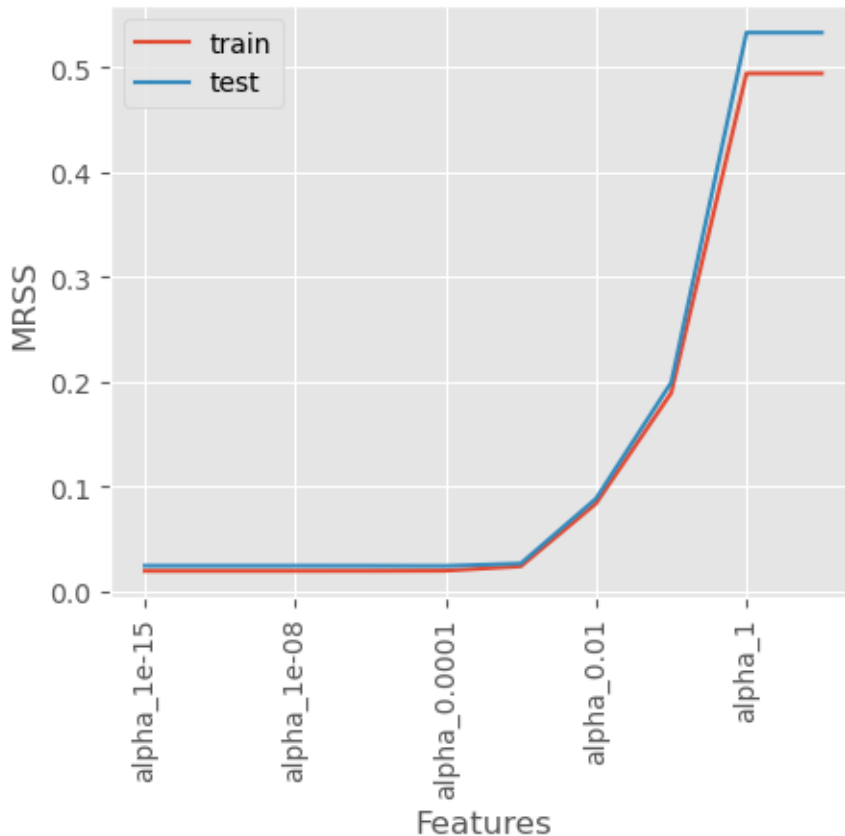


```
coef_matrix_lasso.apply(lambda x: sum(x.values==0),axis=1)
```

alpha_1e-15	0
alpha_1e-10	0
alpha_1e-08	0
alpha_1e-05	2
alpha_0.0001	8
alpha_0.001	11
alpha_0.01	12
alpha_0.1	12
alpha_1	15

```
alpha_5      15  
dtype: int32
```

```
coef_matrix_lasso[['mrss_train', 'mrss_test']].plot()  
plt.xlabel('Features')  
plt.ylabel('MRSS')  
plt.xticks(rotation=90)  
plt.legend(['train', 'test'])
```



Effect of alpha in Lasso Regression - Small alpha (close to 0): The penalty is minimal, and Lasso behaves like ordinary linear regression. - **Moderate alpha:** Some coefficients shrink, and some become exactly zero, performing feature selection. - **Large alpha:** Many coefficients become zero, leading to a very simple model (potentially underfitting).

12.3.7 Elastic Net Regression: Combining L1 and L2 Regularization

12.3.7.1 Mathematical Formulation*

Elastic Net regression combines both **L1 (Lasso)** and **L2 (Ridge)** penalties, balancing feature selection and coefficient shrinkage. The **regularized loss function** for Elastic Net is given by:

$$L_{\text{ElasticNet}}(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \alpha \left((1 - \rho) \frac{1}{2} \sum_{j=1}^p \beta_j^2 + \rho \sum_{j=1}^p |\beta_j| \right)$$

where:

- y_i is the true target value for observation i .
- x_i is the feature vector for observation i .
- β is the vector of model coefficients.
- α is the **regularization strength** parameter in scikit-learn.
- ρ is the **l1_ratio** parameter in scikit-learn, controlling the mix of L1 and L2 penalties.
- $\sum_{j=1}^p |\beta_j|$ is the **L1 norm**, enforcing sparsity.
- $\sum_{j=1}^p \beta_j^2$ is the **L2 norm**, ensuring coefficient stability.

12.3.7.2 Elastic Net Special Cases

- When **l1_ratio = 0**, Elastic Net behaves like **Ridge Regression (L2 regularization)**.
- When **l1_ratio = 1**, Elastic Net behaves like **Lasso Regression (L1 regularization)**.
- When **0 < l1_ratio < 1**, Elastic Net **balances feature selection (L1) and coefficient shrinkage (L2)**.

Now, let's implement an **Elastic Net Regression model** using **scikit-learn** and explore how different values of **alpha** and **l1_ratio** affect the model.

```
from sklearn.linear_model import ElasticNet
alpha_lasso = [1e-15, 1e-10, 1e-8, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1, 5]
l1_ratio=0.5
```

```
# Defining a function which will fit lasso regression model, plot the results, and return the results
def elasticnet_regression(train_x, train_y, test_x, test_y, alpha, models_to_plot={}):

    #fit the model
    if alpha == 0:
        model = LinearRegression()
```

```

else:
    model = ElasticNet(alpha=alpha, max_iter=20000, l1_ratio=l1_ratio)
    model.fit(train_x, train_y)
    train_y_pred = model.predict(train_x)
    test_y_pred = model.predict(test_x)

#check if a plot is to be made for the entered alpha
if alpha in models_to_plot:
    plt.subplot(models_to_plot[alpha])
    # plt.tight_layout()
    plt.plot(*sort_xy(train_x.values[:, 0], train_y_pred))
    plt.plot(train_x.values[:, 0:1], train_y, '.')

    plt.title('Plot for alpha: %.3g'%alpha)

#return the result in pre-defined format
mrss_train = sum((train_y_pred - train_y)**2)/train_x.shape[0]
ret = [mrss_train]

mrss_test = sum((test_y_pred - test_y)**2)/test_x.shape[0]
ret.extend([mrss_test])

ret.extend([model.intercept_])
ret.extend(model.coef_)

return ret

```

```

#initialize a dataframe to store the coefficient:
col = ['mrss_train', 'mrss_test', 'intercept'] + ['coef_VaR_%d'%i for i in range(1, 16)]
ind = ['alpha_%.2g'%alpha_lasso[i] for i in range(0, 10)]
coef_matrix_elasticnet = pd.DataFrame(index=ind, columns=col)

```

```

# Define the number of features for which a plot is required:
models_to_plot = {1e-10:231, 1e-5:232, 1e-4:233, 1e-3:234, 1e-2:235, 0.1:236}

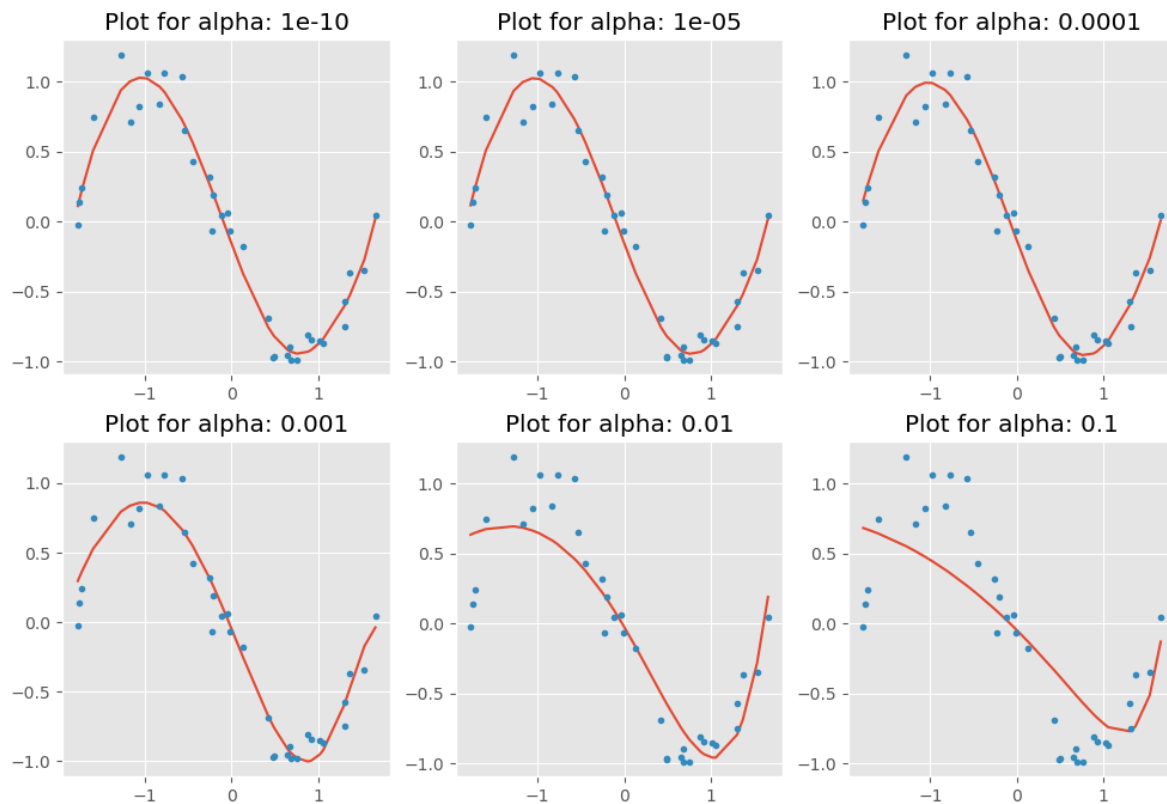
```

```
models_to_plot
```

```
{1e-10: 231, 1e-05: 232, 0.0001: 233, 0.001: 234, 0.01: 235, 0.1: 236}
```

```
#Iterate over the 10 alpha values:
plt.figure(figsize=(12, 8))
for i in range(10):
    coef_matrix_elasticnet.iloc[i,] = elasticnet_regression(X_train_std, train_y, X_test_std
```

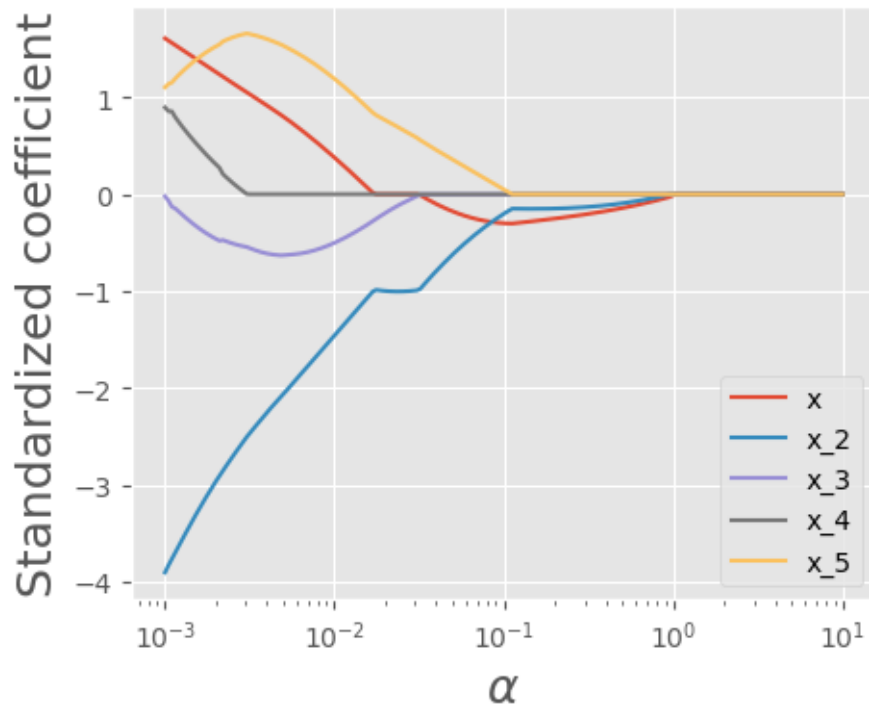
```
c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:131:
  model = cd_fast.enet_coordinate_descent(
c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:131:
  model = cd_fast.enet_coordinate_descent(
c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:131:
  model = cd_fast.enet_coordinate_descent(
c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:131:
  model = cd_fast.enet_coordinate_descent(
c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:131:
  model = cd_fast.enet_coordinate_descent(
c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:131:
  model = cd_fast.enet_coordinate_descent(
```



```
#Set the display format to be scientific for ease of analysis
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_elasticnet
```

	mrss_train	mrss_test	intercept	coef_VaR_1	coef_VaR_2	coef_VaR_3	coef_VaR_4
alpha_1e-15	0.019	0.024	-0.072	2.8	-6.9	0.83	1.5
alpha_1e-10	0.019	0.024	-0.072	2.8	-6.9	0.83	1.5
alpha_1e-08	0.019	0.024	-0.072	2.8	-6.9	0.83	1.5
alpha_1e-05	0.019	0.024	-0.072	2.7	-6.6	0.37	1.7
alpha_0.0001	0.02	0.023	-0.072	2.4	-5.5	-0.6	1.3
alpha_0.001	0.028	0.03	-0.072	1.6	-3.3	-0.6	0
alpha_0.01	0.076	0.081	-0.072	0.31	-1.1	-0.48	-0
alpha_0.1	0.15	0.16	-0.072	-0.19	-0.41	-0.0068	-0
alpha_1	0.48	0.52	-0.072	-0.013	-0	-0	-0
alpha_5	0.49	0.53	-0.072	-0	-0	-0	-0

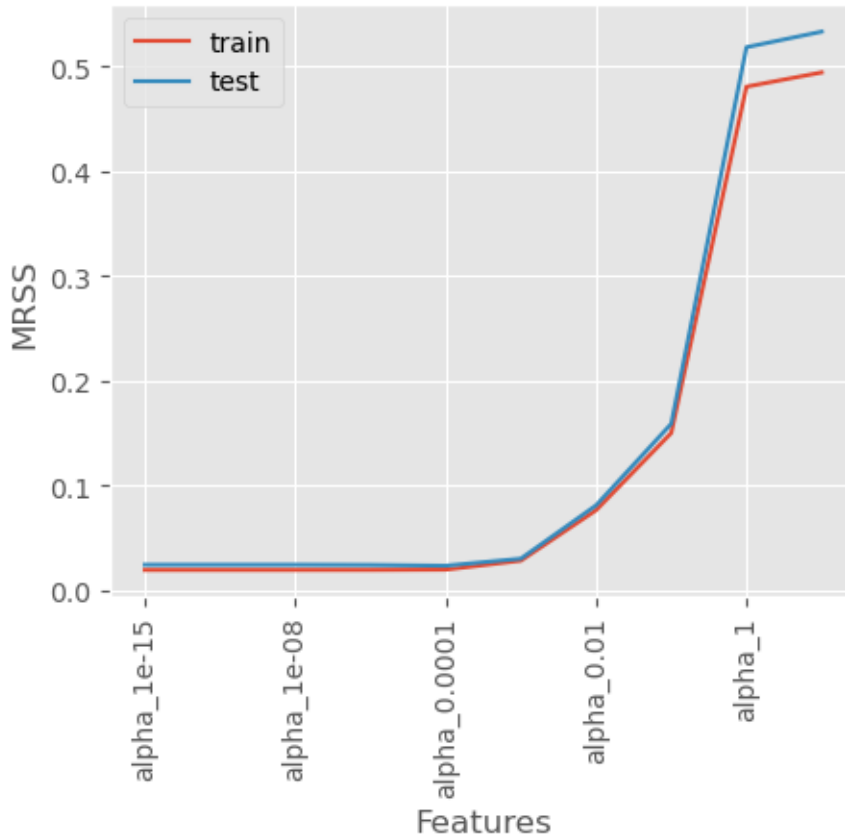
```
def plot_elasticnet_reg_coeff(train_x):
    alphas = np.logspace(1,-3,200)
    coefs = []
    #X_train_std, train_y
    for a in alphas:
        model = ElasticNet(alpha=a, max_iter=20000, l1_ratio=l1_ratio)
        model.fit(train_x, train_y)
        coefs.append(model.coef_)
    #Visualizing the shrinkage in ridge regression coefficients with increasing values of the
    plt.xlabel('xlabel', fontsize=18)
    plt.ylabel('ylabel', fontsize=18)
    plt.plot(alphas, coefs)
    plt.xscale('log')
    plt.xlabel(r'$\alpha$')
    plt.ylabel('Standardized coefficient')
    plt.legend(train_x.columns );
plot_elasticnet_reg_coeff(X_train_std.iloc[:,5])
plt.savefig("test2.png")
```



```
coef_matrix_elasticnet.apply(lambda x: sum(x.values==0),axis=1)
```

```
alpha_1e-15      0
alpha_1e-10      0
alpha_1e-08      0
alpha_1e-05      0
alpha_0.0001     3
alpha_0.001      6
alpha_0.01       7
alpha_0.1        8
alpha_1          14
alpha_5          15
dtype: int32
```

```
coef_matrix_elasticnet[['mrss_train', 'mrss_test']].plot()
plt.xlabel('Features')
plt.ylabel('MRSS')
plt.xticks(rotation=90)
plt.legend(['train', 'test']);
```



ElasticNet is controlled by these key parameters:

- **alpha** (float, default=1.0):
The regularization strength multiplier. Higher values increase regularization.
- **l1_ratio** (float, default=0.5):
The mixing parameter between L1 and L2 penalties:
 - `l1_ratio = 0`: Pure Ridge regression
 - `l1_ratio = 1`: Pure Lasso regression
 - `0 < l1_ratio < 1`: ElasticNet with mixed penalties

12.3.8 RidgeCV, LassoCV, and ElasticNetCV in Scikit-Learn

In Scikit-Learn, `RidgeCV`, `LassoCV`, and `ElasticNetCV` are cross-validation (CV) versions of **Ridge**, **Lasso**, and **Elastic Net** regression models, respectively. These versions **auto-**

atically select the best regularization strength (alpha) by performing internal cross-validation.

12.3.8.1 Overview of RidgeCV, LassoCV, and ElasticNetCV**

Model	Regularization Type	Purpose	How Alpha is Chosen?
RidgeCV	L2 (Ridge)	Shrinks coefficients to handle overfitting, but keeps all features.	Uses cross-validation to select the best alpha .
LassoCV	L1 (Lasso)	Shrinks coefficients, but also removes some features by setting coefficients to zero.	Uses cross-validation to find the best alpha .
ElasticNetCV	L2 (Elastic Net)	Balances Ridge and Lasso.	Uses cross-validation to find the best alpha and l1_ratio .

12.3.8.2 How to Use RidgeCV, LassoCV, and ElasticNetCV

Each model automatically selects the optimal **alpha** value through internal cross-validation without using a loop through the **alpha** values

```
from sklearn.linear_model import LassoCV

alpha_lasso = [1e-15, 1e-10, 1e-8, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1, 5]

# Initialize LassoCV with cross-validation
lasso_cv = LassoCV(alphas=alpha_lasso, cv=5, max_iter=2000000)

# Fit the model using training data
lasso_cv.fit(X_train_std, train_y)

# Make predictions
train_y_pred = lasso_cv.predict(X_train_std)
test_y_pred = lasso_cv.predict(X_test_std)
```

```

# Get the best alpha chosen by cross-validation
best_alpha = lasso_cv.alpha_
print(f"Best alpha selected by LassoCV: {best_alpha}")

# Check if a plot should be made for the selected alpha
if best_alpha in models_to_plot:
    plt.subplot(models_to_plot[best_alpha])
    plt.plot(*sort_xy(train_x[:, 0], train_y_pred))
    plt.plot(train_x[:, 0:1], train_y, '.', label="Actual Data")

    plt.title(f'Plot for alpha: {best_alpha:.3g}')
    plt.legend();

```

12.4 Reference

https://www.linkedin.com/pulse/tutorial-ridge-lasso-regression-subhajit-mondal/?trk=read_related_article-card_title

13 Feature Selection

Feature selection is a crucial step in machine learning that helps improve model performance, reduce overfitting, and speed up training time by selecting the most relevant features from the dataset.

There are three main types of feature selection methods:

- Filter Methods (Based on statistical measures)
- Wrapper Methods (Based on model performance)
- Embedded Methods (Feature selection integrated within the model)

Why feature selection?

- Removes irrelevant or redundant features
- Reduces computational complexity
- Helps avoid the curse of dimensionality
- Improves model interpretability
- Enhances generalization by reducing overfitting

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

13.1 Filter Methods (Statistical approaches)

Filter methods evaluate the relevance of features based on statistical measures before training a model. These methods rank features according to their correlation with the target variable and select the most relevant ones.

13.1.1 Characteristics:

- **Independent of the model:** Feature selection is performed before training the model.
- **Fast and computationally efficient:** Since no model training is required, they are suitable for large datasets.
- **Prone to selecting irrelevant features:** They do not consider interactions between features.
- **Common techniques:**
 - Pearson correlation coefficient
 - Mutual information
 - Chi-square test
 - ANOVA (Analysis of Variance)
 - Variance thresholding

<IPython.core.display.Image object>

13.1.2 Methods

Method	Description	Python Function/Class	Notes
Variance Threshold	Removes low-variance numeric features	<code>VarianceThreshold()</code> (scikit-learn)	Only works on numerical features
Correlation Filter	Removes highly correlated features	Custom implementation + <code>df.corr()</code>	Requires manual analysis (pandas) or <code>correlation_threshold</code> methods
Chi-Square Test	Selects categorical features for classification	<code>chi2()</code> + <code>SelectKBest()</code>	For categorical targets, requires non-negative features
Mutual Information	Measures dependency for classification/regression	<code>mutual_info_classif()</code> or <code>mutual_info_regression()</code>	Use <code>SelectKBest()</code> or <code>SelectPercentile()</code>

```
from sklearn.feature_selection import VarianceThreshold, SelectKBest, f_regression

# Load the data
df = pd.read_csv('./Datasets/Credit.csv', index_col=0)

# Separate features and target
```

```

# Let's use 'Balance' as our target variable
X = df.drop('Balance', axis=1)
y = df['Balance']

# Preprocessing: Encode categorical variables
X_encoded = pd.get_dummies(X, columns=['Gender', 'Student', 'Married', 'Ethnicity'])

# 1. Variance Threshold Feature Selection
def variance_threshold_selection(X, threshold=0.1):
    # Create a VarianceThreshold selector
    selector = VarianceThreshold(threshold=threshold)

    # Fit and transform the data
    X_selected = selector.fit_transform(X)

    # Get the selected feature names
    selected_features = X.columns[selector.get_support()]

    return X_selected, selected_features

# Apply Variance Threshold
X_var_selected, var_selected_features = variance_threshold_selection(X_encoded)

print("1. Variance Threshold Feature Selection:")
print("Selected features:", list(var_selected_features))
print("Number of features reduced from", X_encoded.shape[1], "to", len(var_selected_features))

```

1. Variance Threshold Feature Selection:

Selected features: ['Income', 'Limit', 'Rating', 'Cards', 'Age', 'Education', 'Gender_Male']

Number of features reduced from 15 to 13

```

# 2. Correlation-based Feature Removal
def remove_highly_correlated_features(X, threshold=0.8):
    # Compute the correlation matrix
    corr_matrix = X.corr().abs()

    # Create a mask of the upper triangle of the correlation matrix
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

    # Find features to drop
    to_drop = [column for column in upper.columns if any(upper[column] > threshold)]

```

```

    # Drop highly correlated features
    X_reduced = X.drop(columns=to_drop)

    return X_reduced, to_drop

# Apply Correlation-based Feature Removal
X_corr_removed, dropped_features = remove_highly_correlated_features(X_encoded)

print("\n3. Correlation-based Feature Removal:")
print("Dropped features:", dropped_features)
print("Number of features reduced from", X_encoded.shape[1], "to", X_corr_removed.shape[1])

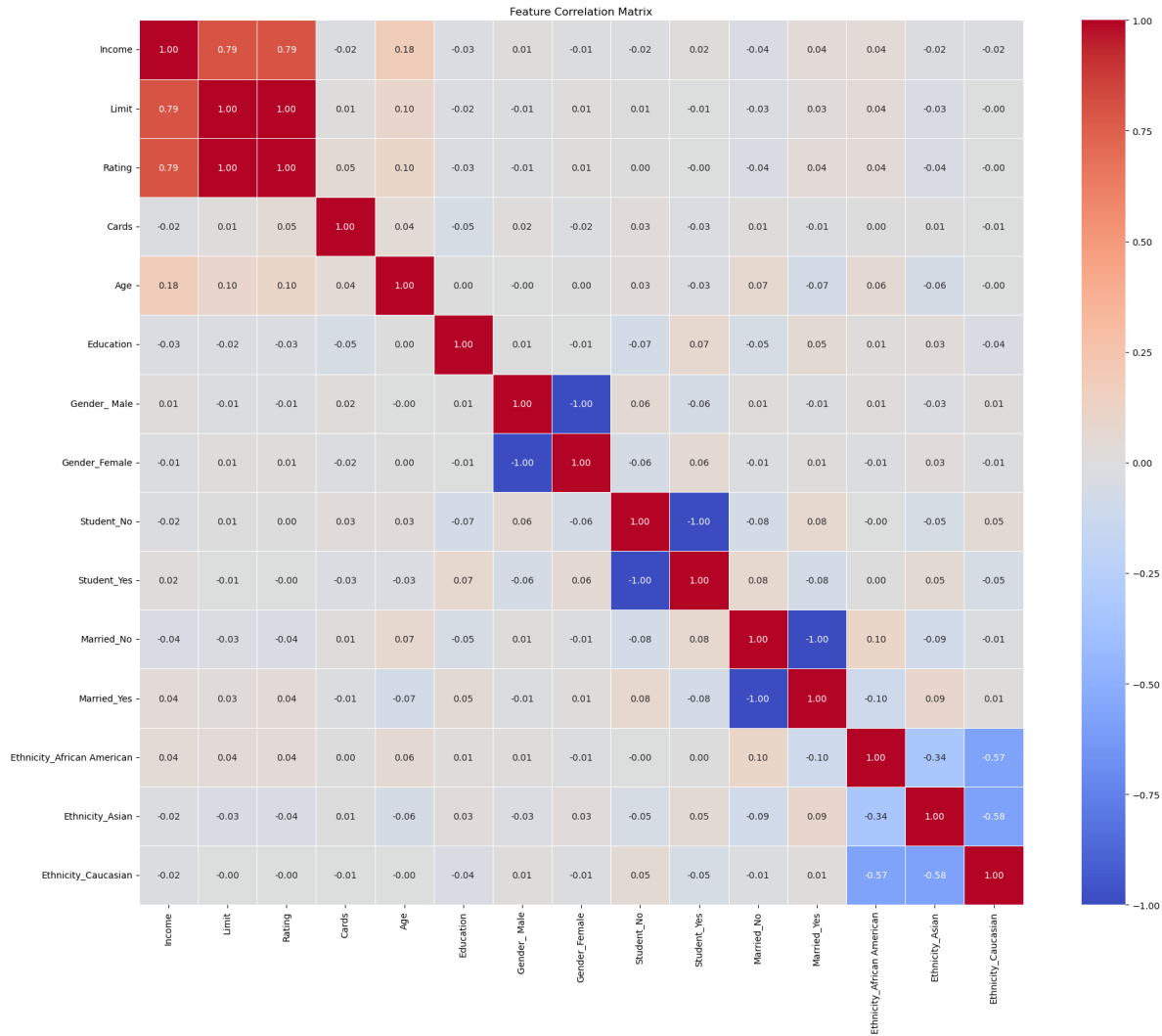
# Visualization of Correlation Matrix
plt.figure(figsize=(20,16))
correlation_matrix = X_encoded.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5, fmt=".2f", square=True)
plt.title('Feature Correlation Matrix')
plt.tight_layout();

```

3. Correlation-based Feature Removal:

Dropped features: ['Rating', 'Gender_Female', 'Student_Yes', 'Married_Yes']

Number of features reduced from 15 to 11



```
# 3. Correlation-based Feature Selection (with target)
def correlation_with_target_selection(X, y, threshold=0.2):
    # Calculate correlation with target
    correlations = X.apply(lambda col: col.corr(y))

    # Select features above the absolute correlation threshold
    selected_features = correlations[abs(correlations) >= threshold].index

    return X[selected_features], selected_features

# Apply Correlation with Target Selection
X_corr_target, corr_target_features = correlation_with_target_selection(X_encoded, y)
```

```
print("\n2. Correlation with Target Feature Selection:")
print("Selected features:", list(corr_target_features))
print("Number of features reduced from", X_encoded.shape[1], "to", len(corr_target_features))
```

2. Correlation with Target Feature Selection:

Selected features: ['Income', 'Limit', 'Rating', 'Student_No', 'Student_Yes']
 Number of features reduced from 15 to 5

```
# Calculate correlations with target
correlations = X_encoded.apply(lambda col: col.corr(y))

# Sort correlations by absolute value in descending order
correlations_sorted = correlations.abs().sort_values(ascending=False)

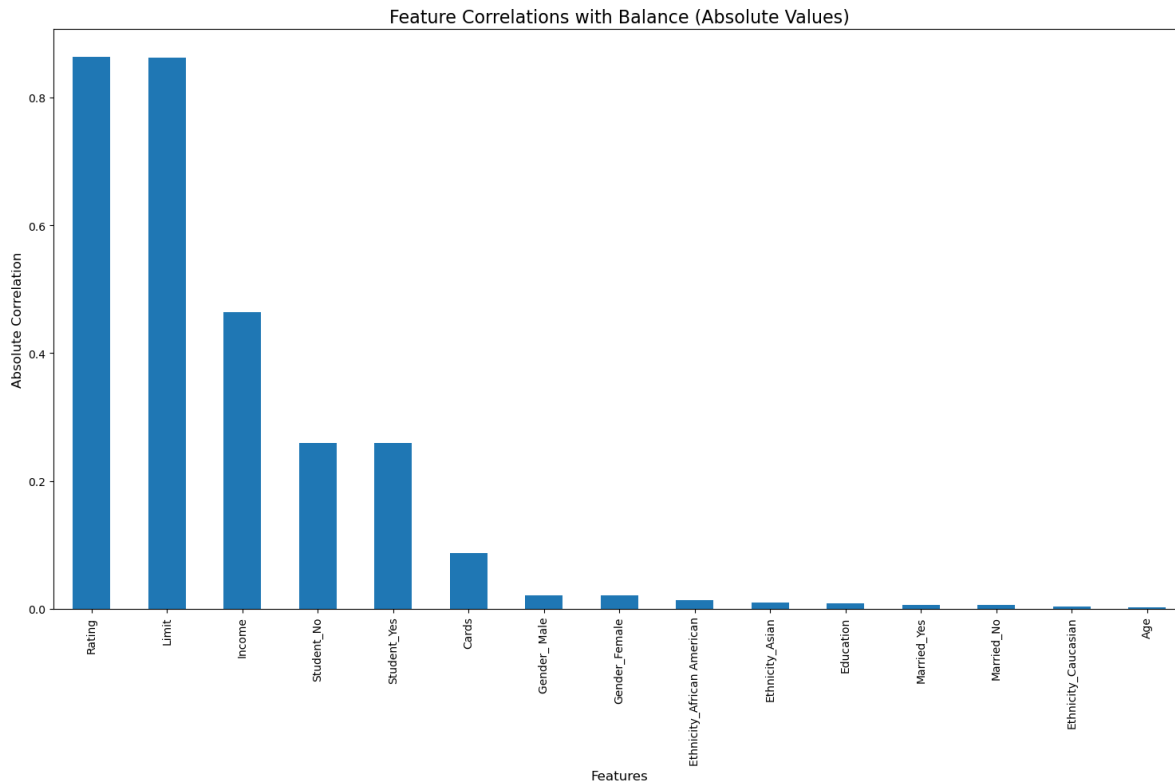
# Prepare the visualization
plt.figure(figsize=(15, 10))
correlations_sorted.plot(kind='bar')
plt.title('Feature Correlations with Balance (Absolute Values)', fontsize=16)
plt.xlabel('Features', fontsize=12)
plt.ylabel('Absolute Correlation', fontsize=12)
plt.xticks(rotation=90)
plt.tight_layout()

# Print out the sorted correlations
print("Correlations with Balance (sorted by absolute value):")
for feature, corr in correlations_sorted.items():
    print(f"{feature}: {corr}")
```

Correlations with Balance (sorted by absolute value):

Rating: 0.863625160621495
 Limit: 0.8616972670153951
 Income: 0.46365645701575736
 Student_No: 0.2590175474501476
 Student_Yes: 0.2590175474501476
 Cards: 0.08645634741861911
 Gender_Male: 0.021474006717338588
 Gender_Female: 0.021474006717338588
 Ethnicity_African American: 0.013719801718047214
 Ethnicity_Asian: 0.00981223592782398

Education: 0.00806157645355343
Married_Yes: 0.005673490217239985
Married_No: 0.005673490217239973
Ethnicity_Caucasian: 0.003288321172097514
Age: 0.0018351188590736563



```
# 4. SelectKBest Feature Selection
def select_k_best_features(X, y, k=5):
    # Create a SelectKBest selector
    selector = SelectKBest(score_func=f_regression, k=k)

    # Fit and transform the data
    X_selected = selector.fit_transform(X, y)

    # Get the selected feature names
    selected_features = X.columns[selector.get_support()]

    return X_selected, selected_features
```

```
# Apply SelectKBest
X_k_best, k_best_features = select_k_best_features(X_encoded, y)

print("\n4. SelectKBest Feature Selection:")
print("Selected features:", list(k_best_features))
print("Number of features reduced from", X_encoded.shape[1], "to", len(k_best_features))
```

4. SelectKBest Feature Selection:

Selected features: ['Income', 'Limit', 'Rating', 'Student_No', 'Student_Yes']

Number of features reduced from 15 to 5

13.2 Wrapper Methods

Wrapper methods evaluate subsets of features by actually training and testing a model on different feature combinations. These methods optimize feature selection based on model performance.

13.2.1 Characteristics:

- **Model-dependent:** They rely on training a model to assess feature usefulness.
- **Computationally expensive:** Since multiple models need to be trained, they can be slow, especially for large datasets.
- **More accurate than filter methods:** They account for feature interactions and can find the optimal subset.
- **Risk of overfitting:** Since they optimize for a specific dataset, the selected features may not generalize well to new data.
- **Common techniques:**
 - Recursive Feature Elimination (RFE)
 - Sequential Feature Selection (SFS) (Forward/Backward Selection)
- Wrapper methods can be explained with the help of following graphic:

<IPython.core.display.Image object>

13.2.2 Recursive Feature Elimination (RFE)

This is how RFE works:

- Start with all features in the dataset.
- Train a model (e.g., Random Forest, SVM, Logistic Regression).
- Rank feature importance using model coefficients (`coef_`) or feature importance scores (`feature_importances_`).
- Remove the least important feature(s).
- Repeat steps 2-4 recursively until reaching the desired number of features.
- Return the best subset of features.

```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestClassifier

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Initialize a model
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Apply Recursive Feature Elimination (RFE)
rfe_selector = RFE(model, n_features_to_select=5) # Select top 5 features
X_rfe_selected = rfe_selector.fit_transform(X, y)

# Get selected feature names
rfe_selected_features = X.columns[rfe_selector.support_]
print("Selected features using RFE:", rfe_selected_features.tolist())
```

Selected features using RFE: ['mean concave points', 'worst radius', 'worst perimeter', 'worst area']

13.2.3 Recursive Feature Elimination with Cross-Validation (RFECV)

RFECV is an extension of Recursive Feature Elimination (RFE) that automatically selects the optimal number of features using cross-validation.

```

from sklearn.feature_selection import RFECV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Initialize model (Random Forest)
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Define cross-validation strategy
cv = StratifiedKFold(n_splits=5)

# Apply RFECV
rfecv_selector = RFECV(estimator=model, step=1, cv=cv, scoring='accuracy', n_jobs=-1)
X_rfecv_selected = rfecv_selector.fit_transform(X, y)

# Get selected feature names
rfecv_selected_features = X.columns[rfecv_selector.support_]
print("Optimal number of features:", rfecv_selector.n_features_)
print("Selected features using RFECV:", rfecv_selected_features.tolist())

# Print feature rankings
feature_ranking = pd.DataFrame({'Feature': X.columns, 'Ranking': rfecv_selector.ranking_})
print(feature_ranking.sort_values(by='Ranking'))

```

Optimal number of features: 16

Selected features using RFECV: ['mean radius', 'mean texture', 'mean perimeter', 'mean area'

	Feature	Ranking
0	mean radius	1
22	worst perimeter	1
23	worst area	1
13	area error	1
24	worst smoothness	1
25	worst compactness	1
10	radius error	1
21	worst texture	1
26	worst concavity	1
7	mean concave points	1
6	mean concavity	1

3	mean area	1
2	mean perimeter	1
1	mean texture	1
27	worst concave points	1
20	worst radius	1
28	worst symmetry	2
5	mean compactness	3
4	mean smoothness	4
29	worst fractal dimension	5
12	perimeter error	6
16	concavity error	7
15	compactness error	8
14	smoothness error	9
18	symmetry error	10
17	concave points error	11
19	fractal dimension error	12
11	texture error	13
8	mean symmetry	14
9	mean fractal dimension	15

13.2.4 Sequential Feature Selection

SFS iteratively selects the most relevant features by **adding them one at a time (forward selection)** or **removing them one at a time (backward elimination)** while evaluating model performance at each step.

The selection strategy is controlled using: - **direction='forward'** for **feature addition** - **direction='backward'** for **feature removal**

```
from sklearn.feature_selection import SequentialFeatureSelector

from sklearn.linear_model import LogisticRegression

# Initialize a model
model = LogisticRegression(max_iter=1000)

# Apply Sequential Feature Selection (SFS)
sfs_selector = SequentialFeatureSelector(model, n_features_to_select=5, direction='forward',
X_sfs_selected = sfs_selector.fit_transform(X, y)

# Get selected feature names
sfs_selected_features = X.columns[sfs_selector.get_support()]
print("Selected features using SFS:", sfs_selected_features.tolist())
```

Selected features using SFS: ['mean radius', 'radius error', 'worst texture', 'worst perimeter']

Let's use them on the *credit* dataset

```
# Load the data
df = pd.read_csv('./Datasets/Credit.csv', index_col=0)

# Separate features and target
X = df.drop('Balance', axis=1)
y = df['Balance']

# Preprocessing
# One-hot encode categorical variables
X_encoded = pd.get_dummies(X, columns=['Gender', 'Student', 'Married', 'Ethnicity'])

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 1. Recursive Feature Elimination with Cross-Validation (RFECV)
def perform_rfecv(X_train, y_train, X_test, y_test):
    # Use both Linear Regression and Random Forest
    estimators = [
        ('Linear Regression', LinearRegression()),
        ('Random Forest', RandomForestRegressor(n_estimators=100, random_state=42))
    ]

    results = {}

    for name, estimator in estimators:
        # RFECV with cross-validation
        rfecv = RFECV(
            estimator=estimator,
            step=1,
            cv=5,
```

```

        scoring='neg_mean_squared_error'
    )

    # Fit RFECV
    rfecv.fit(X_train, y_train)

    # Get selected features
    selected_features = X_train.columns[rfecv.support_]

    # Prepare reduced datasets
    X_train_reduced = X_train[selected_features]
    X_test_reduced = X_test[selected_features]

    # Fit the model on reduced dataset
    estimator.fit(X_train_reduced, y_train)

    # Predict and evaluate
    y_pred = estimator.predict(X_test_reduced)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results[name] = {
        'Selected Features': list(selected_features),
        'Number of Features': len(selected_features),
        'MSE': mse,
        'R2': r2
    }

    return results

# 2. Sequential Feature Selection
def perform_sequential_feature_selection(X_train, y_train, X_test, y_test):
    # Use both Linear Regression and Random Forest
    estimators = [
        ('Linear Regression', LinearRegression()),
        ('Random Forest', RandomForestRegressor(n_estimators=100, random_state=42))
    ]

    results = {}

    for name, estimator in estimators:
        # Sequential Forward Selection

```

```

sfs_forward = SequentialFeatureSelector(
    estimator=estimator,
    n_features_to_select='auto', # will choose based on cross-validation
    direction='forward',
    cv=5,
    scoring='neg_mean_squared_error'
)

# Fit SFS
sfs_forward.fit(X_train, y_train)

# Get selected features
selected_features = X_train.columns[sfs_forward.get_support()]

# Prepare reduced datasets
X_train_reduced = X_train[selected_features]
X_test_reduced = X_test[selected_features]

# Fit the model on reduced dataset
estimator.fit(X_train_reduced, y_train)

# Predict and evaluate
y_pred = estimator.predict(X_test_reduced)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

results[name] = {
    'Selected Features': list(selected_features),
    'Number of Features': len(selected_features),
    'MSE': mse,
    'R2': r2
}

return results

# Run feature selection methods
print("1. Recursive Feature Elimination with Cross-Validation (RFECV):")
rfecv_results = perform_rfecv(X_train, y_train, X_test, y_test)

print("\n2. Sequential Feature Selection:")
sfs_results = perform_sequential_feature_selection(X_train, y_train, X_test, y_test)

```

```
# Print detailed results
def print_feature_selection_results(results):
    for model_name, result in results.items():
        print(f"\n{model_name}:")
        print(f"Number of Selected Features: {result['Number of Features']}")
        print("Selected Features:", result['Selected Features'])
        print(f"Mean Squared Error: {result['MSE']:.4f}")
        print(f"R2 Score: {result['R2']:.4f}")

print("\nRFECV Results:")
print_feature_selection_results(rfecv_results)

print("\nSequential Feature Selection Results:")
print_feature_selection_results(sfs_results)
```

1. Recursive Feature Elimination with Cross-Validation (RFECV):

2. Sequential Feature Selection:

RFECV Results:

Linear Regression:

Number of Selected Features: 15

Selected Features: ['Income', 'Limit', 'Rating', 'Cards', 'Age', 'Education', 'Gender_Male']

Mean Squared Error: 7974.8564

R2 Score: 0.9523

Random Forest:

Number of Selected Features: 5

Selected Features: ['Income', 'Limit', 'Rating', 'Student_No', 'Student_Yes']

Mean Squared Error: 9124.1126

R2 Score: 0.9454

Sequential Feature Selection Results:

Linear Regression:

Number of Selected Features: 7

Selected Features: ['Income', 'Limit', 'Rating', 'Cards', 'Age', 'Student_No', 'Student_Yes']

Mean Squared Error: 8058.6421

R2 Score: 0.9518

Random Forest:

Number of Selected Features: 7

Selected Features: ['Income', 'Limit', 'Rating', 'Gender_Female', 'Student_No', 'Student_Yes']

Mean Squared Error: 9953.4559

R2 Score: 0.9404

13.3 Embedded methods

Embedded methods integrate feature selection directly into the model training process. These methods learn which features are important while building the model, offering a balance between filter and wrapper methods.

13.3.1 Characteristics:

- **More efficient than wrapper methods:** Feature selection is built into the model training, avoiding the need for multiple iterations.
- **Less prone to overfitting than wrapper methods:** Regularization techniques help prevent overfitting.
- **Model-dependent:** The selected features are specific to the chosen model.
- **Common techniques:**
 - **Lasso (L1 Regularization):** Shrinks less important feature coefficients to zero.
 - **Decision Tree-based methods:** Feature importance scores from Random Forest, XGBoost, or Gradient Boosting.
 - **Elastic Net:** A combination of L1 and L2 regularization.

<IPython.core.display.Image object>

```
from sklearn.linear_model import LassoCV
from sklearn.ensemble import RandomForestRegressor

# Load data
df = pd.read_csv('./Datasets/Credit.csv').drop(columns=df.columns[0]) # Drop index column

# Clean categorical variables (trim whitespace)
categorical_cols = ['Gender', 'Student', 'Married', 'Ethnicity']
for col in categorical_cols:
    df[col] = df[col].str.strip()

# Convert categorical variables to dummy variables
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```



```

# Separate features and target
X = df.drop(columns=['Balance'])
y = df['Balance']

# Split data into train/test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize features for Lasso
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Lasso Regression for feature selection
lasso = LassoCV(cv=5, random_state=42)
lasso.fit(X_train_scaled, y_train)

# Get non-zero Lasso coefficients
lasso_feat = pd.Series(lasso.coef_, index=X.columns)
selected_lasso = lasso_feat[lasso_feat != 0].sort_values(ascending=False)

# Random Forest for feature importance
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Get feature importances
rf_feat = pd.Series(rf.feature_importances_, index=X.columns)
selected_rf = rf_feat.sort_values(ascending=False)

# Display results
print("Lasso Selected Features (Non-Zero Coefficients):\n", selected_lasso)
print("\nRandom Forest Feature Importances:\n", selected_rf)

```

```

Lasso Selected Features (Non-Zero Coefficients):
  Limit          408.156450
Student_Yes      106.371166
Cards            36.834149
Ethnicity_Caucasian  6.009746
Gender_Male      -0.254150
Age             -28.253276
dtype: float64

```

Random Forest Feature Importances:

```
Limit          0.458433
Rating         0.418695
Student_Yes    0.046555
Age            0.022970
Unnamed: 0     0.018321
Education      0.014302
Cards          0.010239
Married_Yes    0.003657
Gender_Male    0.002616
Ethnicity_Caucasian 0.002256
Ethnicity_Asian 0.001956
dtype: float64
```

13.4 Comparison of Feature Selection Methods

Method	Model Dependency	Computational Cost	Handles Feature Interactions	Risk of Overfitting
Filter	No	Low	No	Low
Wrapper	Yes	High	Yes	High
Embedded	Yes	Medium	Yes	Medium

13.5 Conclusion

- Use **Filter methods** when working with large datasets or when speed is a priority.
- Use **Wrapper methods** when accuracy is more important and computational cost is not a major concern.
- Use **Embedded methods** when leveraging models that inherently perform feature selection, such as Lasso regression or tree-based models.

By understanding these techniques, you can make better decisions when selecting features for machine learning models.

13.5.1 Reference:

- https://xavierbourretsicotte.github.io/subset_selection.html
- <https://www.kaggle.com/code/prashant111/comprehensive-guide-on-feature-selection>

- <https://www.analyticsvidhya.com/blog/2016/12/introduction-to-feature-selection-methods-with-an-example-or-how-to-select-the-right-variables/>

A Assignment 1: Statistical Learning and Simple Linear Regression

Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Do not write your name on the assignment.
3. Write your code in the **Code cells** and your answers in the **Markdown cells** of the Jupyter notebook. Ensure that the solution is written neatly enough to for the graders to understand and follow.
4. Use [Quarto](#) to render the **.ipynb** file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
5. There is a **bonus** question worth 15 points.
6. **Five points are properly formatting the assignment.** The breakdown is as follows:
 - Must be an HTML file rendered using Quarto **(1 point)**. *If you have a Quarto issue, you must mention the issue & quote the error you get when rendering using Quarto in the comments section of Canvas, and submit the ipynb file.*
 - No name can be written on the assignment, nor can there be any indicator of the student's identity—e.g. printouts of the working directory should not be included in the final submission. **(1 point)**
 - There aren't excessively long outputs of extraneous information (e.g. no printouts of entire data frames without good reason, there aren't long printouts of which iteration a loop is on, there aren't long sections of commented-out code, etc.) **(1 point)**
 - Final answers to each question are written in the Markdown cells. **(1 point)**
 - There is no piece of unnecessary / redundant code, and no unnecessary / redundant text. **(1 point)**

7. The maximum possible score in the assignment is $100 + 15$ (bonus question) $+ 5$ (proper formatting) = 120 out of 100. There is no partial credit for some parts of the bonus question.

A.1 1) Case Studies: Regression vs Classification and Prediction vs Inference (16 points)

For each case below, explain (1) whether it is a classification or a regression problem and (2) whether the main purpose is prediction or inference. **You need justify your answers for credit.**

A.1.1 1a)

You work for a company that is interested in conducting a marketing campaign. The goal of your project is to identify individuals who are likely to respond positively to a marketing campaign, based on observations of demographic variables (such as age, gender, income etc.) measured on each individual. **(2+2 points)**

A.1.2 1b)

For the same company, now you are working on a different project. This one is focused on understanding the impact of advertisements in different media types on the company sales. For example, you are interested in the following question: *‘How large of an increase in sales is associated with a given increase in radio and TV advertising?’* **(2+2 points)**

A.1.3 1c)

A company is selling furniture and they are interested in the finding the association between demographic characteristics of customers (such as age, gender, income etc.) and if they would purchase a particular company product. **(2+2 points)**

A.1.4 1d)

We are interested in forecasting the % change in the USD/Euro exchange rate using the weekly changes in the stock markets of a number of countries. We collect weekly data for all of 2023. For each week, we record the % change in the USD/Euro, the % change in the US market, the % change in the British market, and the % change in the German market. **(2+2 points)**

A.2 2) Examples for Different Regression Metrics: RMSE vs MAE (8 points)

A.2.1 2a)

Describe a regression problem, where it will be more proper to evaluate the model performance using the root mean squared error (RMSE) metric as compared to the mean absolute error (MAE) metric. **You need to justify your answer for credit. (4 points)**

Note: You are not allowed to use the datasets and examples covered in the lectures.

A.2.2 2b)

Describe a regression problem, where it will be more proper to evaluate the model performance using the mean absolute error (MAE) metric as compared to the root mean squared error (RMSE) metric. **You need to justify your answer for credit. (4 points)**

Note: You are not allowed to use the datasets and examples covered in the lectures.

A.3 3) Simple Linear Regression: Formulation (3 points)

When asked to state the simple linear regression model, a student wrote it as follows: $E(Y_i) = \beta_0 + \beta_1 X_i + \epsilon_i$. Is this correct (1 point)? Justify your answer (2 points).

A.4 4) Modeling the Petrol Consumption in U.S. States (58 points)

Read `petrol_consumption_train.csv`. Assume that each observation is a U.S. state. For each observation, the data has the following variables as its five columns:

`Petrol_tax`: Petrol tax (cents per gallon)

`Per_capita_income`: Average income (dollars)

`Paved_highways`: Paved Highways (miles)

`Prop_license`: Proportion of population with driver's licenses

`Petrol_consumption`: Consumption of petrol (millions of gallons)

A.4.1 4a)

Create a pairwise plot of all the variables in the dataset. **(1 point)** Print the correlation matrix of all the variables as well. **(1 point)** Which variable has the highest linear correlation with `Petrol_consumption`? **(1 point)**

Note: Remember that a pairwise plot is a visualization tool that you can find in the seaborn library.

A.4.2 4b)

Fit a simple linear regression model to predict `Petrol_consumption` using the column you found in **part a** as the only predictor. Print the model summary. **(3 points)**

A.4.3 4c)

When asked for a point estimate of the expected petrol consumption for a state where the proportion of population with driver's license is 54.4%, a person gave the estimate 488 million gallons because that is the mean value of `Petrol_consumption` for the two observations of `Prop_license` = 0.544 pieces in the dataset. Is there an issue with this approach? Explain. **(2 points)** If there is an issue, then suggest a better approach and use it to estimate the expected petrol consumption for a state where the proportion of population with driver's license is 54.4%. **(2 points)**

A.4.4 4d)

What is the increase in petrol consumption for an increase of 0.05 in the predictor? **(3 points)**

A.4.5 4e)

Does petrol consumption have a statistically significant relationship with the predictor? **You need to justify your answer for credit. (3 points)**

A.4.6 4f)

How much of the variation in petrol consumption can be explained by its linear relationship with the predictor? **(2 points)**

A.4.7 4g)

Predict the petrol consumption for a state in which 50% of the population has a driver's license. **(2 points)** What are the confidence interval **(2 points)** and the prediction interval **(2 points)** for your prediction? Which interval is wider? **(1 points)** Why? **(2 points)**

A.4.8 4h)

Predict the petrol consumption for a state in which 10% of the population has a driver's license. **(3 points)** Are you getting a reasonable outcome? **(1 point)** Why or why not? **(2 points)**

A.4.9 4i)

What is the residual standard error of the model? **(3 points)**

A.4.10 4j)

Using the trained model, predict the petrol consumption of the observations in `petrol_consumption_test.csv`. **(2 points)** and find the RMSE. **(2 points)** What is the unit of this RMSE value? **(1 point)**

A.4.11 4k)

Based on the answers to **part i** and **part j**, do you think the model is overfitting? **You need to justify your answer for credit. (3 points)**

A.4.12 4l)

Make a scatterplot of `Petrol_consumption` vs. the predictor using `petrol_consumption_test.csv`. **(1 point)** Over the scatterplot, plot the regression line **(1 point)**, the prediction interval **(2 points)**, and the confidence interval. **(2 points)**

Make sure that regression line, prediction interval lines, and confidence interval lines have different colors. **(1 point)** Display a legend that correctly labels the lines as well. **(1 point)** Note that you need two lines of the same color to plot an interval.

A.4.13 4m)

The dataset consists of 40 US States. If you combine this data with the data of the remaining 10 US States, are you likely to obtain narrower confidence and prediction intervals in the plot developed in the previous question, for the same level of confidence? Justify your answer. **(2 points)**.

If yes, then can you guarantee that the width of these intervals will reduce? Justify your answer. If no, then can you guarantee that the width of these intervals will not reduce? Justify your answer. **(2 points)**

A.4.14 4n)

Find the correlation between `Petrol_consumption` and the rest of the variables in `petrol_consumption_train.csv`. Which column would have the lowest R-squared value when used as the predictor for a Simple Linear Regression model to predict `Petrol_consumption`? Note that you can directly answer this question from the correlation values and do not need to develop any more linear regression models. **(2 points)**

A.5 5) Reproducing the Results with Scikit-Learn (15 points)

A.5.1 5a)

Using the same datasets, same response and the same predictor as **Question 4**, reproduce the following outputs with scikit-learn:

- Model RMSE for test data **(3 points)**
- R-squared value of the model **(3 points)**
- Residual standard error of the model **(3 points)**

Note that you are only allowed to use scikit-learn, pandas, and numpy tools for this question. Any other libraries will not receive any credit.

A.5.2 5b)

Which of the model outputs from **Question 4** cannot be reproduced using scikit-learn? Give two answers. **(2+2 points)** What does this tell about scikit-learn? **(2 points)**

A.6 6) Bonus Question (15 points)

Please note that the bonus question requires you to look more into the usage of the tools we covered in class and it will be necessary to do your own research. We strongly suggest attempting it after you are done with the rest of the assignment.

A.6.1 6a)

Fit a simple linear regression model to predict `Petrol_consumption` based on the predictor in **Question 4**, but **without an intercept term**. (5 points - no partial credit)

Without an intercept means that the equation becomes $Y = \beta_1 X$. The intercept term, β_0 , becomes 0.

Note: You must answer this part correctly to qualify for the bonus points in the following parts.

A.6.2 6b)

Predict the petrol consumption for the observations in `petrol_consumption_test.csv` using the model without an intercept and find the RMSE. (1+2 points) Then, print the summary and find the R-squared. (2 points)

A.6.3 6c)

The RMSE for the models with and without the intercept are similar, which indicates that both models are almost equally good. However, the R-squared for the model without intercept is much higher than the R-squared for the model with the intercept. Why? Justify your answer. (5 points - no partial credit)

B Assignment 2: MLR

Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Write your code in the **Code cells** and your answers in the **Markdown cells** of the Jupyter notebook. Ensure that the solution is written neatly enough to for the graders to understand and follow.
3. Use [Quarto](#) to render the **.ipynb** file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
4. **Five points are properly formatting the assignment.** The breakdown is as follows:
 - Must be an HTML file rendered using Quarto (**1 point**). *If you have a Quarto issue, you must mention the issue & quote the error you get when rendering using Quarto in the comments section of Canvas, and submit the ipynb file.*
 - No name can be written on the assignment, nor can there be any indicator of the student's identity—e.g. printouts of the working directory should not be included in the final submission. (**1 point**)
 - There aren't excessively long outputs of extraneous information (e.g. no printouts of entire data frames without good reason, there aren't long printouts of which iteration a loop is on, there aren't long sections of commented-out code, etc.) (**1 point**)
 - Final answers to each question are written in the Markdown cells. (**1 point**)
 - There is no piece of unnecessary / redundant code, and no unnecessary / redundant text. (**1 point**)
5. The maximum possible score in the assignment is $100 + 5$ (proper formatting) = 105 out of 100.

B.1 1) Multiple Linear Regression (24 points)

A study was conducted on 97 male patients with prostate cancer who were due to receive a radical prostatectomy (complete removal of the prostate). The **prostate.csv** file contains data on 9 measurements taken from these 97 patients. Each row (observation) represents a patient and each column (variable) represents a measurement. The description of variables can be found here: <https://rafalab.github.io/pages/649/prostate.html>

B.1.1 1a)

Fit a linear regression model with **lpsa** as the response and all the other variables as the predictors. Print its summary. (2 points) Write down the **optimal equation** that predicts **lpsa** using the predictors. (2 points)

B.1.2 1b)

Is the **overall regression** statistically significant? In other words, is there a statistically significant relationship between the response and at least one predictor? **You need to justify your answer for credit.** (2 points)

B.1.3 1c)

What does the optimal coefficient of **svi** tell us as a numeric output? Make sure you include the predictor, (**svi**) the response (**lpsa**) and the other predictors in your answer. (2 points)

B.1.4 1d)

Check the *p*-values of **gleason** and **age**. Are these predictors statistically significant? **You need to justify your answer for credit.** (2 points)

B.1.5 1e)

Check the 95% Confidence Interval of **age**. How can you relate it to its *p*-value and statistical significance, which you found in the previous part? (2 points)

B.1.6 1f)

This question requires some thinking, and bringing your 303-1 and 303-2 knowledge together.

Fit a **simple** linear regression model on `lpsa` against `gleason` and check the p -value of `gleason` using the summary. **(2 point)** Did the statistical significance of `gleason` change in the absence of other predictors? **(1 point)** Why or why not? **(3 points)**

Hints:

- 1) You need to compare this model with the Multiple Linear Regression model you created above.
- 2) Printing a correlation matrix of all the predictors should be useful.

B.1.7 1g)

Predict the `lpsa` of a **65 year old** man with `lcavol` = 1.35, `lweight` = 3.65, `lbph` = 0.1, `svi` = 0.22, `lcp` = -0.18, `gleason` = 6.75, and `pgg45` = 25. Find the 95% confidence and prediction intervals as well. **(2 points)**

B.1.8 1h)

In the Multiple Linear Regression model with all the predictors, you should see a total of five predictors that appear to be statistically insignificant. Why is it not a good idea to directly conclude that all of them are statistically insignificant? **(2 points)** Implement the additional test that concludes the statistical insignificance of all five predictors. **(2 points)**

Hint: `f_test()` method

B.2 2) Multiple Linear Regression with Variable Transformations (23 points)

The `infmort.csv` file has the infant mortality data of different countries in the world. The `mortality` column represents the infant mortality rate with “deaths per 1000 infants” as the unit. The `income` column represents the per capita income in USD. The other columns should be self-explanatory. (This is an old dataset, as can be seen from some country names.)

B.2.1 2a)

Start your analysis by creating (i) a boxplot of `log(mortality)` for each `region` and (ii) a boxplot of `income` for each `region`. Note that the `region` column has the continent names. **(3 points)**

Note: You need to use `np.log`, which is the natural log. This is to better distinguish the mortality values.

B.2.2 2b)

In the previous part, you should see that Europe has the lowest infant mortality rate on average, but it also has the highest per capita income on average. Our goal is to see if Europe still has the lowest mortality rate if we remove the effect of income. We will try to find an answer for the rest of this question.

Create four scatter plots: (i) `mortality` against `income`, (ii) `log(mortality)` against `income`, (iii) `mortality` against `log(income)`, and (iv) `log(mortality)` against `log(income)`. **(4 points)**

Based on the plots, create an appropriate model to predict the mortality rate as a function of per capita income. Print the model summary. **(2 points)**

B.2.3 2c)

Update the model you created in the previous part by adding `region` as a predictor. Print the model summary. **(2 points)**

B.2.4 2d)

Use the model developed in the previous part to compute a new `adjusted_mortality` variable for each observation in the data. **(5 points)** Adjusted mortality rate is the mortality rate after removing the estimated effect of income. You need to calculate it with the following steps:

- Multiply the (transformed) income column with its optimal coefficient. This is the estimated effect of income.
- Subtract the product from the (transformed) response column. This removes the estimated effect of income.
- You may need to do a inverse transformation to calculate the actual adjusted mortality rate values.

Make a boxplot of `log(adjusted_mortality)` for each `region`. **(2 points)**

B.2.5 2e)

Using the plots in parts **a** and **d**, answer the following questions:

- (i) Does Europe still have the lowest mortality rate on average after removing the effect of income?
- (ii) How did the distribution of values among different continents change after removing the effect of income? How did the comparison of different continents change? Does any non-European country have a lower mortality rate than all the European countries after removing the effect of income?

(5 points)

B.3 3) Variable Transformations and Interactions (33 points)

The `soc_ind.csv` dataset contains many social indicators of a number of countries. Each row is a country and each column is a social indicator. The column names should be clear on what the variables represent. The GDP per capita will be the response variable throughout this question.

B.3.1 3a)

Using correlations, find out the most useful predictor for a simple linear regression model with `gdpPerCapita` as the response. You can ignore categorical variables for now. Let that predictor be P . **(2 points)**

B.3.2 3b)

Create a scatterplot of `gdpPerCapita` vs P . Does the relationship between `gdpPerCapita` and P seem linear or non-linear? **(2 points)**

B.3.3 3c)

If the relationship in the previous part is non-linear, create three models:

- Only with P
- With P and its quadratic term
- With P , its quadratic term and its cubic term

(2x3 = 6 points)

Compare the R -squared values of the models. **(2 points)**

B.3.4 3d)

On the same figure:

- create the scatterplot in part b.
- plot the linear regression line (only using P)
- plot the polynomial regression curve that includes the quadratic and cubic terms.
- add a legend to distinguish the linear and cubic fits.

(3 points)

B.3.5 3e)

Develop a model to predict `gdpPerCapita` using P and `continent` as predictors. (No higher-order terms.)

1. Which continent creates the baseline? Write down its equation. **(2 points)**
2. For a given value of P , are there any continents that **do not** have a statistically significant difference of predicted `gdpPerCapita` from the baseline continent? If yes, then which ones, and why? If no, then why not? You need to justify your answers for credit. **(4 points)**

B.3.6 3f)

The model developed in the previous part has a limitation. It assumes that the increase in predicted `gdpPerCapita` with a unit increase in P does not depend on the `continent`.

Eliminate this limitation by including the interaction of `continent` with P in the model. Print the model summary of the model with interactions. **(2 points)** Which continent has the closest increase in predicted `gdpPerCapita` to the baseline continent with a unit increase in P . Which continent has the furthest? **You need to justify your answers for credit. (5 points)**

B.3.7 3g)

Using the model developed in the previous part, plot the regression lines of all the continents on the same figure. Put `gdpPerCapita` on the y-axis and P on the x-axis. **(4 points)** Use a legend to color-code the continents. **(1 point)**

Hint: For each continent, you need to construct the regression line using the appropriate coefficients from your interaction model. Remember that Africa is the baseline (reference) continent, so its line uses only the intercept and main `lifeFemale` coefficient. For other continents, you need to add their specific interaction terms to both the intercept and slope. Use `np.linspace()` to create x-values and `plt.plot()` for each continent with different colors and labels.

B.4 4) Prediction with Sklearn (20 points)

B.4.1 Instructions

1. Read the `soc_ind.csv` dataset and use the `Index` column as the index of the dataframe.
2. Drop the `geographic_location` and `country` columns since they are not useful for our prediction.
3. We will use **only sklearn and pandas** libraries in this task.
4. `gdpPerCapita` is the response (target) variable.
(2 points)
5. Use **only** `lifeFemale` and `continent` as predictors to match the tasks in the previous problem.
(2 points)
6. Build an sklearn **Pipeline** that includes:
 - A `ColumnTransformer` that applies `PolynomialFeatures(degree=2, include_bias=False)` to `lifeFemale`
 - One-hot encoding for `continent`
 - A `LinearRegression` model **(6 points)**
7. Split the dataset into a **training set (90%)** and a **test set (10%)**.
 - Set `random_state=3` for reproducibility.
(3 points)
8. Fit the pipeline on the training set, then compute **R-squared** and **RMSE** on **both** the training set and the test set.
(4 points)

9. Compare training vs test **R-squared** and **RMSE** and discuss whether your model shows signs of **overfitting** or **underfitting**.
(3 points)

C Datasets, assignment and project files

Datasets used in the book, assignment files, project files, and prediction problems report template can be found [here](#)